

CS620 - Modelling and Simulation

Lesson 1: Introduction [click](#)

[here to download the ppt](#)

INTRODUCTION TO SIMULATION

- A **simulation** is the imitation of the operation of a real-world process or system. Whether done by hand or on a computer, simulation involves the generation of an artificial history of a system and the observation of that artificial history to draw inferences concerning the operating characteristics of the real system.
- The **behavior** of a system as it evolves is studied by developing a simulation model. This model usually takes the form of a set of assumptions concerning the operation of the system. These assumptions are expressed in mathematical, logical, and symbolic relationships between the entities, or objects of interest, of the system.
 - Once developed and validated, a model can be used to investigate a wide variety of "**what if**" questions about the real-world system. Potential changes to the system can first be simulated, to predict their impact on system performance.
 - Simulation can also be used to study systems in the design stage before such systems are built. Thus, simulation modeling can be used both as an analysis tool for predicting the effect of changes to existing systems and as a design tool to predict the performance of new systems under varying sets of circumstances.
- In some instances, a model can be developed which is simple enough to be "solved" by mathematical methods. Such solutions might be found by the use of differential calculus, probability theory, algebraic methods, or other mathematical techniques. The solution usually consists of one or more numerical parameters, which are called **measures of the performance** of the system. However, many real-world systems are so complex that models of these systems are virtually impossible to solve mathematically. In these instances, numerical, computer-based simulation can be used to imitate the behavior of the system over time.

From the simulation, data are collected as if a real system were being observed. This simulation-generated data is used to estimate the measures of performance of the system.

Lesson 2 : When simulation is the appropriate tool

The availability of special-purpose simulation languages, massive computing capabilities at a decreasing cost per operation, and advances in simulation methodologies have made simulation one of the most widely used and accepted tools in operations research and systems analysis.

Circumstances under which simulation is the appropriate tool to use have been discussed by many authors. Simulation can be used for the following purposes:

1. Simulation enables the study of, and experimentation with, the interactions of a complex system or a subsystem within a complex system.
2. Informational, organizational, and environmental changes can be simulated, and the effect of these alterations on the model's behavior can be observed.
3. The knowledge gained during the designing of a simulation model could be of great value toward suggesting improvement in the system under investigation.
4. Changing simulation inputs and observing the resulting outputs can produce valuable insight into which variables are the most important and into how variables interact.
5. Simulation can be used as a pedagogical device to reinforce analytic solution methodologies.
6. Simulation can be used to experiment with new designs or policies before implementation, to prepare for what might happen.
7. Simulation can be used to verify analytic solutions.
8. Simulating different capabilities for a machine can help determine the requirements on it.
9. Simulation models designed for training make learning possible without the cost and disruption of on-the-job instruction.
10. An animation shows a system in simulated operation so that the plan can be visualized.
11. The modern system (factory, wafer fabrication plant, service organization, etc.) is so complex that its internal interactions can be treated only through simulation.

Lesson 3 : When it is not

This section is based on an article by Banks and Gibson [1997], who gave ten rules for evaluating when a simulation is not appropriate.

Rule 1: The first rule indicates that simulation should not be used when the problem can be solved by common sense. An example is given of an automobile tag facility serving customers who arrive randomly at an average rate of 10/hour and are served at a mean rate of 12/hour. To determine the minimum number of servers needed, simulation is not necessary. Just compute $100/12 = 8.33$ indicating that nine or more servers are needed.

Rule 2: The second rule says that simulation should not be used if the problem can be solved analytically. For example, under certain conditions, the average waiting time in the example above can be found from curves that were developed by Hillier and Lieberman [2002].

Rule 3: The next rule says that simulation should not be used if it is easier to perform direct experiments. An example of a fast-food drive-in restaurant is given where it was less expensive to stage a person taking orders using a hand-held terminal and voice communication to determine the effect of adding another order station on customer waiting time.

Rule 4: The fourth rule says not to use simulation if the costs exceed the savings. There are many steps in completing a simulation, as will be discussed, and these must be done thoroughly. If a simulation study costs \$20,000 and the savings might be \$10,000, then a simulation would not be appropriate.

Rule 5 & Rule 6: Rules five and six indicate that simulation should not be performed if the resources or time are not available. If the simulation is estimated to cost \$20,000 and there is only \$10,000 available, the suggestion is not to venture into a simulation study. Similarly, if a decision is needed in two weeks and a simulation will take a month, the simulation study is not advised.

Rule 7: Simulation takes data, sometimes lots of data. If no data is available, not even estimates, simulation is not advised.

Rule 8: The next rule concerns the ability to verify and validate the model. If there is not enough time or if the personnel are not available, simulation is not appropriate.

Rule 9: If managers have unreasonable expectations, if they ask for too much too soon, or if the power of simulation is overestimated, then simulation might not be the most appropriate tool.

Rule 10: Last, if system behavior is too complex or can't be defined, simulation is not appropriate. Human behavior is sometimes extremely complex to model.

Lesson 4 : Advantages and Disadvantage

Simulation is intuitively appealing to a client because it mimics what happens in a real system or what is perceived for a system that is in the design stage. The output data from a simulation should directly correspond to the outputs that could be recorded from the real system. Additionally, it is possible to develop a simulation model of a system without dubious assumptions (such as the same statistical distribution for every random variable) of mathematically solvable models. For these and other reasons, simulation is frequently the technique of choice in problem-solving.

Simulation Models vs. Optimization Models

In contrast to optimization models, simulation models are "run" rather than solved. Given a particular set of input and model characteristics, the model is run and the simulated behavior is observed. This process of changing inputs and model characteristics results in a set of scenarios that are evaluated. A good solution, either in the analysis of an existing system or in the design of a new system, is then recommended for implementation.

Simulation has many advantages, but some disadvantages. These are listed by Pegden, Shannon, and Sadowski [1995].

Some advantages of Simulation:

1. New policies, operating procedures, decision rules, information flows, organizational procedures, and so on can be explored without disrupting the ongoing operations of the real system.
2. New hardware designs, physical layouts, transportation systems, and so on can be tested without committing resources for their acquisition.
3. Hypotheses about how or why certain phenomena occur can be tested for feasibility.
4. Time can be compressed or expanded to allow for a speed-up or slow-down of the phenomena under investigation.
5. Insight can be obtained about the interaction of variables.
6. Insight can be obtained about the importance of variables to the performance of the system.
7. Bottleneck analysis can be performed to discover where work in process, information, materials, and so on are being delayed excessively.
8. A simulation study can help in understanding how the system operates rather than how individuals think the system operates.
9. "What if" questions can be answered. This is particularly useful in the design of new systems.

Lesson 5 : Disadvantages of Simulation

Disadvantages of Simulation

1. Model building requires special training. It is an art that is learned over time and through experience. Furthermore, if two models are constructed by different competent individuals, they might have similarities, but it is highly unlikely that they will be the same.
2. Simulation results can be difficult to interpret. Most simulation outputs are essentially random variables (they are usually based on random inputs), so it can be hard to distinguish whether an observation is a result of system interrelationships or randomness.
3. Simulation modeling and analysis can be time-consuming and expensive. Skimping on resources for modeling and analysis could result in a simulation model or analysis that is not sufficient for the task.
4. Simulation is used in some cases when an analytical solution is possible, or even preferable, as was discussed earlier. This might be particularly true in the simulation of some waiting lines where closed-form queueing models are available.

Counteracting the disadvantages of simulation: In defense of simulation, these four disadvantages, respectively, can be offset as follows:

1. Vendors of simulation software have been actively developing packages that contain models that need only input data for their operation. Such models have the generic tag "simulator" or "template."
2. Many simulation software vendors have developed output-analysis capabilities within their packages for performing very thorough analyses.
3. Simulation can be performed faster today than yesterday and will be even faster tomorrow, because of advances in hardware that permit rapid execution of scenarios and because of advances in many simulation packages. For example, some simulation software contains constructs for modeling material handling that uses such transporter as fork-lift trucks, conveyors, and automated guided vehicles.
4. Closed-form models are not able to analyze most of the complex systems that are encountered in practice. In many years of consulting practice by two of the authors, not one problem was encountered that could have been solved by a closed-form solution.

Lesson 6 : Areas of application

The applications of simulation are vast. The Winter Simulation Conference (WSC) is an excellent way to learn more about the latest in simulation applications a theory. There are also numerous tutorials at both the beginning and the advanced levels. WSC is sponsored by six

technical societies and the National Institute of Standards and Technology (IST). The technical societies are the American Statistical Association (ASA), Association for Computing Machinery/Special Interest Group on Simulation (ACMIGSI), Institute of Electrical and Electronics Engineers: Computer Society (IEECS), Institute of Electrical and Electronics Engineers: Systems, Man, and Cybernetics Society (IEE, SMCS), Institute of Industrial Engineers (IIE), Institute for Operations Research and the Management Sciences: College on Simulation (INFORMS/CS) and The Society for Computer Simulation (SCS). Note that IEEE is represented by two bodies. Information about the upcoming WSC can be obtained from www.wintersim.org. WSC programs with full papers are available from www.informscs.org/wscpapers.html. Some presentations, by area, from a recent WSC are listed next:

Manufacturing Applications

- Dynamic modeling of continuous manufacturing systems, using analogies to electrical systems
- Benchmarking of a stochastic production planning model in a simulation testbed
- Paint line color change reduction in automobile assembly
- Modeling for quality and productivity in steel cord manufacturing
- Shared resource capacity analysis in biotech manufacturing
- Neutral information model for simulating machine shop operations

Semiconductor Manufacturing

- Constant time interval production planning with application to work-in-process control
- Accelerating products under due-date oriented dispatching rules
- Design framework for automated material handling systems in 300mm wafer fabrication factories
- Making optimal design decisions for next-generation dispensing tools • Application of cluster tool modeling in a 300mm wafer fabrication factory.
- Resident-entity based simulation of batch chamber tools in 300mm semiconductor manufacturing

Construction Engineering and Project Management

- Impact of multitasking and merge bias on procurement of complex equipment
- Application of lean concepts and simulation for drainage operations maintenance crews
- Building a virtual shop model for steel fabrication
- Simulation of the residential lumber supply chain

Lesson 7 : Military applications

Military Applications

- Multinational Intra-Theatre Logistics Distribution
- Examining Future Sustainability of Canadian Forces Operations
- Feasibility Study for Replacing the MK19 Automatic Grenade Launching System
- Training Joint Forces for Asymmetric Operations
- Multi-Objective Unmanned Aerial Vehicle Mission Planning
- Development of Operational Requirements Driven Federations **Logistics,**

Transportation, and Distribution

- Operating Policies for a Barge Transportation System
- Dispensing Plan for Emergency Medical Supplies in the Event of Bioterrorism
- Analysis of a Complex Mail Transportation Network
- Improving the Performance of Container Terminals
- Yard Crane Dispatching Based on Real-Time Data
- Unit Loading Device Inventory in Airline Operations
- Inventory Systems with Forecast Based Policy Updating
- Dock Allocation in a Food Distribution Center
- Operating Policies for a Barge Transportation System **Business Processing**
- A New Policy for the Service Request Assignment Problem
- Process Execution Monitoring and Adjustment Schemes
- In-Store Merchandising of Retail Stores
- Sales Forecasting for Retail Small Stores **Health Care**
- Interventions to Reduce Appointment Lead-Time and Patient No-Show Rate
- Supporting Smart Thinking to Improve Hospital Performance
- Verification of Lean Improvement for Emergency Room Process
- Reducing Emergency Department Overcrowding
- Inventory Modeling of Perishable Pharmaceuticals
- Implementation of an Outpatient Procedure Center
- Infectious Disease Control Policy
- Balancing Operating Room and Post-Anesthesia Resources
- Cost-Effectiveness of Colorectal Cancer Screening Tests

Conclusion

Simulation can be performed faster today than yesterday and will be even faster tomorrow, because of advances in hardware that permit the rapid execution of scenarios and because of advances in many simulation packages.

For example, some simulation software contains constructs for modeling material handling that uses such transporters as fork-lift trucks, conveyors, and automated guided vehicles

Closed-form models are not able to analyze most of the complex systems that are encountered in practice.

In many years of consulting practice, not one problem was encountered that could have been solved by a closed-form solution. We have seen such a wide variety of areas where simulation has applications.

To model a system, it is necessary to understand the concept of a **system and the system boundary**.

SYSTEM

- A **system** is defined as a group of objects that are joined together in some regular interaction or interdependence toward the accomplishment of some purpose. An example is a production system

manufacturing automobiles. The machines, component parts, and workers operate jointly along an assembly line to produce a high-quality vehicle.

SYSTEM ENVIRONMENT

A system is often affected by changes occurring outside the system. Such changes are said to occur in the system environment [Gordon, 1978].

- In modeling systems, it is necessary to decide on the boundary between the system and its environment. This decision may depend on the purpose of the study.
- In the case of the factory system, for example, the factors controlling the arrival of orders may be considered to be outside the influence of the factory and therefore part of the environment. However, if the effect of supply on demand is to be considered, there will be a relationship between factory output and the arrival of orders, and this relationship must be considered an activity of the system.
- Similarly, in the case of a bank system, there could be a limit on the maximum interest rate that can be paid. For the study of a single bank, this would be regarded as a constraint imposed by the environment.
- In a study of the effects of monetary laws on the banking industry, however, the setting of the limit would be an activity of the system [Gordon, 1978]

Lesson 9 : System Concepts

- **System:** An organized entity made up of interrelated and interdependent parts.

- **Boundaries:** Barriers that define a system and distinguish it from other systems in the environment.
- **Homeostasis:** The tendency of a system to be resilient towards external factors and maintain its key characteristics.
- **Adaptation:** The tendency of a self-adapting system to make the internal changes needed to protect itself and keep fulfilling its purpose.
- **Reciprocal Transactions:** Circular interactions that systems engage in such that they influence one another.
- **Feedback Loop:** The process by which systems self-correct based on reactions from other systems in the environment.
- **Throughput:** Rate of energy transfer between the system and its environment during the time it is functioning.
- **Microsystem:** The system closest to the client.
- **Mesosystem:** Relationships among the systems in an environment.
- **Exosystem:** A relationship between two systems that has an indirect effect on a third system.
- **Macrosystem:** A larger system that influences clients, such as policies, administration of entitlement programs, and culture.
- **Chronosystem:** A system composed of significant life events that can affect adaptation.

Lesson 10 : Components of a system

To understand and analyze a system, several terms need to be defined.

ENTITY: An entity is an object of interest in the system.

ATTRIBUTE: An attribute is a property of an entity.

ACTIVITY: An activity represents a time period of a specified length.

EXAMPLE: If a bank is being studied, customers might be one of the entities, the balance in their checking accounts might be an attribute, and making deposits might be an activity.

The collection of entities that compose a system for one study might only be a subset of the overall system for another study [Law, 2007]. For example, if the aforementioned bank is being studied to determine the number of tellers needed to provide for paying and receiving, the system can be defined as that portion of the bank consisting of the regular tellers and the customers waiting in line. If the purpose of the study is expanded to determine the number of special tellers needed (to prepare cashier's checks, to conduct commercial transactions, etc.), the definition of the system must be expanded.

STATE: The state of a system is defined to be the collection of variables necessary to describe the system at any time, relative to the objectives of the study.

- In the study of a bank, possible state variables are the number of busy tellers, the number of customers waiting in line or being served, and the arrival time of the next customer.

EVENT: An event is defined as an instantaneous occurrence that might change the state of the system.

ENDOGENOUS VS. EXOGENOUS EVENTS: The term endogenous is used to describe activities and events occurring within a system, and the term exogenous is used to describe activities and events in the environment that affect the system.

- In the bank study, the arrival of a customer is an exogenous event, and the completion of the service of a customer is an endogenous event.

Table 1 lists examples of entities, attributes, activities, events, and state variables for several systems. Only a partial listing of the system components is shown. A complete list cannot be developed unless the purpose of the study is known. Depending on the purpose, various aspects of the system will be of interest, and then the listing of components can be completed.

Table 1 Examples of Systems and Their Components

<i>System</i>	<i>Entities</i>	<i>Attributes</i>	<i>Activities</i>	<i>Events</i>	<i>State Variables</i>
Banking	Customers	Checking-account balance	Making deposits	Arrival; departure	Number of busy tellers; number of customers waiting
Rapid rail	Riders	Origin; destination	Traveling	Arrival at station; arrival at destination	Number of riders waiting at each station; number of riders in transit
Production	Machines	Speed; capacity; breakdown rate	Welding; stamping	Breakdown	Status of machines (busy, idle, or down)
Communications	Messages	Length; destination	Transmitting	Arrival at destination	Number waiting to be transmitted
Inventory	Warehouse	Capacity	Withdrawing	Demand	Levels of inventory; backlogged demands

Lesson 11 : Discrete and Continuous

[click here to download the ppt](#)

SYSTEM CATEGORIES: Systems can be categorized as discrete or continuous. “Few systems in practice are wholly discrete or continuous, but since one type of change predominates for most systems, it will usually be possible to classify a system as being either discrete or continuous” [Law, 2007].

DISCRETE SYSTEM: A discrete system is one in which the state variable(s) change only at a discrete set of points in time. The bank is an example of a discrete system: The state variable, the number of customers in the bank, changes only when a customer arrives or when the service provided by a customer is completed. **Figure 1** shows how the number of customers changes only at discrete points in time.

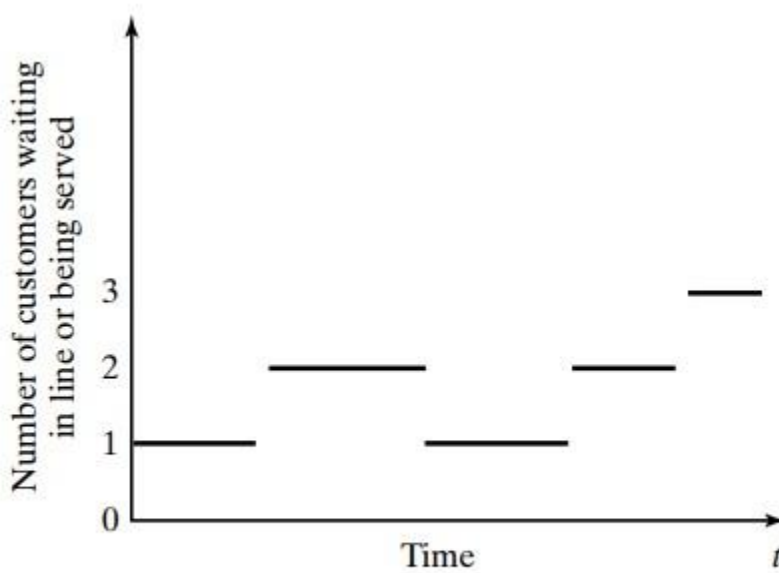


Figure 1 Discrete-system state variable.

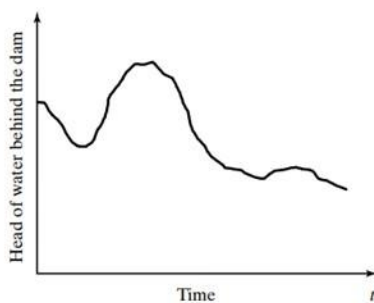


Figure 2 Continuous-system state variable.

CONTINUOUS SYSTEM: A continuous system is one in which the state variable(s) change continuously over time. An example is the head of water behind a dam. During and for some time after a rainstorm, water flows into the lake behind the dam. Water is drawn from the dam for flood control and to make electricity. Evaporation also decreases the water level. Figure 2 shows how the state variable head of water behind the dam changes for this continuous system.

Lesson 12 : Model of a system

Why Model?

Sometimes it is of interest to study a system to understand the relationships between its components or to predict how the system will operate under a new policy. To study the system, it is sometimes possible to experiment with the system itself. However, this is not always an option. A new system might not yet exist; it could be in a hypothetical form only or at a design stage. Even if the system exists, it might be impractical to experiment with it.

BANK EXAMPLE: In the case of a bank, reducing the number of tellers to study the effect on the length of waiting lines might infuriate the customers so greatly that they will move their accounts to a competitor. Consequently, studies of systems are often accomplished with a model of a system.

Example: Consulting job for the simulation of a redesigned port in Western Australia

At US\$300 million for a loading/unloading berth, it is not advisable to invest that amount only to find that the berth is inadequate for the task.

Model: A model is defined as a representation of a system to study that system. For most studies, it is only necessary to consider those aspects of the system that affect the problem under investigation. These aspects are represented in a model of the system; the model, by definition, is a simplification of the system.

On the other hand, the model should be sufficiently detailed to permit valid conclusions to be drawn about the real system. Different models of the same system could be required as the purpose of investigation changes.

Similar to the components of a system including entities, attributes, and activities, models are developed to represent them. However, the model contains only those components that are relevant to the study.

Lesson 13 : Types of Models

Types of Models: Models can be classified as being mathematical or physical.

Mathematical Model: A mathematical model uses the symbolic notation and mathematical equations to represent a system. A simulation model is a particular type of mathematical model of a system.

Physical model: A physical model is a larger or smaller version of an object such as the enlargement of an atom or a scaled-down version of the solar system.

Classification of Simulation Models: Simulation models may be further classified as being static or dynamic, deterministic or stochastic, and discrete or continuous.

Static Simulation Model: A static simulation model, sometimes called a Monte Carlo simulation, represents a system at a particular point in time.

Dynamic Simulation Model: Dynamic simulation models represent systems as they change over time. The simulation of a bank from 9:00 A.M. to 4:00 P.M. is an example of a dynamic simulation.

Deterministic Simulation Models: Simulation models that contain no random variables are classified as deterministic. Deterministic models have a known set of inputs, that will result in a unique set of outputs. Deterministic arrivals would occur at a dentist's office if all patients arrived at their scheduled appointment times.

Stochastic Simulation Model: A stochastic simulation model has one or more random variables as inputs. Random inputs lead to random outputs. Since the outputs are random, they can be considered only as estimates of the true characteristics of a model. The simulation of a bank would usually involve random interarrival times and random service times. Thus, in a stochastic simulation, the output measures—the average number of people waiting, the average waiting time of a customer—must be treated as statistical estimates of the true characteristics of the system.

Discrete and continuous Models: Discrete and continuous models are defined analogously to discrete and continuous systems discussed earlier. However, a discrete simulation model is not always used to model a discrete system, nor is a continuous simulation model always used to model a continuous system.

Tanks and pipes might be modeled discretely, even though we know that fluid flow is continuous. In addition, simulation models may be mixed, both discrete and continuous. The choice of whether to use a discrete or continuous (or both discrete and continuous) simulation model is a function of the characteristics of the system and the objective of the study. Thus, a communication channel could be modeled discretely if the characteristics and movement of each message were deemed important.

Conversely, if the flow of messages in aggregate over the channel were of importance, modeling the system via continuous simulation could be more appropriate. The models emphasized in this course are dynamic, stochastic, and discrete.

Lesson 14 : Discrete Event Simulation

Discrete-event system simulation: It is the modeling of systems in which the state variable changes only at a discrete set of points in time. The simulation models are analyzed by numerical rather than analytical methods.

- **Analytical methods:** These methods employ the deductive reasoning of mathematics to “solve” the model. For example, differential calculus can be used to compute the minimum cost policy for some inventory models.
- **Numerical methods:** These methods employ computational procedures to “solve” mathematical models. In the case of simulation models, which employ numerical methods, models are “run” rather than being “solved”—that is, an artificial history of the system is generated from the model assumptions, and observations are collected to be analyzed and to estimate the true system performance measures.

Real-world simulation models are rather large, and the amount of data stored and manipulated is vast, so these “runs” are usually conducted with the aid of a computer. In summary, in discrete event system simulation, the models of interest are analyzed numerically, usually with the aid of a computer.

Lesson 15 : Steps in a sim. Study: Planning

STEPS IN A SIMULATION STUDY: **Figure 3** shows a set of steps to guide a model builder in a thorough and sound simulation study

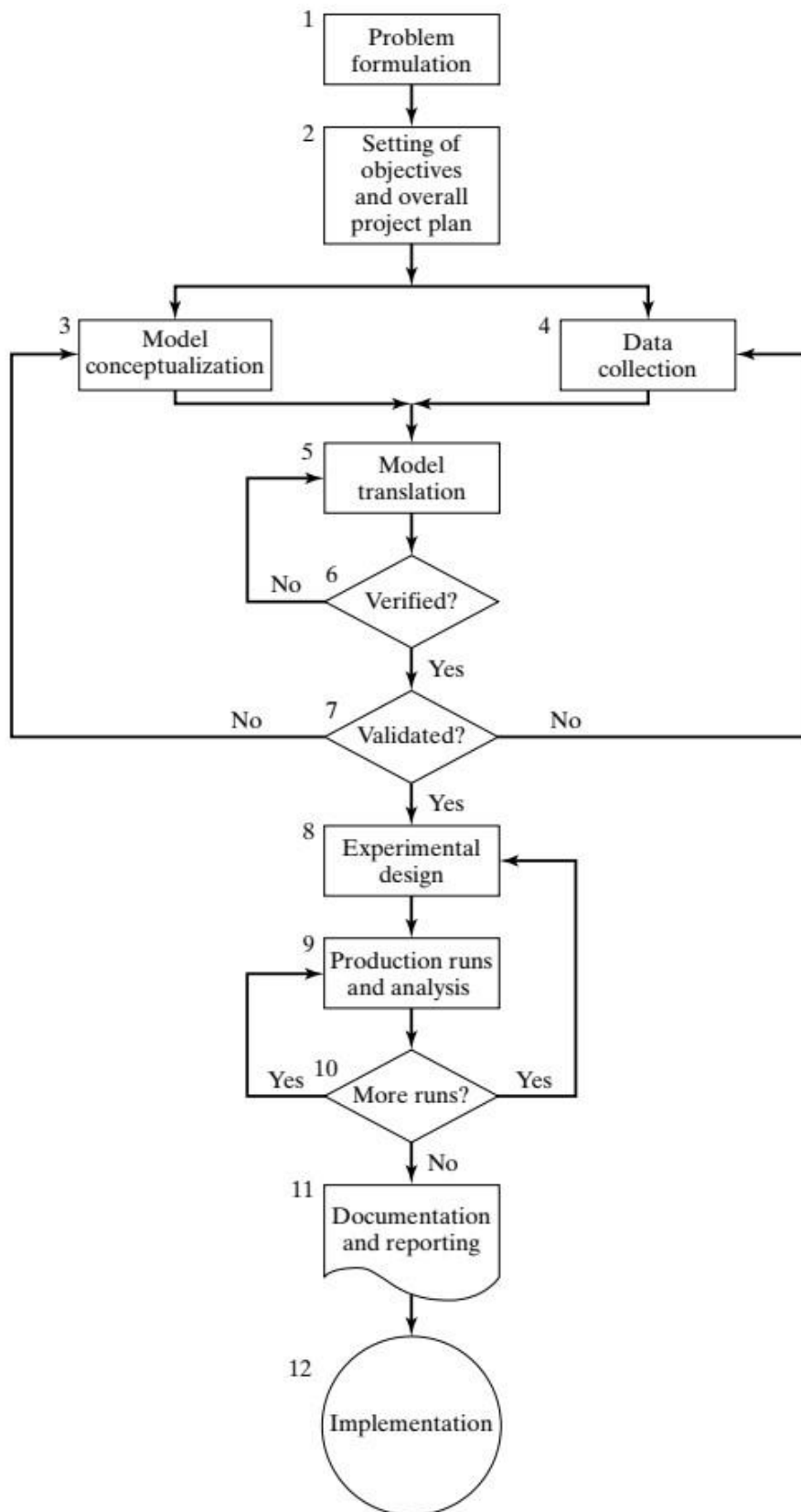


Figure 3 Steps in a simulation study.

y. The steps in a

simulation study are as follows:

1. Problem formulation: Every study should begin with a statement of the problem. If the statement is provided by the policymakers or those that have the problem, the analyst must ensure that the problem being described is clearly understood. If a problem statement is being developed by the analyst, the policymakers must understand and agree with the formulation.

Although not shown in Figure 3, there are occasions where the problem must be reformulated as the study progresses. In many instances, policymakers and analysts are aware that there is a problem long before the nature of the problem is known.

2. Setting of objectives and overall project plan: The objectives indicate the questions to be answered by simulation. At this point, a determination should be made concerning whether simulation is the appropriate methodology for the problem as formulated and the objectives as stated.

Assuming that it is decided that simulation is appropriate, the overall project plan should include a statement of the alternative systems to be considered and of a method for evaluating the effectiveness of these alternatives. It should also include the plans for the study in terms of the number of people involved, the cost of the study, and the number of days required to accomplish each phase of the work, along with the results expected at the end of each stage.

Lesson 16 : Steps in a sim. Study: Conceptualization

STEPS IN A SIMULATION STUDY

3. Model conceptualization: The construction of a model of a system is probably as much art as science. Pritsker [1998] provides a lengthy discussion of this step. “Although it is not possible to

provide a set of instructions that will lead to building successful and appropriate models in every instance, there are some general guidelines that can be followed” [Morris, 1967]. The art of modeling is enhanced by an ability to abstract the essential features of a problem, to select and modify basic assumptions that characterize the system, and then to enrich and elaborate the model until useful approximation results. Thus, it is best to start with a simple model and build toward greater complexity. However, the model complexity need not exceed that required to accomplish the purposes for which the model is intended. Violation of this principle will only add to modelbuilding and computer expenses. It is not necessary to have a one-to-one mapping between the model and the real system. Only the essence of the real system is needed.

Involving the Model User in Conceptualization: It is advisable to involve the model user in model conceptualization. Involving the model user will both enhance the quality of the resulting model and increase the confidence of the model user in the application of the model.

Lesson 17 : Steps in a sim. Study: Data Collection and Model Translation

STEPS IN A SIMULATION STUDY

3. Model conceptualization: The construction of a model of a system is probably as much art as science. Pritsker [1998] provides a lengthy discussion of this step. “Although it is not possible to provide a set of instructions that will lead to building successful and appropriate models in every instance, there are some general guidelines that can be followed” [Morris, 1967]. The art of modeling is enhanced by an ability to abstract the essential features of a problem, to select and modify basic assumptions that characterize the system, and then to enrich and elaborate the model until useful approximation results. Thus, it is best to start with a simple model and build toward greater complexity. However, the model complexity need not exceed that required to accomplish the purposes for which the model is intended. Violation of this principle will only add to modelbuilding and computer expenses. It is not necessary to have a one-to-one mapping between the model and the real system. Only the essence of the real system is needed.

Involving the Model User in Conceptualization: It is advisable to involve the model user in model conceptualization. Involving the model user will both enhance the quality of the resulting model and increase the confidence of the model user in the application of the model.

Lesson 18 : Steps in a sim. Study: Verification and Validation

6. Verified? Verification pertains to the computer program that has been prepared for the simulation model. Is the computer program performing properly? With complex models, it is difficult, if not impossible; to translate a model successfully in its entirety without a good deal of debugging; if the input parameters and logical structure of the model are correctly represented in the computer, verification has been completed. For the most part, common sense is used in completing this step.

7. Validated? Validation usually is achieved through the calibration of the model, an iterative process of comparing the model against actual system behavior and using the discrepancies between the two, and the insights gained, to improve the model. This process is repeated until model accuracy is judged acceptable. In the previously mentioned example of a bank, data were collected concerning the length of the waiting lines under current conditions. Does the simulation model replicate this system measure? This is one means of validation.

Lesson 19 : Steps in a sim. Study: Experimental Design

8. Experimental design: The alternatives that are to be simulated must be determined. Often, the decision concerning which alternatives to simulate will be a function of runs that have been completed and analyzed. For each system design that is simulated, decisions need to be made concerning the length of the initialization period, the length of simulation runs, and the number of replications to be made of each run. (Sanchez [2007] discusses this topic extensively.)

9. Production runs and analysis: Production runs and their subsequent analysis, are used to estimate measures of performance for the system designs that are being simulated. (Software to aid in this step, include AutoStat (in AutoMod), OptQuest (in several pieces of simulation software), and SimRunner (in ProModel).

10. More runs? Given the analysis of runs that have been completed, the analyst determines whether additional runs are needed and what design those additional experiments should follow

Lesson 20 : Steps in a sim. Documentation

11. Documentation and reporting: There are two types of documentation: program and progress.

Program documentation: It is necessary for numerous reasons. If the program is going to be used again by the same or different analysts, it could be necessary to understand how the program operates. This will create confidence in the program, so that model users and policymakers can make decisions based on the analysis. Also, if the program is to be modified by the same or a different analyst, this step can be greatly facilitated by adequate documentation. One experience with an inadequately documented program is usually enough to convince an analyst of the necessity of this important step. Another reason for documenting a program is so that model users can change parameters at will to learn the relationships between input parameters and output measures of performance or to discover the input parameters that “optimize” some output measure of performance.

Musselman [1998] discusses progress reports that provide the important, written history of a simulation project.

Lesson 21 : Steps in a sim. Study: Logs & Reports

Logs and Reports: Project reports give a chronology of work done and decisions made. This can prove to be of great value in keeping the project on course.

Musselman suggests frequent reports (monthly, at least) so that even those not involved in the day-to-day operation can be kept abreast. The awareness of these others can often enhance the successful completion of the project by surfacing misunderstandings early when the problem can be solved easily. Musselman also suggests maintaining a project log to provide a comprehensive record of accomplishments, change requests, key decisions, and other items of importance.

On the reporting side, Musselman suggests frequent deliverables. These may or may not be the results of major accomplishments. His maxim is that “it is better to work with many intermediate milestones than with one absolute deadline.” Possibilities before the final report include a model specification, prototype demonstrations, animations, training results, intermediate analyses, program documentation, progress reports, and presentations. He suggests that these deliverables should be timed judiciously over the life of the project.

The results of all the analyses should be reported clearly and concisely in a final report. This will allow the model users (now the decision-makers) to review the final formulation, the alternative systems that were addressed, the criteria by which the alternatives were compared, the results of

the experiments, and the recommended solution(s) to the problem. Furthermore, if decisions have to be justified at a higher level, the final report should provide a vehicle of certification for the model user/decision-maker and add to the credibility of the model and the model-building process.

12. Implementation: The success of the implementation phase depends on how well the previous eleven steps have been performed. It is also contingent upon how thoroughly the analyst has involved the ultimate model user during the entire simulation process. If the model user has been involved during the entire model-building process and if the model user understands the nature of the model and its outputs, the likelihood of a vigorous implementation is enhanced [Pritsker, 1995].

Conversely, if the model and its underlying assumptions have not been properly communicated, implementation will probably suffer, regardless of the simulation model's validity.

Lesson 22 : Steps in a sim. Study: Phases Overview

Simulation-model building process in Phases: The simulation-model building process shown in Figure 3 can be broken down into four phases.

Phase 1: The first phase, consisting of steps 1 (Problem formulation) and 2 (Setting of objective and overall design), is a period of discovery or orientation. The initial statement of the problem is usually quite “fuzzy,” the initial objectives will usually have to be reset, and the original project plan will usually have to be fine-tuned. These recalibrations and clarifications could occur in this phase or perhaps will occur after or during another phase (i.e., the analyst might have to restart the process).

Phase 2: The second phase is related to model building and data collection and includes steps 3 (Model conceptualization), 4 (Data collection), 5 (Model translation), 6 (Verification), and 7 (Validation). A continuing interplay is required among the steps. Exclusion of the model user during this phase can have dire implications at the time of implementation.

Phase 3: The third phase concerns the running of the model. It involves steps 8 (Experimental design), 9 (Production runs and analysis), and 10 (More runs). This phase must have a

comprehensively conceived plan for experimenting with the simulation model. A discrete-event stochastic simulation is, in fact, a statistical experiment. The output variables are estimates that contain random error, and therefore a proper statistical analysis is required. Such a philosophy is in contrast to that of the analyst who makes a single run and draws an inference from that single data point.

Phase 4: The fourth phase, implementation, involves steps 11 (Documentation and reporting) and 12 (Implementation). Successful implementation depends on the continual involvement of the model user and the successful completion of every step in the process. Perhaps the most crucial point in the entire process is Step 7 (Validation) because an invalid model is going to lead to erroneous results, which, if implemented, could be dangerous, costly, or both.

Lesson 23 : Simulation Case Studies Basic steps

Simulation Case Studies

- In this module, we shall see an overview of various case studies

Variety of Examples

- We shall be taking a look at a couple of examples.
- The variety will include hands-on as well as spreadsheet examples.
- Key objective: Take a look at Discrete Event Simulation and also descriptive

statistics for the prediction of system performance **General Steps**

- Every simulation that we observe will be based on 3 key steps
- The idea is to examine the examples using a methodological approach

First Step

- Determine the characteristics of each of the inputs to the simulation.
- Quite often, these are modeled as probability distributions, either continuous or discrete.

Second Step

- Construct a simulation table.
- Each simulation table is different, for each is developed for the problem at hand.

Example: Simulation Table

- An example of a simulation table is shown in Table.
- In this example, there are p inputs, $x_{ij}, j = 1, 2, \dots, p$, • One response y_i for each of repetitions (or, trials) • $i = 1, 2, \dots, n$.
- Initialize the table by filling in the data for repetition 1

Table

Repetitions	Inputs						Response
	x_{i1}	x_{i2}	...	x_{ij}	...	x_{ip}	
1							
2							
3							
⋮							
⋮							
⋮							
n							

Step 3

- For each repetition, i , generate a value for each of the p inputs and evaluate the function
- Calculate a value of the response y_i .
- The input values may be computed by sampling values from the distributions chosen in step 1
- A response typically depends on the inputs and one or more previous responses.
- Also, if the program is to be modified by the same or a different analyst, this step can be greatly facilitated by adequate documentation.
- One experience with an inadequately documented program is usually enough to convince an analyst of the necessity of this important step.

Overview of Examples

- Queuing
 - Inventory
 - Reliability
 - Network analysis.
 - The two queuing examples provide ○ single-server ○ two-server system
- First Example Overview**

- The first of the inventory examples involves a problem that has a closed-form solution;
- What it means is that the simulation solution can be compared to the mathematical solution.

Second Inventory Example

- The second inventory example pertains to the classic order-level model.

More examples

- Next, there is an example that introduces the concept of random normal numbers
- Also, there is a model for the simulation of lead-time demand.
- The examples conclude with the analysis of a network.

Summary

- In this module, we have examined how simulation examples give us a clear picture of how to do simulations
- We also took a look at the type of examples that we shall be examining in our next module

Lesson 24 : Basics of a Spreadsheet Simulation: Simulating Randomness

Basics of Spreadsheet Simulation

- In this module, we shall see an overview of spreadsheet simulation
- We shall also take a look at simulating randomness **How to Simulate**

Randomness

- Almost every simulation we encounter needs at least one random variable •
E.g. Coin Toss (Head vs. Tail)



Another Example: Queue

- **Arrival and departure rate**



Taking a Step Back: What is Random?

- In the real world, the number of factors involved in any event is so numerous that it can be very difficult to predict such events.
- In a computer, this can be very hard to generate.
- Hence the concept of “pseudorandomness”

General Steps

- Generate a number between 0 and 1 – referred to as the “random number”
- “Random variable” is any randomly generated quantity with a specified statistical distribution.
- A method of generating a sequence of random numbers is called a Random Number Generator (RNG)

Important Statistical Properties

- The number should be uniformly distributed between 0 and 1
- Subsequent numbers should be statistically independent of all previous numbers

VBA in MS Excel

- We use the RAND formula = RAND()
- More complex formulas can be used e.g. IF(RAND() <= 0.5, 0.1)
- Here IF checks the first condition and if true, returns the second one
- Another option is RANDBETWEEN()
- Rnd01
- Discrete Uniform(min, max)
- DiscreteEmp(rCumProb, rValues)
- Uniform(low, high)
- NORMSINV(Rnd01())
- For manual simulation, we can use coin toss or dice (Impractical for anything large, however)

Summary

- In this module, we have examined the basics of simulation in a spreadsheet

Lesson 25 : Simulating a Coin Toss

How can we simulate a coin toss in a spreadsheet model? The solution shows how to use a uniform random number generator in a spreadsheet simulation.

Example 1: Coin Tossing

We want to simulate a sequence of 10 coin tosses. The coin is a “fair” coin meaning that the chance of getting ahead is the same as getting a tail. Each has a probability of 0.5 of occurring. You should run the simulation multiple times and compare the results with your expectations from real-life coin tossing. This is an example of a Monte Carlo simulation; no events or clock times are being tracked.

The spreadsheet solution, shown in Table 1, in fact, consists of four solutions, the first two using the RAND() random number generator, and the last two using the supplied VBA function Rnd01(). It is left as an exercise to devise a justification for the statistical equivalence of the 4 solutions, even though on any click of the button ‘Generate New Trial’, the 4 sets of results are different.

The spreadsheet logic for coin tossing is simple. The second solution (Column E) is:

=IF(RAND()<=0.5,"H","T") while the

fourth solution (Column I) is:

=IF(Rnd01()<=0.5,"H","T")

Each call to RAND() or Rnd01() produces a new random number. The ‘One Trial’ worksheet records the frequency for heads versus tails in the 10 tosses and graphs the results in a histogram.

Table 1 Simulation Table for Coin Tossing

	B	C	D	E	F	G	H	I
11		Solution #1A		Solution #1B		Solution #2A		Solution #2B
12	Toss	RAND()	Solution using RN from ColC	Solution using RAND()	Toss	Rnd01()	Solution using RN from ColG	Solution using Rnd01()
13								
14								
15	1	0.7317089	T	T	1	0.9871114	T	T
16	2	0.8285837	T	T	2	0.0225598	H	H
17	3	0.0701168	H	T	3	0.0008356	H	T
18	4	0.2147295	H	H	4	0.2127686	H	T
19	5	0.8613179	T	H	5	0.8586159	T	H
20	6	0.4159098	H	H	6	0.5503362	T	T
21	7	0.7492317	T	H	7	0.5243730	T	H
22	8	0.9671661	T	T	8	0.8884351	T	H
23	9	0.4823089	H	T	9	0.2111118	H	T
24	10	0.6738897	T	H	10	0.7680427	T	H

The first and third solutions are two steps, an approach that is useful when a generated random number is used more than once. Columns C and G contain the generated random number, and columns D and H the result (“H” or “T”).

Lesson 26 : Simulating a Random Service Time

How to Simulate a Random Service Time

The following example shows how to generate random samples from an arbitrary discrete distribution.

The example is simple, having just 3 possible values of service time. The key principle, namely, the transformation from a random number to a service time of a prescribed probability, easily generalizes to any discrete distribution with a finite number of possible values.

Example 2: Random Service Times

An automated telephone information service spends either 3, 6, or 10 minutes with each caller. The proportion of calls for each service length is 30%, 45%, and 25%, respectively. We want to simulate these service times in a spreadsheet. Our actual purpose is to learn how to generate random samples from a discrete distribution, in preparation for the queuing, inventory, and other examples to follow.

This also is an example of a Monte Carlo simulation.

Table 2 shows the input specification and a portion of the simulation table (the first 4 callers), taken from the spreadsheet model.

Table 2 Input Specification and Simulation Table for Service Times

	A	B	C	D
4		Service Time	Probability	Cumulative Probability
5				
6				
7		3	0.30	0.30
8		6	0.45	0.75
9		10	0.25	1.00
10				
11		Number of Callers=		25
12		Simulation Table		
13		Step	Activity	
14				
15		Caller	Service Time	
16		1	6	
17		2	6	
18		3	10	
19		4	6	

The input specification defines the probabilities for the service times, and also includes the cumulative probabilities. Cumulative probabilities always increase to 1.0; as we shall see, the generation method uses the cumulative

probabilities. The method always starts with a random number and ends with the desired value, in this case, a random service time with the specified distribution. The simulation table in the spreadsheet shows the resulting frequency distribution (or histogram) of the generated service times for 25 callers.

Lesson 27 : Simulating a Random Arrival Time

How to Simulate a Random Arrival Time

In this section, our goal is to learn how to generate random interarrival times (times between successive arrivals) and compute arrival times of customers or other entities to a system. The interarrival times, as well as the service times in Example 2, are examples of simulation-generated time durations, called **activity times** or **activities**. These are distinguished from **arrival times**, which are times on a simulation clock representing the time of arrival to a facility.

A simulation clock is a key component of every dynamic discrete-event simulation. In all of the Excel spreadsheet solutions, the clock times are labeled just above the appropriate column titles of the simulation table, so that you can easily distinguish between the activity times (service or interarrival times, for example) and the computed clock times. In all cases, a clock time represents the time of occurrence of an event, such as arrival, a service beginning, or a service completion in a queueing model.

Example 3: Random Arrival Times

Telephone calls to the telephone information service, where service times are defined in Example 2, occur at random times defined by a discrete distribution for which the interarrival times have values 1, 2, 3, or 4 minutes, all with equal probability. Our purpose here is to show how to generate both interarrival times and arrival times.

Since this example has one event, namely, the arrival event, it is our first example of a dynamic, event-based model (despite its simplicity).

Dynamic simply means time-based, with system state changing over time; dynamic, event-based means that the model tracks the progression of the event occurrences over time.

The interarrival times can be generated randomly in the same manner as done in Example 2 for service times. The spreadsheets, however, supply a second VBA function to simplify random generation from a discrete uniform distribution. **Uniform** means that all the values have equal probabilities; in this example, all the interarrival times may have one of the values 1, 2, 3, or 4 minutes, each occurring with a probability of 0.25. For such discrete uniform distributions, the spreadsheets supply the VBA function, DiscreteUniform(), which takes two arguments: either the cell containing the minimum value followed by the cell containing the maximum value of the desired range of integers, or the actual low and high values, as in:

```
= DiscreteUniform($G$5,$G$6) or  
= DiscreteUniform(1,4)
```

The DiscreteUniform(low, high) function is implemented by a single line of VBA code, as follows:

DiscreteUniform = low + Int((high – low + 1)*Rnd()) which becomes, for this example:
DiscreteUniform = 1 + Int(4 * Rnd()) where Rnd() generates a random number between 0 and 1 (in VBA, it is always less than one but may equal zero), and Int() is a VBA function that truncates (rounds down) its argument. (Note that 4 * Rnd() generates real values between 0.0 and 3.99999, so truncating and then adding 1 yield the desired range 1 to 4. Why do we expect equal probabilities?)

Table 3 Input Specification and Simulation Table for Arrival Times.

	A	B	C	D
4		Interarrival Times (minutes)		
5				
6		Minimum	1	
7		Maximum	4	
8				
9		Number of Callers=		25
10		Simulation Table		
11		Step	Activity	Clock
12		Caller	Interarrival Time	
13				Arrival Time
14		1		0
15		2	2	2
16		3	1	3
17		4	3	6

Table 3 shows the input specification for interarrival times and a portion of the simulation table, taken from the spreadsheet model in “Example2.3ArrivalTimes.xls”. As shown in Step 1 (row 14), we assume that the first arrival occurs at simulation time 0. (The assumed arrival time for the first customer is model-dependent; it could be any other specified time, constant or random.) The interarrival times are randomly and independently generated using the DiscreteUniform(1, 4)function. The arrival times are computed by a simple formula, by adding the current customer’s interarrival time to the previous customer’s arrival time. (You should verify the spreadsheet formula for arrival times.)

Example 3 is the first that contains an event, namely, the arrival event, and the event time associated with that event, namely, the CLOCK time for the time of arrival. Note that Table 3 distinguishes the activity times from the CLOCK times.

Lesson 28 : Framework for Spreadsheet Simulation

The simulation table provides the structure, and the steps outlined below provide a set of general guidelines for carrying out a spreadsheet simulation. We have seen several examples of a simulation table, with Table 3 in Example 3 being the best so far—since it contains a clock time for the arrival the event, making it one of the simplest possible dynamic, event-based simulations.

First, the model developer must define the model inputs, the system states, and the model outputs.

Inputs: Inputs are the exogenous variables that are (usually) defined independently of other system characteristics; examples include the probability of a head-on a coin toss, the service time, and the inter-arrival time distributions in a queuing system, or the demand distribution in an inventory model. These input specifications are used to generate random activity times and other random variables. Other inputs, such as policy or decision parameters, may be constants.

Outputs: Outputs are used to compute measures of system performance (also known as *model responses*, or simple *responses*).

For example, an output could be an individual customer's waiting time in a queue, or the cost of an individual transaction in an inventory system.

Associated Responses: The associated responses might be average waiting time in a queue or the average cost per unit time in an inventory system, respectively.

Lesson 29 : Framework Simulation Table Design

Simulation Table Design: Each simulation table is custom designed for the problem at hand. Each column in the table is one of the following types:

1. *An activity time associated with a model input;*
2. *Any other random variable defined by a model input;*
3. *A system state;*
4. *An event, or the clock time of an event;*
5. *A model output;*
6. *Sometimes, a model response.*

For the spreadsheet simulations here, the above-listed items are sufficient.

Model Responses: In general, model responses, also known as *measures of system performance*, are computed outside the simulation table after the simulation has been completed. For example, in a waiting line model, the delay in a queue of each customer is a *model output*, while the average delay overall customers is a *model response*. The model output is tracked in the simulation table; the response is computed afterward.

Activity Time: An activity time is a time defined by a constant or a statistical distribution, such as a service time, or an interarrival time. An activity is a duration of time, in contrast to event times, which are clock times.

System States: System states include, for example, server status (busy or idle) in a queueing model and inventory levels in an inventory model. In general, the set of system states includes any information needed for the simulation to proceed through time (or from one step to the next), as well as any quantity of information that makes the simulation logic simpler or more transparent and thus easier to understand. The greatest simplicity possible, as long as the model logic is correct, assists in ease in understanding that can be of tremendous value, especially when trying to verify that a model's logic is indeed correct.

Table 4 A Generic Simulation Table

<i>Step</i>	<i>Activities and System States</i>						<i>Outputs</i>
	x_{i1}	x_{i2}	$\dots$	x_{ij}	$\dots$	x_{ip}	y_i
1							
2							
3							
.							
.							
.							
n							

Each row in the simulation table represents a step in the simulation and is usually associated with the occurrence of one or more events, or the progress of one entity (such as a customer) through

the system. The simulation table is designed so that a given step depends solely on model inputs and/or one or more previous steps or previously computed values in the current step.

In general, a modeler develops and runs a spreadsheet or manual simulation by following these guidelines:

1. Determine the characteristics of each of the inputs to the simulation. Often encountered types of inputs include constants as well as probability distributions.
2. Determine the activities, events, and system states relevant to the problem.
3. Determine model responses or summary performance measures, depending on objectives and specific questions to meet those objectives.
4. Determine the model outputs needed for computing model responses.
5. Construct a simulation table. A generic representation of a simulation table is shown in Table 4. In this table, there are p activities and system states, x_{ij} , $j = 1, 2, \dots, p$, and one output, y_i , for each simulation step $i = 1, 2, \dots, n$. Initialize the table by filling in the data for Step 1. When doing a spreadsheet model, the initial step or steps (usually just Step 1) are typically different from subsequent steps, which are defined by formulas.
6. For each Step i , generate a value for each of the activities, then compute the system states and outputs y_i . Random input values may be computed by sampling values from the distributions chosen in Step 1. An output typically depends on the inputs, system states, and possibly other outputs.
7. When the simulation is finished, use the outputs to compute the model responses or measures of performance.

Lesson 30 : Framework using Discrete event simulation: General Idea

This module develops a common framework for the modeling of complex systems by using discrete event simulation. It covers the basic building blocks of all discrete-event simulation models: entities and attributes, activities, and events.

Discrete-Event Simulation: In discrete-event simulation:

System: A system is modeled in terms of its state at each point in time; of the entities that pass through the system and;

Entities: The entities that represent system resources; and of the activities and events that cause the system state to change.

Discrete-event models are appropriate for those systems for which changes in system state occur only at discrete points in time.

Framework using Discrete event simulation – I

Lesson 31 :

Concepts In Discrete-event Simulation

Here we are exclusively dealing with dynamic, stochastic systems (i.e., involving time and containing random elements) that change discretely. This module expands on these concepts and proposes a framework for the development of a discrete-event model of a system. The major concepts are briefly defined and then illustrated by examples:

System A collection of entities (e.g., people and machines) that interact together over time to accomplish one or more goals.

Model An abstract representation of a system, usually containing structural, logical, or mathematical relationships that describe a system in terms of state, entities and their attributes, sets, processes, events, activities, and delays.

System state A collection of variables that contain all the information necessary to describe the system at any time.

Entity Any object or component in the system that requires explicit representation in the model (e.g., a server, a customer, a machine).

Attributes The properties of a given entity (e.g., the priority of a waiting customer, the routing of a job through a job shop).

List A collection of (permanently or temporarily) associated entities, ordered in some logical fashion (such as all customers currently in a waiting line, ordered by “first come, first served,” or by priority).

Event An instantaneous occurrence that changes the state of a system (such as an arrival of a new customer).

Event notice A record of an event to occur at the current or some future time, along with any associated data necessary to execute the event; at a minimum, the record includes the event type and the event time.

Event list A list of event notices for future events, ordered by time of occurrence; also known as the future event list (FEL).

Framework using Discrete event simulation – II

Lesson 32 :

Concepts In Discrete-event Simulation

Activity A duration of time of specified length (e.g., a service time or interarrival time), which is known when it begins (although it may be defined in terms of a statistical distribution).

Delay A duration of time of unspecified indefinite length, which is not known until it ends (e.g., a customer's delay in a last-in-first-out waiting line which, when it begins, depends on future arrivals).

Clock A variable representing simulated time, called CLOCK in the examples to follow.

Different simulation packages use different terminology for the same or similar concepts—for example, **lists** are sometimes called **sets**, **queues**, or **chains**.

Sets or lists: Sets or lists are used to hold both entities and event notices.

Entities on a List: The entities on a list are always ordered by some rule, such as first-in-first-out or last-in-first-out, or are ranked by some entity attribute, such as priority or due date.

Event List: The future event list is always ranked by the event time recorded in the event notice.

Activity: An activity typically represents a service time, an interarrival time, or any other processing time whose duration has been characterized and defined by the modeler.

Duration of an Activity: An activity's duration may be specified in several ways:

- a. **Deterministic**—for example, always exactly 5 minutes;
- b. **Statistical**—for example, as a random draw from the set {2, 5, 7} with equal probabilities;
- c. **A function depending on system variables and/or entity attributes**—for example, loading time for an iron ore ship as a function of the ship's allowed cargo weight and the loading rate in tons per hour.

However, it is characterized, the duration of an activity is computable from its specification at the instant it begins. Its duration is not affected by the occurrence of other events (unless, as is allowed by some simulation packages, the model contains logic to cancel or postpone an activity in progress).

Framework using Discrete event simulation – III

Lesson 33 :

Keeping Track of Activities: To keep track of activities and their expected completion time, at the simulated instant that an activity duration begins, an **event notice** is created having an event time equal to the activity's completion time.

Example: For example, if the current simulated time is $CLOCK = 100$ minutes and an inspection time of exactly 5 minutes is just beginning, then an event notice is created that specifies the type of event (an end-of-inspection event), and the event time ($100 + 5 = 105$ minutes).

Delays: In contrast to an activity, a delay's duration is not specified by the modeler ahead of time, but rather is determined by system conditions. Quite often, a delay's duration is measured and is one of the desired outputs of a model run. Typically, a delay ends when some set of logical conditions becomes true or one or more other events occur. For example, a customer's delay in a waiting line may be dependent on the number and duration of service of other customers ahead in line as well as the availability of servers and equipment.

Delay or Conditional Wait: A *delay* is sometimes called a *conditional wait*, an activity an *unconditional wait*.

Primary Event: The completion of an activity is an event, often called a *primary event*, that is managed by placing an event notice on the FEL.

In contrast, delays are managed by placing the associated entity on another list, perhaps representing a waiting line, until when the system conditions permit the processing of the entity.

Secondary Event: The completion of a delay is sometimes called a *conditional or secondary event*, but such events are not represented by event notices, nor do they appear on the FEL.

Dynamic Nature of the System:

The systems considered here are *dynamic*, that is, *changing over time*. Therefore, *system state*, *entity attributes* and the *number of active entities*, the *contents of sets*, and the *activities and delays*

Framework using Discrete event simulation – IV

currently in progress are all *functions of time* and are constantly changing over time. Time itself is represented by a variable called **CLOCK**.

Example: Call Center, Revisited

Consider the Able–Baker call center system of Example from the Simulation Examples in a Spreadsheet chapter. A discrete-event model has the following components:

System state

$LQ(t)$, the number of callers waiting to be served at time t ; $LA(t)$, 0 or 1 to indicate Able as being idle or busy at time t ; $LB(t)$, 0 or 1 to indicate Baker as being idle or busy at time t .

Entities Neither the callers nor the servers need to be explicitly represented, except in terms of the state variables, unless certain caller averages are desired (compare Examples 4 and 5).

Events

Arrival event;

Service completion by Able; Service completion by Baker.

Lesson 34 : Framework using Discrete event simulation - IV

Example: Call Center, Revisited (Continued...)

Activities

Interarrival time, defined in Table from the Simulation Examples in a Spreadsheet chapter;
Service time by Able, defined in Table from the Simulation Examples in a Spreadsheet chapter;
Service time by Baker, defined in Table from the Simulation Examples in a Spreadsheet chapter.

Delay

A caller waits in the queue until Able or Baker becomes free

The definition of the model components provides a static description of the model. In addition, a description of the dynamic relationships and interactions between the components is also needed.

Some questions that need answers include:

1. How does each event affect system state, entity attributes, and set contents?
2. How are activities defined (i.e., deterministic, probabilistic, or some other mathematical equation)? What event marks the beginning or end of each activity? Can the activity begin regardless of the system state, or is its beginning conditioned on the system being in a certain state? (For example, a machining “activity” cannot begin unless the machine is idle, not broken, and not in maintenance.)
3. Which events trigger the beginning (and end) of each type of delay? Under what conditions does a delay begin or end?
4. What is the system state at time 0? What events should be generated at time 0 to “prime” the model—that is, to get the simulation started?

Discrete Event Simulation

A *discrete-event simulation* is the modeling over time of a system all of whose state changes occur at discrete points in time—those points when an event occurs. A discrete-event simulation (hereafter called a **simulation**) proceeds by producing a sequence of system snapshots (or system images) that represent the evolution of the system through time.

Lesson 35 : Event Scheduling Time advance algo

The Event Scheduling/Time Advance Algorithm

The mechanism for advancing simulation time and guaranteeing that all events occur in correct chronological order is based on the future event list (FEL). This list contains all event notices for events that have been scheduled to occur at a future time. Scheduling a future event means that, at the instant an activity begins, its duration is computed or drawn as a sample from a statistical distribution; and that the end-activity event, together with its event time, is placed on the future event list. In the real world, most future events are not scheduled but merely happen—such as random breakdowns or random arrivals. In the model, such random events are represented by the end of some activity, which in turn is represented by a statistical distribution.

At any given time t , the FEL contains all previously scheduled future events and their associated event times (called $t_1, t_2, \dots$ in Figure 1). The FEL is ordered by event time, meaning that the events are arranged chronologically—that is, the event times satisfy

$$t < t_1 \leq t_2 \leq t_3 \leq \dots \leq t_n.$$

CLOCK	System state	Entities and attributes	Set 1	Set 2	...	Future event list (FEL)	Cumulative statistics and counters
t	$(x, y, z, \dots)$					$(3, t_1)$ — Type 3 event to occur at time t_1 $(1, t_2)$ — Type 1 event to occur at time t_2 $\cdot$ $\cdot$ $\cdot$ $\cdot$ $\cdot$ $\cdot$	

Figure 1 Prototype system snapshot at simulation time t .

Time t is the value of CLOCK, the current value of simulated time. The event associated with time t_1 is called the imminent event; that is, it is the next event that will occur. After the system snapshot at simulation time $\text{CLOCK} = t$ has been updated, the CLOCK is advanced to simulation time $\text{CLOCK} = t_1$, the imminent event notice is removed from the FEL, and the event is executed. Execution of the imminent event means that a new system snapshot for time t_1 is created, one based on the old snapshot at time t and the nature of the imminent event. At time t_1 , new future events may or might not be generated, but if any are, they are scheduled by creating event notices and putting them into their proper position on the FEL. After the new system snapshot for time t_1 has been updated, the clock is advanced to the time of the new imminent event and that event is executed. This process repeats until the simulation is over.

Lesson 36 : Event Scheduling Time advance algo-II

The sequence of actions that a simulator (or simulation language) must perform to advance the clock and build a new system snapshot is called the ***event-scheduling/time-advance algorithm***, whose steps are listed in Figure 2 (and explained thereafter).

Old system snapshot at time t

CLOCK	System state	...	Future event list	...
t	(5, 1, 6)		$(3, t_1)$ – Type 3 event to occur at time t_1 $(1, t_2)$ – Type 1 event to occur at time t_2 $(1, t_3)$ – Type 1 event to occur at time t_3 $(2, t_n)$ – Type 2 event to occur at time t_n	

Event-scheduling/time-advance algorithm

- Step 1.** Remove the event notice for the imminent event (event 3, time t_1) from FEL.
- Step 2.** Advance CLOCK to imminent event time (i.e., advance CLOCK from t to t_1).
- Step 3.** Execute imminent event: update system state, change entity attributes, and set membership as needed.
- Step 4.** Generate future events (if necessary) and place their event notices on FEL, ranked by event time.
(Example: Event 4 to occur at time t^* , where $t_2 < t^* < t_3$.)
- Step 5.** Update cumulative statistics and counters.

New system snapshot at time t_1

CLOCK	System state	...	Future event list	...
t_1	(5, 1, 5)		$(1, t_2)$ – Type 1 event to occur at time t_2 $(4, t^*)$ – Type 4 event to occur at time t^* $(1, t_3)$ – Type 1 event to occur at time t_3 $(2, t_n)$ – Type 2 event to occur at time t_n	

Figure 2 Advancing simulation time and updating system image.

List Processing: The length and contents of the FEL are constantly changing as the simulation progresses, and thus its efficient management in a simulation software package will have a major impact on the model's computer runtime. The management of a list is called list processing.

Cancellation Of An Event: The major list processing operations performed on an FEL are removal of the imminent event, the addition of a new event to the list, and occasionally the removal of some event (called *cancellation of an event*). As the imminent event is usually at the top of the list, its removal is as efficient as possible.

The addition of a new event (and cancellation of an old event) requires a search of the list. The efficiency of this search depends on the logical organization of the list and on how the search is conducted. In addition to the FEL, all the sets in a model are maintained in some logical order, and the operations of addition and removal of entities from the set also require efficient list-processing techniques.

The removal and addition of events from the FEL are illustrated in Figure 2. Event 3 with event time t_1 represents, say, a service completion event at server 3. Since it is the imminent event at time t , it is removed from the FEL in Step 1 (Figure 2) of the event-scheduling/time-advance algorithm.

When event 4 (say, an arrival event) with event time t^* is generated at step 4, one possible way to determine its correct position on the FEL is to conduct a top-down search:

If $t^* < t_2$,	place event 4 at the top of the FEL.
If $t_2 \leq t^* < t_3$,	place event 4 second on the list.
If $t_3 \leq t^* < t_4$,	place event 4 third on the list.
$\vdots$	
If $t_n \leq t^*$,	place event 4 last on the list.

The least efficient way to maintain the FEL is to leave it as an unordered list (additions placed arbitrarily at the top or bottom), which would require at Step 1 of Figure 2 a complete search of the list for the imminent event before each clock advance. (The imminent event is the event on the FEL with the lowest event time).

Lesson 37 : Event Scheduling Time advance algo-III

Exogenous Events: The system snapshot at time 0 is defined by the initial conditions and the generation of the so-called exogenous events. The specified initial conditions define the system state at time 0. For example, in Figure 2, if $t = 0$, then the state (5, 1, 6) might represent the initial number of customers at three different points in the system. An exogenous event is a happening “outside the system” that impinges on the system. An important example is an arrival to a queueing system. At time 0, the first arrival event is generated and is scheduled on the FEL (meaning that its event notice is placed on the FEL). The interarrival time is an example of an activity. When the clock eventually is advanced to the time of this first arrival, a second arrival event is generated. First, an interarrival time, call it a^* , is generated; it is added to the current time, $\text{CLOCK} = t$; the resulting (future) event time, $t + a^* = t^*$, is used to position the new arrival event notice on the FEL. This method of generating an external arrival stream is called **bootstrapping**; it provides one example of how future events are generated in Step 4 of the event-scheduling/time-advance algorithm. Bootstrapping is illustrated in Figure 3

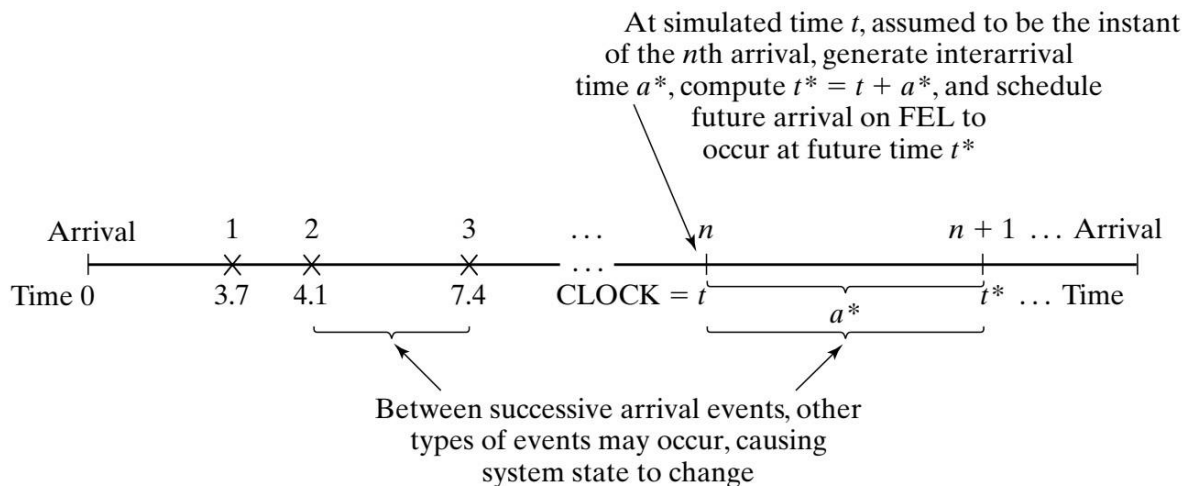


Figure 3 Generation of an external arrival stream by bootstrapping.

A second example of how future events are generated (Step 4 of Figure 2) is provided by a service completion event in a queueing simulation. When one customer completes service, at current time $\text{CLOCK} = t$, if the next customer is present, then a new service time, s^* , will be generated for the next customer. The next service completion event will be scheduled to occur at future time $t^* = t + s^*$, by placing onto the FEL a new event notice, of type *service completion*, with event time t^* .

Lesson 38 : Event Scheduling Time advance algo-IV

Beginning service is a conditional event because its occurrence is triggered only on the condition that a customer is present and a server is free. Service completion is an example of a primary event. Note that a conditional event, such as beginning service, is triggered by a primary event occurring and by certain conditions prevailing in the system. Only primary events appear on the FEL.

A third important example is the alternate generation of runtimes and downtimes for a machine subject to breakdowns. At time 0, the first runtime will be generated, and an end-of-runtime event will be scheduled. Whenever an end-of-runtime event occurs, downtime will be generated, and an end-of-downtime event will be scheduled on the FEL. When the CLOCK is eventually advanced to the time of this end-of-downtime event, a runtime is generated, and an end-of-runtime event is scheduled on the FEL. In this way, runtimes and downtimes continually alternate throughout the simulation. A runtime and downtime are examples of activities, and the *end of runtime* and *end of downtime* are primary events.

Every simulation must have a stopping event, here called E , which defines how long the simulation will run. There are generally two ways to stop a simulation:

1. At time 0, schedule a stop simulation event at a specified future time T_E . Thus, before simulating, it is known that the simulation will run over the time interval $[0, T_E]$. Example: Simulate a job shop for $T_E = 40$ hours.
2. Run-length T_E is determined by the simulation itself. Generally, T_E is the time of occurrence of some specified event E . Examples: T_E could be the time of the 100th service completion at a certain service center. T_E could be the time of breakdown of a complex system. T_E could be the time of disengagement or total kill (whichever occurs first) in a combat simulation. T_E could be the time at which a distribution center ships the last carton in a day's orders.

In case 2, T_E is not known ahead of time. Indeed, it could be one of the statistics of primary interest to be produced by the simulation.

Lesson 39 : World views – I

When using a simulation package or even when doing a manual simulation, a modeler adopts a world view or orientation for developing a model. The most prevalent world views are the *event scheduling world view*, as discussed in the previous section, *the process-interaction world view*, and the activity-scanning world view. Even if a particular package does not directly support one or more of the world views, understanding the different approaches could suggest alternative ways to model a given system.

When using a package that supports the process-interaction approach, a simulation analyst thinks in terms of processes.

The analyst defines the simulation model in terms of entities or objects and their life cycle as they flow through the system, demanding resources and queueing to wait for resources.

More precisely, a process is the life cycle of one entity. This life cycle consists of various events and activities. Some activities might require the use of one or more resources whose capacities are limited. These and other constraints cause processes to interact, the simplest example being an entity forced to wait in a queue (on a list) because the resource it needs is busy with another entity. The process-interaction approach is popular because it has intuitive appeal and because the simulation packages that implement it allow an analyst to describe the process flow in terms of high-level block or network constructs, while the interaction among processes is handled automatically.

In more precise terms, a process is a time-sequenced list of events, activities, and delays, including demands for resources, that define the life cycle of one entity as it moves through a system. An example of a “customer process” is shown in Figure 4. In this figure, we see the interaction between two customer processes as customer $n + 1$ is delayed until the previous customer’s “end service event” occurs. Usually, many processes are active simultaneously in a model, and the interaction among processes could be quite complex.

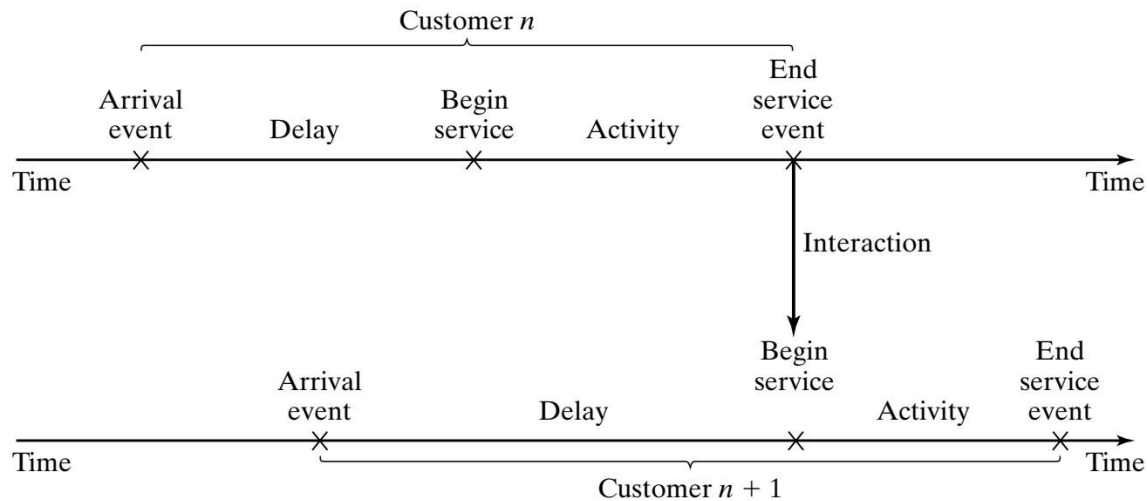


Figure 4 Two interacting customer processes in a single-server queue.

With the activity-scanning approach, a modeler concentrates on the activities of a model and those conditions, simple or complex, that allow an activity to begin. At each clock advance, the conditions for each activity are checked, and, if the conditions are true, then the corresponding activity begins. Proponents claim that the activity-scanning approach is simple in concept and leads to modular models that are more easily maintained, understood, and modified by other analysts at later times. They admit, however, that the repeated scanning to discover whether an activity can begin results in slow runtime on computers.

Lesson 40 : World views – II

The pure activity-scanning approach has been modified (and made conceptually somewhat more complex) by what is called the three-phase approach, which combines some of the features of event scheduling with activity scanning to allow for variable time advance and the avoidance of scanning when it is not necessary, but keeps the main advantages of the activity-scanning approach.

In the three-phase approach, events are considered to be activities of duration zero-time units.

With this definition, activities are divided into two categories, which are called *B* and *C*.

B activities: activities bound to occur; all primary events and unconditional activities.

C activities: activities or events that are conditional upon certain conditions being true.

The B-type activities and events can be scheduled ahead of time, just as in the event-scheduling approach. This allows variable time advance. The FEL contains only B-type events. Scanning to

learn whether any C-type activities can begin or C-type events occur happens only at the end of each time advance, after all, B-type events have been completed.

In summary, with the three-phase approach, the simulation proceeds with repeated execution of the 3 phases until it is completed:

Phase A Remove the imminent event from the FEL and advance the clock to its event time.

Remove from the FEL any other events that have the same event time.

Phase B Execute all B-type events that were removed from the FEL. (This could free several resources or otherwise change the system state.) **Phase C** Scan the conditions that trigger each C-type activity and activate any whose conditions are met. Rescan until no additional C-type activities can begin and no events occur.

Lesson 41 : Manual Simulation

In the conducting of an event-scheduling simulation, a simulation table is used to record the successive system snapshots as time advances.

Example 3: Single-Channel Queue

Consider the grocery store with one checkout counter that is simulated in Example 5 from the Simulation Examples in a Spreadsheet chapter by an ad hoc method. The system consists of those customers in the waiting line plus the one (if any) checking out. A stopping time of 60 minutes is set for this example. The model has the following components:

System state ($LQ(t)$, $LS(t)$), where $LQ(t)$ is the number of customers in the waiting line, and $LS(t)$ is the number being served (0 or 1) at time t .

Entities The server and customers are not explicitly modeled, except in terms of the state variables.

Events

Arrival (A);

Departure (D);

Stopping event (E), scheduled to occur at time 60.

Event notices

(A, t), representing an arrival event to occur at future time t ; (D, t), representing a customer departure at future time t ;
(E, 60), representing the simulation stop event at future time 60.

Activities

Interarrival time, defined in Table 9 from the Simulation Examples in a Spreadsheet chapter;
Service time, defined in Table 10 from the Simulation Examples in a Spreadsheet chapter.

Delay Customer time spent in the waiting line.

The event notices are written as (*event type, event time*). In this model, the FEL will always contain either two or three event notices. The effect of the arrival and departure events is shown in detail in Figures 5 and 6.

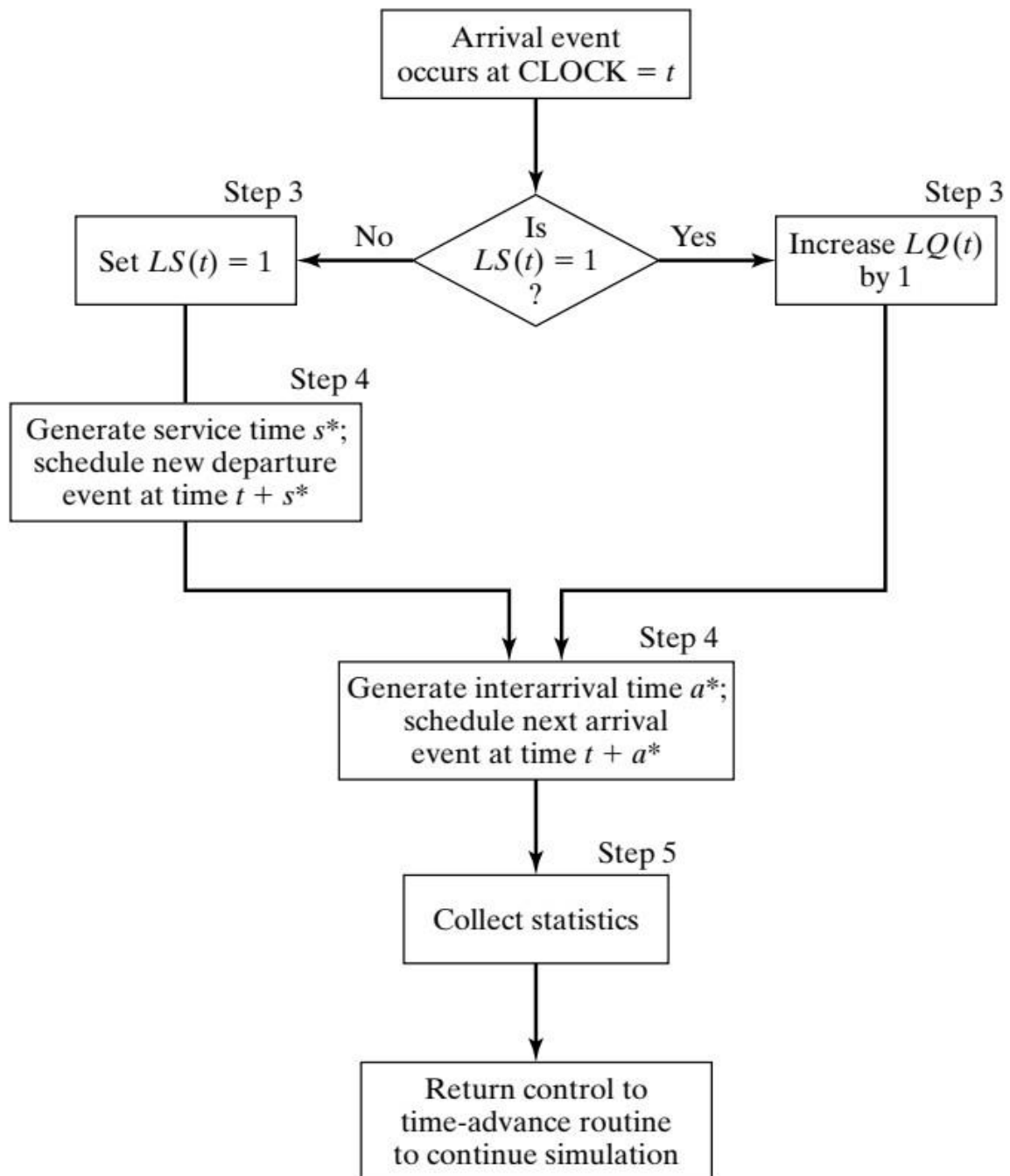


Figure 5 Execution of the arrival event.

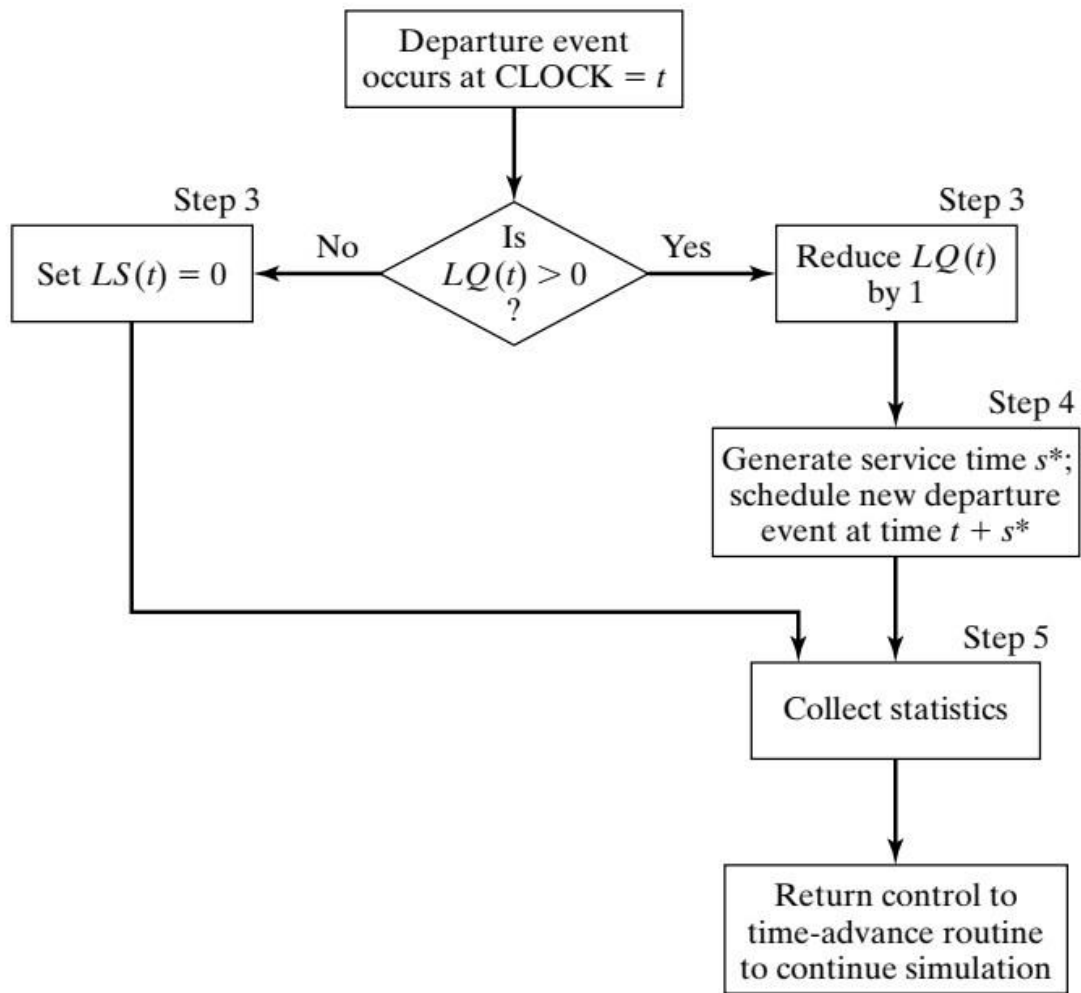


Figure 6 Execution of the departure event.

The simulation table for the checkout counter is given in Table 1. The reader should cover all system snapshots except one, starting with the first, and attempt to construct the next snapshot from the previous one and the event logic in Figures 5 and 6.

Interarrival Times	1	1	6	3	7	5	2	4	1...
Service Times	4	2	5	4	1	5	4	1	4...

Lesson 42 : Manual Simulation-II

Execution Of The Arrival Event

Initial conditions are that the first customer arrives at time 0 and begins service. This is reflected in **Table 1** by the system snapshot at time zero (CLOCK = 0), with $LQ(0) = 0$, $LS(0) = 1$, and both a departure event and arrival event on the FEL.

Also, the simulation is scheduled to stop at time 60. Only two statistics, server utilization, and maximum queue length will be collected.

Server Utilization

Server utilization is defined by total server busy time (B) divided by total time (T_E).

Total busy time, B , and maximum queue length, MQ , will be accumulated as the simulation progresses.

A column headed “Comments” is included to aid the reader (a_* and s_* are the generated interarrival and service times, respectively).

Table 1 Simulation Table for Checkout Counter

CLOCK	System state		Future event list	Comment	Cumulative statistics	
	$LQ(t)$	$LS(t)$			B	MQ
0	0	1	(A, 1) (D, 4) (E, 60)	First A occurs ($a^* = 1$) Schedule next A ($s^* = 4$) Schedule first D	0	0
1	1	1	(A, 2) (D, 4) (E, 60)	Second A occurs: (A, 1) ($a^* = 1$) Schedule next A (Customer delayed)	1	1
2	2	1	(D, 4) (A, 8) (E, 60)	Third A occurs: (A, 2) ($a^* = 6$) Schedule next A (Two customers delayed)	2	2
4	1	1	(D, 6) (A, 8) (E, 60)	First D occurs: (D, 4) ($s^* = 2$) Schedule next D (Customer delayed)	4	2
6	0	1	(A, 8) (D, 11) (E, 60)	Second D occurs: (D, 6) ($s^* = 5$) Schedule next D	6	2
8	1	1	(D, 11) (A, 11) (E, 60)	Fourth A occurs: (A, 8) ($a^* = 3$) Schedule next A (Customer delayed)	8	2
11	1	1	(D, 15) (A, 18) (E, 60)	Fifth A occurs: (A, 11) ($a^* = 7$) Schedule next A Third D occurs: (D, 11) ($s^* = 4$) Schedule next D (Customer delayed)	11	2
15	0	1	(D, 16) (A, 18) (E, 60)	Fourth D occurs: (D, 15) ($s^* = 1$) Schedule next D	15	2
16	0	0	(A, 18) (E, 60)	Fifth D occurs: (D, 16)	16	2
18	0	1	(D, 23) (A, 23) (E, 60)	Sixth A occurs ($a^* = 5$) Schedule next A ($s^* = 5$) Schedule next D	16	2
23	0	1	(A, 25) (D, 27) (E, 60)	Seventh A occurs: (A, 23) ($a^* = 2$) Schedule next Arrival Sixth D occurs: (D, 23)	21	2

As soon as the system snapshot at time $CLOCK = 0$ is complete, the simulation begins. At time 0, the imminent event is (A,1).

The $CLOCK$ is advanced to time 1, and (A,1) is removed from the FEL.

Because $LS(t) = 1$ for $0 \leq t \leq 1$ (i.e., the server was busy for 1 minute), the cumulative busy time is increased from $B = 0$ to $B = 1$. By the event logic in **Figure 6**, set $LS(1) = 1$ (the server becomes busy). The FEL is left with three future events, (A, 2), (D, 4), and (E, 60). The simulation $CLOCK$ is next advanced to time 2, and an arrival event is executed. The interpretation of the remainder of **Table 1** is left as an exercise.

The simulation in **Table 1** covers the time interval $[0,23]$. At simulated time 23, the system empties, but the next arrival also occurs at time 23.

The server was busy for 21 of the 23-time units simulated, and the maximum queue length was two.

This simulation is, of course, too short to draw any reliable conclusions.

Note that the simulation table gives the system state at all times, not just the listed times.

For example, from time 11 to time 15, there is one customer in service and one in the waiting line.

Lesson 43 : Manual Simulation-III

The Dump-Truck Problem

Six dump trucks are used to haul coal from the entrance of a small mine to the railroad. Figure 7 provides a schematic of the dump-truck operation. Each truck is loaded by one of two loaders. After loading, the truck immediately moves to the scale, to be weighed as soon as possible. Both the loaders and the scale have a first-come-first-served waiting line (or queue) for trucks. Travel time from a loader to the scale is considered negligible. After being weighed, a truck begins a travel time (during which time the truck unloads) and then afterward returns to the loader queue.

The distributions of loading time, weighing time, and travel time are given in Tables 3, 4, and 5, respectively.

The purpose of the simulation is to estimate the loader and scale utilization (percentage of the busy time). The model has the following components:

System state

$[LQ(t), L(t), WQ(t), W(t)]$, where

$LQ(t)$ = number of trucks in loader queue;

$L(t)$ = number of trucks (0, 1, or 2) being loaded

$WQ(t)$ = number of trucks in weigh queue;

$W(t)$ = number of trucks (0 or 1) being weighed, all at simulation time t .

Entities

The six dump trucks ($DT1, \dots, DT6$).

Event notices

(ALQ, t, DTi) , DTi arrives at loader queue (ALQ) at time t ;

(EL, t, DTi) , DTi ends loading (EL) at time t ; (EW, t, DTi) , DTi ends weighing (EW) at time t .

Lists

Loader queue, all trucks waiting to begin loading, ordered on a first-come-first-served basis; Weigh queue, all trucks waiting to be weighed, ordered on a first-come-first-served basis.

Activities

Loading time, weighing time, and travel time.

Delays

Delay at loader queue, and delay at scale.

Table 3 Distribution of Loading Time for the Dump Trucks

Loading Time	Probability	Cumulative Probability	Random Number Interval
5	0.30	0.30	$0.0 \leq R \leq 0.3$
10	0.50	0.80	$0.3 < R \leq 0.8$
15	0.20	1.00	$0.8 < R \leq 1.0$

Table 4 Distribution of Weighing Time for the Dump Trucks

Weighing Time	Probability	Cumulative Probability	Random Number Interval
12	0.70	0.70	$0.0 \leq R \leq 0.7$
16	0.30	1.00	$0.7 < R \leq 1.0$

Table 5 Distribution of Travel Time for the Dump Trucks

Travel Time	Probability	Cumulative Probability	Random Number Interval
40	0.40	0.40	$0.0 \leq R \leq 0.4$
60	0.30	0.70	$0.4 < R \leq 0.7$
80	0.20	0.90	$0.7 < R \leq 0.9$
100	0.10	1.00	$0.9 < R \leq 1.0$

Lesson 44 : Statistical Models in Sim

Review of Terminology and Concepts

Discrete Random Variables: Let X be a random variable. If the number of possible values of X is finite, or countably infinite, X is called a discrete random variable. The possible values of X may be listed as $x_1, x_2, \dots$. In the finite case, the list terminates; in the countably infinite case, the list continues indefinitely.

Example 1

The number of jobs arriving each week at a job shop is observed. The random variable of interest is X , where

X = number of jobs arriving each week

The possible values of X are given by the range space of X , which is denoted by R_X . Here $R_X = \{0, 1, 2, \dots\}$.

Let X be a discrete random variable. With each possible outcome x_i in R_X , a number $p(x_i) = P(X = x_i)$ gives the probability that the random variable equals the value of x_i . The numbers $p(x_i)$, $i = 1, 2, \dots$, must satisfy the following two conditions:

a. $p(x_i) \geq 0$, for all i

b. $\sum_{i=1}^{\infty} p(x_i) = 1$

The collection of pairs $(x_i, p(x_i))$, $i = 1, 2, \dots$ is called the probability distribution of X , and $p(x_i)$ is called the probability mass function (pmf) of X .

Example 2

Consider the experiment of tossing a single die. Define X as the number of spots on the up face of the die after a toss. Then $R_X = \{1, 2, 3, 4, 5, 6\}$. Assume the die is loaded so that the probability that a given face lands up is proportional to the number of spots showing. The discrete probability distribution for this random experiment is given by:

x_i	1	2	3	4	5	6
$p(x_i)$	1/21	2/21	3/21	4/21	5/21	6/21

The conditions stated earlier are satisfied—that is, $p(x_i) \geq 0$ for $i = 1, 2, \dots, 6$ and $\sum_{i=1}^{\infty} p(x_i) = 1/21 + \dots + 6/21 = 1$. The distribution is shown graphically in Figure 1.

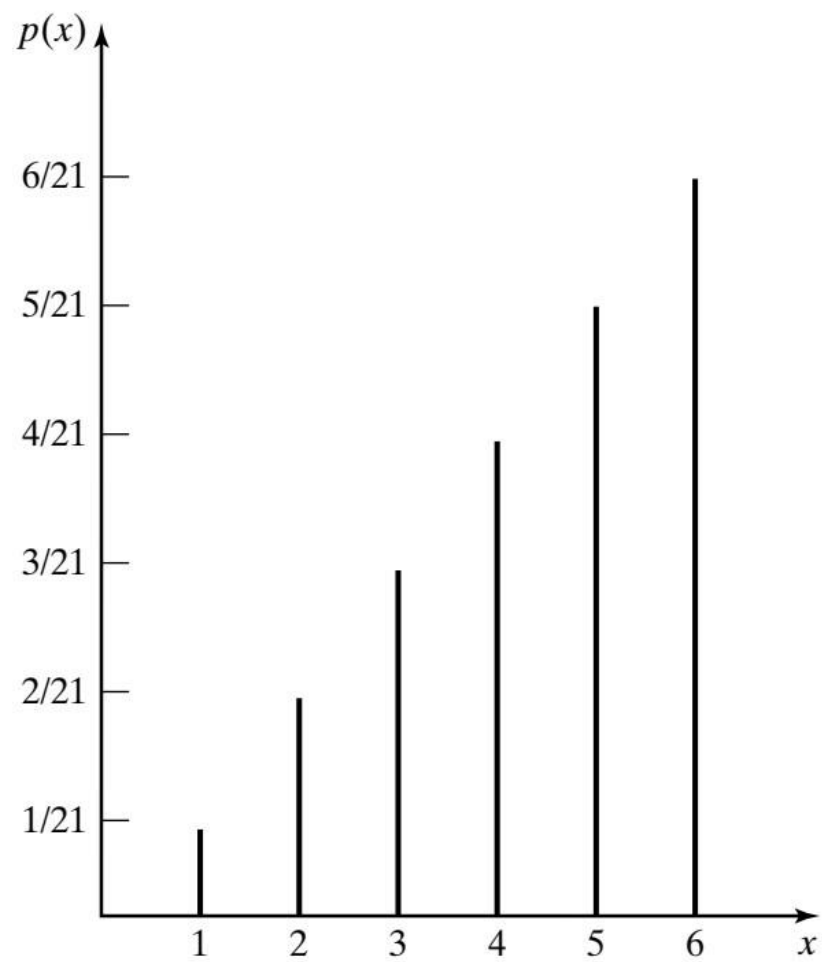


Figure 1 Probability mass function for loaded-die example.

Review of Terminology and Concepts

Continuous Random Variables

If the range space R_X of the random variable X is an interval or a collection of intervals, X is called a continuous random variable. For a continuous random variable X , the probability that X lies in the interval $[a, b]$ is given by:

$$P(a \leq X \leq b) = \int_a^b f(x) dx \quad (1)$$

The function $f(x)$ is called the probability density function (pdf) of the random variable X . The pdf satisfies the following conditions:

- a. $f(x) \geq 0$ for all x in R_X
- b. $\int_{R_X} f(x) dx = 1$
- c. $f(x) = 0$ if x is not in R_X

As a result of Equation (1), for any specified value x_0 , $P(X = x_0) = 0$, because

$$\int_{x_0}^{x_0} f(x) dx = 0$$

$P(X = x_0) = 0$ also means that the following equations hold:

$$P(a \leq X \leq b) = P(a < X \leq b) = P(a \leq X < b) = P(a < X < b) \quad (2)$$

The graphical interpretation of Equation (1) is shown in Figure 2. The shaded area represents the probability that X lies in the interval $[a, b]$.

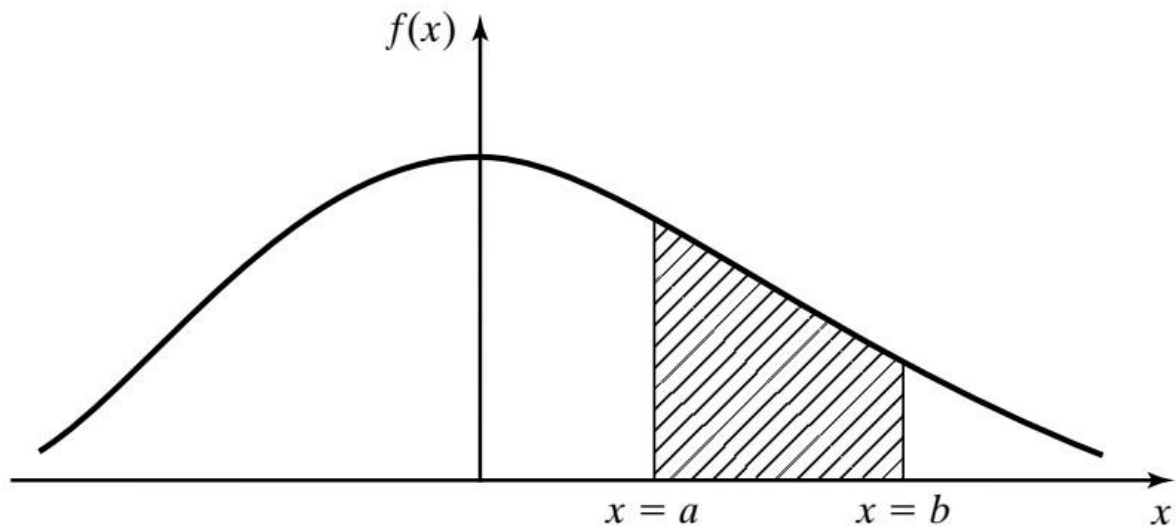


Figure 2 Graphical interpretation of $P(a < X < b)$.

Example 3

The life of a device used to inspect cracks in aircraft wings is given by X , a continuous random variable assuming all values in the range $x \geq 0$. The pdf of the device's lifetime, in years, is as follows:

$$f(x) = \begin{cases} \frac{1}{2}e^{-x/2}, & x \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

This pdf is shown graphically in Figure 3. The random variable X is said to have an exponential distribution with a mean of 2 years.

The probability that the life of the device is between 2 and 3 years is calculated as:

$$\begin{aligned}
 P(2 \leq X \leq 3) &= \frac{1}{2} \int_2^3 e^{-x/2} dx \\
 &= -e^{-3/2} + e^{-1} \\
 &= -0.223 + 0.368 \\
 &= 0.145
 \end{aligned}$$

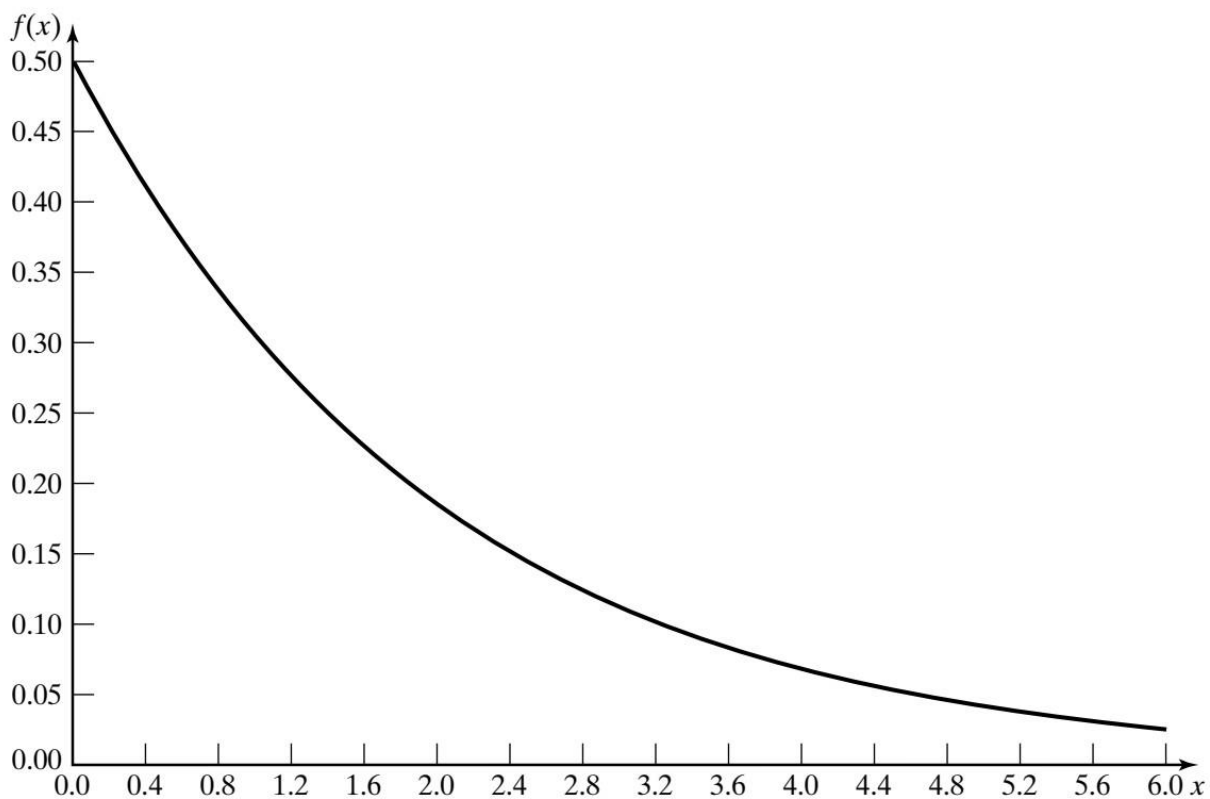


Figure 3 pdf for inspection-device life.

Lesson 46 : Review of Terms-II

Review of Terminology and Concepts

Cumulative distribution function

The cumulative distribution function (cdf), denoted by $F(x)$, measures the probability that the random variable X assumes a value less than or equal to x , that is, $F(x) = P(X \leq x)$. If X is discrete, then

$$F(x) = \sum_{\substack{\text{all} \\ x_i \leq x}} p(x_i) \quad (3)$$

If X is continuous, then

$$F(x) = \int_{-\infty}^x f(t) dt \quad (4)$$

Some properties of the *cdf* are listed here:

- a. F is a nondecreasing function. If $a < b$, then $F(a) \leq F(b)$.
- b. $\lim_{x \rightarrow \infty} F(x) = 1$
- c. $\lim_{x \rightarrow -\infty} F(x) = 0$

All probability questions about X can be answered in terms of the cdf. For example,

$$P(a < X \leq b) = F(b) - F(a) \quad \text{for all } a < b \quad (5)$$

For continuous distributions, not only does Equation (5) holds but also the probabilities in Equation (2) are equal to $F(b) - F(a)$.

Example 4

The die-tossing experiment described in Example 2 (Module 44) has a cdf given as follows:

x	$(-\infty, 1)$	$[1, 2)$	$[2, 3)$	$[3, 4)$	$[4, 5)$	$[5, 6)$	$[6, \infty)$
$F(x)$	0	1/21	3/21	6/21	10/21	15/21	21/21

where $[a, b) = \{a \leq x < b\}$. The *cdf* for this example is shown graphically in Figure 4.

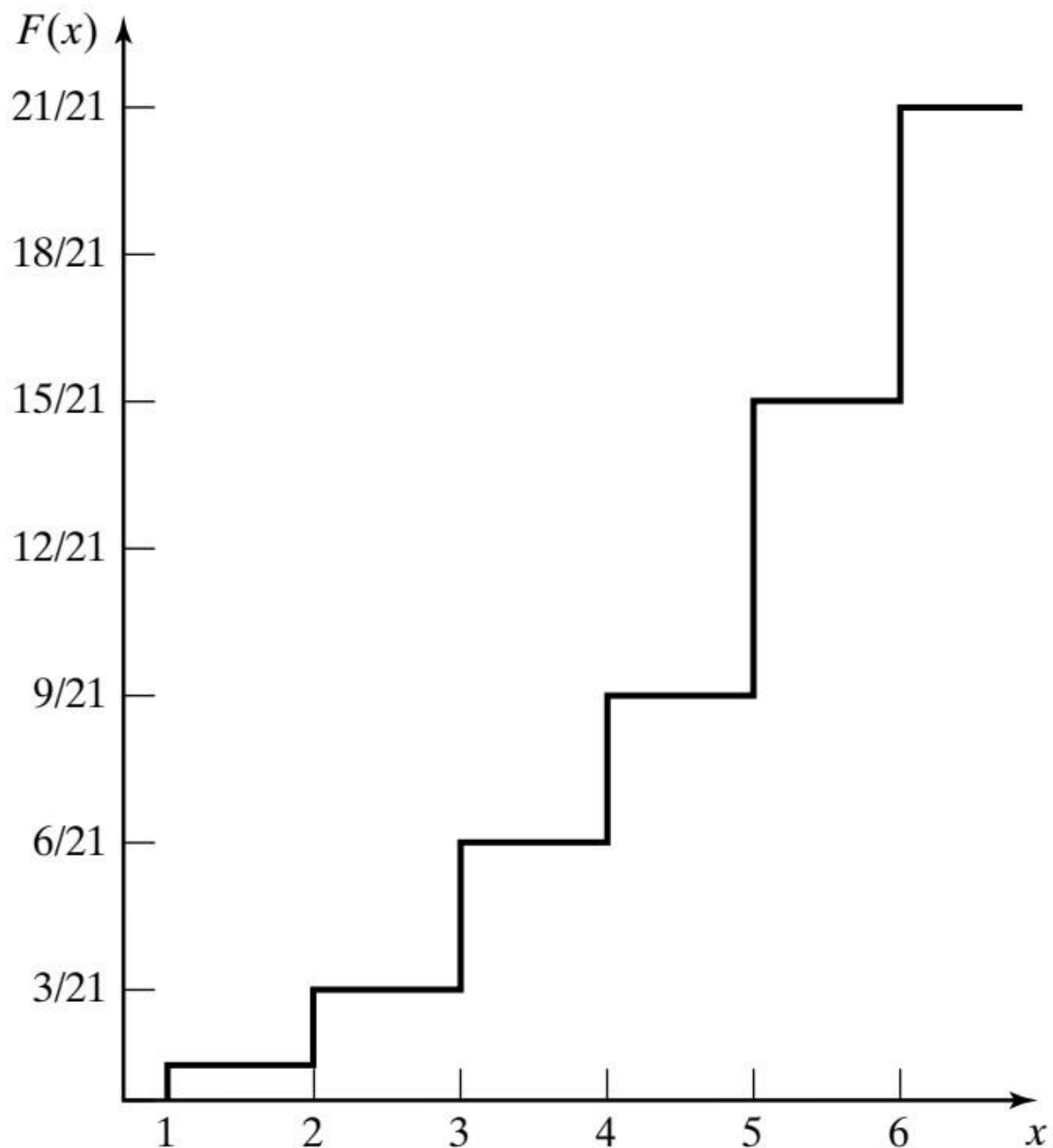


Figure 4 cdf for loaded-die example.

If X is a discrete random variable with possible values $x_1, x_2, \dots$, where $x_1 < x_2 < \dots$, the cdf is a step function. The value of the cdf is constant in the interval $[x_{i-1}, x_i)$ and then takes a step, or jump, of size $p(x_i)$ at x_i . Thus, in Example 4, $p(3) = 3/21$, which is the size of the step when $x = 3$.

Example 5

The *cdf* for the device described in Example 3 (Module 45) is given by

$$F(x) = \frac{1}{2} \int_0^x e^{-t/2} dt = 1 - e^{-x/2}$$

The probability that the device will last for less than 2 years is given by

$$P(0 \leq X \leq 2) = F(2) - F(0) = F(2) = 1 - e^{-1} = 0.632$$

The probability that the life of the device is between 2 and 3 years is calculated as

$$\begin{aligned} P(2 \leq X \leq 3) &= F(3) - F(2) = (1 - e^{-3/2}) - (1 - e^{-1}) \\ &= -e^{-3/2} + e^{-1} = -0.223 + 0.368 = 0.145 \end{aligned}$$

as found in Example 3.

Lesson 47 : Review of Terms-III

Review of Terminology and Concepts

Expectation

An important concept in probability theory is that of the expectation of a random variable. If X is a random variable, the expected value of X , denoted by $E(X)$, for discrete and continuous variables, is defined as follows:

$$E(X) = \sum_{\text{all } i} x_i p(x_i) \quad \text{if } X \text{ is discrete} \quad (6)$$

and

$$E(X) = \int_{-\infty}^{\infty} x f(x) dx \quad \text{if } X \text{ is continuous} \quad (7)$$

The expected value $E(X)$ of a random variable X is also referred to as the mean, μ , or the first moment of X . The quantity $E(X^n)$, $n \geq 1$, is called the n th moment of X , and is computed as follows:

$$E(X^n) = \sum_{\text{all } i} x_i^n p(x_i) \quad \text{if } X \text{ is discrete} \quad (8)$$

and

$$E(X^n) = \int_{-\infty}^{\infty} x^n f(x) dx \quad \text{if } X \text{ is continuous} \quad (9)$$

The variance of a random variable X denoted by $V(X)$ or $\text{var}(X)$ or σ^2 , is defined by

$$V(X) = E[(X - E[X])^2]$$

A useful identity in computing $V(X)$ is given by

$$V(X) = E(X^2) - [E(X)]^2 \quad (10)$$

The mean $E(X)$ is a measure of the central tendency of a random variable. The variance of X measures the expected value of the squared difference between the random variable and its mean.

Thus, the variance, $V(X)$, is a measure of the spread or variation of the possible values of X around the mean $E(X)$. The standard deviation, σ , is defined to be the square root of the variance, σ^2 . The mean, $E(X)$, and the standard deviation, $\sigma = \sqrt{V(X)}$, are expressed in the same units.

Example 6

The mean and variance of the die-tossing experiment described in Example 2 are computed as follows:

$$E(X) = 1 \left(\frac{1}{21} \right) + 2 \left(\frac{2}{21} \right) + \cdots + 6 \left(\frac{6}{21} \right) = \frac{91}{21} = 4.33$$

To compute $V(X)$ from Equation (10), first compute $E(X^2)$ from Equation (8) as follows:

$$E(X^2) = 1^2 \left(\frac{1}{21} \right) + 2^2 \left(\frac{2}{21} \right) + \cdots + 6^2 \left(\frac{6}{21} \right) = 21$$

Thus,

$$V(X) = 21 - \left(\frac{91}{21} \right)^2 = 21 - 18.78 = 2.22$$

and

$$\sigma = \sqrt{V(X)} = 1.49$$

Example 7

The mean and variance of the life of the device described in Example 3 are computed as follows:

$$\begin{aligned} E(X) &= \frac{1}{2} \int_0^{\infty} x e^{-x/2} dx = -x e^{-x/2} \Big|_0^{\infty} + \int_0^{\infty} e^{-x/2} dx \\ &= 0 + \frac{1}{1/2} e^{-x/2} \Big|_0^{\infty} = 2 \text{ years} \end{aligned}$$

To compute $V(X)$ from Equation (10), first compute $E(X^2)$ from Equation (9) as follows:

$$E(X^2) = \frac{1}{2} \int_0^{\infty} x^2 e^{-x/2} dx$$

Thus,

$$E(X^2) = -x^2 e^{-x/2} \Big|_0^{\infty} + 2 \int_0^{\infty} x e^{-x/2} dx = 8$$

giving

$$V(X) = 8 - 2^2 = 4 \text{ years}$$

and

$$\sigma = \sqrt{V(X)} = 2 \text{ years}$$

With a mean life of 2 years and a standard deviation of 2 years, most analysts would conclude that actual lifetimes, X , have a fairly large variability.

Lesson 48 : Review of Terms- IV

Review of Terminology and Concepts

The mode

The mode is used in describing several statistical models that appear in statistical models in simulation.

Mode in Discrete Case: In the discrete case, the mode is the value of the random variable that occurs most frequently.

Mode in Continuous Case: In the continuous case, the mode is the value at which the pdf is maximized.

Bimodal Distribution: The mode might not be unique; if the modal value occurs at two values of the random variable, the distribution is said to be *bimodal*.

Lesson 49 : Useful models – I

Useful Statistical Models

Numerous situations arise in the conduct of a simulation where an investigator may choose to introduce probabilistic events.

In a queuing system, interarrival and service times are often probabilistic.

Inventory model: In an inventory model, the time between demands and the lead times (time between placing and receiving an order) can be probabilistic.

Reliability model: In a reliability model, the time to failure could be probabilistic. In each of these instances, the simulation analyst desires to generate random events and to use a known statistical model if the underlying distribution can be found.

Queueing systems

In queueing examples, interarrival- and service-time patterns, the times between arrivals and the service times are often probabilistic. However, it is possible to have a constant interarrival time (as in the case of a line moving at a constant speed in the assembly of an automobile), or a constant service time (as in the case of robotized spot welding on the same assembly line). The following example illustrates how probabilistic interarrival times might occur

Example 8

Mechanics arrive at a centralized tool crib as shown in Table 1. Attendants check-in and check out the requested tools to the mechanics. The collection of data begins at 10:00 A.M. and continues until 20 different interarrival times are recorded. Rather than record the actual time of day, the absolute time from a given origin could have been computed. Thus, the first mechanic could have arrived at time 0, the second mechanic at time 7:13 (7 minutes, 13 seconds), and so on.

Table 1 Mechanics' Arrival Data

Arrival Number	Arrival (hour:minutes::seconds)	Interarrival Time (minutes::seconds)
1	10:05::03	—
2	10:12::16	7::13
3	10:15::48	3::32
4	10:24::27	8::39
5	10:32::19	7::52
6	10:35::43	3::24
7	10:39::51	4::08
8	10:40::30	0::39
9	10:41::17	0::47
10	10:44::12	2::55
11	10:45::47	1::35
12	10:50::47	5::00
13	11:00::05	9::18
14	11:04::58	4::53
15	11:06::12	1::14
16	11:11::23	5::11
17	11:16::31	5::08
18	11:17::18	0::47
19	11:21::26	4::08
20	11:24::43	3::17
21	11:31::19	6::36

Lesson 50 : Useful models – II

Useful Statistical Models

Example 9

Another way of presenting interarrival data is to find the number of arrivals per time period. Here, such arrivals occur over approximately 1.5 hours; it is convenient to look at 10-minute time intervals for the first 20 mechanics. That is, in the first 10-minute time period, one arrival occurred at 10:05::03.

In the second time period, two mechanics arrived, and so on. The results are summarized in Table 2.

Table 2 Arrivals in Successive Time Periods

Time Period	Number of Arrivals	Time Period	Number of Arrivals
1	1	6	1
2	2	7	3
3	1	8	3
4	3	9	2
5	4	—	—

This data could then be plotted in a histogram, as shown in Figure 5.

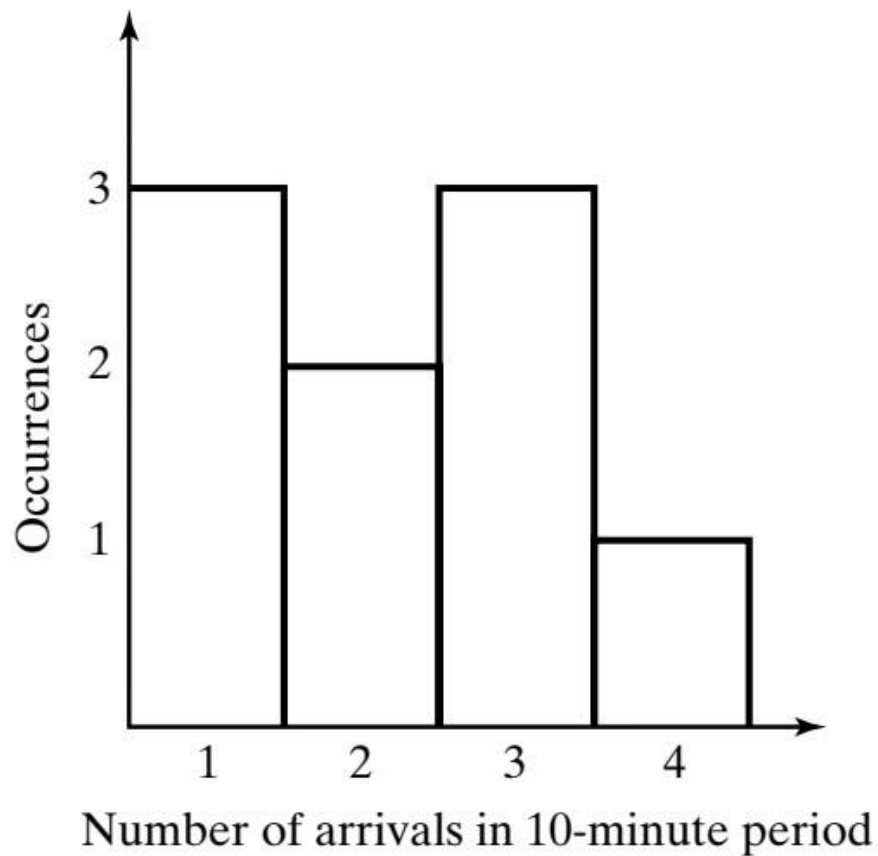


Figure 5 Histogram of arrivals per time period.

The distribution of time between arrivals and the distribution of the number of arrivals per time period is important in the simulation of waiting-line systems. “*Arrivals*” occur in numerous ways; as machine breakdowns, as jobs coming into a job shop, as units being assembled on a line, as orders to a warehouse, as data packets to a computer system, as calls to a call center, and so on.

Agent-Based Modelling

Before we discuss why agent-based modeling is important, we briefly describe what agent-based modeling is.

Agent: An *agent* is an autonomous computational individual or object with particular properties and actions.

Agent-based modeling: It is a form of computational modeling whereby a phenomenon is modeled in terms of agents and their interactions.

In the past two decades, scientists have increasingly used agent-based modeling methods to conduct their research.

- ABM is a species of computation, growing up alongside the maturation of computer technology.
- **The combination of large data, cheap computation, and high connectivity allow agent-based models to be constructed with millions of individual agents** whose properties and behaviors have been validated. Moreover, computational representations are dynamic and executable, allowing for greater interactivity between the user and representation. Perhaps, even more important, agent-based representations have particular advantages in that they are easy for people to understand.
- **Agent-based representations are easier to understand than mathematical representations of the same phenomenon.** This is because agent-based models are constructed out of individual objects and simple rules for their movement or behavior, as opposed to equational models that are constructed from mathematical symbols. In our natural discourse, we commonly describe our experience in terms of the interactions of individuals, as opposed to in terms of the rates of change of aggregates as in differential equations.
- **Individual Agents:** In thinking about individual agents, we can make sense of them by projecting our bodily experience onto the agents. Thus, the language and concepts we use in agent-based modeling are much closer to natural language and our natural thinking.

Steps for Conceptualization: As a first step to presenting this conceptualization, we look back historically at changes to representational tools and practices used in science and the significant benefits they had for both scientists and learners.

Lesson 52 : A Thought Experiment

A Thought Experiment

Imagine a country (let's call it Foo) where people represented numbers as the Romans did, using symbols such as MCMXLVIII. Fooian scientists worked laboriously to quantify and more accurately calculate basic science such as planetary motion and dynamical forces.

Number of Different Approaches: Business people had great difficulties with their financial calculations, tradespeople struggled with their measurements, and consumers labored to assess their purchases. Educational researchers and policymakers in this imaginary country were very concerned with the difficulty of learning to handle numbers, and they worked hard to make these skills accessible to more of their citizens. They engaged in several different approaches.

The Sort of Innovation: A first step is to name the sort of innovation associated with the shift from Roman to Hindu-Arabic representations of numbers. It is not sufficient, for example, to say that we have a new approach to learning and working with numbers.

Structuration: By *structuration*, we mean the encoding of the knowledge in a domain as a function of the representational infrastructure used to express the knowledge.

Restructuration: A change from one structuration of a domain to another resulting from such a change in representational infrastructure we call a *restructuration*.

Examples of Restructuration: The development of Arabic numerals and the transformation of kinematics via algebra were empowering and democratizing, enabling significant progress in science and widening the range of people who could make sense of previously formidable topics and skills.

Lesson 53 : Complex Systems and Emergence

COMPLEX SYSTEMS AND EMERGENCE

What areas are widely thought to be difficult for people to comprehend and potentially ripe for restructuration? One such area is complex systems.

Its very name suggests that it is a difficult area for comprehension.

However, even if the level of complexity in our life remained constant over the ages, our continual quest for knowledge would ultimately lead us to study complex systems.

From Simple to Complex System

As we gain facility and a more complete understanding of simple systems, we naturally progress to trying to make sense of increasingly complex systems.

Simple Population Dynamics Models

Simple population dynamics models, for example, make the implicit assumption that all members of a species are the same, but later, it becomes important to explore the manifold complexity of the food web and how each individual interacts with every other individual. Thus, our need to understand complex systems is also a natural result of the growth of human knowledge.

Complex Systems Theory: Complex systems theory develops principles and tools for making sense of the world's complexity and defines complex systems as systems that are composed of multiple individual elements that interact with each other yet whose aggregate properties or behavior is not predictable from the elements themselves.

Emergent Phenomenon: Through the interaction of the multiple distributed elements an “emergent phenomenon” arises. The phenomenon of emergence is characteristic of complex systems. The term “emergent” was coined by the British philosopher and psychologist G. H. Lewes.

Emergence: Since Lewes's time, scholars have struggled with how to best define emergence — some definitions are succinct, others more involved. For our purposes, we define emergence as *the arising of novel and coherent structures, patterns, and properties through the interactions of multiple distributed elements*.

Features of Emergence: Important features of emergence include:

- the global pattern's spontaneous arising from the interaction of elements,
- and the absence of an orchestrator or centralized coordinator — the system “selforganizes.”

Macrostructures Are Emergent: Structure (or rules) at the micro-level leads to ordered patterns at the macro-level. Because the macrostructures are emergent, composed of many elements, they are dynamic and perturbing them often results in them dynamically reforming.

Lesson 54 : Understanding Complex Systems and Emergence

Understanding Complex Systems and Emergence

We have said that understanding complex systems and emergence is hard for people.

Fundamental And Distinct Challenges: Emergence, in particular, presents two fundamental and distinct challenges.

- The first difficulty lies in trying to figure out the aggregate pattern when one knows how individual elements behave.

Integrative Understanding: We sometimes call this *integrative* understanding, as it parallels the cumulative integration of small differences in calculus.

- A second difficulty arises when the aggregate pattern is known and one is trying to find the behavior of the elements that could generate the pattern.

Differential or Compositional Understanding: We sometimes call this *differential* understanding (aka *compositional* understanding), as it parallels the search in calculus for the small elements that produce an aggregate graph when accumulated.

Lesson 55 : Example 1: Integrative Understanding

Example 1: Integrative Understanding

Figure 0.1 presents a system composed of a few identical elements following one rule.

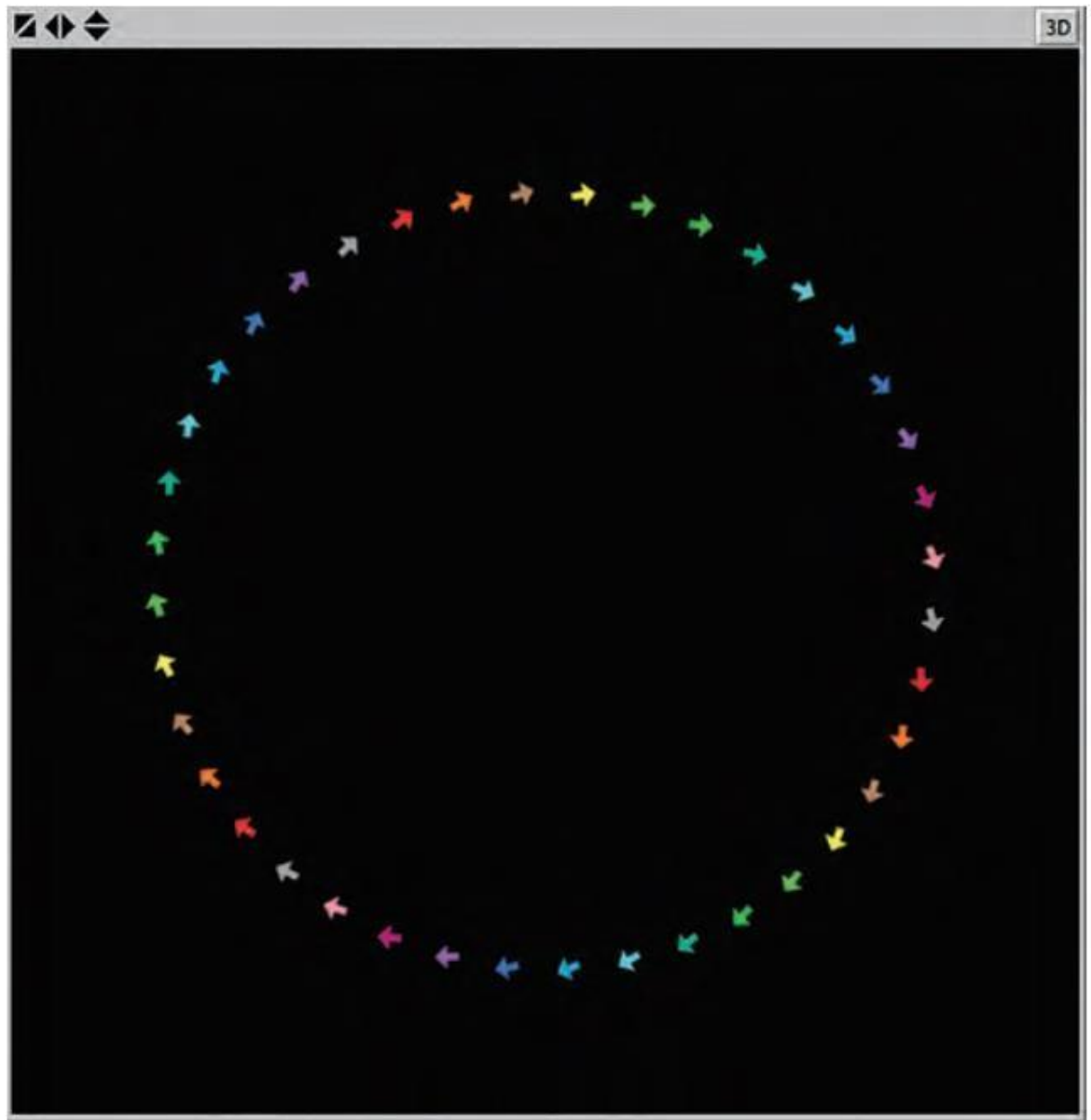


Figure 0.1

Some arrows moving clockwise around a circle of radius 20.

Each element is a small arrow. We imagine a clock ticking and at each tick of the clock, the arrows follow their rule. We initialize the system so that each arrow starts on a circle (of radius 20 units). We start them all facing clockwise on the circle. Now, we give them one movement behavior (or rule). At every tick of the clock, they move forward 0.35 units then turn right one degree. As the clock ticks, they continue to move and turn, move and turn, moving clockwise along the circle.

Now suppose that we slightly alter these rules. Instead of moving forward 0.35 units, we have them move 0.5 units while still turning one degree. What will be the aggregate pattern that we see?

The pattern that emerges is a pulsating circle. All the arrows stay in a circle, but the circle changes its radius, first expanding, then contracting, and repeating this cycle forever.

Lesson 56 : Example 2: Differential Understanding

Example 2: Differential Understanding

Now let's consider the flip side of these difficulties. There are many coherent, powerful, and beautiful patterns we observe in the world. What accounts for their prevalence? How do they originate? The secret to understanding the formation of these patterns is to understand that they are emergent, arising from the interactions of distributed individual elements. One such prevalent (and often beautiful) pattern is the flocking of birds. Birds fly together in many different formations, from the classic V formation of

goose flocks to the large, very dense flocks of starlings that resemble insect swarms. How and why do these flocks emerge?



Figure 0.3
Flock of geese flying in a classic V formation.

Deterministic-centralized Mindset

In the 1980s and early 90s, Wilensky and Resnick interviewed people from a wide range of backgrounds, asking them to explain how such patterns arise. The results suggested a phenomenon, or cognitive pattern, called the deterministic-centralized mindset, or DC mindset. The pattern stemmed from two main empirical findings:

- (1) Most subjects did not see any role for randomness in creating these structures, randomness was seen as destructive of pattern, not a force for creating a pattern (the D component); and
- (2) Most subjects described these patterns as arising from the actions of a centralized controller or orchestrator (the C component).

Lesson 57 : Agent-Based Modeling as Representational Infrastructure for

...

Thinking In Levels: In further analyses of the interviews, Wilensky and Resnick (1999) identified a key component of the DC mindset and an obstacle to thinking about emergent phenomena: the problem of “thinking in levels.”

Emergent phenomena’s two levels: Emergent phenomena can be described as existing on at least two levels: the level of the individual elements (cars, birds, people, etc.); and the level of system or aggregate patterns (flocks, traffic jams, housing patterns, etc.). Most people fail to distinguish between these levels, instead “ slipping ” between levels to attribute the properties of one level to the other.

Consider a V- flock, which appears to be stable and to have a consistent shape.

Appearance Of Stability: The appearance of stability often leads people to conclude that the individual elements of the flock (the birds) are stable and have a consistent place in the flock.

Complex Phenomena: Wilensky and Resnick also showed a host of examples where this level's slippage interfered with both the integrative and differential understanding of complex phenomena in the natural and human social worlds.

Lesson 58 : Example: Predator-Prey Interactions

Agent-Based Modeling as Representational Infrastructure for Restructurations: With the aid of new computer-based modeling environments, we can simulate complex patterns and better understand how they arise in nature and society. Whereas in many areas of science we have relied on simplified descriptions of complexity

Agent-based modeling

Agent-based modeling is a computational methodology that enables one to model complex systems.

As the name suggests, agent-based models are composed of *agents*: computational entities that have properties, or *state variables and values* (e.g., position, velocity, age, wealth, etc.). **Agents** usually also have a graphical component so you can see them on the computer screen. An agent can represent any element of a system.

Gas Molecule Agent

A gas molecule agent, for instance, might have properties such as “mass” with a value of 30 atomic mass units, “speed” with a value of 10 meters per second, and “heading” with a value of the angle it is facing.

A sheep agent

A sheep agent, by contrast, might have properties such as “speed” with a value of 3 mph, weight with a value of 30 lbs., and fleece with a value of “full” (a discrete-textual rather than numerical value).

Rules of Behavior

In addition to their properties, agents also have rules of behavior. A gas molecule agent might have the rule to collide with another molecule; a sheep agent might have a rule to eat grass if there is grass available nearby.

Universal Clock

In an agent-based model, we imagine a universal clock. When the clock ticks, all agents invoke their rules.

If the conditions of the rules are satisfied, (e.g., they are at the edge of a box, or grass is nearby), they enact the behavior (i.e., bounce or eat grass).

Lesson 59 : Example: Forest Fires

Agents And Rules

The goal of agent-based modeling is to create agents and rules that will generate a target behavior. Sometimes the rules are not well known, or you just want to explore the system’s behavior. In that case, ABM can be used to help you better understand a phenomenon through experimentation with rules and properties.

ABM’s Enable Restructurations Of Complex Systems

We contend that ABM enable restructurations of complex systems so that the

- (a) understanding of complex systems can be democratized and
- (b) the science of complex systems can be advanced.

This hypothesis begets a design challenge: Can we design a suitable representational language that supports both parts of the claim, enabling scientists to author scientific models in this language while simultaneously enabling a wider audience to gain access to (and understand) complex systems?

NetLogo

The computer language used in this text, NetLogo (*Wilensky, 1999*), was developed by Uri Wilensky for these express purposes. It is a general-purpose agent-based modeling language designed to be “ low-threshold ” — that is, novices can quickly employ it to do meaningful and useful things— but also “ high-ceiling ” — meaning that scientists and researchers can use it to design cutting-edge scientific models. The language borrows much of its syntax from the Logo language, which was designed to be accessible to children.

Turtle:

Like Logo, NetLogo calls its prototypical agent a “ turtle. ” However, while in Logo, the user directs the turtle to draw geometric figures, in NetLogo, this is generalized to thousands of turtles. Instead of drawing with pens, they typically draw with their bodies, moving according to rules, and the configuration of their bodies presents a visualization of the modeled phenomenon. NetLogo was first developed in the late 1990s, and it is now in use by hundreds of thousands of users worldwide.

Netlogo Used for Constructing And Exploring Models

Thousands of scientific papers have utilized NetLogo to construct and explore models in a wide variety of disciplines. It has also been employed by policymakers to model policy choices, business practitioners to model business decisions, and students to model subject matter in their coursework across virtually the entire curriculum. Many NetLogo-based courses have sprung up in both universities and secondary schools.

Lesson 60 : What Is Agent-Based Modeling?

[click here to download the modelling](#)

To illustrate the potential power of widespread agent-based modeling literacy and restructuration, we will look briefly at two examples derived from different content domains: predator-prey interactions and the spread of forest fires.

Example: Predator-Prey Interactions

Let us start with the study of predator-prey interactions. This domain is often first introduced qualitatively in high school, then quantitatively at the university level.

Lotka-Volterra Differential Equations

In its quantitative form, the population dynamics of a single predator and prey are introduced by the classic Lotka-Volterra differential equations, a pair of coupled differential equations that proceed as follows:

$$\frac{dPred}{dt} = K_1 * Pred * Prey - M * Pred$$

$$\frac{dPrey}{dt} = B * Prey - K_2 * Pred * Prey$$

The first equation says that the number of predators increases as predators interact with prey (by fixed constant K_1) and decreases by a constant mortality rate (M).

The second equation says that the number of prey increases by a constant birthrate (B) and decreases in interaction with predators (by a fixed constant K_2).

The solution to these equations resembles the classic sinusoidal curves that show the cycling of the predator populations with one ascendant when the other is at a trough.

Lesson 61 : Ants

[click here to download the ppt](#)

An Agent-based representation of Predator and Prey

An agent-based representation of predator and prey, such as the one illustrated in figure 0.6, would typically employ simple algorithmic models.

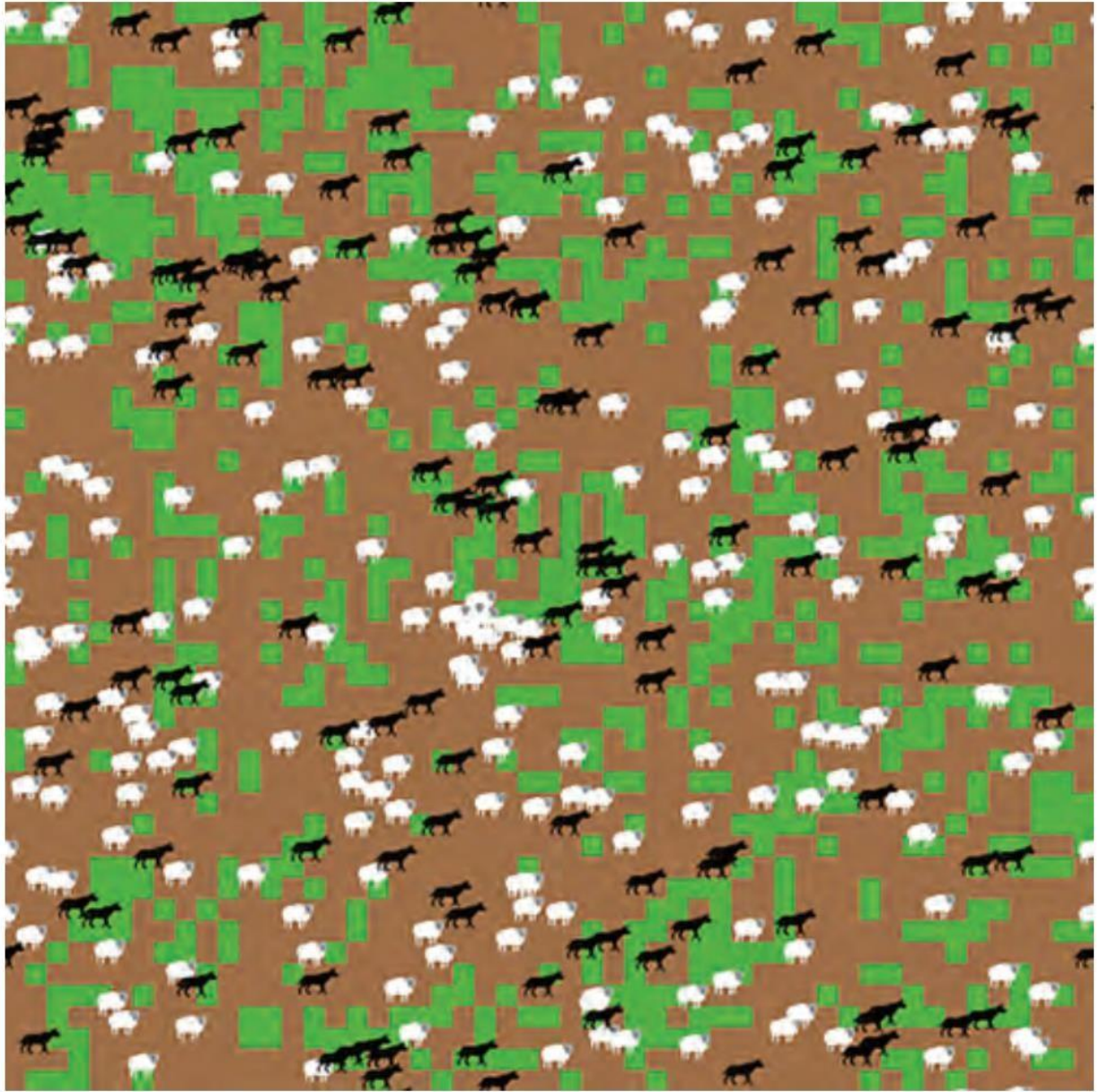


Figure 0.6

An agent-based Wolf Sheep Predation model. (See *Wilensky, 1997c*)

They might give each predator and prey a store of energy that is depleted when they move and increased when they eat. If their energy dips too low, they die. If it is high enough, they might reproduce.

And when they move, if they encounter food (which, for the predator, is the prey), they eat it. These instructions are explicitly stated in an easy-to-read language that instructs each agent in how to behave and accompanies the visual model.

Explicit Mechanism

This kind of representation makes its mechanisms explicit; as such, they are usually quite understandable by even young children. They also can be challenged more easily and tested. We will explore such predator-prey models in detail.

Advantages To Agent-based Model Representations

There are many advantages to agent-based representations. First, it does not require knowing calculus. Requiring calculus to be able to reason about predator-prey interactions sets the entry threshold quite high.

In the United States, only a small percentage of adults have ever taken calculus, and far fewer are familiar with differential equations. So calculus

serves as a gatekeeper limiting access to important content to a small minority of adults and almost all children. Yet the content of the predator-prey model can be made quite accessible to a non-calculus audience through ABM. This gatekeeper problem may seem less formidable when thinking about an audience of scientists who have likely had calculus. Even with this audience in mind, it is still often difficult to uncover the mechanisms behind the equations, to challenge them, and to propose alternate mechanisms and new equations.

Agent-based models can serve as powerful complements to equation-based models. They are particularly effective entry points into scientific domains.

Disadvantage Of ABM Representation

But they also have some disadvantages as compared to equations. • For an expert, an equation can more compactly represent a phenomenon than can an ABM.

- Moreover, when the equation is solvable, it enables the direct calculation of results without the need to run a model.

When a model requires a large number of agents, its execution time can be so long as to be impractical as a way of calculating results. The corresponding equation-based models must often make many simplifying assumptions to gain this increase in speed.

These simplifications are most justifiable when the agents are sufficiently homogenous that it can be advantageous to treat them as average quantities as opposed to the heterogeneous individuals often used in ABM.

Disadvantages of ABM

We now know from the works of biologists such as Gause (1936) that the equational model is less accurate than an ABM in the isolated predator-prey situation for which it was intended. In particular, the equational model underrepresents extinctions, since the model uses real numbers to represent the population densities.

Lesson 62 : Creating the Ant Foraging Model

Example: Forest Fires

Our second example is about the spread of a forest fire.

This domain is not usually present in the K-12 or university curriculum, but when taught, it typically falls under the subject matter of physics, described in terms of two classic partial differential equations.

- **Classic heat equation • Spread of a forest fire**

- The first is the classic heat equation, which describes the distribution of heat in a given region over time, where theta represents the thermal diffusivity of the material through which the heat is traveling.

$$\frac{dH(x,t)}{dt} = \theta \frac{d^2 H(x,t)}{dx^2}$$

- The second equation physicists use to describe the spread of a forest fire treats the fire as if it were a potentially turbulent fluid, thus using the Reynolds equation of fluid flow.

$$\frac{dU_i}{dt} + U_j \frac{dU_i}{dx_j} = -\frac{1}{\rho} \frac{dP}{dx_i} + \nu \frac{d^2 U_i}{dx_j dx_j} - \frac{d}{dx_j} \overline{u_i^t u_j^t}$$

ABM Approach To Modeling Forest Fires

Contrast these two equations with the ABM approach to modeling forest fires (illustrated in figure 0.7), which would typically model the environment as a grid of cells with trees occupying certain cells.

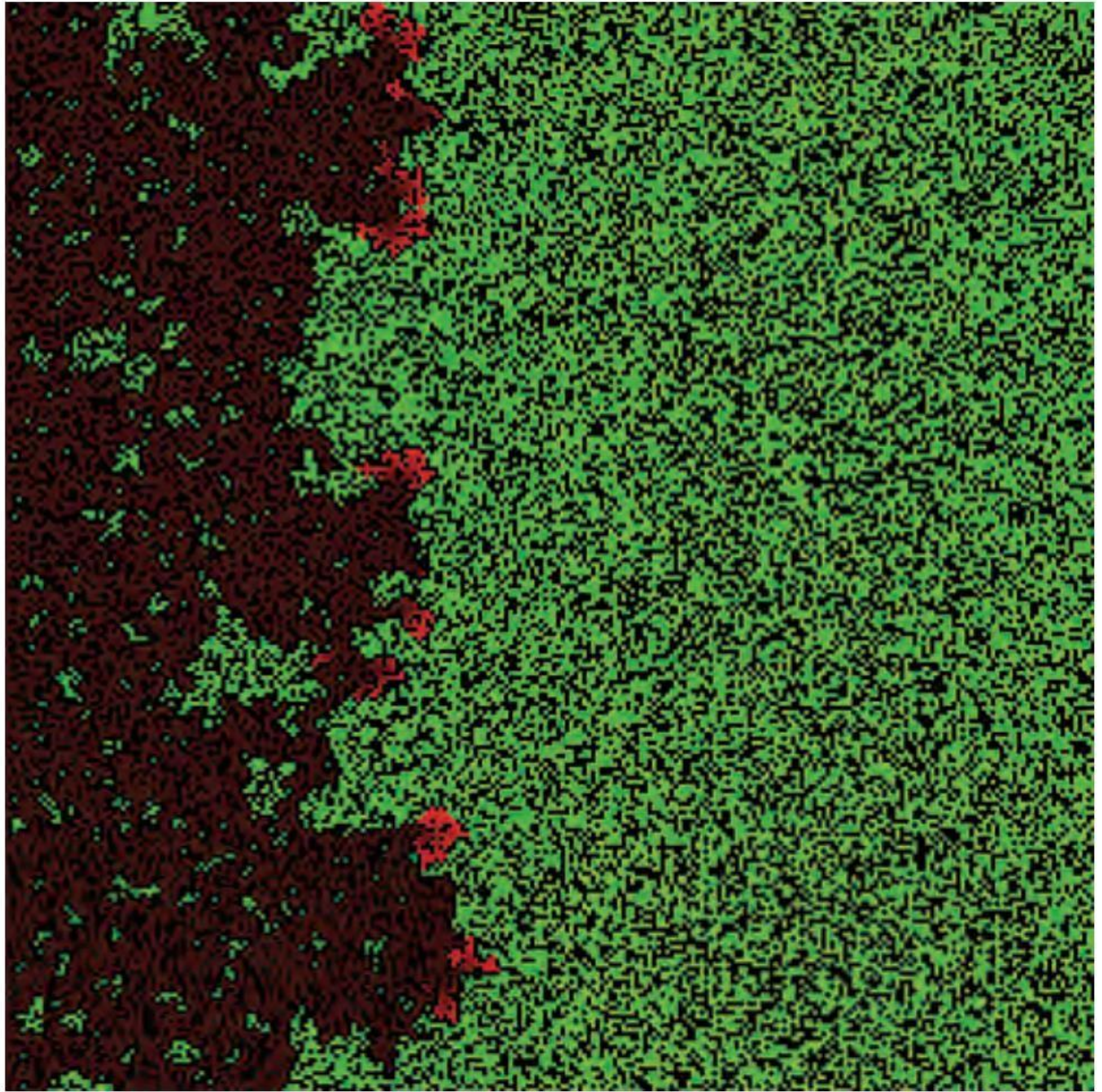


Figure 0.7

An agent-based model of a forest fire. (See *Wilensky, 1997b*)

Modeling the spread of fire consists simply of giving rules to the cells that are on fire as to when to spread to neighboring tree cells.

A very simple ABM of forest fire spread would not correctly model a particular fire, but it does give insight into the dynamics of any fire and once we know the details of a particular fire, we can add in whatever data or rules that apply to the situation.

ABM Provides Several Representational Benefits

ABM methods are starting to be used to model and fight real forest fires (see, e.g., www.simtable.com for a company that does agent-based modeling of emergency management including wildfires).

The restructuration of these systems using ABM provides several representational benefits.

ABM uses Discrete rather than Continuous Representations

They make use of discrete rather than continuous representations, which are more easily comprehensible, more closely match real-world situations, and require much less formal mathematics to employ.

They are easier to explore and much easier to modify. They present immediate feedback with visualizations that allow researchers and practitioners to understand and critique them at two levels, the level of the overall aggregate pattern, such as the fire spread or predator population levels, but also the level of the behavior of the individual animals, and the fire spread to particular trees. Though these two examples highlight some of the advantages of agent-based restructurations, the full potential of ABM restructuration is not yet evident either in these examples or in science as a whole.

Ants (Foraging For Food)

Ants

- The ant opens her eyes and looks around. There are many of her siblings nearby, but there is no food.
- The ant is hungry, so she heads out from the ant colony and starts to wander around.
- She sniffs a little to the left and a little to the right, and still, she cannot smell any food. So she keeps wandering.
- She passes by several of her sisters, but they do not interest her right now; she has food on her mind. • She keeps wandering. Sniff! Sniff! Mmmm ...good!
- She gets a whiff of some of that delightful pheromone stuff.
- She heads in the direction of the strongest pheromone scent; in the past, there have been food at the end of that trail.
- Sure enough, as the ant proceeds along the trail, she arrives at some delicious food.
- She grabs some food and heads back to the colony, making sure to drop some pheromone along the way.
- On the way back, she runs into many of her sisters, each sniffing her way along pheromone trails, repeating the same process they will carry out all-day

What we have just fancifully described is an ant foraging for food. The above paragraph of this section is in itself a model of ant behavior.

Model of Ant Behavior

Model: By a model, we mean an abstracted description of a process, object, or event. Models can take many distinct forms. However, certain forms of models are easier to manipulate than other forms.

Textual Description: The textual description in our first paragraph cannot easily be manipulated to answer questions about the ant colony's behavior — for example, what would we see if all the ants in the colony followed the behavior we described? It is difficult to extrapolate from the textual description of one ant to a description of an ant colony. The textual model is not sufficiently generative to answer such questions. It is a fixed description — always “behaving” the same, thus not bestowing any insight into the range of variation in behaviors. And it is not combinatorial — we cannot use the description to understand the interaction of the ants with each other or with the environment.

Lesson 64 : What Good Is an Ant Model?

Computational Model

One way to make the preceding model more generalizable and gain insight into the behavior of an ant colony would be to implement the model in a **computational form**.

A **computational model** is a model that takes certain input values, algorithmically manipulates those inputs, and generates outputs.

In **computational form**, it would be easy to run the “Ants” model with large numbers of ants and to observe the model’s outputs given many possible different inputs.

We use the term **model implementation** to refer to this process of transforming a textual model into a working computational simulation (written in some form of computer “code”).

Besides textual models, other forms of conceptual models describe processes, objects, or events but are not computational; conceptual models can also be **diagrammatic** or **pictorial**.

Agent

This description lends itself particularly well to being implemented since it results from a particular standpoint, that of an individual ant, or ant *agent*. By the word **agent**, we mean an autonomous individual element of a computer simulation.

- These individual elements have properties, states, and behaviors. The ant is an agent.
- It has properties such as its *appearance* and its *rate of movement*. It has characteristic behaviors such as *moving*, *sniffing*, *picking up food*, and *dropping pheromone*.
- It has states such as whether or not it is carrying food (*a binary state*) and whether it can sense how much pheromone is in the environment around it (*a multivalued state*)

Agent-based modeling (ABM)

- Agent-based modeling (ABM) is a computational modeling paradigm that enables us to describe how any agent will behave.
- The methodology of ABM encodes the behavior of individual agents in simple rules so that we can observe the results of these agents’ interactions. This technique can be used to model and describe a wide variety of processes,
- phenomena, and situations, but It is most useful when describing these phenomena as complex systems.

Lesson 65 : What Is Agent Based Modeling?

The Ant Foraging Model

- Many biologists and entomologists have observed ants in the wild (Hölldobler & Wilson, 1998; Wilson, 1974)
- and have described how ants seemed to form trails to and from food sources and their nest.

Complexity and Complex Systems • The study of complex systems has become an important scientific frontier.

- By the term complex system, we mean a system composed of many distributed interacting parts.
- The field of complex systems or complexity science arose in the mid-1980s and was born out of a variety of disparate fields from economics to physics to ecology.
- Complex systems science provides a set of tools and frameworks for viewing phenomena.
- Any phenomenon can be viewed as a complex system, and choosing when to do so depends on assessing when it is most useful to use the lens and/or methodologies of complex systems.
- One important methodology of complexity science is agent-based modeling.

Creating the Ant Foraging Model (Hypothesis)

In the next few paragraphs, we will describe some hypotheses about how ants accomplish this behavior and what mechanisms are at work that enables ants to find food in this way.³ Perhaps the trails form as follows:

- After an ant finds food, it goes back to the nest, drops the food off, and communicates to the queen ant, and this queen ant tells the other ants where the food is and sends them off to collect it.
- Suppose a researcher notices that the food-finding ant never communicates with any kind of “boss” ant before leaving the nest again, so the researcher might reason that food gathering was not working through a centralized leader or controller but rather through distributed control. **Another Hypothesis**
- Another hypothesis that could be advanced is that perhaps the ant did not communicate with a central leader,
- but rather simply communicated with other ants, and those other ants were able to “diffuse” the information about the food source throughout the nest.
- There is reason to believe such a hypothesis might be true since bees work similarly;
- when a bee finds a food source, it returns to the hive and conducts a complicated dance that tells the other bees where to find the food (Gould & Gould, 1988).
- However, though ants do have some methods of communicating information directly (Hölldobler & Wilson, 1998)
- Most species do not communicate the exact route to a food source in this way.

Lesson 66 : Agent-Based Models vs. Other Modeling Forms

Basic Hypothesis

- Now that we have a basic hypothesis (in the form of a textual model) that describes the behavior of the ant,
- how do we implement the model so that we can test out this hypothesis and see if our computational model adequately accounts for what we observe in nature?

Implemented Model

- The first step is to create a more algorithmic description of the preceding textual model.
- This is just another model itself, but one that is more easily translated into an implemented model.
- Here is one set of rules that an individual ant could follow to operate per the model described earlier.

Set of Rules

- We describe the rules from the point of view of an individual ant.

1. If I am not carrying food, I check if there is food where I am; if there is, I pick it up; if there is no food right here, I try to sense a pheromone trail nearby; if I find one, then I face “uphill” along the pheromone gradient toward its strongest source.
2. If I am carrying food, I turn back toward the nest and drop pheromone on the ground below me.
3. I turn randomly a small amount and move forward a step.

NetLogo • These rules are easily implemented in a computer language.

- There are many computer languages in use for many purposes, but only a few are specially tailored to work with agent-based modeling.
- One of these is NetLogo (Wilensky, 1999a).
- ***NetLogo*** is both a modeling language and an integrated environment designed to make agent-based models easy to build.
- NetLogo is so easy to use that, rather than describe algorithms and models in pseudocode, throughout this course we will use NetLogo instead.
- Describing the preceding rules in the NetLogo language is straightforward.
- Here, for example, is one translation of rules 1 – 3 into NetLogo code:

```
If not carrying-food? [ look-for-food ]      ;; if not carrying food, look for it
if carrying-food? [ move-towards-nest ]      ;; if carrying food turn back towards the nest
wander                                       ;; turn a small random amount and move forward
```

This code snippet is not the complete implemented model but it does describe the core components that go into the Ants model. To complete the model, we need to describe each of the subcomponents (such as “move-towards-nest,” and “look-for-food,” and “wander” and “lookfor-food” will need to describe the ant’s sniffing for pheromone). Each of which is a small amount of code. The result will be a fully implemented computational model. (See figure 1.2.)

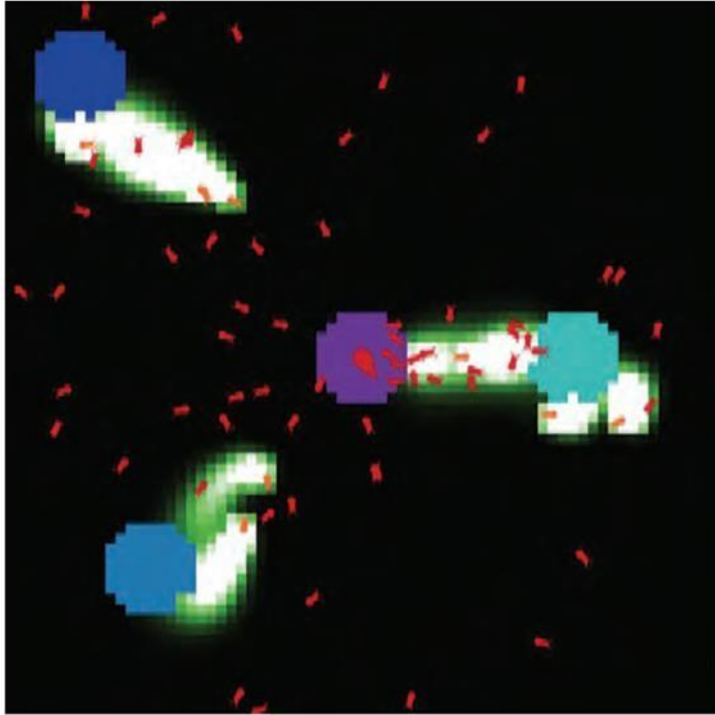


Figure 1.2
NetLogo “Ants” model of foraging behavior.⁸

Lesson 67 : Randomness vs. Determinism

Results and Observations from the Ant Model

When running the Ants model, the results are initially surprising for someone who has only seen the rules for the individual ants. The “ aggregate ” or “ macro ” behavior of the model shows systematic food gathering behavior. It is as if the ant colony has a clear plan for how to gather the food. Yet, we have seen that the Ants model rules do not contain any systematic foraging plan. • If we look closely at the model running, we observe that initially, the ants wander around at random.

- Then some ants will wander into a nearby food source.

- Once they find that food source, they will start to bring food back to the nest, laying a pheromone trail beneath them.
- If just one lone ant finds the food source, the pheromone trail will not be strong enough for other ants to follow it;
- but as more and more ants find the food source, the trail will become stronger and stronger.
- Eventually, the actions of many ants will create a strong pheromone trail from the nest to the food source, and so any ant can easily find the trail to the food source.
- The ants as a group appear to exploit food sources optimally.
- That is, they first gather food from the nearest food source, then the second nearest, and so on.
- This appears to be a conscious plan of the ant colony; but as we know from the ant rules, this is not the case.
- As you observe the model closely, you will note that sometimes ants operate almost at cross-purposes with other ants, creating additional pheromone trails to farther food sources and distracting some of the ants that are currently harvesting the closer food source. The ants do not have a centralized controller; instead, the nearest food sources are the most likely to be found first by the ants randomly wandering from the nest.
- The nearest food sources also require the least amount of pheromone since the pheromone only has to cover the shortest path from the nest to the food. Once a sufficient number of ants have found a particular food source, the pheromone trail to it stabilizes and thus attracts more ants to it.⁹ When the food source has been completely consumed, the ants no longer lay pheromone; hence, the trail dissipates, and the ants are released to search for other food sources.

Lesson 68 : When Is ABM Most Beneficial?

Exploration & Exploitation • This optimal exploitation of food sources could be placed within a larger context.

- In many ways, the colony of ants seems to balance exploration and exploitation (Dubins & Savage, 1976).
- In any situation in which an entity is operating in an unknown environment, the entity must spend some time exploring the environment to understand how its actions affect its rewards, and sometimes exploiting the environment, that is taking actions that it knows have produced the best rewards in the past.

- By allocating a large number of ants to exploit the current nearest food source while other ants continue to explore, the ant colony as a whole successfully balances exploration and exploitation.

Emergent Phenomenon

- These “trails” that the ants build to the food source, the “optimal” behavior that they exhibit, and the “balance” between exploration and exploitation are not coded into anyone ant.
- There is nothing that tells the ant to build a trail; there is nothing that tells the ant to go to the nearest food source first; there is nothing that tells some ants to explore while others exploit.
- The “trails,” “optimal,” and “balance” behavior of the ants is not coded into any of the ants but is instead an emergent phenomenon of the model (Holland, 1998; Anderson, 1972; Wilensky & Resnick, 1999).

Lesson 69 : Trade-offs of ABM

What Good Is an Ant Model?

The Ant model is our first example of an agent-based model.

- Now that we have gone through the process of understanding how a model of ant foraging works,
- what knowledge have we gained from that process and how can we profitably use the model?
- It may seem at first that the only thing the model does is provide a visualization of one particular textual model.

Eight Main Uses For Agent-based Models

We describe eight main uses for agent-based models:

- (1) description,
- (2) explanation,
- (3) experimentation,
- (4) providing sources of analogy,
- (5) communication/education,
- (6) providing focal objects or centerpieces for scientific dialogue,
- (7) as thought experiments, and
- (8) prediction.

A Model Is Descriptive

A model is *descriptive* of a real-world system.

- Granted, it is a simplification of the real world and does not contain all of the details and inconsistencies that are present in the real world.
- But all models are coarse-grained descriptions of reality; and models that are not coarsegrained descriptions are useless as descriptions because they are indistinguishable from the real world and therefore do not assist in our understanding of complex systems (Korzybski, 1990).

If your model includes all aspects of the real phenomenon, it is more efficient to simply observe reality, since it saves you the time of building the

model. The function of a model is to help us to understand and examine phenomena that exist in the real world in more tractable and efficient ways than by simply observing reality. Even if you have never observed a real ant colony, the Ants model helps — it can help you know what to look for and generate hypotheses that you can confirm or disconfirm by observation. **Models Are Explanatory**

- Models are explanatory in that they point out the essential mechanisms underlying a phenomenon.
- They can function as proof that hypothesized mechanisms are sufficient to account for an observation.

- Models provide us with a proof of concept that something is possible. For example, once we built the Ants model and observed the results, we proved that an ant colony could exhibit characteristics such as “ trails, ” “ optimality, ” and “ balance ” without a centralized controller (Resnick & Wilensky, 1993; Resnick, 1994; Wilensky & Resnick, 1999). These characteristics are all emergent outcomes of low-level mechanisms.

Lesson 70 : What Is Needed to Understand ABM?

The key function of ABMs • A key function of ABMs is to explicate the power of the emergence.

- In general, it is difficult for people to understand how such simple rules can lead to complex observed phenomena, and ABMs make this connection explicit.

Even if this were not the way ants worked, the model illustrates that this is one mechanism that could be used. We can also compare and contrast alternative hypotheses. We could, for example, build the other food-gathering hypotheses discussed earlier as computational models and compare their results to the Ants model and to real-world observations to determine the most plausible explanation.

Models Facilitate Experimentation

Models facilitate *experimentation*.

- Models can be run repeatedly to note variations in their dynamics and their outputs.
- Some models have very little variation from run to run.
- Others exhibit *path dependency* and can therefore vary tremendously from run to run.

Experimentation

Model parameters can be varied to see their effect on model behavior and outputs. For example, in the Ants model, we could change the evaporation rate of the pheromone and see what effect that has on the performance of the ants. Alternatively, we could model the return to nest behavior in more detail, by placing a sun in the environment and having the ants calculate their path back based on the sun.

Effects of modifications on the system

Thus agent-based models enable us to easily examine different mechanisms and attributes of a system and see what effect those modifications have on the overall behavior of the system.

Model's Behavior Classification

By varying these different attributes or parameters and observing their effect on the behavior of the system, we can classify model behavior into output regimes, or characteristic output behaviors of the model.

Models Provide Us with Analogies

Since models are simplifications of reality, they enable us to find similarities with other such simplifications even if the modeled phenomena are very different. In this way, we can apply reasoning gained in one domain

- of knowledge to other knowledge domains. For instance, in 1996, Schoonderwoerd, extending Dorigo's work on ant colonies as optimization devices, found that the problem of ants efficiently finding food sources is similar to the problem of packets on networks efficiently seeking out their destinations (Schoonderwoerd et al., 1996).

Lesson 71 : Explorations

Communication And Education • Agent-based models can be used as a vehicle for communication and education.

- We can show the model to people who have never seen an ant colony before and they can explore how ant colonies behave.
- Models provide us with an educational tool that encapsulates knowledge that may not be readily available from real-world observation.
- Moreover, an agent-based model allows us to expand beyond static knowledge, enabling learners to conduct experiments much as scientists do.
- If someone has a hypothesis as to how a particular mechanism works, they can implement that mechanism and see whether it can account for observed behavior.

Thought Experiments

Models can sometimes present new phenomena that are not necessarily about some real-world phenomenon but are *thought experiments* on possible computations.

Some might not use the word “model” for this kind of computational artifact, but many scientists do refer to them as models. Some classic examples of these kinds of models are cellular automata, fractals, and particle swarms.

Prediction

A common conception of computer modeling is that its major purpose is *prediction*.

- Indeed, we often use modeling to think about possible future scenarios, and to the extent that any modeling tool can be used for discussing the future, agent-based modeling can be used in this way.
- However, like any modeling tool, an agent-based model's predictions depend on the accuracy of its input data.
- This is especially true when the events we are interested in are the results of complex systems, for which small changes in inputs can often lead to very different results.
- Many times, while modelers may claim to want to use a model to predict the outcome of a system, such as an ant colony, they use the model to *describe* past patterns of behavior, such as what food source was first eaten, and to *explain* future patterns that might arise, such as how ants might move around on the landscape.
- Sometimes modelers start motivated by generating predictions but end up finding greater power for explanation and description.
- Agent-based models, in particular, are distinctive from other modeling approaches in that they were designed to understand and explain complex phenomena that otherwise could not be explained through traditional approaches.

Lesson 72 : Beginner NetLogo Explorations

The Reality of Agents

The profitable uses of the Ants model that we described earlier are not particular to that one model. These are generally applicable affordances of the methodology used to develop the model, which is agent-based modeling. The core idea of Agent-Based Modeling is that many (if not most) phenomena in the world can be effectively modeled with agents, an environment, and a description of agent-agent and agent-environment interactions.

What is an Agent?

An agent is an autonomous individual or object with particular properties, actions, and possibly goals.

What is the environment?

- The environment is the landscape on which agents interact and can be geometric, networkbased, or drawn from real data.

What about the interactions?

The interactions that occur between these agents or with the environment can be quite complex.

- Agents can interact with other agents or with the environment,
- and not only can the agent's interaction behaviors change in time, but so can the strategies used to decide what action to employ at a particular time.
- These interactions are constituted by the exchange of information. As a result of these interactions, agents can update their internal state or take additional actions.

Agent-Based Models vs. Other Modeling Forms

What makes agent-based models distinct from other models?

The most common form of scientific models is the equation form.

Differences between Agent-based Modeling (ABM) and equation-based modeling (EBM)

Many differences between ABM and equation-based modeling (EBM) have been discussed. • One distinction is that because ABM models individuals it can model a heterogeneous population, whereas equational models typically must make assumptions of homogeneity.

In many models, most notably in social science models, heterogeneity plays a key role.

- Furthermore, when you model individuals, the interactions and results are typically discrete and not continuous. Continuous models do not always map well onto real-world situations. For instance, equation-based models of population dynamics treat populations as if they are continuous quantities when in fact they are populations of discrete individuals.

When simulating population dynamics it is very important to know if you have a sustainable population. After all, a wolf population cannot continue if there are fewer than two wolves left; in reality, a millionth of a wolf cannot exist and certainly cannot reproduce, but it can result in an increased wolf population in EBMs. The mismatch between the continuous nature of EBMs and the discrete nature of real populations causes this “ nano-wolf ” problem (Wilson, 1998). As a result, for EBMs to work correctly, they must assume that the population size is large and that spatial effects are unimportant.

Lesson 73 : Ants and Other Model Explorations

Randomness

One important feature of agent-based modeling, and computational modeling in general, is that it is easy to incorporate randomness into your models.

Benefits of Randomness

Many equation-based models and other modeling forms require that each decision in the model be made deterministically.

In agent-based models this is not the case; instead, the decisions can be made based on a probability.

Example of Randomness

For instance, in the Ants model, as the ants move around the landscape, their decisions are not completely determined; instead, at each time step, they modify their heading by a small amount based on a random number.

Ant Model and Randomness

- As a result, each ant follows a unique, irregular path.
- In reality, ants might be affected by small changes in elevation, the presence or absence of twigs and stones, and even the light of the sun.

- Finally, there are often times when we simply do not know enough about how a complex system works to build a completely deterministic model.
- In many of these cases, the only type of model that we can build is a model with some random elements.
- Agent-based modeling and other modeling forms that allow you to incorporate random features are essential to studying these kinds of systems.

Lesson 74 : Concept Explorations

ABM Benefits

- Agent-based modeling has some benefits over other modeling techniques, but, as with any tool, there are contexts in which it is more useful than others.
- ABM can be used to model just about any natural phenomenon (e.g., you could describe any phenomenon by describing the interaction of its subatomic particles).
- However, there are some contexts for which the cost of building an ABM exceeds the benefits, and there are other times when the benefits are extraordinary given the costs.

Examples

Some problems with a large number of homogenous agents are often better modeled (i.e., they will provide more accurate solutions to aggregate problems faster) using an aggregate solution like mean-field theory or system dynamics modeling (Oppen & Saad, 2001; Forrester, 1968).

For instance, if you are concerned about the temperature in a room, then tracking every individual molecule and its history is not necessary.

General Rules

On the other hand, if a problem has only a handful of interacting agents, then you usually do not need to bring to bear the full power of ABM and instead can write detailed equations describing the interaction — two billiard balls colliding, for example, does not require ABM.

As a rule of thumb, agent-based models are most useful when there are a medium number (tens to millions) of interacting agents (Casti, 1995).

Lesson 75 : NetLogo Explorations

ABM Tradeoffs

Agent-based modeling provides some benefits over other methods of modeling, but, in any particular situation, choosing a modeling methodology is a case of choosing the appropriate tool at the appropriate time, and sometimes agent-based modeling is not the right tool for the job.

For example, ABM can be computationally intensive. Simulating thousands or millions of individuals can require great computing power.

Equation-based models, by contrast, are often very simple to run and essentially just require repetitive mathematical calculations. This is true only for simple equation-based models; numerically solving complicated equation-based models may take as much computational time as agent-based models.

Still, ABM may be Better

The computational expense of running an ABM is a price one pays for having

the benefits of rich individual-level data. The additional computational power needed for running ABMs is the same power that allows the tracking and development of rich histories of individuals.

Examples

For example, in the Ants model, we could write a simple equation that describes the rate at which food is gathered based on its distance from the nest.

However, through ABM, we can observe the behavior of the individual ants and understand how they form trails to the food source. **Other cases**

- ABM does require us to gain an understanding of the micro behavior of a system, but without modeling the micro behaviors, ABM would not provide as elaborate a model as it does.
- While ABM does require us to know or guess about the individual-level mechanisms operating, it does not require knowledge of the aggregate-level mechanism.

Lesson 76 : Tradeoffs of ABM

ABM Tradeoffs

Agent-based modeling provides some benefits over other methods of modeling, but, in any particular situation, choosing a modeling methodology is a case of choosing the appropriate tool at the appropriate time, and sometimes agent-based modeling is not the right tool for the job.

For example, ABM can be computationally intensive. Simulating thousands or millions of individuals can require great computing power.

Equation-based models, by contrast, are often very simple to run and essentially just require repetitive mathematical calculations. This is true only for simple equation-based models; numerically solving complicated equation-based models may take as much computational time as agent-based models.

Still, ABM may be Better

The computational expense of running an ABM is a price one pays for having

the benefits of rich individual-level data. The additional computational power needed for running ABMs is the same power that allows the tracking and development of rich histories of individuals.

Examples

For example, in the Ants model, we could write a simple equation that describes the rate at which food is gathered based on its distance from the nest.

However, through ABM, we can observe the behavior of the individual ants and understand how they form trails to the food source. **Other cases**

- ABM does require us to gain an understanding of the micro behavior of a system, but without modeling the micro behaviors, ABM would not provide as elaborate a model as it does.
- While ABM does require us to know or guess about the individual-level mechanisms operating, it does not require knowledge of the aggregate-level mechanism.

Lesson 77 : Creating Simple Agent-Based Models: Life-I

Now We will learn to construct a few simple agent-based models. These simple

models, sometimes referred to as “ toy models, ” are not meant to be models of real phenomena, but instead are intended as “ thought experiments. ” They are offered, as Seymour Papert puts it, as “ objects to think with ” (1980). Our purpose is to show that it is relatively easy to create simple agent-based models, yet these simple models still exhibit interesting and surprising emergent behavior. We will construct three such models: “ Life, ” “ Heroes and Cowards, ” and “ Simple Economy. ”

Game of Life: History

In 1970, the British mathematician John Horton Conway (described in Conway, 1976) created a cellular automaton that he called the “ Game of Life. ” Martin Gardner (1970) popularized this game in his Scientific American column. Subsequently, millions of readers played the game.

Game of Life

The game is played on a large grid, such as a checkerboard or graph paper. Let’s say we are playing on a square grid with 51 squares (or “ cells ”) on a side. Each cell can be either “ alive ” or “ dead. ” This is called the “ state ” of the cell. Every cell is surrounded by eight “ neighbor ” cells. The grid is considered to “ wrap around ” so that a cell on the left edge has three (3) neighbor cells on the right edge and, similarly, a cell on the top edge has three (3) neighbor cells on the bottom edge. There is a central clock. The clock ticks

establish a unit of time. In the game of Life, the unit of time is called a generation. More generally, in agent-based models, the unit of time is referred to as a tick. Whenever the clock ticks, each cell updates its state according to the following rules:

- Each cell checks the state of itself and its eight neighbors and then sets itself to either alive or dead.

In the rule descriptions that follow, blue cells are “ dead, ” green cells are “ alive ” and the yellow stars indicate the cells affected by the rule described.

- (1) If the cell has less than two (2) alive neighbors, it dies
- (2) If it has more than three (3) alive neighbors, it also dies
- (3) If it has exactly two (2) alive neighbors, the cell remains in the state it is in
- (4) If it has exactly three (3) alive neighbors, the cell becomes alive if it is dead, or stays alive if it is already alive.

Lesson 78 : Life-II

The Game of Life is played on a grid of cells, so we will consider each NetLogo patch as a different cell. Life has two kinds of cells in it, “ live ” cells and “ dead ” cells. We choose to model live cells as green patches and dead cells as blue patches. Once we have thought through what the model will look like we still need to create the model instructions. To do this, we will need to write NetLogo instructions (or code) in the Code tab. We select

the Code tab and begin to write our code. NetLogo code takes the form of modules known as “ procedures. ” Each procedure has a name and begins with the word TO and ends with the word END.

Our Life model (Life-Simple in the IABM Textbook folder of the NetLogo models library) will consist of two procedures: **SETUP**, which initializes the game, and **GO**, which advances the clock by one tick. **Neighbors**

In agent-based modeling, we often model local interactions. This is accomplished by having cells in the grid talk to their neighbor cells. Most often, we are using two-dimensional grids or lattices with square cells.

In a square lattice, there are two commonly used neighborhoods: the **von Neumann neighborhood** and the **Moore neighborhood**.

A cell's **von Neumann neighborhood** consists of the **four cells** that share an edge with it, the cells to the north, south, east, and west (the green squares in the right figure that follows).

A **Moore neighborhood** consists of the **eight cells** that touch it, adding the cells to the northeast, southeast, northwest, and southwest (the entire green area in the left figure that follows).

In NetLogo, the primitive NEIGHBORS refers to a patch's Moore neighbors, and NEIGHBORS4 refers to a patch's von Neumann neighbors.

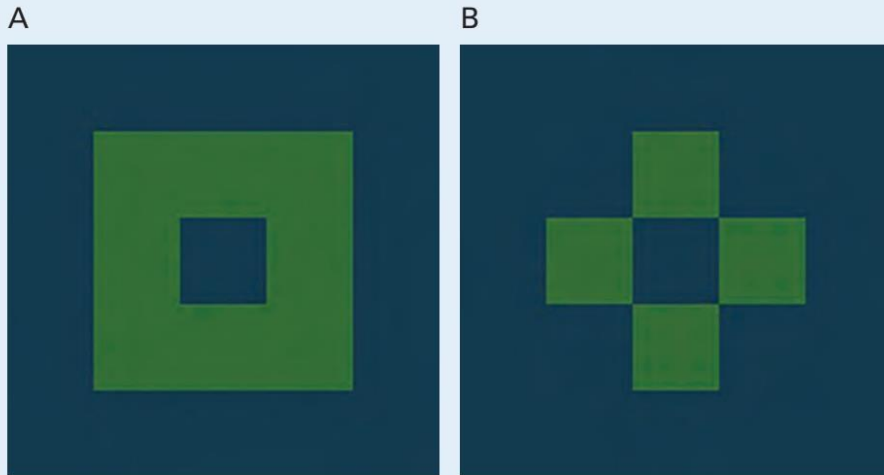


Figure 2.10

(A) Moore Neighborhood. (B) von Neumann Neighborhood. This will be further elaborated in chapter 5.

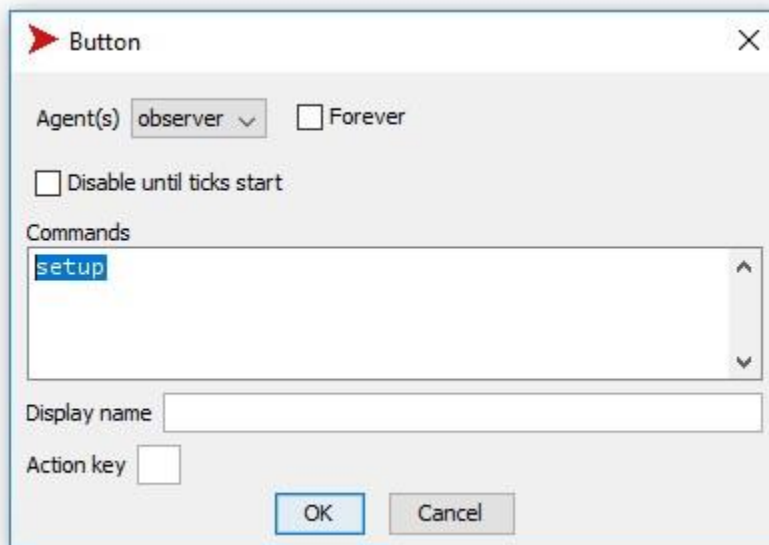
Lesson 79 : Life-III

Overview

- Observing how Game of Life works
- Examining emergent patterns

User Interface

- Adding a Button



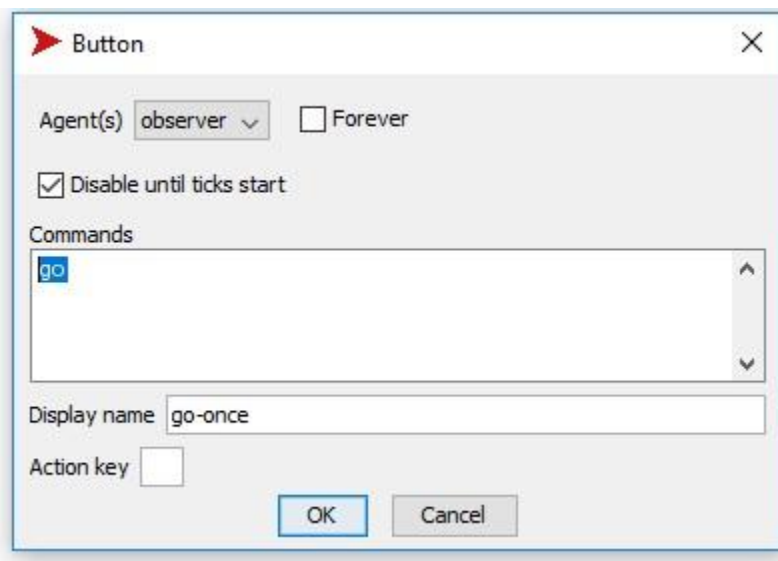
The 'Button' dialog box is shown with the following settings:

- Agent(s): `observer` (selected from a dropdown)
- ☐ Forever
- ☐ Disable until ticks start
- Commands: `setup` (entered in the text field)
- Display name: (empty text field)
- Action key: (empty text field)

Buttons: OK, Cancel

Go-Once button

- Add another button



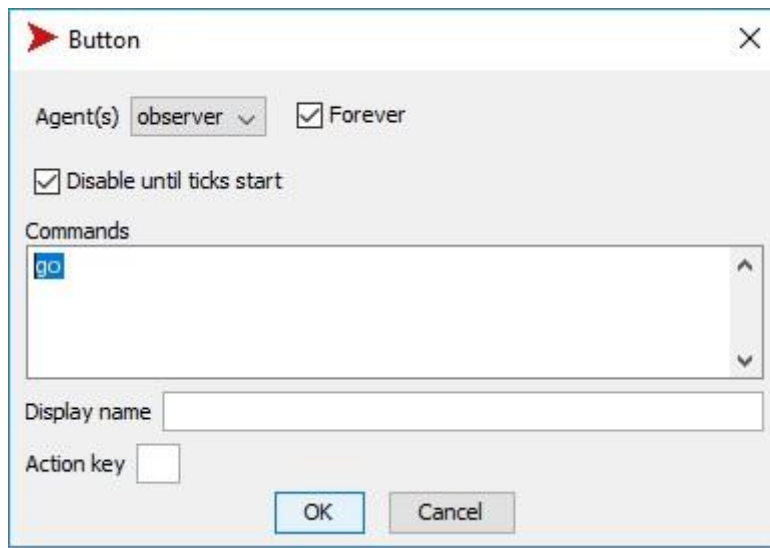
The 'Button' dialog box is shown with the following settings:

- Agent(s): `observer` (selected from a dropdown)
- ☐ Forever
- ☒ Disable until ticks start
- Commands: `go` (entered in the text field)
- Display name: `go-once` (entered in the text field)
- Action key: (empty text field)

Buttons: OK, Cancel

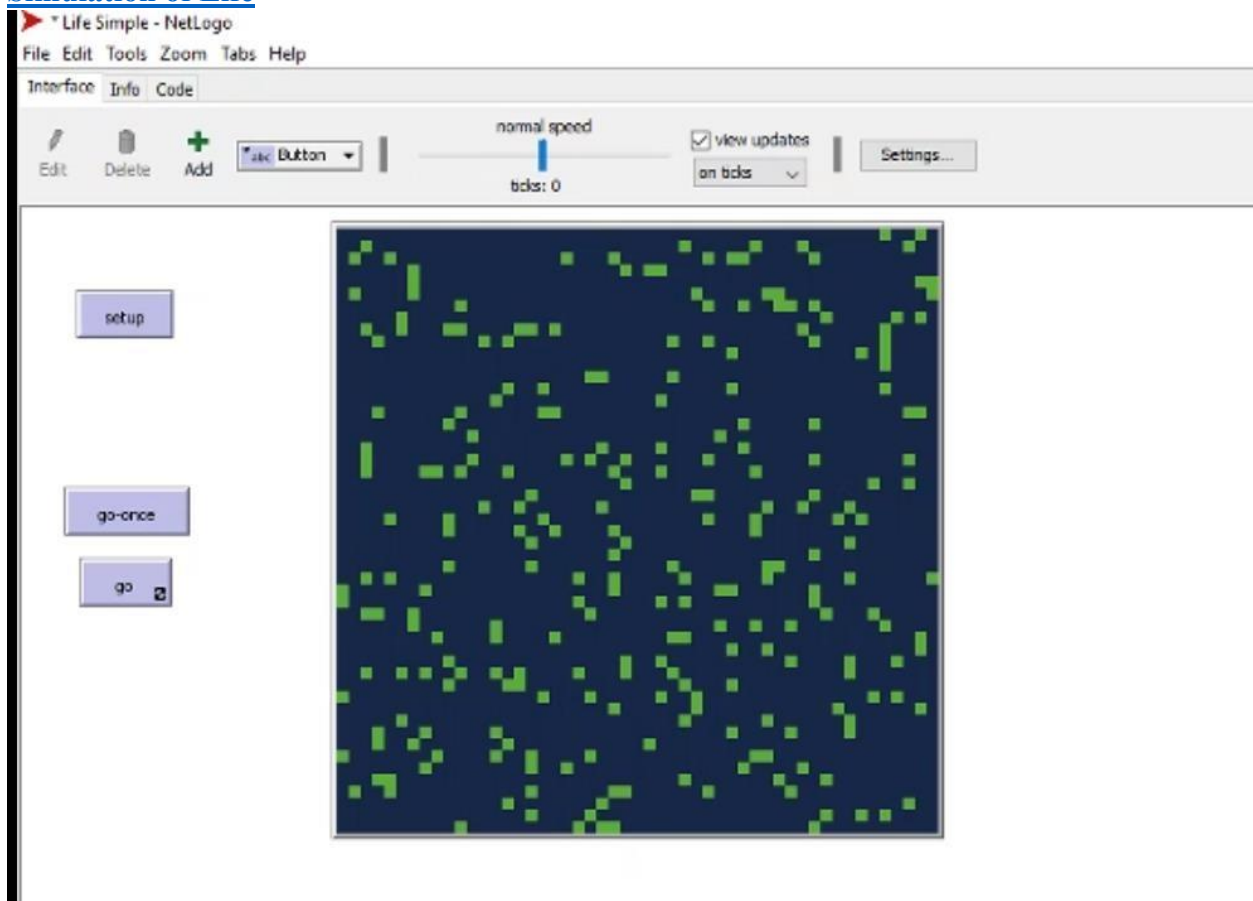
Go button

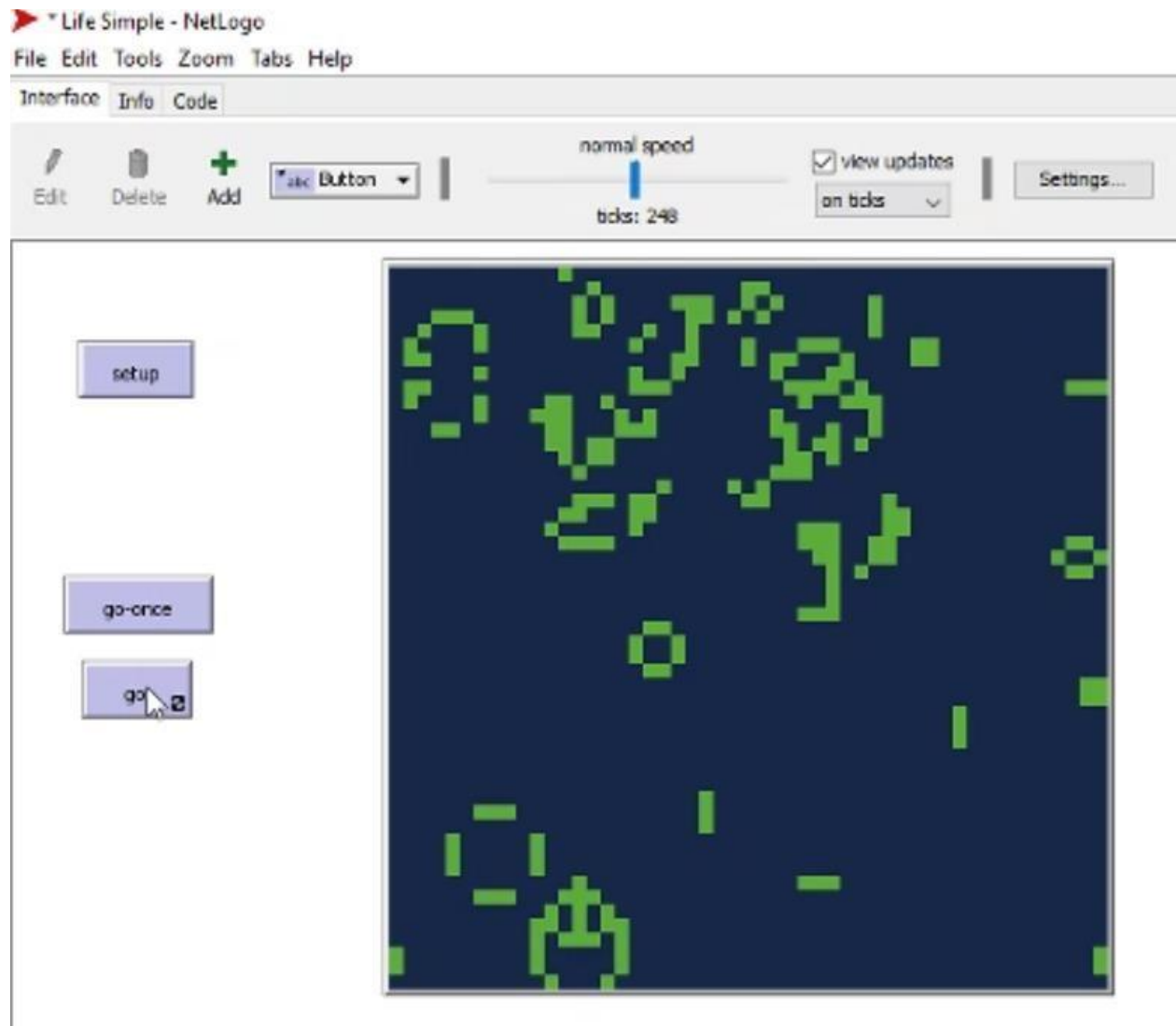
- Add yet another button



Understanding the Output

Simulation of Life





- **Emergent Pattern: Glider**



- **Another emergent pattern**



Summary

- We have examined outcomes from the game of life
- We have seen how emergent patterns emerge unexpectedly

Lesson 80 : Simple Economy-I

In the past two decades, there has been increasing interest in and use of agent-based modeling in the social sciences. Several of the models that we have already discussed, such as Ants, the Game of Life and Heroes, and Cowards have been applied to understanding social systems.

Why ABM in Other Sciences?

Agent-based methods may be particularly valuable in the social sciences where agents are heterogeneous and mathematical descriptions often do not offer sufficient descriptive power.

Organizations

Several prominent communities have organized around using complex systems methods and agent-based modeling in the social sciences. Among these are:

- the Complex Systems Social Sciences Association (CSSSA) and •
the World Congress on Social Simulation (WCSS),

both of which also organize conferences. Several such events are included as part of other meetings.

Other Organizations

In recent years, organizations such as

- the American Education Research Association (AERA),
- Marketing Science,
- Eastern Economics Association, and
- American Association of Geographers (AAG)

The above-mentioned organizations have hosted sessions on the intersection of agent-based modeling and social science.

Economics Using ABM

One area that has been receiving increasing attention from the ABM community in economics. As economies consist of heterogeneous actors such as buyers and sellers, there is a natural mapping of ABM methods to economics. In 1996, the economists, Josh Epstein and Robert Axtell published a book that depicted a world they called SugarScape, which was populated by economic agents.

Simple Economy Description

In this section, we will build a very simple economic model that has some surprising results.

Suppose you have a fixed number of people, say 500, each starting out with the same amount of money, say \$100. At every tick, each person gives one of his or her dollars to any other person at random. What will happen to the distribution of money? An important constraint is that the total amount of money remains fixed, so no one can have less than zero money. If you run out of money, you cannot give any away until you get some back. Refining the question a little further, we ask: Is there a stable limiting distribution of the money? If so, what is it? For instance, will all the wealth be concentrated in a few hands or will it be equitably distributed?

Simple Economy Intuition

Many people, when posed with this question, have an intuition that there is a limiting distribution and that it is relatively flat. The reasoning behind the intuition is that since no

person starts with an advantage and the selection of people to whom money is transferred is random, no person should have much of an advantage over any others. Thus, the resulting wealth distribution should be relatively flat: Everyone should wind up with roughly the same amount of money he or she started with.

Alternate Intuition

Other people have an intuition that wealth should be normally distributed.

Lesson 81 : Simple Economy-II

[click here to download the ppt](#)

Overview

- Go in more detail about the “Simple Economy” model
- Understand the code for Simple Economy

Simple Economy

```
turtles-own [ wealth ]

to setup
  clear-all
  create-turtles 500 [
    set wealth 100
    set shape "circle"
    set color green
    set size 2

    ;; visualize the turtles from left to right in ascending order of wealth
    setxy wealth random-ycor
  ]
  reset-ticks
end
```

```

3 to go
  ;; transact and then update your location
  ask turtles with [ wealth > 0 ] [ transact ]
  ;; prevent wealthy turtles from moving too far to the right
  ask turtles [ if wealth <= max-pxcor [ set xcor wealth ] ]
  tick
end

3 to transact
  ;; give a dollar to another turtle
  set wealth wealth - 1
  ask one-of other turtles [ set wealth wealth + 1 ]
end

3 to-report top-10-pct-wealth
  report sum [ wealth ] of max-n-of (count turtles * 0.10) turtles [ wealth ]
end

3 to-report bottom-50-pct-wealth
  report sum [ wealth ] of min-n-of (count turtles * 0.50) turtles [ wealth ]
end

```

Summary

- We have examined the implementation of the Simple Economy Model
- We have seen how turtles can be used to develop a model

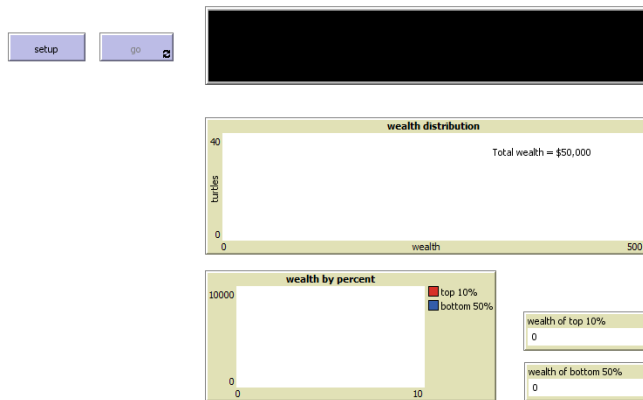
Lesson 82 : Simple Economy-III

[click here to download the ppt](#)

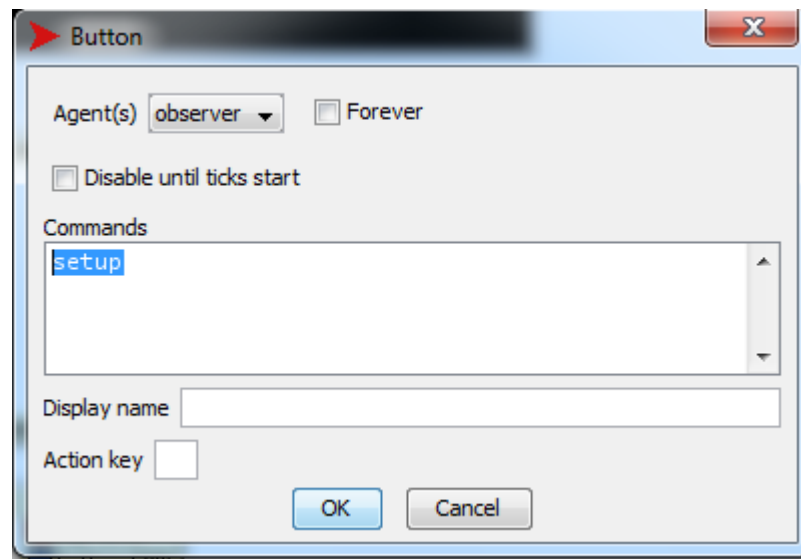
Overview

- Understanding the User Interface of Simple Economy
- Understand the execution of Simple Economy

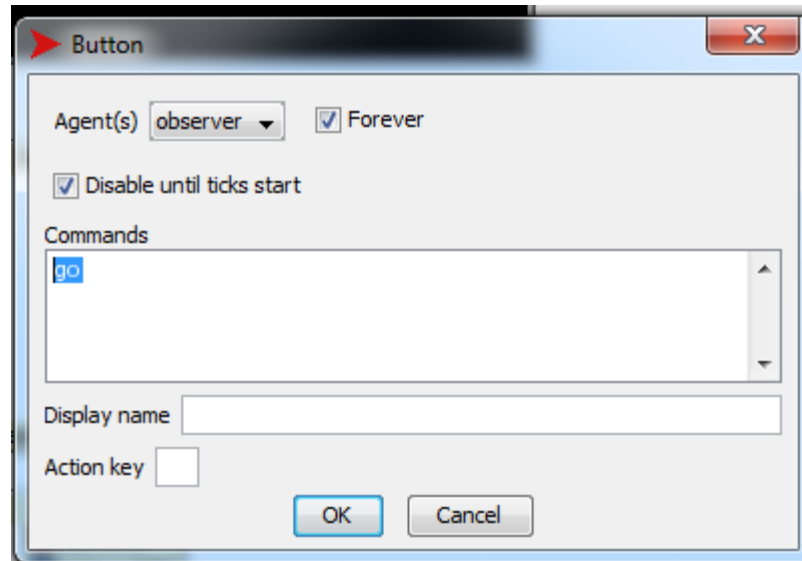
Simple Economy



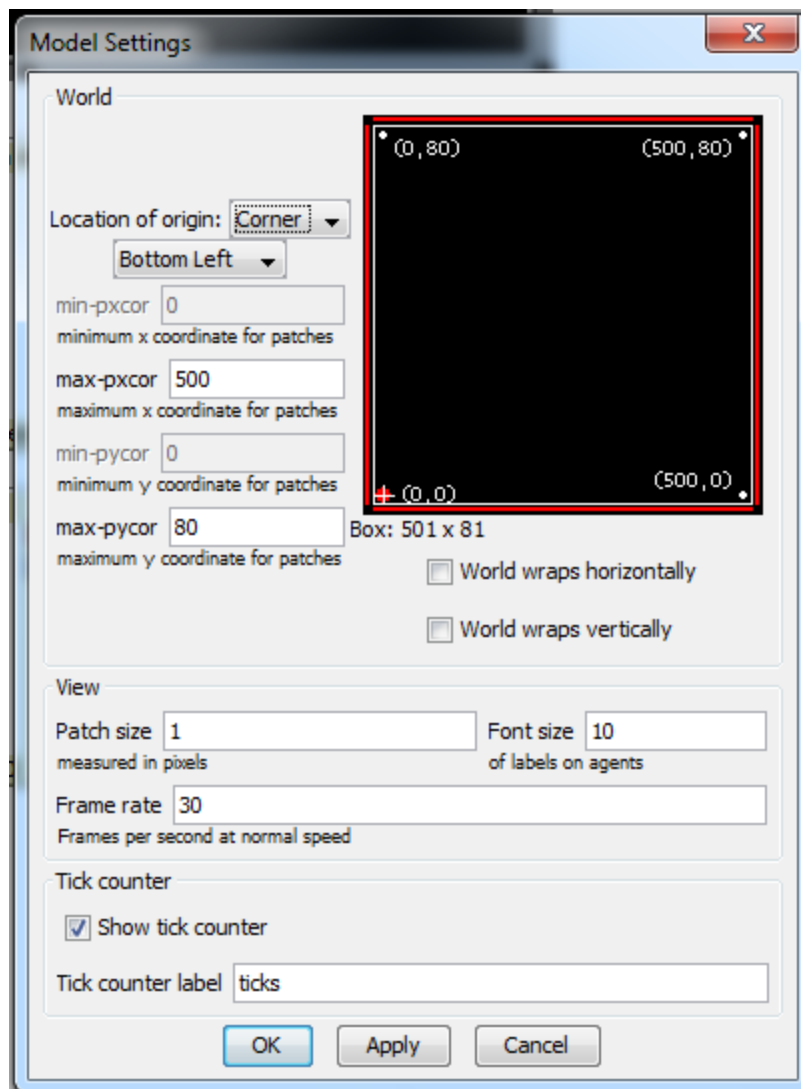
- First we add a button



- Another button



- Next, let us adjust the world settings



- Now, we add a plot

Plot

Name:

X axis label: X min: X max:

Y axis label: Y min: Y max:

☒ Auto scale? ☐ Show legend?

Plot setup commands

Plot update commands

Plot pens

Color	Pen name	Pen update commands	
	current	<code>set-plot-y-range 0 40</code> <code>histogram [wealth] of turtles</code>	 

- Now, we add another plot

Plot

Name:

X axis label: X min: X max:

Y axis label: Y min: Y max:

☒ Auto scale? ☒ Show legend?

Plot setup commands

Plot update commands

Plot pens

Color	Pen name	Pen update commands		
	top 10%	plot top-10-pct-wealth		
	bottom 50%	plot bottom-50-pct-wealth		

- Now, we add a monitor

Monitor

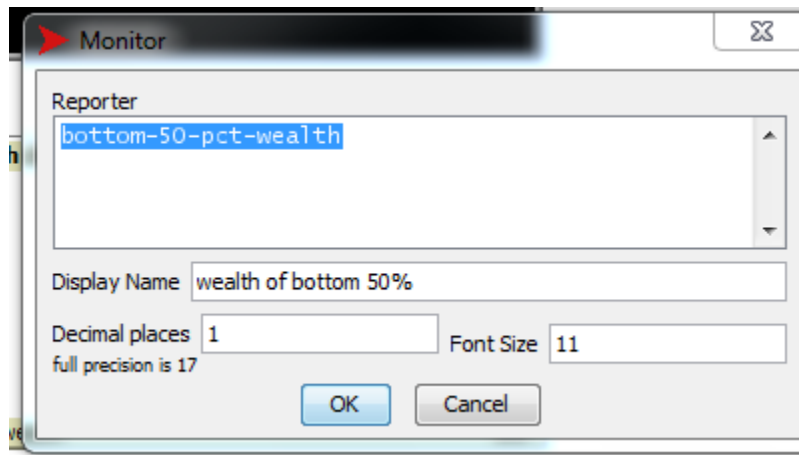
Reporter:

Display Name:

Decimal places: Font Size:

full precision is 17

- Now, we add another monitor



Summary

- We have examined the design of the User interface for the Simple Economy Model

Lesson 83 : Simple Economy-IV

[click here to download the ppt](#)

Overview

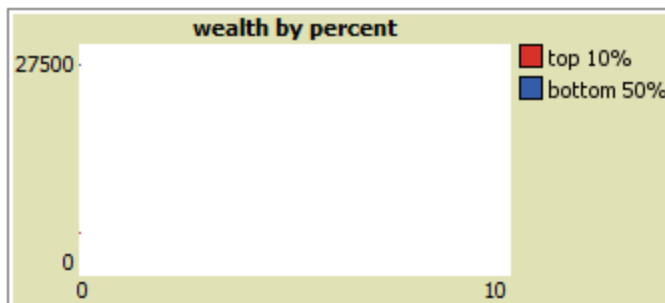
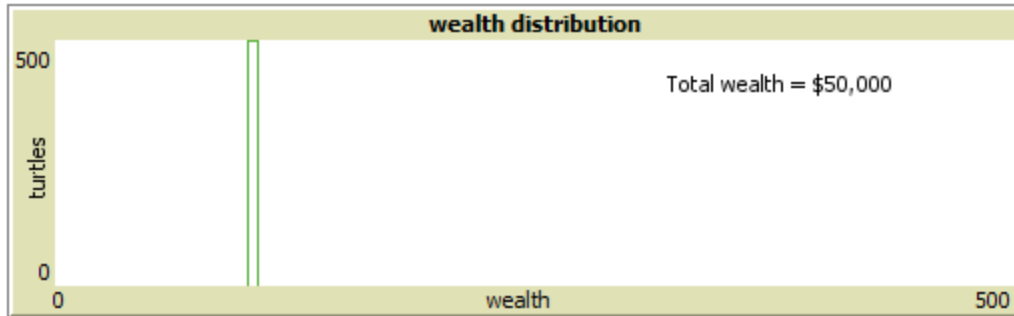
- Understand the Complexity dynamics of the Simple Economy Model

Simulation Run

- Let us see how this works in NetLogo

Running the Model

- Let us see the initial conditions



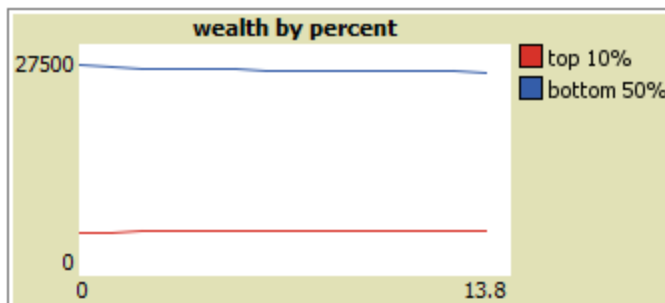
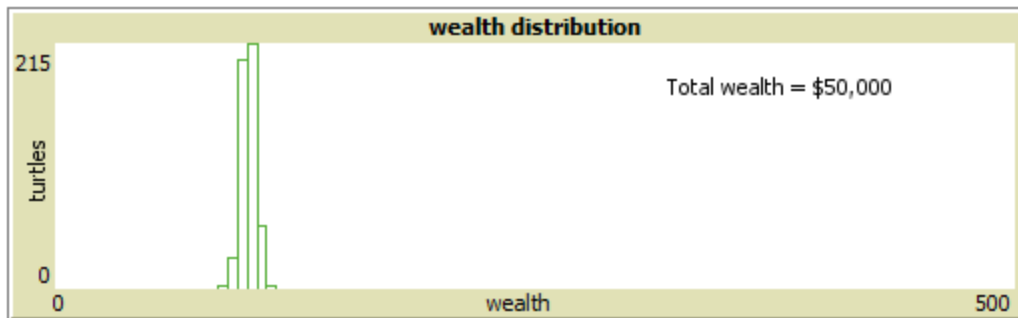
wealth of top 10%

5000

wealth of bottom 50%

25000

- After running

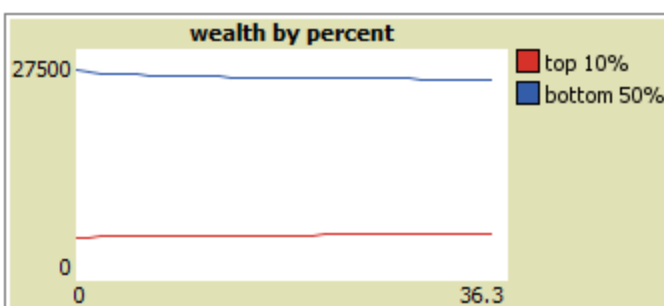
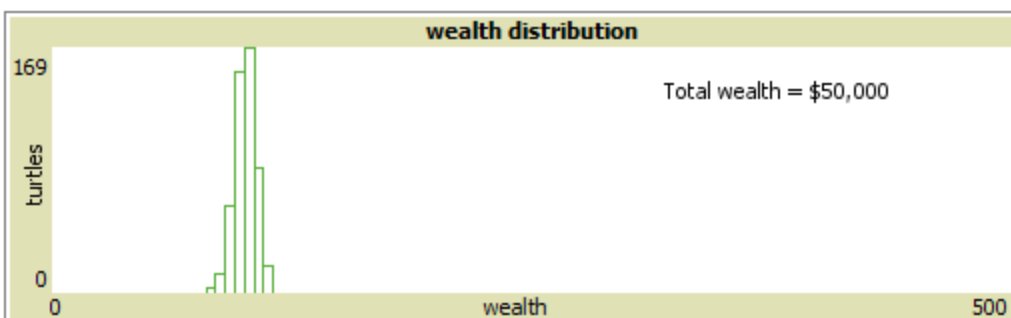


wealth of top 10%

5334

wealth of bottom 50%

24263



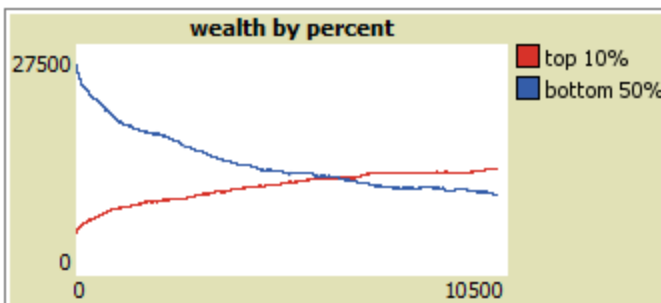
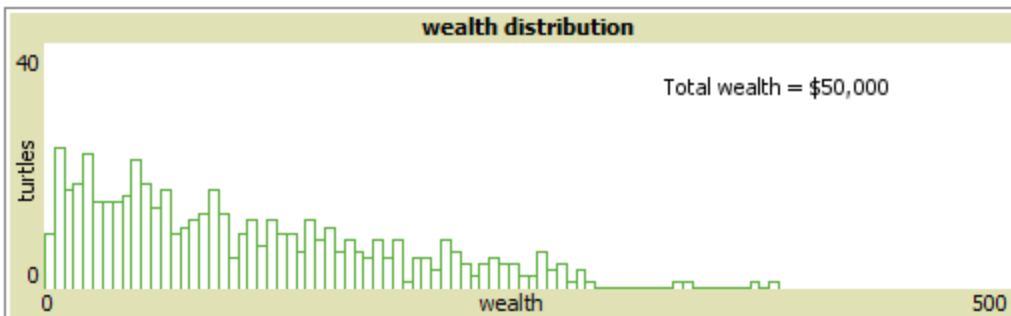
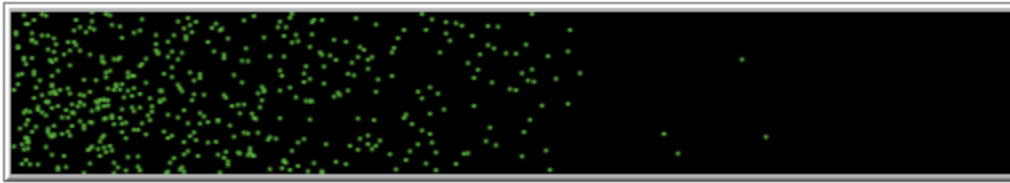
wealth of top 10%

5478

wealth of bottom 50%

23924

- After long run: 10,000



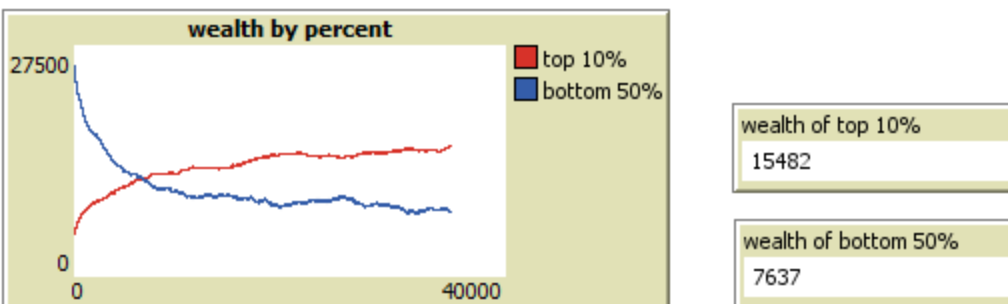
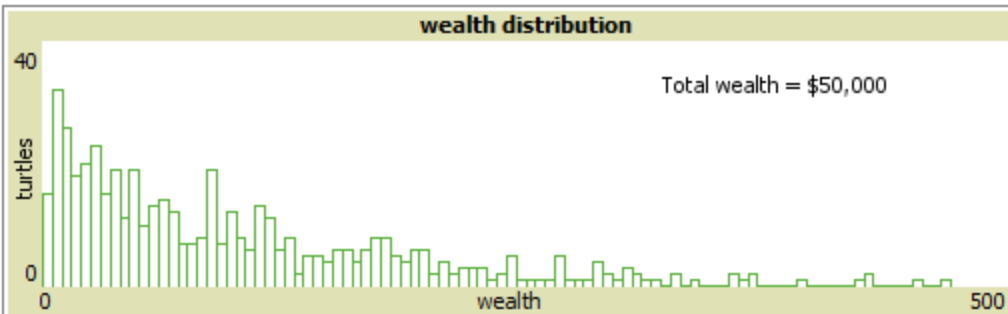
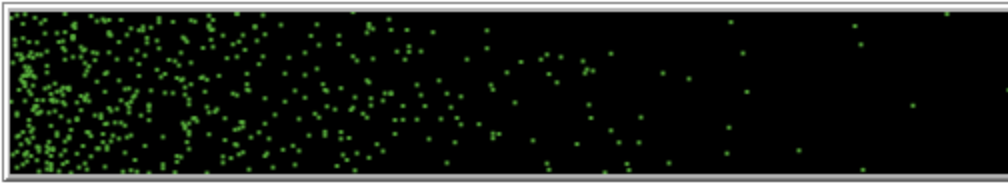
wealth of top 10%

12693

wealth of bottom 50%

9613

- After long run: 35,000



Understanding Results

- We can see that the distribution is not at all flat.
- There are a few very wealthy individuals and many poor agents.
- At this time step in this model run, the wealth of the top 10 percent of the agents is a total of \$12,633, or an average of \$253 per person
- Whereas the total wealth of the entire bottom 50 percent is only \$10,166, or an average of \$41 per person.
- The crossover point at which the wealth of the top 10 percent of all agents exceeded the wealth of the bottom half, or 50 percent, of all agents was at 5,600 ticks.
- If we run it for 25,000 more ticks, the gap continues to grow and now the top 10 percent have more than twice as much in total (and more than 10 times as much per individual) than the bottom 50 percent.
- The distribution of money eventually converges to a stationary distribution.
- This distribution has been shown to be exponential, which means that there is great inequality in monetary wealth.
- The key condition that creates this stationary distribution is the conservation of money.

Summary

- We have examined the Execution of the Simple Economy Model
- We have seen how surprising results can appear in relatively simple models

Lesson 84 : Econophysics

[click here to download the ppt](#)

The basis

- The distribution we have just seen in the Simple Economy model is one instance of a broader class of models in which a conserved quantity is exchanged in a closed system.
- The great nineteenth-century Austrian physicist Ludwig Boltzmann first examined such models when he worked out the physics of gas molecules in a box.
- He imagined these molecules as “billiard balls” moving and colliding.
- When they collide, they change speed and direction, but because of the conservation of energy, the total energy after a collision remains the same.
- He derived an equation, now known as the Boltzmann equation, for the distribution of the energies of the billiard balls and solved the equation to show that their energies formed a stationary exponential distribution, known as Boltzmann distribution, the Boltzmann-Gibbs distribution, and the Maxwell-Boltzmann distribution.
- The Boltzmann distribution is a central component of statistical mechanics.
- Statistical mechanics uses probability theory to understand the thermodynamic behavior of systems composed of a large number of particles.
- Statistical mechanics provides a framework for relating the microscopic properties of individual atoms and molecules to the macroscopic bulk properties of materials that can be observed in everyday life.
- In physics, the methods of statistical mechanics have been used with a range of “particles” from single atoms interacting to complex molecules, and they can be used to explain macroscopic concepts such as work, heat, and entropy.
- Since statistical mechanics gives scientists the ability to reason from microscopic particles to macroscopic patterns, it has been extremely useful in generating a comprehensive theory of physics.
- In many ways, agent-based modeling can be seen as extending this perspective to content domains beyond physics.

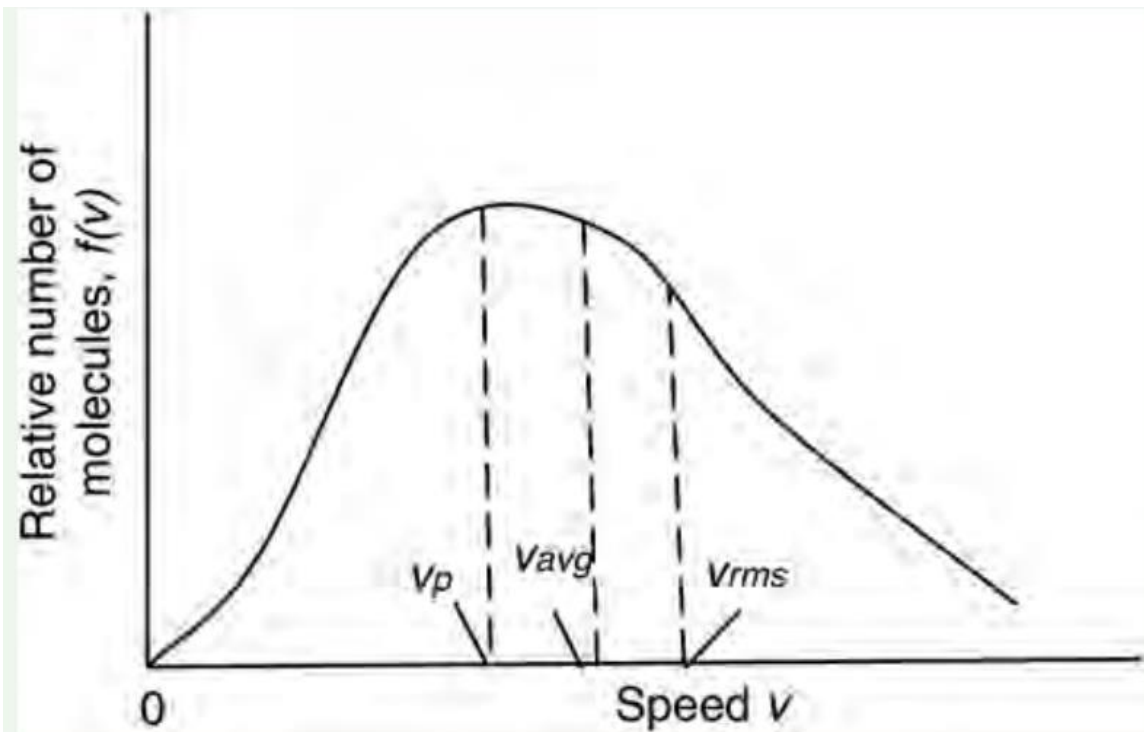


Figure 2.41

Maxwell-Boltzmann distribution of molecule speeds (illustration from Giancoli, 1984).

Recent Work

- In recent years, theories and methods from physics have been applied to economics. The physicist Gene Stanley (1996) coined the term “econophysics” for this interdisciplinary research field.
- In their 2000 paper, Dragulescu and Yakovenko describe several variants of the kind of monetary exchange we have explored in this model.
- They show that if agents are allowed to go into debt (subject to a fixed credit limit), there is still a stationary exponential distribution, but one that results in even greater inequality.
- We have referred to our agents as owning wealth. A more specific term for what we are modeling is *monetary wealth*

Dragulescu and Yakovenko caution:

It is tempting to identify the money distribution with the distribution of wealth. However, money is only one part of wealth, the other part being material wealth. Material products have no conservation law: They can be manufactured, destroyed, consumed, etc. Moreover, the monetary value of a material product (the price) is not constant. The same applies to stocks, which economics textbooks explicitly exclude from the definition of money. So, in general, we do not expect the Boltzmann-Gibbs law for the distribution of wealth.

Lesson 85 : NetLogo: Agentsets and Lists

[click here to download the ppt](#)

Overview

- Understand some commonly used NetLogo concepts

Understanding Complex Results

- Understanding complex results often requires advanced modeling techniques
- E.g. in Simple economy, we have seen complex results
- Some of these results need correlation with the real world
- Empirical studies have shown that wealth distribution in most countries is not distributed exponentially,
- Instead it follows a power law known as the Pareto distribution (Pareto, 1964).
- This distribution is even more unequal than an exponential distribution.
- Power law distributions arise in a surprising number of contexts
- They are typically associated with preferential attachment processes (Barabási, 2002)
- The amount received is proportional to the existing amount.

Lists and Agentsets

- To model such complexity we often need advanced programming concepts
- Some of the most used constructs (data structures) in NetLogo are:
- Agentsets
- Lists
- An agentset is a set of agents.
- Agentsets can have turtles, patches, or links, but they cannot mix agent types.
- You can ask an agentset to perform some commands.
- NetLogo comes with three special agentsets built in: “ turtles, ” “ patches, ” and “ links”
- What makes agentsets so powerful is that you can create your own agentsets
- For example, an agentset of all the red turtles,
- Or an agentset of all the patches in the upper right quadrant.
- Agentsets are always in random order.
- So if you ask an agentset, several different times, to execute some commands, each time the order in which the agents take turns executing the commands will be different.
- Lists are ordered collections of data.
- We can have a list of numbers
- A list of words, even a list of lists, also even a list of agents.
- Since they are in a list, these agents will have a particular order, so you can use the list to execute their commands in any order you might like.

Summary

- We have examined the basis of complex results in the Simple Economy Model
- We have seen how concepts from one domain can be used in a completely different domain

Lesson 86 : Exploring and Extending Agent-Based Models

[click here to download the ppt](#)

Part of the inhumanity of the computer is that, once it is competently programmed and working smoothly, it is completely honest.

—Isaac Asimov

The perfect journey is never finished, the goal is always just across the next river, round the shoulder of the next mountain. There is always one more track to follow, one more mirage to explore.

—Rosita Forbes

Sailors on a becalmed sea, we sense the stirring of a breeze.

—Carl Sagan

Characteristic Features

There are four characteristic features of agent-based modeling:

1. Simple rules can be used to generate complex phenomena
2. Randomness in individual behavior can result in consistent patterns of population behavior
3. Complex patterns can “self-organize” without any leader orchestrating the behavior
4. Different models emphasize different aspects of the world

1. Simple rules can be used to generate complex phenomena

Many of the models that we will look into have very simple rules that do not require complex mathematical formulas or a deep understanding of the knowledge domain that they are attempting to model. Nonetheless, they are able to reproduce complex phenomena that are observed in the real world. For instance, a model of fire spread may have only a simple rule to describe fire spread from one tree to another, but it still may have interesting things to say about how likely a fire is to spread across an entire forest.

2. Randomness in individual behavior can result in consistent patterns of population behavior

It is common for people when they see an ordered population level behavior such as a flock of birds to assume that there must be deterministic processes that govern the behavior

of the individuals (Wilensky & Resnick, 1999). In the case of the birds, people tend to believe that there must be specific social rules or communications that tell each bird how

to place itself in the flock. However, nature has some surprises for us: Many times the individual level rules are quite simple (see point 1) and do not necessarily tell the bird where to position itself in the flock. Instead, the rules often contain a certain amount of nondeterminism and are robust to perturbations in the initial conditions. Despite the stochastic nature of these systems, they can still result in the generation of predictable high-level behavior like the flocking of birds.

Lesson 87 : Characteristic Features of ABM

[click here to download the file](#)

Characteristic Features of ABM

3. Complex patterns can “self-organize” without any leader orchestrating the behavior

Similarly, it is common for people when they see a flock of birds, to assume that there must be a leader who orchestrates the behavior — a leader bird who tells each follower bird what to do (Resnick & Wilensky, 1993 ; Resnick, 1994a; Wilensky & Resnick, 1999).¹ However, nature again surprises: a population of individuals, each following very simple rules, can “ self-organize, ” generating complex and beautiful patterns without any orchestrator or centralized controller — these patterns are called “emergent” (Wilensky & Reisman, 2006).

4. Different models emphasize different aspects of the world

Even after we have completed a good working model of a particular phenomenon, we have not finished with the modeling process. Every model foregrounds certain aspects of

the world and backgrounds other aspects. There can be many models of the same phenomenon, and they may each have something interesting to say about the way the world works. For instance, a model of

residential location preferences may emphasize how likely a person is to move into a neighborhood based on how she likes the other people in the neighborhood. Such a model may have very interesting things to say about how urban populations develop, but it may say nothing about school districts, location of retail businesses, or development of parks, all of which are also affected by residential location preferences.

Lesson 88 : Fire Model

[click here to download the ppt](#)

The Fire Model

Critical Threshold or Tipping Point: Many complex systems tend to exhibit a phenomenon known as a “critical threshold” (Stauffer & Aharony, 1994) or a “tipping point” (Gladwell, 2000).

Tipping Point: Essentially, a tipping point occurs when a small change in one parameter results in a large change in an outcome.

Tipping Point Example: One model that clearly contains a tipping point is the early agent-based model of a *forest fire*.

Forest Fire Model: This model is easy to understand, yet exhibits some interesting behavior.

- Besides being interesting in its own right, the model of forest fire spread is highly relevant to other natural phenomena such as the spread of a disease, percolation of oil in rock, or diffusion of information within a population (Newman, Girvan & Farmer, 2002).
- This simple model is highly sensitive to one parameter. When observing the resultant outcome of whether or not a fire will burn from one side of a forest to another (percolate), the output is mainly dependent on the percentage of the ground that is covered by trees (see figure 3.1). As this parameter increases, there will be little to no effect on the system for a long time, but then all of a sudden, the fire will leap across the world. This is a “tipping point” in the system.



Figure 3.1
A forest fire consuming trees in a forest.

Benefits of Knowing that a System has a Tipping Point: Knowing that a system has tipping points can be useful for a variety of reasons.

1. First, if you know a system exhibits a tipping point, you know that continuing to put effort into the system, even if you are not seeing any results yet, may yet bear fruit.
2. Second, if you know where the tipping point is, and if you know how close you are to it, then you can determine whether or not it is worth putting additional effort into the system.
3. If you are far away from the tipping point, then it might not be worthwhile trying to change the state of the system, whereas if you are close to the tipping point it may take only a small amount of effort to make a big change in the state of the system.

Description of the Fire Model: The Fire model arose from a number of independent efforts to understand *percolation phenomena*.

Percolation: In percolation, a substance (such as oil) moves through another material (such as rock), which has some porosity.

Broadbent and Hammersley (1957) first posed this problem, and since then many mathematicians and physicists have worked on it. Influenced by cellular automata models, they introduced a percolation model using the example of a porous stone immersed in a bucket of water. The question they focused on was: What is the probability that the center of the stone becomes wet?

A fire moving through a forest can be thought of as a kind of percolation where the fire is like the oil and the forest is like the rock, with the empty places in the forest analogous to the porosity of the rock. A similar question to Broadbent and Hammersley's is: If you start with some burning trees on one edge of the forest, how likely is the fire to spread all

the way to the other side of the forest? Many scientists created and studied such fire models. In 1987, the Danish physicist and complex systems theorist Per Bak and his colleagues showed that the spread of the fire depended on a critical parameter, the density of the forest. Because this parameter arises naturally, the complexity of the fire can arise spontaneously and was therefore a possible mechanism to explain the world's naturally arising complexity. Bak and his colleagues called this phenomenon "self-organizing criticality" and demonstrated it in a number of contexts including, famously, the emergence of avalanches in sand piles.

Herein, we will explore a version of the Fire model adapted by Wilensky (1997a) that is developed in the NetLogo modeling language and is distributed in the Earth Science section of the NetLogo models library.

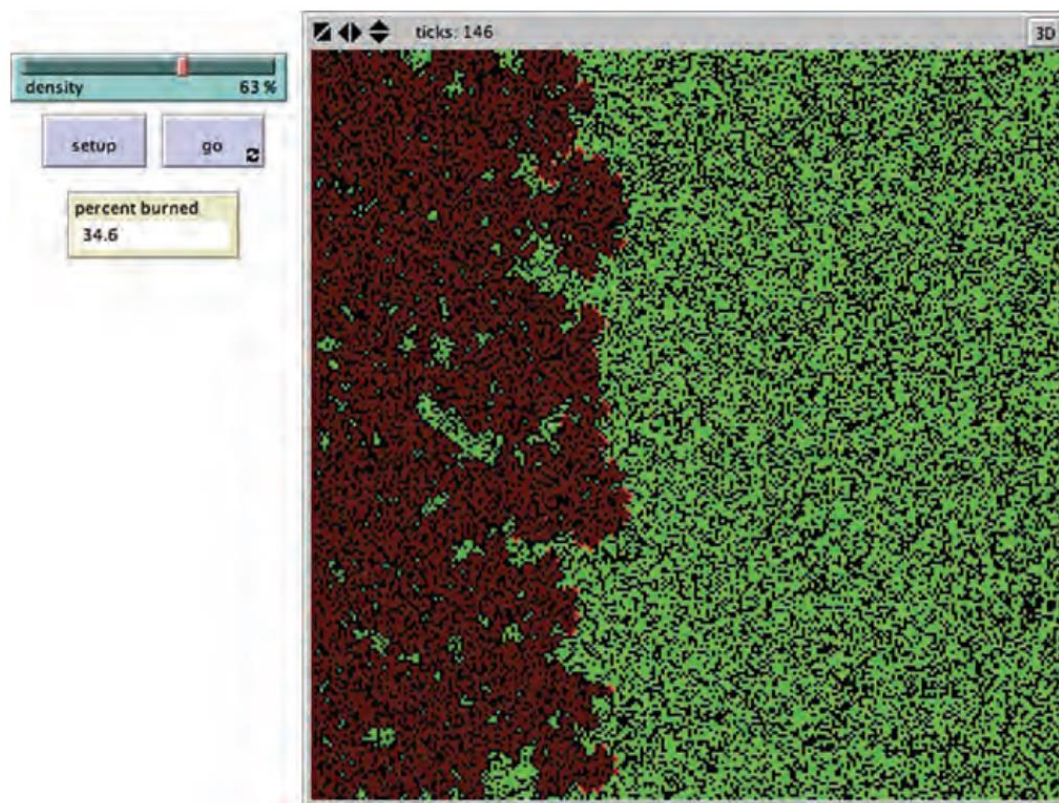


Figure 3.2

NetLogo Fire Simple Model. Based on NetLogo Fire model (Wilensky, 1997). <http://ccl.northwestern.edu/netlogo/models/Fire>.

This version of the Fire model contains only patches, no turtles. The patches can have four distinct states. They can be (1) green, indicating an unburned tree, (2) red, indicating a burning tree, (3) brown, indicating a burned tree, or (4) black, indicating empty space (illustrated in figure 3.2). When the model is first set up the left edge of the world is all red, indicating that it is “on fire.” When the model starts running, the fire will ignite any “neighboring” tree — that is, a tree to its right, left, below, or above it that is not already

burned and not already on fire. This will continue until the fire runs out of trees that it can

ignite. The only “control parameter” in this model is the density of trees in the world. This density parameter is not an exact measure of the number of trees in the world. Rather, it

is a probability that determines whether or not each patch in the world contains a tree.

Because the density is probabilistic and not deterministic, even if you run the model multiple times with the same density setting, you will get different results.

Lesson 89 : Fire Model: Basic Model [click here to download the ppt](#)

Understand the code of the Forest Fire Model

Let us examine the simple rules that govern the Fire model. The code that initializes the model is as follows:

```
to setup
  clear-all
  ;; make some green trees
  ask patches [
    if (random 100) < density
      [ set pcolor green ]
    ;; make a column of burning trees at the left-edge
    if pxcor = min-pxcor
      [ set pcolor red ]
  ]
  ;; keep track of how many trees there are
  set initial-trees count patches with [pcolor = green]
  reset-ticks
end
```

Just as in the Life Simple model we saw earlier, the Fire Simple model will use only stationary agents, patches, and no moving agents, turtles. Besides these basic types of agents, agent-based models can have many other types, including user created agent types.

Explanation of code of the Forest Fire Model

- The first line of the procedure is a command, **CLEAR-ALL** (or CA for short in the NetLogo language), which resets the world to its initial state — it resets the model's clock, kills all moving agents, and restores the default values to the stationary agents.
- The rest of this code issues commands to the patches. First, it populates the world with trees, and second, it makes a column of burning trees (by setting their color to red, the indication that they are on fire) at the left edge of the world. We have made many modeling choices here. Foremost among these are (1) modeling empty space in the forest as black patches, (2) modeling nonburning trees in the forest as green patches, and (3) modeling fire as burning trees, which are represented as red patches.
- Once these modeling choices are made, we can write the code to populate the model. To set up the trees, we have asked the patches to perform:

```
if (random 100) < density
  [ set pcolor green ]
```

To help us in understanding the setup code, suppose that the density is set to 50. This code tells each patch to roll a hundred-sided “die” (or, an alternative metaphor, spin a spinner with 100 equal sectors). Let us take the point of view of a patch. If I’ m a patch, I throw the die. If that die comes up less than 50, then “I become a tree,” otherwise “I don’t do anything.” So, half of the time I will become a tree (turn green) and the other half of the time I’ll remain black. Note that each patch throws its own independent die so theoretically all of them (or none of them) could become trees. But, on average, half will become trees. If the density were higher or lower, the same code would work to populate the model with roughly the appropriate tree density. This trick of taking the point of view of the agent, what we called *agent-centric thinking* earlier, is one we will employ frequently and is a good habit to form for facility with ABM.

Once the model is set up or initialized, we must define what the model does at each “tick” of the clock. This is typically done in a procedure named GO.

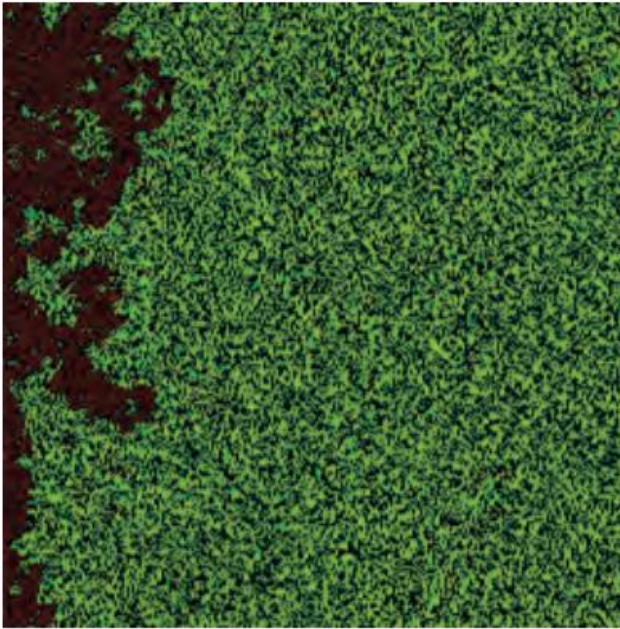
The code for the GO procedure is then as follows:

```
to go
  ;; stop the model when done
  if all? patches [pcolor != red]
    [ stop ]

  ;; ask the burning trees to set fire to any neighboring non-burning trees
  ask patches with [pcolor = red] [ ;; ask the burning trees
    ask neighbors4 with [pcolor = green] [ ;; ask their non-burning neighbor trees
      set pcolor red ] ;; to catch on fire
    set pcolor red - 3.5 ;; once the tree is burned, darken its color
  ]
  tick ;; advance the clock by one “tick”
end
```

Now run the Fire Simple model several times. If we run it with a low density of trees, we will see, as expected, very little spread of the fire. If we run it with a very high density of trees, we will see, as expected, the forest being decimated by the inexorable march of the fire. What should we expect at medium densities? Many people surmise that if the density is set to 50 percent, then the fire will have a 50 percent probability of reaching the right edge of the forest. If we try it, however, we see that at 50 percent density, the fire does not spread much. If we raise it to 57 percent, the fire burns more, but still doesn’t usually reach the other side of the forest (see figure 3.3).

A



B

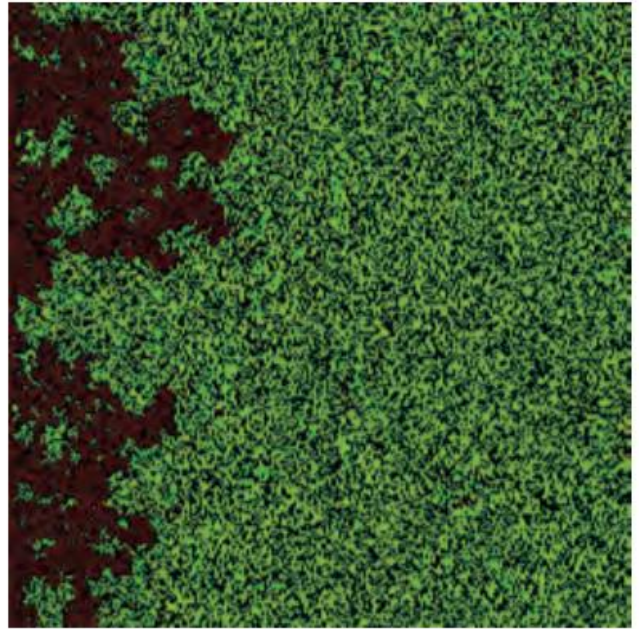


Figure 3.3

Two typical runs of the Fire Simple model with density set to 57 percent.

However, if we raise the density to 61 percent, just 2 percent more density, the fire inevitably reaches the other side (see figure 3.4).

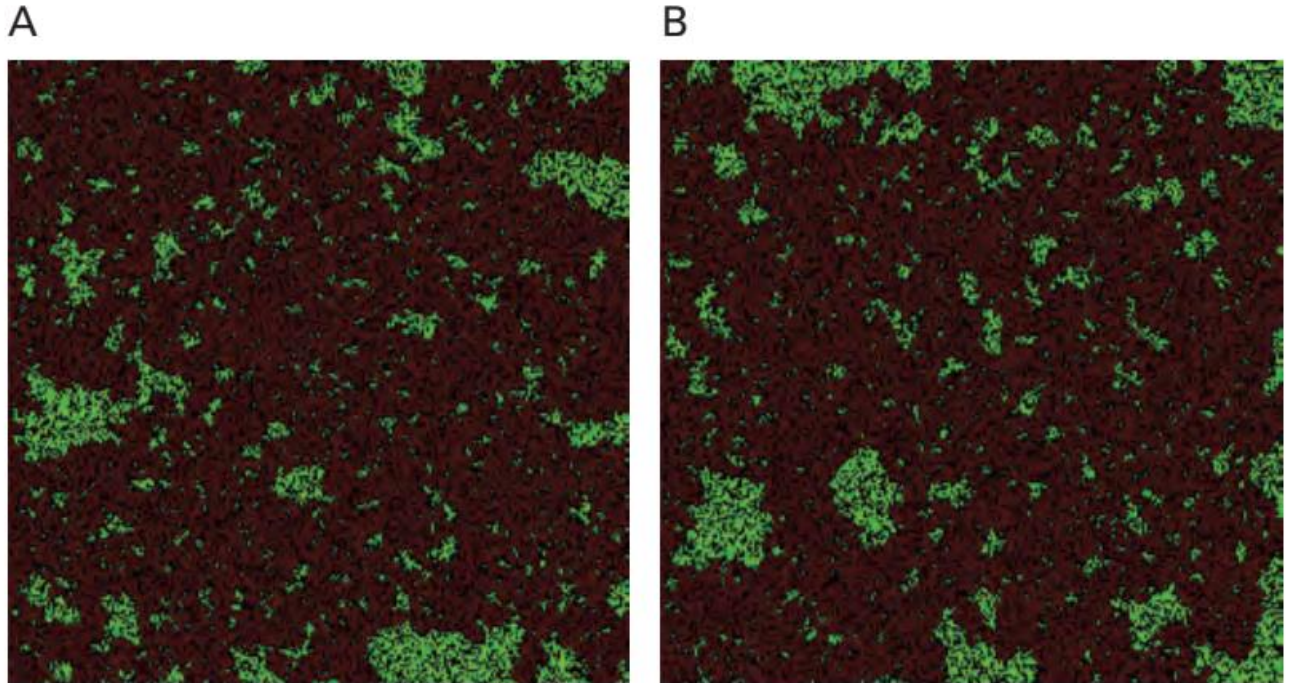


Figure 3.4

Two typical runs of the Fire Simple model with density set to 61 percent.

This is unexpected. We would expect a small change in density to have a relatively small effect on the spread of the fire. But, it turns out, the Fire model has a “critical parameter” of 59 percent density. Below 59 percent density the fire does not spread that much; above it, it spreads dramatically farther. This is an important and prevalent property of complex systems: They exhibit nonlinear behavior where a small change in input can lead to a very large change in output.

Lesson 90 : First Extension: Probabilistic Transitions

[click here to download the ppt](#)

First Extension: Probabilistic Transitions

The Fire Simple model, like any model, makes many simplifying assumptions about the

way the world works. In particular, fire in the real world does not spread deterministically from one tree to another tree, but rather it jumps to another tree based on a variety of factors such as wind, the type of wood, even how close the branches are to each other.

Often when there are many factors that affect a process like this one, we simplify the mechanism by using random numbers. We know that it is not certain that the fire will spread from one tree to another, but we do not know exactly what the probability of the spread is. We can model the mechanism by which fire spreads from one tree to another via a probability, and we can make this probability a parameter of the system. By experimenting with different values of the probability we can explore what effect the probability has on the overall spread of the fire.

To do this we first create a slider in the NetLogo model called PROBABILITY-OF-SPREAD. We set up this slider to go from 0 to 100. This is the variable that will control

how frequently the fire spreads from one tree to another. Simply adding the slider is not

enough — we need to modify the code itself to take this parameter into account. To do that we must first decide how we want this parameter to affect the model. Every time the model “spreads” the fire, we want it to create a random number between 0 and 100, if that number is less than the PROBABILITY-OF-SPREAD then we spread the fire as before. If it is not, then the fire does not spread in that direction. Let’s examine the GO procedure in the original model. The fire spread occurs when all the fire agents (which are the red patches) ignite their neighboring tree patches by setting their color to red. This seems like the logical place to insert our new variable. The old code looks like this:

```
ask patches with [pcolor = red] [  
  ask neighbors4 with [pcolor = green] [  
    set pcolor red  
  ]  
  ...
```

To modify this model to take our new parameter into account, all we have to do is add one additional check:

```
ask patches with [pcolor = red] [  
  ask neighbors4 with [pcolor = green] [  
    ;; only burn if a random draw is greater than the probability of spread  
    if random 100 < probability-of-spread [ set pcolor red ]  
  ]  
  ...
```

As before the fire agents look to their four neighbors and pick out those that are unburned trees. But this time, instead of just burning those trees, each tree executes RANDOM 100 which will generate an integer between 0 and 99; if that number is less than the value of PROBABILITY-OF-SPREAD the model proceeds as before. If not, then the fire does not spread to that tree. It is important to note that even if one

of these probabilities “fails” it is still possible that the tree in question may be burned down by one of its other burning neighbors.

Try setting both the DENSITY and the PROBABILITY-OF-SPREAD parameters to a variety of values. You can reproduce the results of the original model by setting the PROBABILITY-OF-SPREAD slider to 100 percent. It is a good rule of thumb to extend your model so you can reproduce previous results. By doing so, you can check to see if your new model is consistent with your old model and verify that you did not introduce any coding errors into the new model. You will often want to be able to generate the old results in order to compare them to the new results — ensuring that the new model can replicate the old mechanisms means you do not have to keep multiple versions of the model around.

Change these parameters and let the model run a few times. What happens to the model?

If you set the tree densities to values that, in the original model, would have allowed the fire to burn through the forest, and set the PROBABILITY-OF-SPREAD at 50 percent, the fire rarely burns through the forest. In fact, with a 50 percent spread probability, it takes a very high density for the fire to burn completely through the forest. While experimenting, you might now notice that the density parameter has an upper bound of 99

percent. In the original model, this made sense as 100 percent density was uninteresting, but now, since fire spread between trees is probabilistic, 100 percent density is also interesting. Modify the upper bound of the slider, set the density to 100 percent and observe the fire spread. Our new parameter has dramatically altered the numerical results of the model. A screenshot of the Fire Simple model after completing this first extension is shown in figure 3.5 .

Lesson 91 : Second Extension: Adding Wind

[click here to download the PPTs](#)

Second Extension: Adding Wind

Sometimes we can take an ABM for which we used a probability to model a suite of mechanisms and refine it so that some of those mechanisms are modeled in a more physical way. Our first extension hides a host of possible mechanisms in the PROBABILITYOF-SPREAD. We can pull out one of those mechanisms and model it in a more refined way. Wind is a good example of a process that we can model more specifically. We can think of the effect of wind on a fire as increasing the chance of fire spread in the direction it is blowing, decreasing the chance of fire spread in the direction it is not blowing, and having little effect on fire spread that occurs perpendicular to the direction of its movement. Of course, this too is an oversimplification — there are often local effects of wind, such as turbulence. As we have seen, all models are simplifications, and playing the modeling game means we will accept this one for now.

To implement wind in our model, we will create two sliders. One will control the speed of the wind from the south (a negative value will indicate a wind from the north) and one will control the speed of the wind from the west (a negative value will indicate a wind from the east). We create these two sliders and set them up to go from – 25 to 25. How do we use these new parameters in the code? If we think back to our first modification, we want these new parameters to affect the PROBABILITY-OF-SPREAD.

This effect will be based on the direction that the fire is attempting to spread. Since we have set the wind speed to vary from – 25 to 25, we could conceive of those numbers as percentages by which to modify the probability of spread which is expressed as a percent. In order to do this, let us first create a local variable called PROBABILITY that will initially be set to PROBABILITY-OF-SPREAD. A local variable is one that has a value only within a limited context, usually inside the procedure in which it is defined. If we only need to reference a variable in one procedure, then it is best to use a local variable. But, remember, you will not be able to see the value of the variable elsewhere in your program or in the command center. We define a local variable with the “let” primitive and can modify it with the “set” primitive. We can modify PROBABILITY to take into account WIND-SPEED by increasing or decreasing it by the WIND-SPEED in the direction the fire is burning. When we put this all together, we get the following code:

to go

```
if all? patches [pcolor = red]
  [ stop ]

;; each burning tree (red patch) checks its 4 neighbors.
;; If any are unburned trees (green patches), change their probability
;; of igniting based on the wind direction
ask patches with [pcolor = red] [
  ;; ask the unburned trees neighboring the burning tree
  ask neighbors4 with [pcolor = green] [
    let probability probability-of-spread      ;; define a local variable

    ;; compute the direction you (the green tree) are from the burning tree
    ;; (NOTE: "myself" is the burning tree (the red patch) that asked you
    ;; to execute commands)
    let direction towards myself
    ;; the burning tree is north of you
    ;; so the south wind impedes the fire spreading to you
    ;; so reduce the probability of spread
    if (direction = 0 ) [
      set probability probability - south-wind-speed ]

    ;; the burning tree is east of you
    ;; so the west wind impedes the fire spreading to you
    ;; so reduce the probability of spread
    if (direction = 90 ) [
      set probability probability - west-wind-speed ]

    ;; the burning tree is south of you
    ;; so the south wind aids the fire spreading to you
    ;; so increase the probability of spread
    if (direction = 180 ) [
      set probability probability + south-wind-speed ]

    ;; the burning tree is west of you
    ;; so the west wind aids the fire spreading to you
    ;; so increase the probability of spread
    if (direction = 270 ) [
      set probability probability + west-wind-speed ]
    if random 100 < probability [ set pcolor red ]
  ]
  set pcolor red - 3.5
]
tick
end
```

This code is a little tricky. Essentially, what it does is modify the probability of spread, increasing it in the direction of the wind and decreasing it in the opposite direction. It calculates the change in the probability by first determining which direction the fire is trying to spread and then determining which of the winds

will affect it. Once this probability is calculated, it is then used to determine whether or not the fire spreads to the neighboring tree.

This modification can lead to quite interesting patterns of spread. For example, set the density at 100 percent, and make the wind blow strong from the south and west. At the same time, set the PROBABILITY-OF-SPREAD fairly low, say around 38 percent. This

creates a triangular spread, and, all else being equal, the fire should spread to the northeast.

(See figure 3.6.)

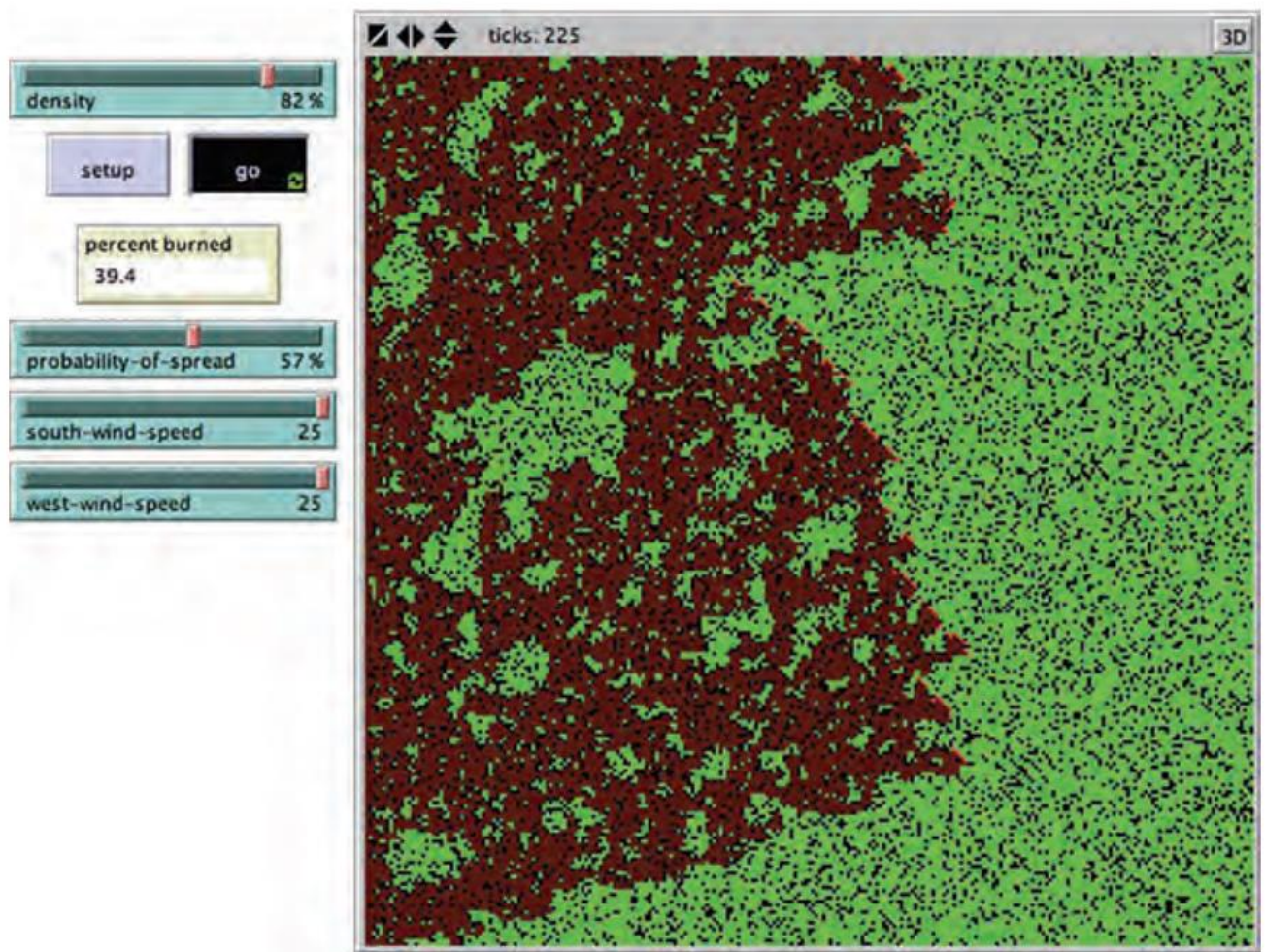


Figure 3.6
NetLogo Fire model after second extension.

Lesson 92 : Third Extension: Allow Long-Distance Transmission

[click here to download the ppt](#)

Third Extension: Allow Long-Distance Transmission

We have modeled the effect of wind as pushing the fire to spread in one direction. Another possible effect of wind is to enable the fire to jump over long distances and start fires where there are no surrounding burning trees. That might be an interesting process to model. In order to make sure the revised model can replicate our old results, we add a switch to control the jumping. We add a Boolean (TRUE or FALSE) switch labeled BIG-JUMPS? Switches in NetLogo provide a way to have direct control over variables that can only be true or false.

This particular switch allows us to turn on and off the jumping behavior. With the switch

off, the revised model should produce the same results as Fire Simple Extension 2. One way to model fire jumping due to wind would be to ignite a new fire at some distance in the direction of the wind. We can do this by modifying the code inside our earlier GO procedure as follows picking up after we do all of the changes in probability based on wind:

```
...
if random 100 < probability [
    set pcolor red

    ;; if big-jumps is on, then sparks can fly farther
    if big-jumps? [
        let target patch-at ( west-wind-speed / 5 ) ( south-
        wind-speed / 5 )
        if target != nobody and [pcolor] of target = green [
            ask target [
                set pcolor red ;; ignite the target patch
            ]
        ]
    ]
]
...
```

The first part of this code turns the current patch to red, which is the same as the previous version, but the second part after IF BIG-JUMPS? is what has changed. If BIG-JUMPS? is true then it looks for a target patch that is some distance away in the direction of the wind. If there is an unburned (green) tree at that location, then that tree is also set on fire (due to the spark having landed there). A more detailed model could include explicit spark agents, which travel according to the wind, and catch trees on fire when they land, but in this case, we just model the effects of flying sparks without modeling the sparks themselves.

Insert this code and see what effect it has on the model. You start to see lines in the direction of the wind as it jumps across gaps in the forest (as shown in figure 3.7). This extension can have visually dramatic

effects. Explore different sets of parameters and observe the different patterns of spread you can get. This modification increases the probability that the fire will reach the right edge of the world (our measure), but the resultant pattern is no longer the same; there are big chunks of the world that are not burned anymore. As new patterns emerge, we may need to reevaluate the questions that drove us to create the model, which may change the measures we collect about the model.

Each extension can lead to many more questions, which in turn call for new extensions.

In our third fire extension, it is of course also possible to have big jumps be probabilistic in the same way that the neighboring spreads are, or to have the sparks from the big jumps land in a random rather than a determined location.

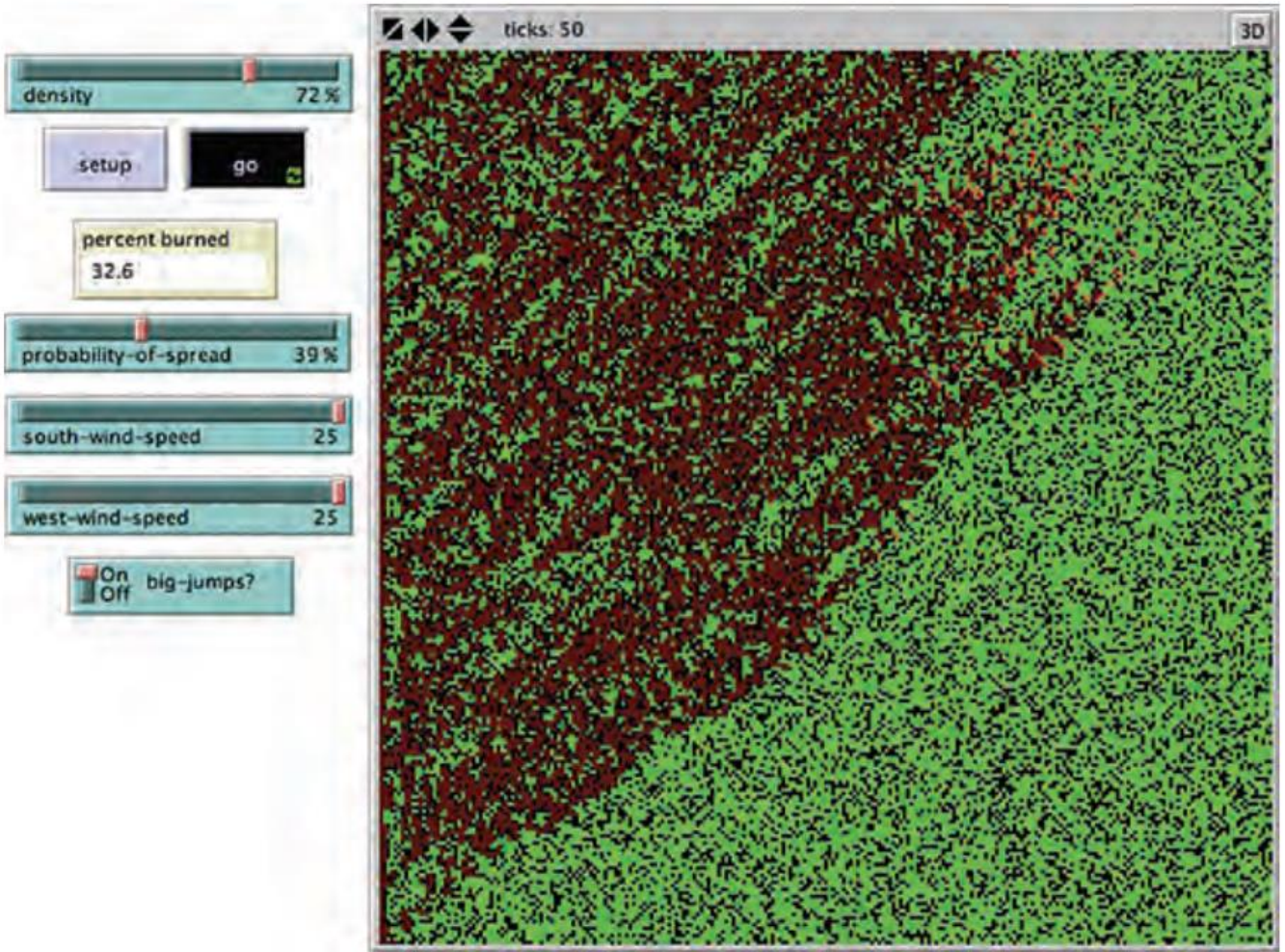


Figure 3.7
NetLogo Fire model after third extension.

Summary of the Fire Model

We have made three different changes to the model, which have affected our tipping point in three different ways.

1. The final change to the model points out that the measure we were observing (whether or not the fire made it to the right edge of the world) might not even be the measure we want to observe.
2. As we change the fire spread mechanisms, other measures such as “percent burned” may become more important. These new measures may or may not have tipping points. This illustrates an important feature of tipping points.
3. They are defined by an input and an output measure. A model does not have a tipping point in and of itself — the tipping point is relative to the choice of inputs and output measures.

Lesson 93 : The Diffusion-Limited Aggregation (DLA) Model

[click here to download the ppt](#)

The Diffusion-Limited Aggregation (DLA) Model

As we discussed earlier, the ABM perspective enables a new and different understanding of complex systems. As demonstrated with the Fire Simple model, it is possible to think about inanimate objects (such as trees) as agents. By reconceptualizing atoms and molecules as agents, and then developing procedural rules to describe how these agents interact we can gain a deeper understanding of many different types of physical phenomena.

For instance, the formation of complex beautiful patterns in nature has mystified and amazed humans for many years (see figure 3.9), yet many of these patterns can be generated using simple rules. Often in ABMs we will see that the complexity of a resulting pattern bears little direct relationship to the complexity of the underlying rules.



Figure 3.9

A DLA copper aggregate formed from a copper sulfate solution in an electrode position cell (Kevin R. Johnson, 2006, http://commons.wikimedia.org/wiki/File:DLA_Cluster.JPG).

In fact, in some cases the results will seem to be exactly the opposite; simpler rules will often create more complex patterns. The focus of our interest should not necessarily be on the complexity of the rules, but instead on the interaction those rules produce. Many agent-based models are interesting not because of what each agent does, but because of what the agents do together.

In this section, we will look at a very simple model where all that the agents do is move randomly around the world, and eventually stop moving when a basic condition is met; despite the simplicity of the rules, interesting complex phenomena can emerge.

Description of Diffusion-Limited Aggregation

In many physical processes from the creation of clouds, to snowflakes, to soot, smoke, and dust, particles aggregate in interesting ways.

- Diffusion-limited aggregation (DLA) is an idealization of this process, and was first examined as a computational model in the early eighties (Witten & Sander, 1981, 1983).
- DLA models were able to generate patterns that resemble many found in nature, such as crystals, coral, fungi, lightning, and growth of human lungs, as well as social patterns such as growth of cities (Garcia-Ruiz et al., 1993; Bentley & Humphreys, 1962; Batty & Longley, 1994).

Netlogo Model of DLA

A NetLogo version of diffusion-limited aggregation was one of the first models written for the NetLogo models library (Wilensky, 1997b). This model starts with a large number of red particle agents and one green patch in the center. The green patch at the center of the screen is stationary, but when you press GO, all of the red particles move randomly by “wiggling” left and right (resulting in a small random turn) and then moving forward one step. After a particle moves, if any of its neighbors (i.e., nearby patches) are green, the particle dies, turning the patch it is on green (the particle can do so because all turtles have direct access to the variables of the patch they are on). If you let the model run long enough, you will get an interesting fractal-like pattern forming from the green patches (see figure 3.10).

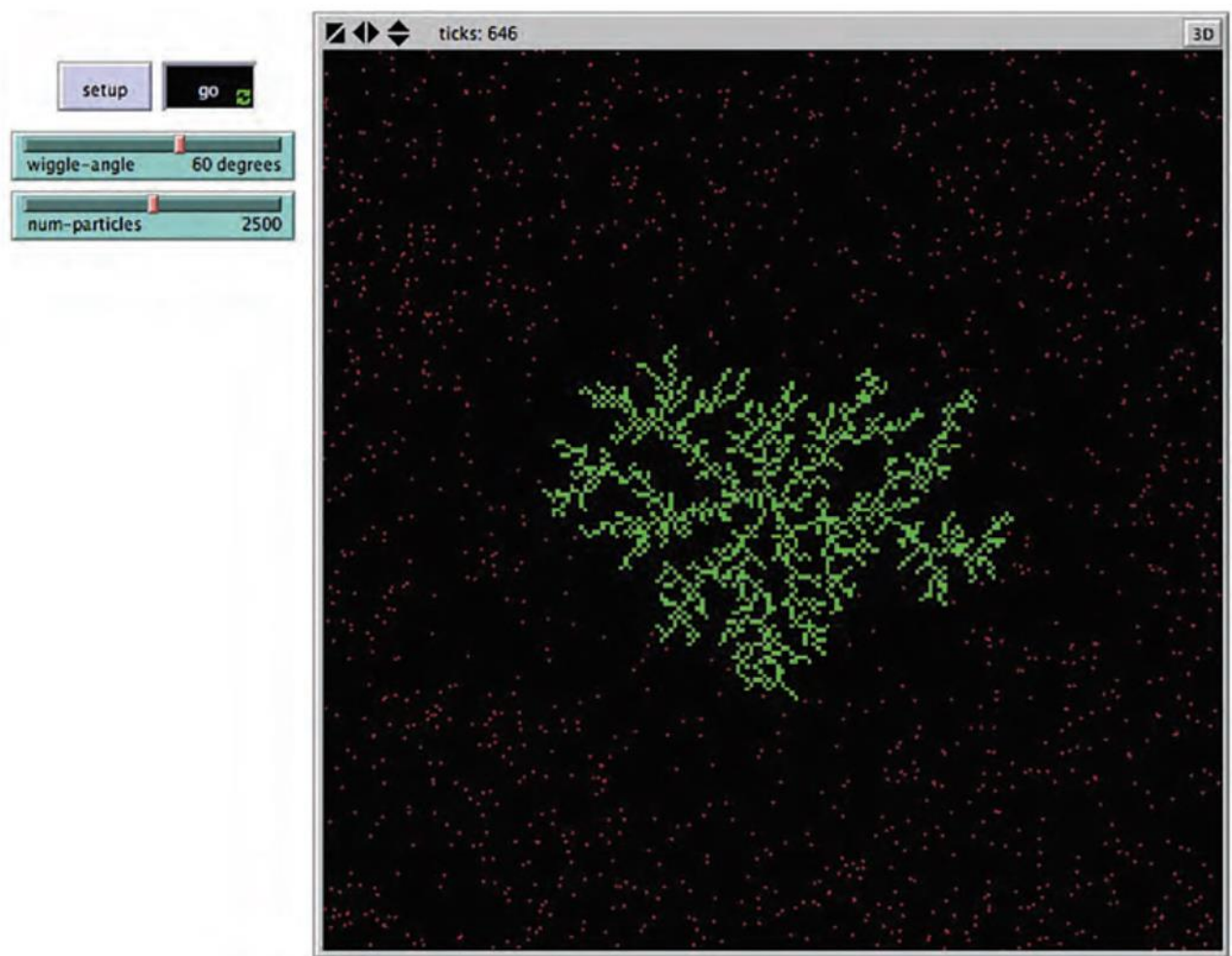


Figure 3.10

NetLogo model of diffusion limited aggregation. <http://ccl.northwestern.edu/netlogo/models/DLA> (based on Wilensky, 1997b).

Here is the NetLogo code for the GO procedure for DLA Simple. WIGGLE-ANGLE is a global variable, the value of which is set by a slider in the interface.

```

to go
  ask turtles
    ;; turn a random amount right and left
    [ right random wiggle-angle
      left random wiggle-angle
      forward 1
      ;; if you are touching a green patch, turn your own patch green and
      ;; then die
      if ( any? neighbors with [pcolor = green] )
        [ set pcolor green
          die ] ]
  tick
end

```

As the code shows, if any of the neighbors are green the particle stops moving, changes the color of the patch that it is on to green and dies. Turtles have direct access to the variables of the patch they are on — that is how the turtle can set the color of the patch it

is on (the PCOLOR in NetLogo).

Lesson 94 : First Extension: Probabilistic Sticking

[click here to download the ppt](#)

We will now create three extensions of the DLA Simple model.

First Extension: Probabilistic Sticking

The DLA Simple model has very simple rules. The code consists of only two procedures, and both of them are very small, yet these two procedures can generate many interesting results. In these extensions we will examine how adding a few more simple rules can enable us to generate more interesting patterns.

After you run the model for a while, you will realize that it always produces thin and wispy types of structures. Often the “stems” and “trunks” of the structures are only a single patch wide. This is because as soon as a particle touches anything green it will stop moving.

It is much more likely that it will touch something toward the edge of the structure rather than near the interior of the structure.

However, we can change this. One way to think of this system is that if a particle comes into contact with a stationary object, then it becomes stationary itself with 100 percent probability. What if we decrease this probability? That is exactly what we will do in this extension. We will allow the user to control the probability of a particle becoming stationary.

In the original code, if any of the neighbors are green, the particle stops moving, changes the color of the patch that it is on to green, and dies. What we need to do is add another test to this rule, but we also have to make one other change to the code at the same time.

Since stopping will be probabilistic, it is now possible that a particle could be on top of another green particle, and, in that case, we do not want the particle to stop. So, we also need to make sure we are not on a green particle before we stop, which involves adding an additional condition to the model:

```
to go
  ask turtles
    [ right random wiggle-angle
      left random wiggle-angle
      forward 1
      if (pcolor = black) and ( any? neighbors with [pcolor = green] )
      and ( random-float 1.0 < probability-of-sticking )
        [ set pcolor green
          die ] ]
  tick
end
```

Now, a particle will only stick if a random number it generates is less than a given probability, but where is this probability defined? If you add this code into the GO procedure and then move to the Interface tab, NetLogo will generate an error because we have not defined this parameter yet. So we will do that now. We go back to the Interface tab and create a slider called PROBABILITY-OF-STICKING, and we give it a minimum value of 0.0, a maximum value of 1.0 and an increment of 0.01.³ Now we are finished with this extension, and we can run the model with a PROBABILITY-OF-STICKING of 0.5, as seen in figure 3.11.

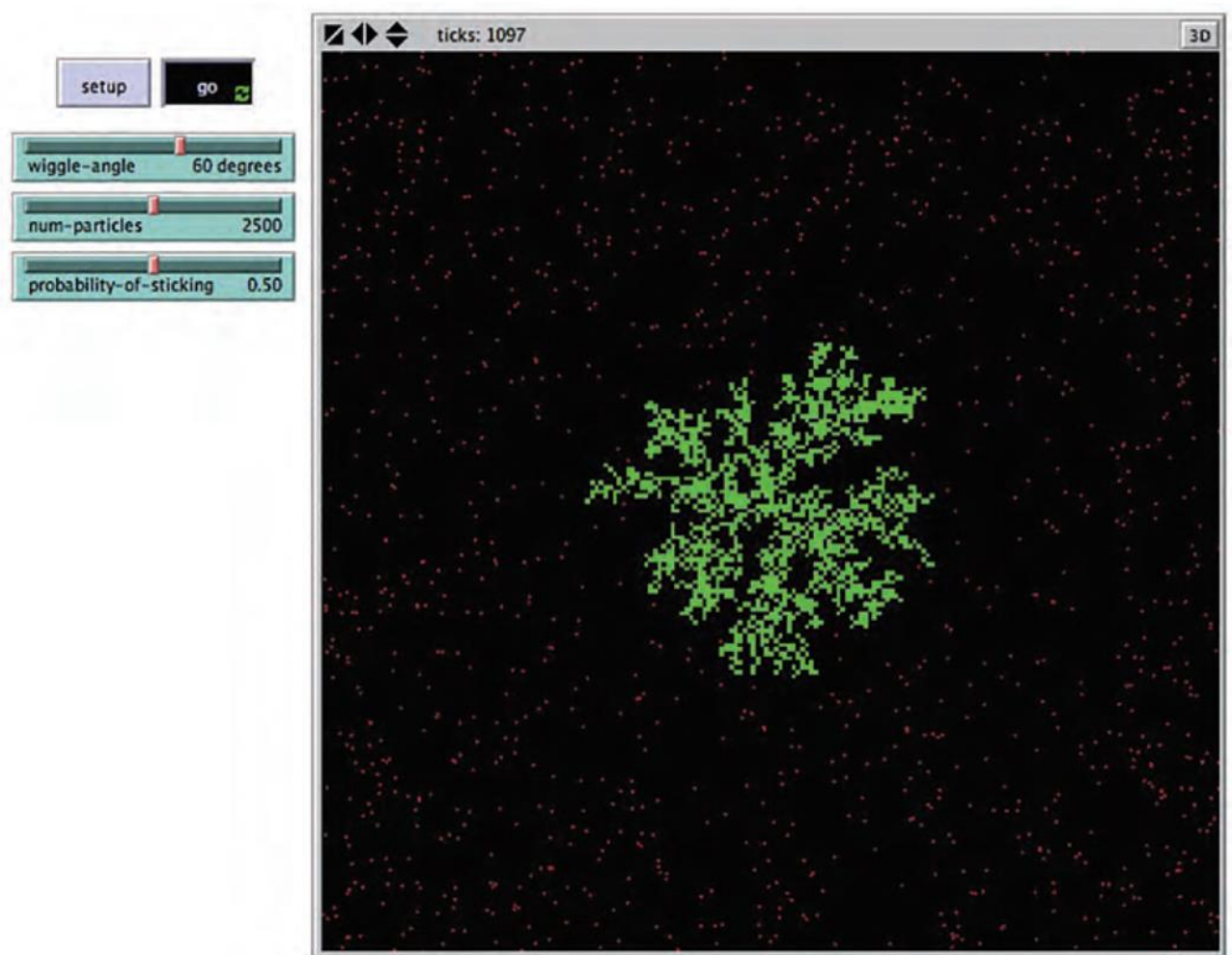


Figure 3.11
DLA model after first extension.

You will notice that the branches of the structure become thicker when the probability is 0.5. This is because particles have a smaller probability of getting stuck on the outside

of the structure and can wander deeper in to the structure before stopping. If you want you can still generate the original results by setting the probability to 1.0. As with the extensions to the Fire Simple model, it is often helpful to make sure that when extending

a model you can still generate the previous results. Not only does this allow you to inspect whether any errors were introduced by your code change, but also, as you continue to add new parameters and mechanisms, you will often want to go back to your original results for comparison.

Lesson 95 : Second Extension: Neighbor Influence

[click here to download the ppt](#)

Second Extension: Neighbor Influence

The addition of our probability parameter enables us to explore a whole new range of structures, while not interfering with our ability to add additional simple rules to this model. One commonly explored extension to the DLA model is to explore how the probability of sticking is related to the number of neighbors that are already stationary (the green patches) (Witten & Sander, 1983). If a particle is moving, it is more likely that it will stick somewhere if two or three of its neighbors are at rest than if just one of its neighbors is at rest. Thus, we want the probability of stopping to increase as the number of stopped neighbors increases.

Let us begin this extension by adding a switch to our interface, which we will call NEIGHBOR-INFLUENCE?. This switch will determine whether or not we are taking the number of neighbors in to account when determining if a particle should stop moving.

NEIGHBOR-INFLUENCE? is a Boolean variable, that is, it has one of two values, TRUE or FALSE. It is conventional to append a question mark character to the end of the variable name to point out to the reader that the variable is Boolean. After we add this switch, we can go back and look at our GO procedure. We left the GO procedure looking like this:

```
to go
  ask turtles
    [ right random wiggle-angle
      left random wiggle-angle
      forward 1
      if ( pcolor = black ) and ( any? neighbors with [pcolor = green] ) and
        ( random-float 1.0 < probability-of-sticking )
        [ set pcolor green
          die ] ]
  tick
end
```

We need to modify the PROBABILITY-OF-STICKING based on the NEIGHBORINFLUENCE? switch. To accomplish this, we must first create a variable, which we will call LOCAL-PROB If NEIGHBOR-INFLUENCE? is turned off then LOCAL-PROB will be the same as PROBABILITY-OF-STICKING. However, if NEIGHBOR-INFLUENCE? Is turned on, then we will reduce the probability of sticking for small numbers of green neighbors. We accomplish this by multiplying the PROBABILITY-OF-STICKING by the fraction of its eight neighbors that are green. For example, if PROBABILITY-OF-STICKING is set to 0.5 and if a particle encounters a neighborhood with four green neighbors, then we multiply PROBABILITY-OF-STICKING by 4/8 to get a sticking probability for that encounter of 0.25. We do this in order to create a relationship such that if many neighbors are green the probability approaches the PROBABILITY-OF-STICKING and if few neighbors are green the probability approaches zero.⁴ As a further exploration, try some other functions and see how it affects this result; to facilitate this, you may want to make the number of neighbors that affect a particle a parameter. The new code looks like this:

```

    to go
      ask turtles
    [ right random wiggle-angle
      left random wiggle-angle
      forward 1
      let local-prob probability-of-sticking

      ;; if neighbor-influence is TRUE then make the probability proportionate
      ;; to the number of green neighbors, otherwise use the slider as before
      if neighbor-influence? [
        ;; increase the probability of sticking the more green neighbors there are
        set local-prob probability-of-sticking *
        (count neighbors with [pcolor = green] / 8)
      ]

      if (pcolor = black) and ( any? neighbors with [pcolor = green] )
      and ( random-float 1.0 < local-prob )
      [ set pcolor green
        die ] ]
    tick
  end

```

When we run the model with this change in the code, we notice

- (1) that it takes much longer to form a structure because there is a much lower probability of a mobile particle stopping, and
- (2) the structures that emerge are very thick and almost bloblike. In addition, there are many fewer one patch-wide branches, because the probability of stopping when in contact with only one other stationary patch is very small. You can see the results in figure 3.12 .

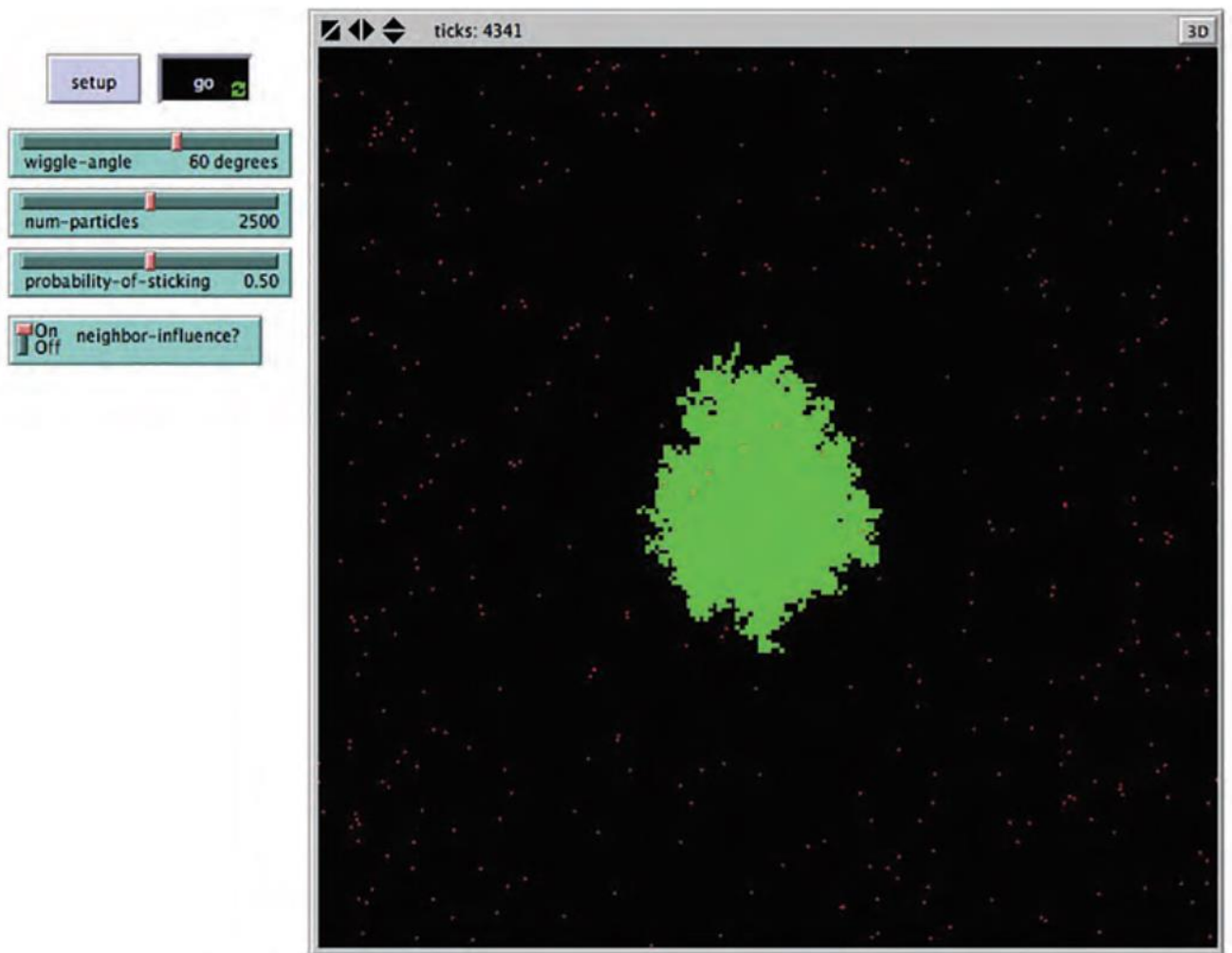


Figure 3.12
DLA model after second extension.

Lesson 96 : Third Extension: Different Aggregates

[click here to download the ppt](#)

Third Extension: Different Aggregates

The previous two extensions have concentrated on deciding how a particle should stop when it is moving — there is another way that we can control how a particle stops. We can simply give it more places to stop at the beginning. If we look at the SETUP procedure in the previous version of the model, we can see that it creates one green patch in the middle of the world:

```

to setup
  clear-all
  ;; ask the middle patch to turn green
  ask patch 0 0
    [ set pcolor green ]
  create-turtles num-particles
    [ set color red
      set size 1.5    ;; make the particle bigger so it's easier to see
      setxy random-xcor random-ycor ]
end

```

Patch 0 0 is the unique name for the patch that is at x-coordinate 0, and y-coordinate 0, which is at the center of the NetLogo world by default. However, what if we want to create multiple green patches at the start? If there are multiple green patches, then there will be more places for the moving particles to come to rest, and we can generate different patterns of aggregation. To begin with, we need to create a slider so we can control the number of different aggregates that we create; we will call this slider NUM-SEEDS. We give this slider a minimum of 1 (since we need at least one seed), a maximum of 10, and an increment of 1.

Once we have this slider, we can ask NUM-SEEDS patches to turn green in the setup:

```

to setup
  clear-all
  ;; start with NUM-SEEDS green patches as "seeds"
  ask n-of num-seeds patches
    [ set pcolor green ]
  create-turtles num-particles
    [ set color red
      set size 1.5
      setxy random-xcor random-ycor ]
  reset-ticks
end

```

The N-OF reporter selects a random set of NUM-SEEDS patches. We then ask these

randomly chosen patches to turn green, making them seeds for the aggregates. This is different from what we did in the Fire model; in that model, we asked each of the patches to determine if they should become a tree (a seed) using a random variable. The Fire model method will generate roughly the fraction of trees specified by the slider; the N-OF method will always generate exactly the number of seeds specified by the slider. In agent-based modeling, the probabilistic method used in the Fire model is often considered

more realistic since nature does not specify exact numbers. However, the N-OF method gives us more precise control over the model's behavior.

This code works, but note that it no longer re-creates the exact results of the original model. Even if we set NUM-SEEDS to 1, we will no longer always create a seed at the center of the world. Instead, the seed will be randomly placed somewhere in the world. In this case it is simpler to deviate from the original version and not much is lost by doing so. There is really nothing special about the center of the world, and having the seed start anywhere randomly will be sufficiently similar to the original result for testing and exploration purposes. If we really desire to have the seed start at

the center of the world, we could add another parameter; this is left as an exercise for

the reader. The final version of the simple DLA model, after all three extensions, is shown in figure 3.13. With multiple seeds, the patterns look somewhat like frost forming on a cold window.

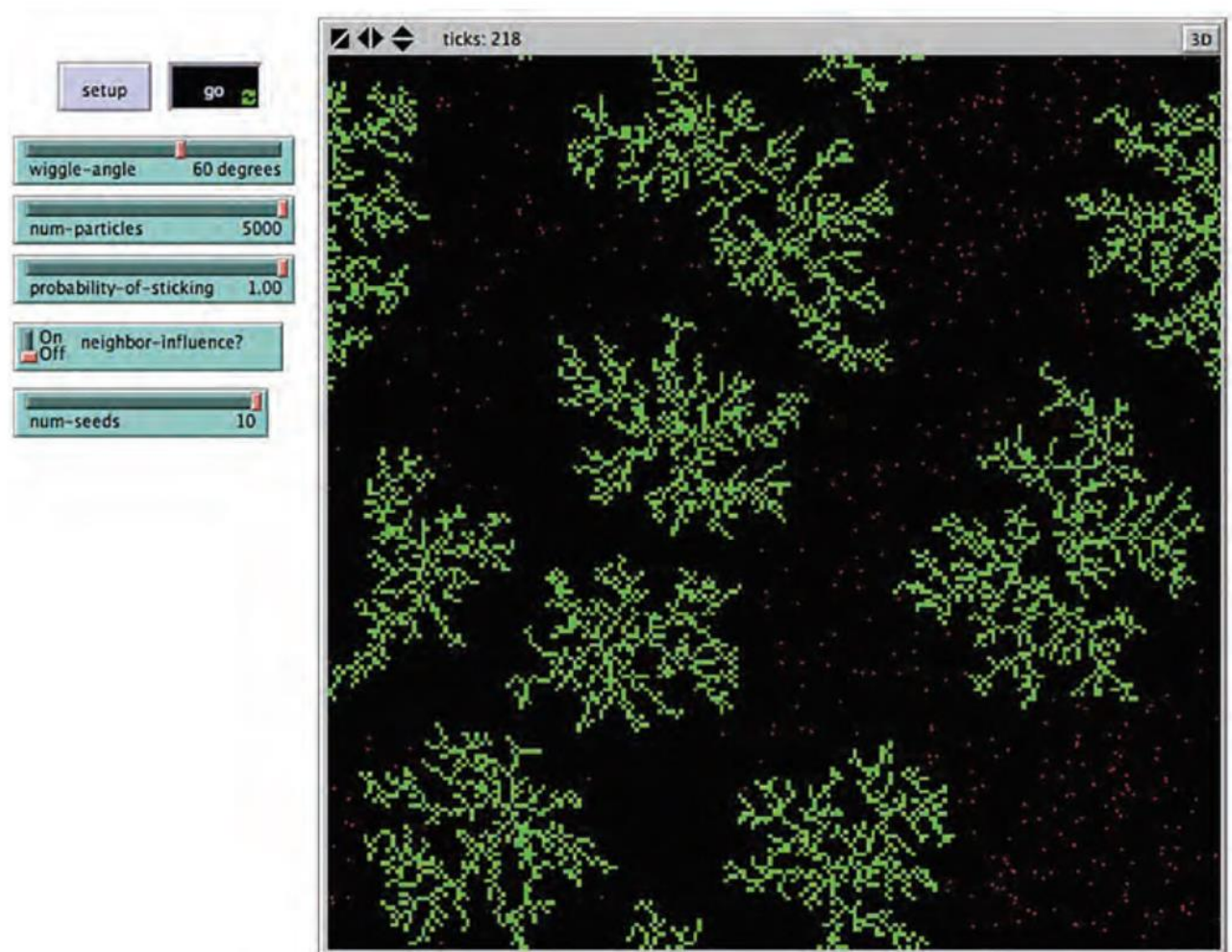


Figure 3.13
DLA model after third extension.

Lesson 97 : Segregation Model

[click here to download the ppt](#)

The Segregation Model

Fundamental Approaches to ABM

There are two fundamental approaches to starting to build an agent-based model.

- The first approach, more common in science contexts, is to start with a known phenomenon to be modeled. This approach is called **phenomena-based modeling**. Usually when engaged in phenomena-based modeling, we have an aggregate pattern, termed a *reference pattern* that we are trying to generate with agent rules.
- Another way to begin is to start with some simple rules and play them out to see what patterns develop. This approach is sometimes referred to as **exploratory modeling**.

Frequently in pursuing ABM, we take some combination of these two approaches.

Schelling Model

In his investigations into the nature of segregation, Thomas Schelling took a more exploratory modeling approach with his Neighborhood Tipping model (1971).

Schelling wondered what would happen if you assume that everyone in the world wanted to live in a place where at least a reasonable fraction of their neighbors were like themselves, i.e., they have an aversion to being an extreme minority (which he called weakly prejudiced; see Anas, 2002). He explored this model using a checkerboard as a grid, with pennies to represent one race and dimes to represent another. He counted by hand the number of pennies surrounding each dime and divide by the number of neighbors surrounding that dime. If this value exceeded a certain threshold value, he would move the dime to a random empty location on the board. He repeated this process hundreds of times and observed the outcome. (See figure 3.15.)

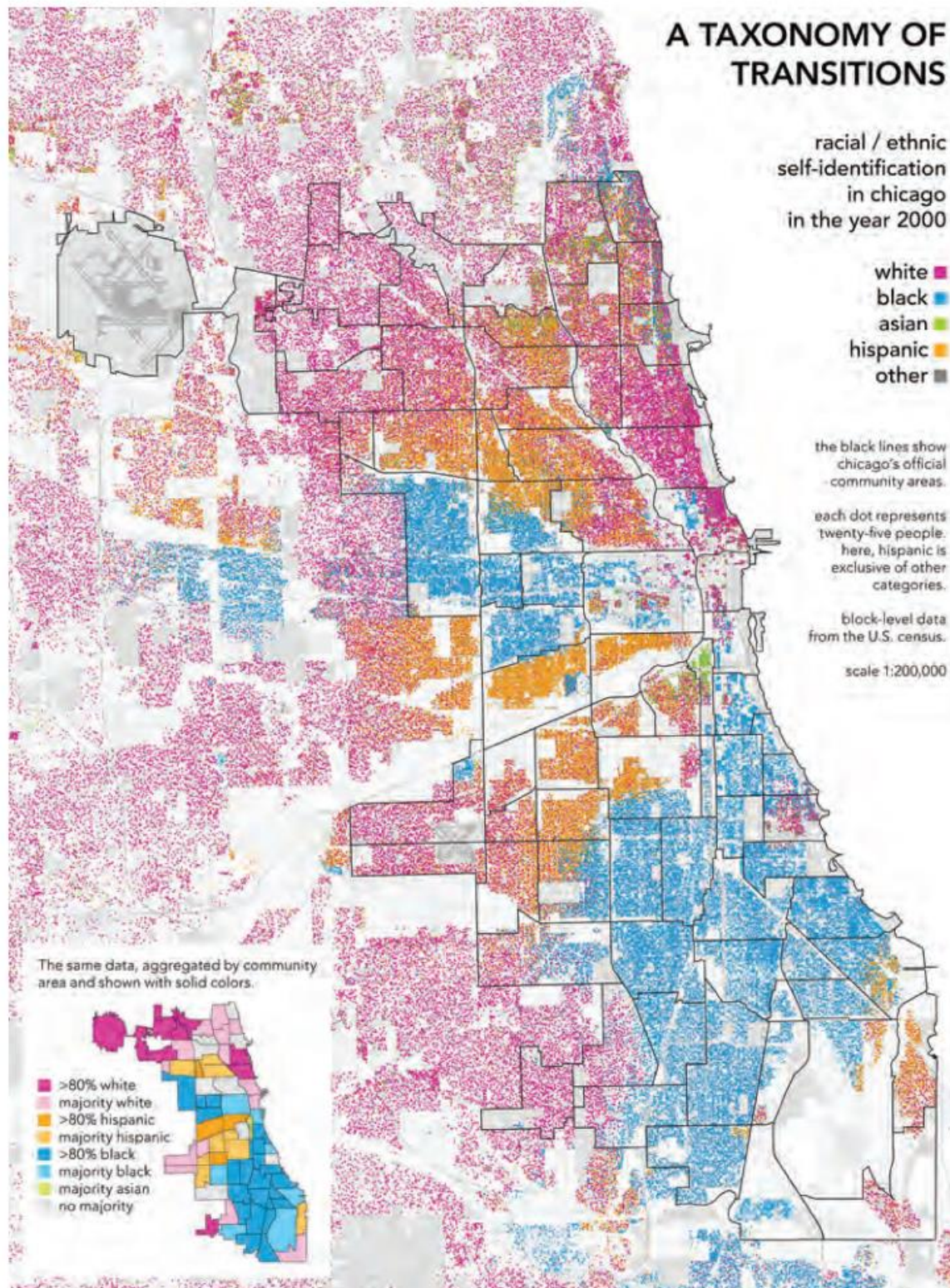


Figure 3.15

A map of Chicago with each dot representing twenty-five people. By representing the data at this finer level than traditional maps that aggregate demographics are mapped at a high level, Rankin illustrates nuances of segregation much like ABM enables us to explore nuances of individual-level behavior.

Lesson 98 : Description of the Segregation Model-I

[click here to download the ppt](#)

Schelling Model

For high levels of the threshold, the model confirmed what he predicted. The checkerboard would quickly segregate into areas of all pennies and all dimes. What surprised Schelling about this model (and many other people at the time) was that even at low levels of the threshold, he would still see groups of pennies and groups of dimes together in dense clusters.

This global segregation occurred despite the relative tolerance of each individual. There was no single individual in the Schelling model who wanted a segregated neighborhood, but the group as a whole moved toward segregation. Schelling described this as “macrobehavior” derived from “micromotives” (1978).

At the time when Schelling introduced this model, it was very controversial for several reasons. First, it was widely believed at the time that housing segregation was caused by individuals being prejudiced. Schelling’s work seemed to be “excusing” people from

prejudice, saying that prejudice itself is not the cause of segregation. The cause of segregation is an emergent effect of the aggregation of people’s weak prejudice. But Schelling’s point was not to excuse people for their prejudice, his model demonstrates that prejudice is not a “leverage point” of the housing system. Unless you can reduce prejudice to close to zero, so that everyone is perfectly comfortable being the sole member of their race in their neighborhood, this segregation dynamic will emerge. So, if your goal is to reduce housing segregation, it may not be effective to work directly on reducing individual’s prejudice.

Another controversial aspect of Schelling’s model is that it modeled the people as behaving with very simple rules. Critics argued that “people are not ants” ; they have complex cognition and social arrangements, and you cannot model them with such simple rules.

To be sure, Schelling’s model was highly oversimplified model of people’s housing choices. Nevertheless, it did reveal a hitherto unknown important dynamic. Eventually,

Schelling carried the day. For his large body of work on exploring the relations of micromotives and macrobehavior and his work on conducting game-theory analyses of

conflict and cooperation, Schelling was awarded the Nobel Prize in economics in 2005.

To this day, it remains controversial how much of human behavior can be modeled with simple rules.

Lesson 99 : Description of the Segregation Model-II

[click here to download the ppt](#)

SEGREGATION MODEL

A NetLogo version of the segregation model is distributed in the IABM Textbook section of the models library. This model is shown in figure 3.16 .

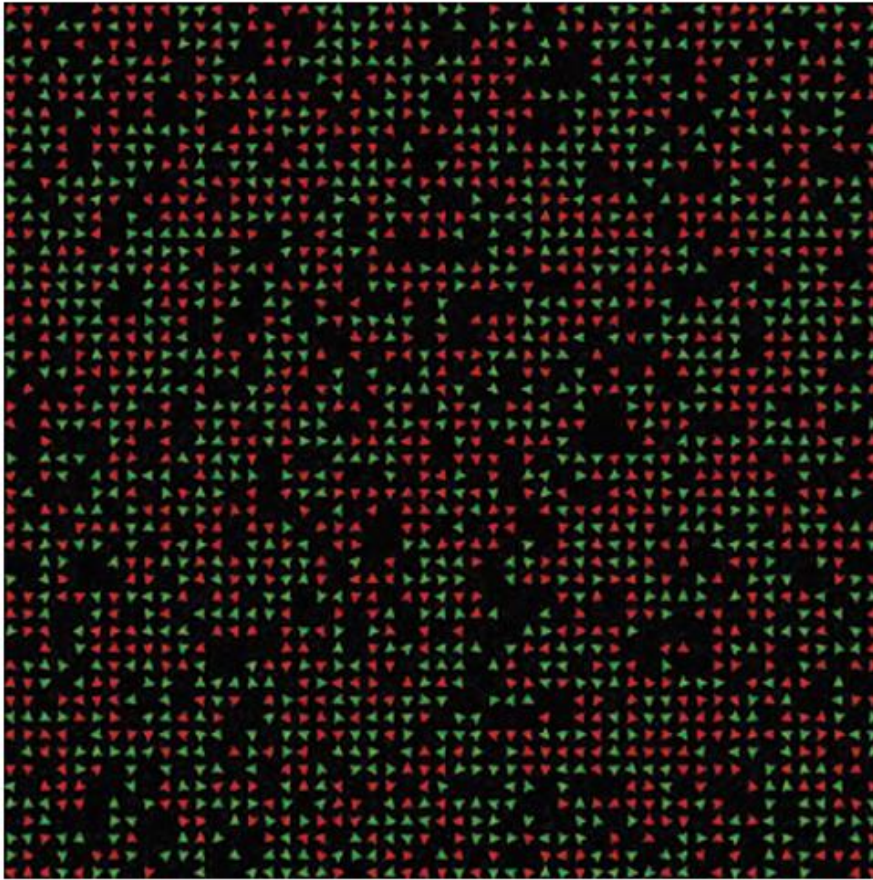


Figure 3.16

Initial state of the NetLogo Segregation model. Red and green turtles are distributed at random. <http://ccl.northwestern.edu/netlogo/models/Segregation> (Wilensky, 1997d).

When you press the SETUP button, an approximately equal number of red and green turtles will appear at random locations in the world.

Each turtle will determine if it is happy or not, based on whether the percentage of neighbors that are the same color as itself meets the %-SIMILAR-WANTED threshold.

When you press the GO button, the model will check if there are any unhappy turtles.

Each unhappy turtle will move to a new location. It moves by turning a random amount and moving forward a random amount from 0 to 10. If the new location is not occupied then the turtle settles down, if it is occupied, it moves again. After all the unhappy turtles have moved, every turtle again determines whether or not it is happy and the process repeats itself. (See figure 3.17.)

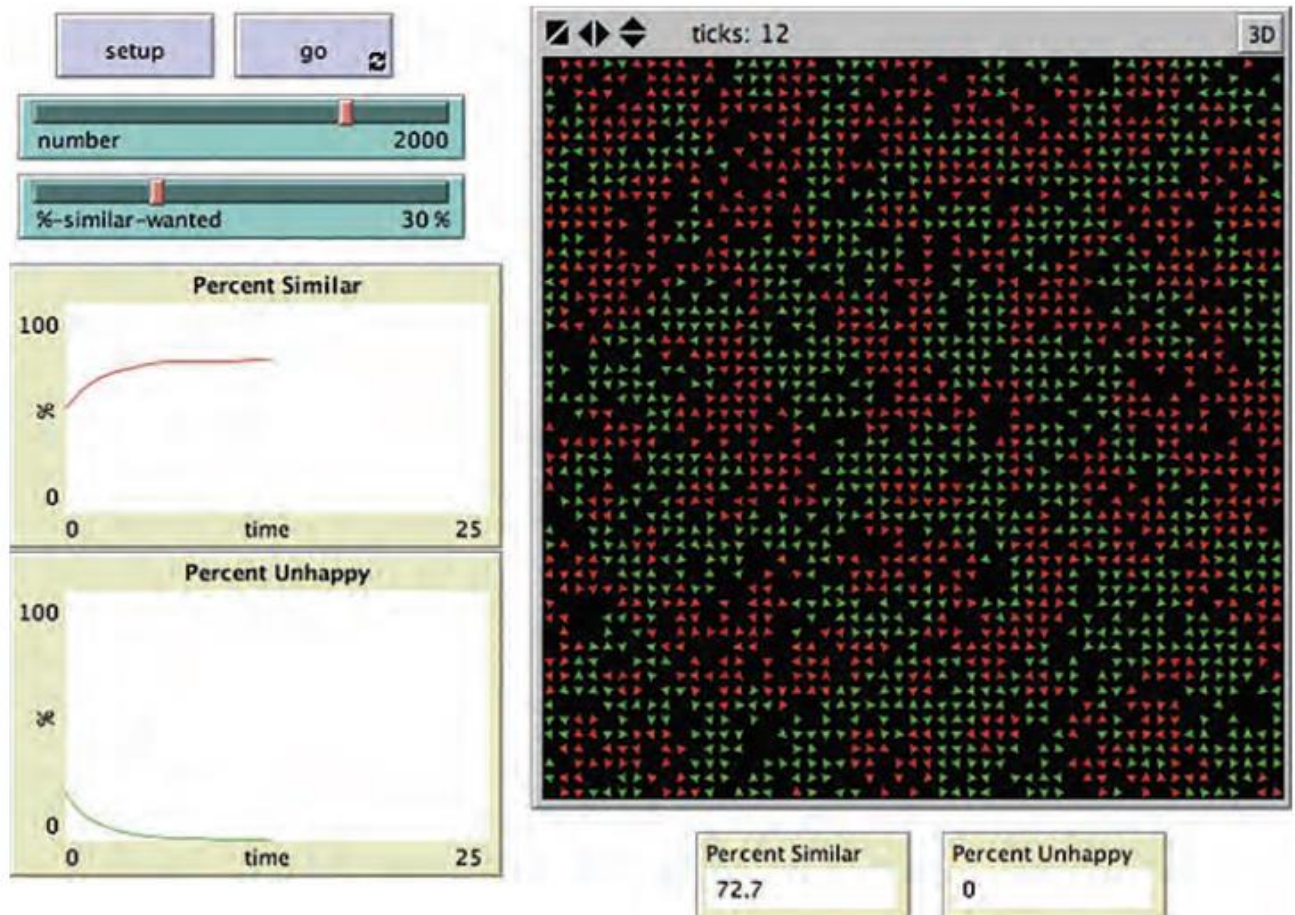


Figure 3.17

A run of the Segregation model with a “tolerance” level of 30 percent. Even though most agents are comfortable in an integrated neighborhood, the housing gets very segregated with 72.7 percent of agents surrounded by their same color agents.

The SETUP procedure for this model is as follows:

```

to setup
  clear-all
  ;; create a turtle on NUMBER randomly selected patches.
  ask n-of number patches
    [ sprout 1 ]

  ask turtles [
    ;; make approximately half the turtles red and the other half green
    set color one-of [red green]
  ]

  update-variables ;; update the turtles and the global variables
  reset-ticks
end

```

The code asks a random set of patches to each sprout a red turtle. The size of the patch set is determined by the NUMBER slider, so after sprouting, the number of turtles is exactly the value on the NUMBER slider, each turtle on its own patch. The code then asks the turtles to turn red or green with equal probability, resulting in an approximately equal number of red and green turtles.

Lesson 100 : Simulation of the Segregation Model

[Click here to download .pptx](#)

SEGREGATION MODEL CONTINUED

The GO procedure is:

```

to go
  if all? turtles [happy?] [ stop ]
  move-unhappy-turtles
  update-variables
tick
end

```

This code stops the simulation when all the turtles are happy. If there are any unhappy turtles, it asks them to move, and then all turtles recalculate their happiness.

The happiness calculation takes place in the update-turtles procedure, which follows:

```

to update-turtles
  ask turtles [
    ;; in next two lines, we use "neighbors" to test the eight patches
    ;; surrounding the current patch

    ;; count the number of my neighbors that are the same color as me
    set similar-nearby count (turtles-on neighbors)
      with [color = [color] of myself]

    ;; count the total number of neighbors
    set total-nearby count (turtles-on neighbors)

    ;; I'm happy if there are at least the minimal number of
    ;;   same-colored neighbors
    set happy? similar-nearby >= ( %-similar-wanted * total-nearby / 100 )
  ]
end

```

This code asks each turtle to count how many of its neighbors are same-colored turtles and how many are differently colored turtles. It then sets its “ happy? ” variable to TRUE

if the percentage of same-colored neighboring turtles is at least equal to the value set by the %-SIMILAR-WANTED slider.

The PERCENT SIMILAR monitor displays what percentage of a turtle ’ s neighbors are the same color as it, on average, while the PERCENT UNHAPPY monitor display what percentage of all turtles are unhappy.

Lesson 101 : First Extension: Adding Multiple Ethnicities

[Download the powerpoint by clicking here](#)

First Extension: Adding Multiple Ethnicities

One simple extension to this model is to add a third, fourth, and even fifth ethnicity. This can be done by modifying the SETUP code. Currently the model sets all the turtles to red initially, then asks half of them to become green. Recall the code that sets up the turtles:


```
;; create turtles on random patches
ask n-of number patches
  [ sprout 1 ]
ask turtles [
  ;; make half the turtles red and the other half green
  set color one-of [red green]
]
```

We need to figure out how to modify this code so that it works with more than two ethnicities. Since we are now allowing the number of colors to vary we need to go beyond red and green turtles, sometimes we will need blue, yellow, and orange turtles as well.

Therefore, we begin by defining the colors we will allow the turtles to have. We do this

by establishing a global variable *colors*:

```
globals [
  percent-similar    ;; on average, what percent of a turtle's neighbors
                      ;; are the same color as that turtle?
  percent-unhappy    ;; the percent of the turtles that are unhappy
  colors              ;; a list of colors we use to color the turtles
]
```

The last line, “colors,” is the only new global variable. Now we need to define what the value of this global is. We can do this in the SETUP procedure:

```
set colors [red green yellow blue orange ]
```

This line of code initializes the global variable *colors* to be a list of the five colors. Next, we would like to allow the model user to control of the number of ethnicities in the system, and we can do this by adding a NUMBER-OF-ETHNICITIES slider. We will initially set the bounds of this slider from 2 to 5. Now we have to make the code use this slider. We do this by modifying the turtle coloring code. We replace all of the SETUP code above with the following:

```
;; create a turtle on NUMBER randomly selected patches.
ask n-of number patches
  [ sprout 1 ]

;; assign a color to each turtle from the list of our colors
ask turtles
  [ set color (item (random number-of-ethnicities) colors) ]
```

This code tells a number (equal to the interface slider NUMBER) of random patches to

sprout a turtle. Each turtle picks its color randomly from the list of colors we just initialized. But the turtles can only pick the colors in the list up to the number of ethnicities.

For example, if the NUMBER-OF-ETHNICITIES is 3, only the first three colors in the list are used.

Now that we have code that allows multiple ethnicities, try running the model with different numbers of ethnicities and observe what effect this has on the segregation of the system. You will notice that as you increase the NUMBER-OF-ETHNICITIES, it takes longer for the system to settle — that is, for all the turtles to be happy. (See figure 3.18.)

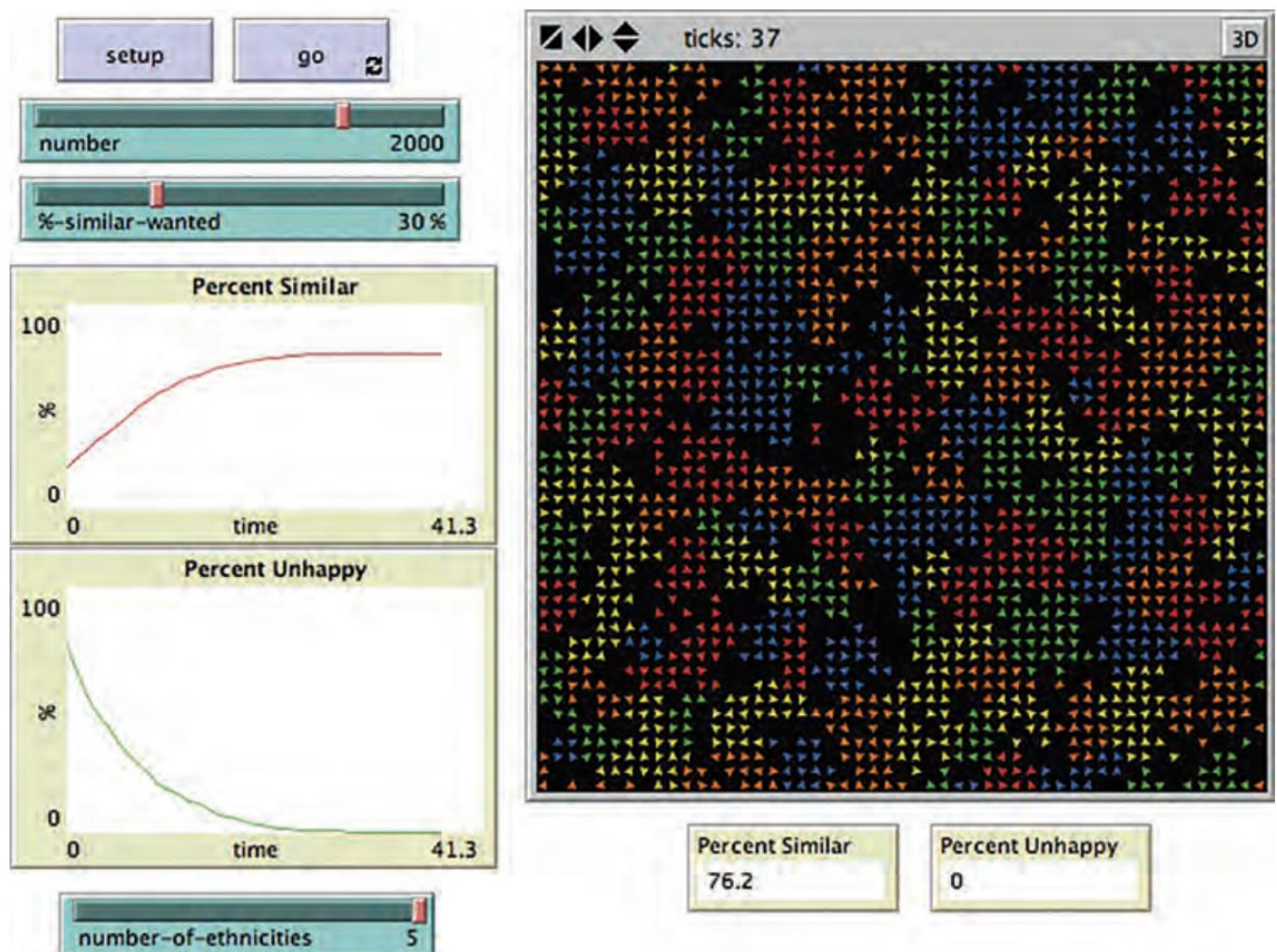


Figure 3.18
NetLogo Segregation model after first extension.

However, once the system has settled, the final PERCENT SIMILAR displayed in the monitor remains pretty much the same regardless of how many ethnicities there are. Can

you explain why this might be?

Lesson 102 : Second Extension: Allowing Diverse Thresholds

[Click here to download ppt](#)

Second Extension: Allowing Diverse Thresholds

In the original version of Schelling ' s model, every agent in the world had exactly the same similarity threshold. This meant that every agent had the same level of tolerance for having other ethnicities in its neighborhood. Although relaxing this assumption might have been difficult to enact with a checkerboard, it is straightforward to do using an ABM language.

It's likely that different individuals in a real population would have different levels of tolerance. And with an ABM language we can easily give each individual agent its own personal characteristics. Agents can act on these personal characteristics and make decisions based on them. This means that even when agents in an ABM are running the exact same code, their behavior can be quite different.

So how can we make the individuals in the Segregation model more diverse? One way would be to give them a range of similarity thresholds rather than requiring them all to have the same value for that threshold. To do this, we need to first give the turtles the ability to have their own %-SIMILAR-WANTED. We do this by modifying the turtles-own properties. As we showed in chapter 2, any variable in a turtles-own declaration is a variable that is a property of every turtle, that means every turtle can have a different value for that variable.

```
turtles-own [  
  happy?           ;; for each turtle, indicates whether at least  
  ;; %-SIMILAR-WANTED percent of that turtle's neighbors  
  ;; are the same color as the turtle  
  similar-nearby   ;; how many neighboring patches have a turtle with  
  ;; my color?  
  total-nearby    ;; sum of previous two variables  
  my-%-similar-wanted ;; the threshold for this particular turtle  
]
```

Now, in addition to the other properties that the turtles have, they each have a MY-%- SIMILAR-WANTED property, but as of right now this value is not set. We need to modify

the SETUP procedure in order to initialize the turtles with their own MY-%-SIMILARWANTED values.

```
;; assign a color to each turtle from the list of our colors using number-of-  
;; ethnicities to maximize the number of colors and assign an  
;; individual level of %-similar-wanted  
ask turtles  
[ set color (item (random number-of-ethnicities) colors)  
  set my-%-similar-wanted random %-similar-wanted5 ]
```

We insert this new code directly after specifying the color that we changed in the first extension. This value is set to a random value between 0 and the value from the %-SIMILAR-WANTED slider. This means that the %-SIMILAR-WANTED variable no

longer specifies the tolerance of each agent, but rather it specifies a maximum value that

any agent can have for its tolerance.

However, the code that determines whether or not a turtle is happy has not yet changed, and so all of these individual tolerance values will be ignored. Let us change the UPDATE-TURTLES code to use these new values. This requires only one small change

to the code:

```
set happy? similar-nearby >= ( my-%-similar-wanted * total-nearby / 100 )
```

All we do is replace %-SIMILAR-WANTED with MY-%-SIMILAR-WANTED to tell the turtle to use its own agent variable instead of the global variable.

Now that we have all of the code for the agents to use their own thresholds, run the model several times with different values of %-SIMILAR-WANTED. Do you see any different results? If you play with the model enough, you will notice that the PERCENT

SIMILAR monitor ends up with a lower value at the end of the run than it did after the first extension. This result is logical, since no individual can be less tolerant in this extension than before, but some individuals will be more tolerant allowing them to find

locations where they are happy even though there are more ethnically different individuals around them. In fact, the average of the MY-%-SIMILAR-WANTED values of the turtles will be approximately half of the global %-SIMILAR-WANTED slider. Comparing the original Segregation model with this extension of the model using twice the %-SIMILAR-WANTED value is left as further exploration for interested readers. (See figure 3.19.)

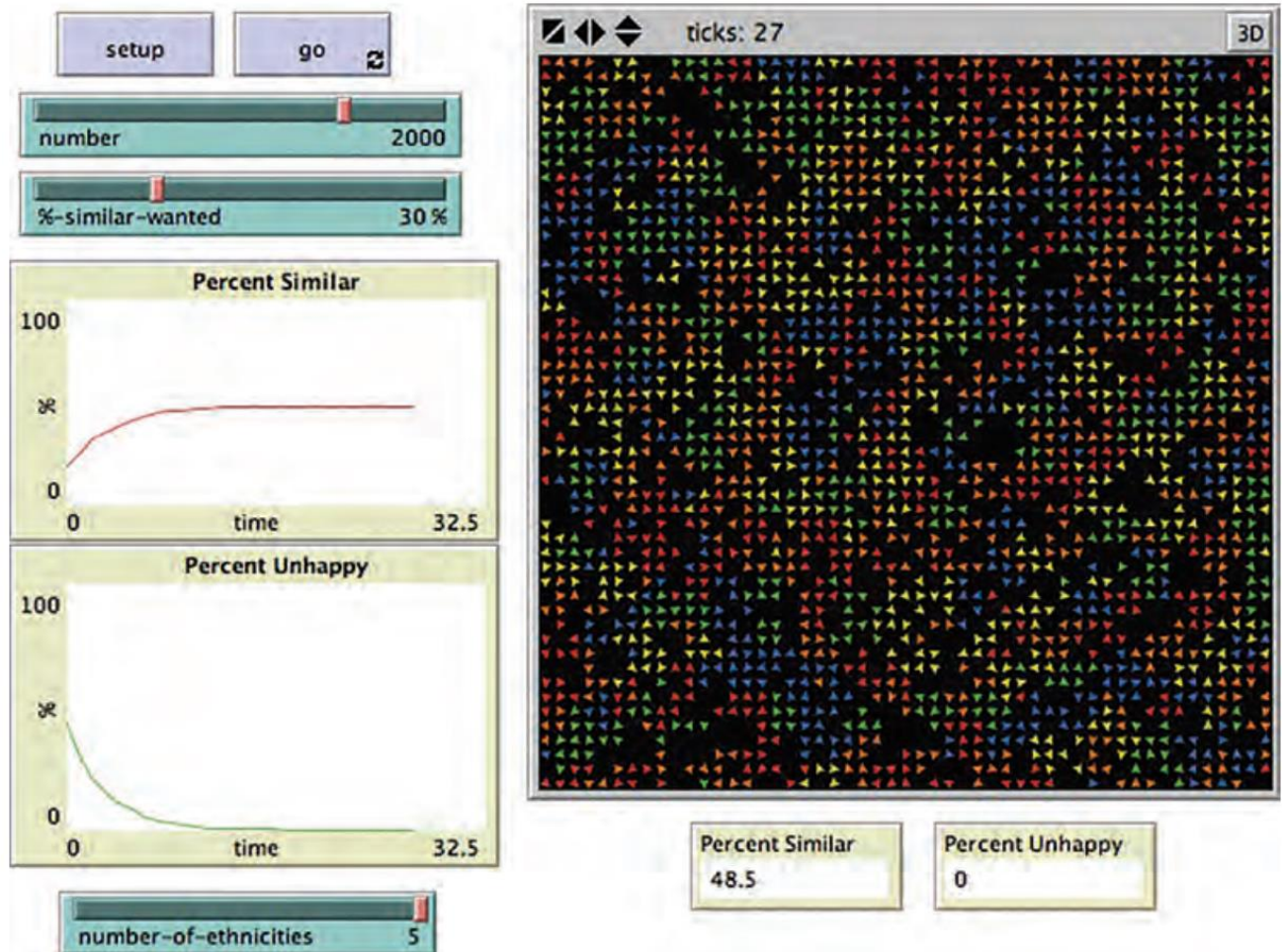


Figure 3.19
NetLogo Segregation model after second extension.

Lesson 103 : Third Extension: Adding Diversity-Seeking Individuals□

[click here to download ppt](#)

Third Extension: Adding Diversity-Seeking Individuals

Another simplification that the original segregation model makes about the world is that individuals are concerned about too much diversity but that no agents actively seek diversity in their neighborhoods. An easy way to overcome this simplification is to allow individuals to have a %-DIFFERENT-WANTED property just as they have a %-SIMILAR-WANTED property. We begin by creating a slider for %-DIFFERENTWANTED that is just like the %-SIMILAR-WANTED slider. After that we only need to make one additional change to the code. We've already been calculating the number of ethnically different

turtles in the neighborhood and so we only have to modify the `update-turtles` procedure to make use of the new parameter:

```
;; count the number of my neighbors that are a different color than me
let other-nearby count (turtles-on neighbors)
    with [color != [color] of myself]

set happy? similar-nearby >= ( my-%-similar-wanted * total-nearby / 100 )
    and other-nearby >= ( %-different-wanted * total-nearby / 100 )
```

With the addition of the new clause (shown in bold), the new happiness calculation tells the agents that they are only happy if the number of similar agents nearby is greater than their %-SIMILAR-WANTED threshold *and* the number of other agents nearby is greater than the %-DIFFERENT-WANTED threshold. So for an agent to be happy it cannot be too much in a minority or too great a majority. Now that we have added this code to the model, run it a few times and observe the results. As you play around with these sliders, it becomes quickly clear that it is much easier for the PERCENT SIMILAR results to decrease after this extension has been implemented. The reason is that all of the agents actually seek out diversity now, so they are deliberately taking actions that decrease the PERCENT SIMILAR results. Moreover, as you manipulate the sliders, you can easily find states of the parameters where the system

never settles down. For instance, if you put both the %-SIMILAR-WANTED and %-

DIFFERENT-WANTED sliders over 50 percent, the model will probably never reach

equilibrium. This is because some agents (those with MY-%-SIMILAR-WANTED greater than 50) will be trying to satisfy impossible demands; they will never find a location where more than half the agents around them are similar to themselves and more than half the agents around them are different than themselves.⁶ In general you will find that it takes this new system much longer to settle down to an equilibrium state. This is because the agents are now pickier in terms of where they are happy. They are seeking both diversity and similarity. (See figure 3.20.)

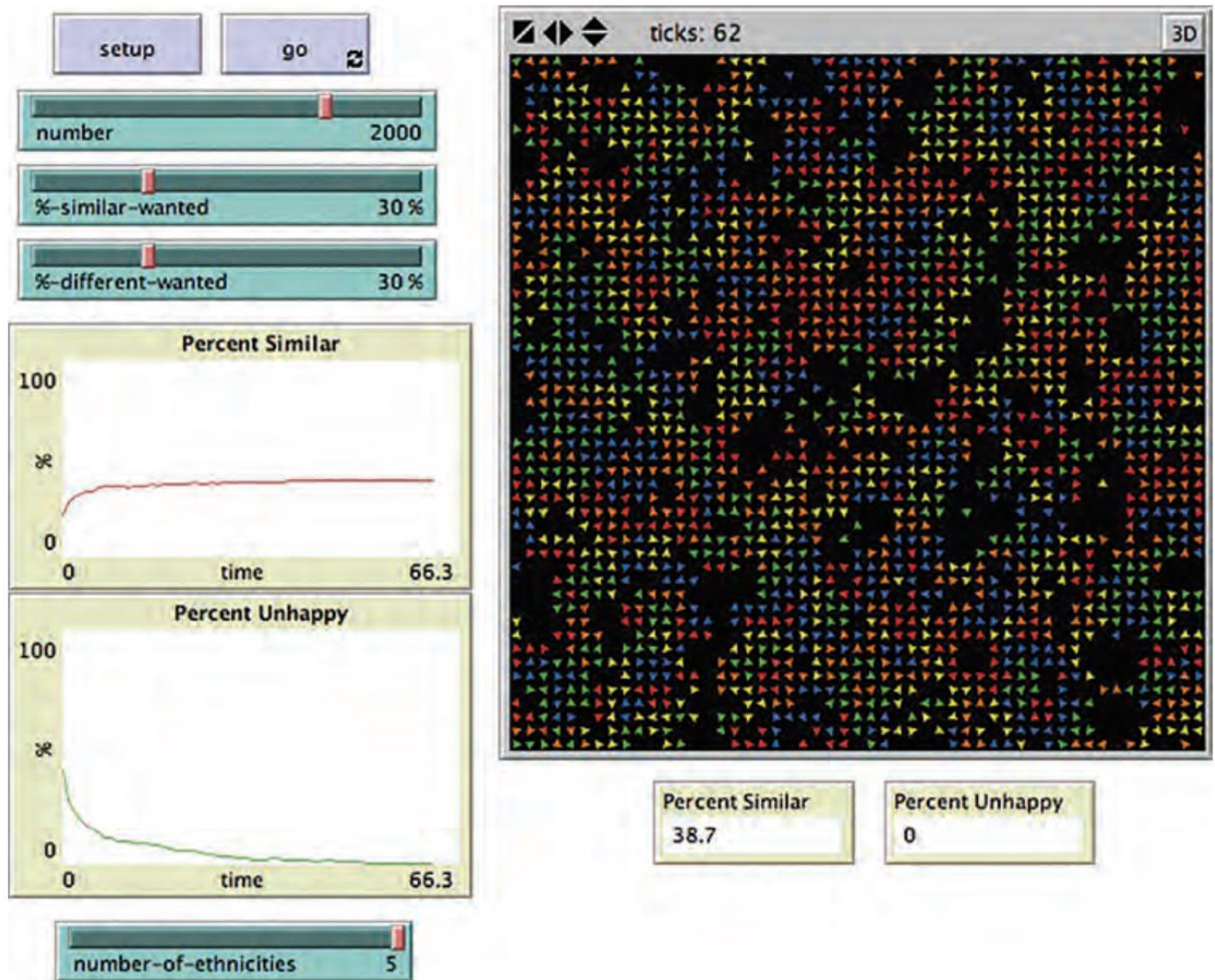


Figure 3.20
NetLogo Segregation model after third extension.

Lesson 104 : CITIES Project

[Click here to download .pptx](#)

Advanced Urban Modeling Applications

Agent-based modeling has frequently been used for the modeling of urban landscapes.

“Residential Preference”

The Segregation model described earlier can be seen as one particular example of what are generally called “residential preference” models, which are a major component of urban modeling systems.

CITIES project

These models can be combined with commercial, industrial, and governmental models of urban policy to create an integrated model of a city. One such example is the CITIES project (see figure 3.21) carried out by the Center for Connected Learning and Computer-Based Modeling at Northwestern University. The goal of the CITIES project was to procedurally develop realistic cities that could be used as tools in education and exploration (Lechner et al., 2006).

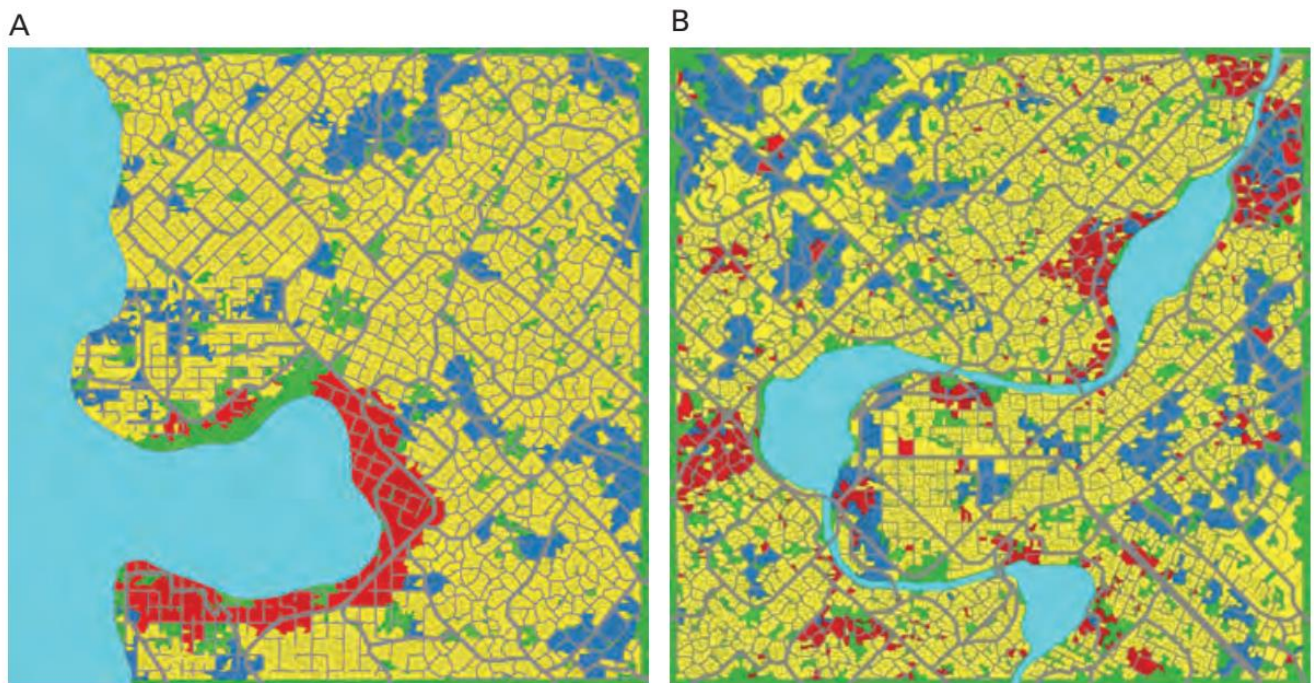


Figure 3.21
Some runs of the CITIES model in NetLogo (Lechner et al., 2006).

Goal Of Urban Modeling

Another goal of urban modeling is to explore the ecological impact of cities and their footprint on the environment.

SLUCE Project

The SLUCE project at the University of Michigan built several models of residential and tract development to explore land-use policies and their effect on the surrounding environs in southeastern Michigan (Brown et al., 2005; Rand et al., 2003).

Both of these projects can be seen as a subset of the larger category of land-use modeling. For a detailed review of agent-based models of land use, see Parker et al. (2003) and Batty (2005).

Lesson 105 : El Farol Bar

[Click here to download .pptx](#)

The El Farol Model

Occasionally, when working on agent-based models, you will find a model that reproduces phenomena that you are interested in but does not quite provide the output or data that you want. In this section we will take a model of an economic scenario, the El Farol model, and we will add additional reporters to this model. In this way we can gain new information from the model that was not previously available. This is possible because data collection in agent-based models is fairly simple, and the capabilities of ABMs for visually displaying information are diverse.

This section is a good example of how you do not need to understand every part of an agent-based model in order to work with it. The El Farol model uses “ lists ” and mathematical regression, but the modifications we will make to this model do not require knowledge of how these particular methods work. Although it is almost always desirable to understand the basic rules of the model, for the purposes of a particular extension, it may be sufficient to develop that understanding from reading the Info tab rather than trying to puzzle out the details of the code. When extending models, it is useful, though not easy, to develop the skill of understanding only the code you will need to modify and know which code is safe to ignore. Our extensions to this model will not require much understanding of the mechanics of the model.

Description of the El Farol Model

W. Brian Arthur was an Irish economist who, at age thirty-seven, became the youngest

holder of an endowed chair at Stanford University. He was a pioneer of using complexity methods in economics and is on the founders’ board of the Santa Fe Institute. In 1991,

Arthur posed the El Farol Bar problem as an exploration of bounded rationality and inductive reasoning.

" Optimal " Equilibrium

Traditional neoclassical economics presupposes that humans are completely rational: that is, they have access to perfect information and maximize their utility in every situation (Arthur, 1994). When each agent behaves this way, the aggregate can achieve an “optimal” equilibrium. But this is an idealization and people do not really conform to this normative description.

Therefore, Arthur suggested the El Farol Bar problem as an example of a system where agents do not perfectly optimize, yet a classical economic equilibrium is achieved. Arthur’ s example is even more

striking, because it is a situation in which it would be difficult for the “ideal” economic model to achieve equilibrium.

El Farol Bar

On Canyon Road in Santa Fe, New Mexico, there is a bar called the El Farol. This bar was popular with researchers from the Santa Fe Institute, including Arthur. One night each week, the El Farol had live Irish music, and Arthur enjoyed going on these nights, but occasionally it got crowded, and he did not enjoy it on those nights. Arthur wondered,

how do people decide if they should go to the bar or not? He imagined that there were one hundred citizens of Santa Fe who liked Irish music, and that each week they each tried to predict if the El Farol would be crowded. If they thought it was going to be crowded, specifically that more than sixty people would go to the bar, then they would stay at home, but if they thought it was not going to be crowded, then they would go to the El Farol.

Assuming that the attendance information at the bar each week was readily available, but that each citizen could only remember a limited number of weeks of the attendance,

Arthur’s hypothesis

Arthur hypothesized that one way to model this situation would be to give each agent a bag of strategies.

Strategies

Each of these strategies would be some rule of thumb about what the attendance was that week, e.g., “twice last week’s attendance,” “half the attendance of two weeks ago,” or “an average of the last three weeks attendance.” Each agent had a group of these strategies, and he would see how well these strategies would work had he used them in the previous weeks. The agents would use whichever strategy would have worked the best to predict this coming week’s attendance, and they would decide

whether to attend the bar based on this prediction. When Arthur wrote this up as an agentbased model and examined the results, he found that the average attendance at the bar was around 60. So despite all of the agents using different strategies and not having perfect information, they managed to optimally utilize the bar as a resource.

Netlogo Model

This model is available in the NetLogo models library under Sample Models > IABM

Textbook > Chapter Three > El Farol (Rand & Wilensky, 2007; <http://ccl.northwestern.edu/netlogo/models/ELFarol>). This model has a few controls. You can adjust the MEMORYSIZE of the agents, which controls how many weeks of attendance they remember.

You can also change the NUMBER-STRATEGIES, which is the number of strategies each agent has in its bag of strategies. Finally, you can adjust the OVERCROWDINGTHRESHOLD, which is the number of

agents needed to make the bar crowded. If you run the model as is, you will see the agents moving back and forth from the bar (the blue area) to their homes (green area), and the attendance is plotted on the left. The Interface tab of the model can be seen in figure 3.22.

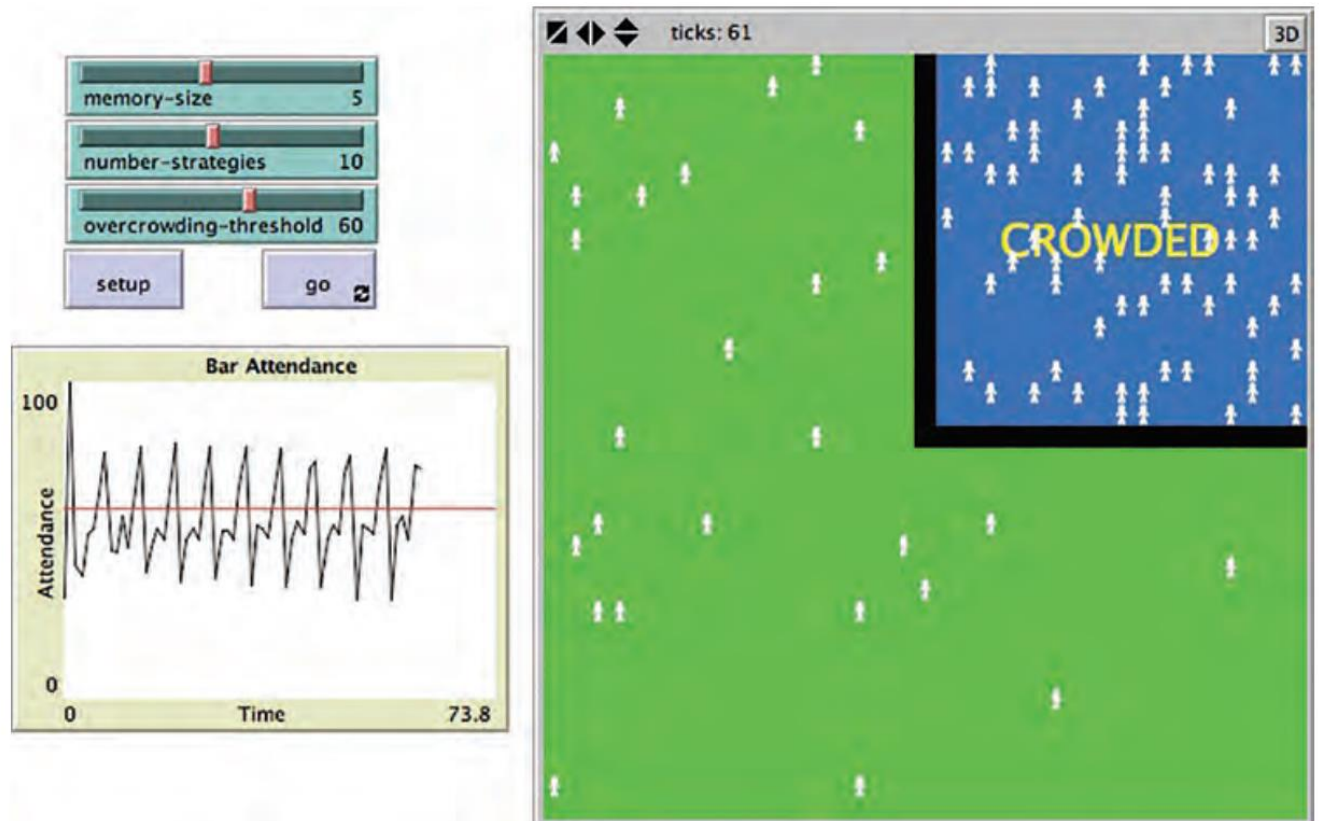


Figure 3.22

The El Farol model. Rand & Wilensky (2007). <http://ccl.northwestern.edu/netlogo/models/ElFarol>.

Lesson 106 : First Extension: Color Agents that are more successful predictors

[Click here to download .pptx](#)

First Extension: Color Agents That Are More Successful Predictors

The original model tells you how many agents are attending the bar, and you can get the visual reference of them moving to and from the bar, but there is nothing in this model to indicate which agents are better at choosing good times to attend the bar. Are some agents better at deciding when the bar will not be crowded than others? In order to find this out, we first need each agent to keep track of how often they go

to the bar when it is not crowded; to do this, we add a property to each agent. We will modify the turtles ' properties to add one additional item, REWARD:

```
turtles-own [  
    strategies          ;; list of strategies  
    best-strategy      ;; index of the current best strategy  
    attend?            ;; true if the agent currently plans to attend the bar  
    prediction          ;; current prediction of the bar attendance  
    reward              ;; the amount that each agent has been rewarded  
]
```

We are calling this new property REWARD, since the agents get rewarded if they attend

the bar when it is not crowded. Now we have to initialize this property, since each agent

will start with reward of 0. To do this we modify the create turtles part of the SETUP procedure, to initialize the REWARD to 0:

```
;; create the agents and give them random strategies  
create-turtles 100 [  
    set color white  
    move-to-empty-one-of home-patches  
    set strategies n-values number-strategies [random-strategy]  
    set best-strategy first strategies  
    set reward 0  
    update-strategies  
]
```

We also need to update the reward whenever the agent goes to the bar and it is not crowded. If we look at the GO procedure there is some code that determines if the bar is overcrowded or not and, if it is, the word CROWDED is displayed on a patch in the bar:

```
if attendance > overcrowding-threshold [  
    ask crowded-patch [ set plabel "CROWDED" ] ;; label the bar as crowded  
]
```

We want to update the reward of turtles who attend the bar when this condition is not true, so we can make this IF statement, an IFELSE statement, and then in the ELSE part

of the IFELSE we can ask all the turtles who attended the bar to increase their reward:

```

ifelse attendance > overcrowding-threshold [
  ask crowded-patch [ set plabel "CROWDED" ]
]
[ ;; if the bar is not overcrowded, reward the turtles that are attending
  ask turtles with [ attend? ] [
    set reward reward + 1
  ]
]

```

Now that we have given rewards to agents attending the bar when it is uncrowded, the final step is to modify the visualization to display these rewards appropriately. One way

to do that is to give each agent a different color based on how much reward it has accumulated. To make sure that we can keep track of relative differences in the model, we are going to color the agents relative to the maximum reward obtained. This means that we need to recolor every agent not just the ones that have gathered new rewards this time step.

If you look farther up in the GO procedure you will find some code where we ask each agent to predict the attendance that week. This is a good place to have the agents update

their color. NetLogo has a command called SCALE-COLOR that allow you to do this, we can set each agent's color to a shade of a color based on their reward. The SCALE-COLOR primitive takes four inputs: (1) a base color, which we will make red, (2) the variable that we are linking the color to, REWARD in this case, (3) a first range value, in this case we set it to slightly more than the maximum reward so far, and (4) a second range value, which we set to 0. If the first range value for SCALE-COLOR is less than the second one, then the larger the number of the linked variable the lighter the color will be. If the second range value is less than the first one (as it is in this case), then the larger the number of the linked variable, the darker the color will be. In this model, we set the first value to be the larger of the two so that everyone starts as white, and then turns a darker color as they accumulate rewards. This makes the model more consistent with the basic version, where all the agents are white:

```

ask turtles [
  set prediction predict-attendance best-strategy sublist history 0 memory-size
  ;; set the Boolean variable
  set attend? (prediction <= overcrowding-threshold)
  set color scale-color red reward (max [ reward ] of turtles + 1) 0
]

```

In this case, the darkest agents will be the one with the lowest reward, and the lightest agents will be the ones with the most reward (see figure 3.23).

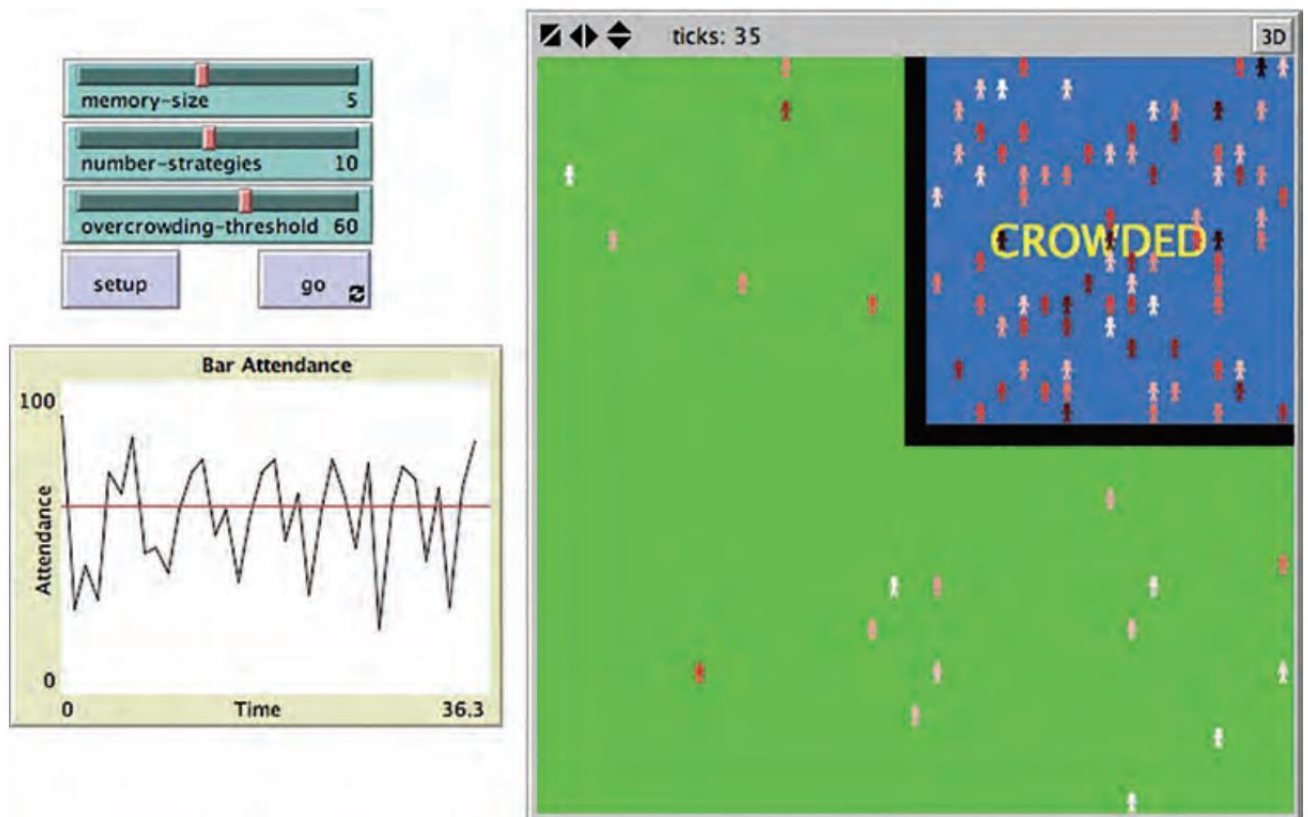


Figure 3.23
El Farol model: first extension.

Lesson 107 : Second Extension: Average, Min and Max Rewards

[Click here to download .pptx](#)

Second Extension: Average, Min, and Max Rewards

The first extension enables us to visually see which agents are doing better at going to the bar when it is not crowded. However, we do not have any hard numbers for the agents.

We could simply inspect each agent by right clicking on it and selecting inspect agent.

This would enable us to see the reward for each agent, but a better way might be to constantly display some information about the agents. We can do this by using monitors, which display values about the agent-based models.

To begin with we add a monitor to the Interface tab. In the first monitor, we can calculate the maximum reward collected by any agent. In the reporter area of the monitor editing window, we can put the following code snippet:

```
max [ reward ] of turtles
```

We can also give this monitor the name “Max Reward.” After this we can add another monitor for the minimum reward with the following code:

```
min [ reward ] of turtles
```

And give this monitor the name “Min Reward.” Finally, we can create a third monitor and calculate the average or mean reward using the following code:

```
mean [ reward ] of turtles
```

We can give this monitor the name “Avg. Reward.” The model interface window now should look like figure 3.24 . When we run this model we now get data on the average,

maximum, and minimum rewards over time. It quickly becomes apparent that though both the average and maximum reward increase over time, the max reward grows faster than the average reward meaning that some agents do better and better as time goes on. Though this extension did not take much coding, it gives you information and insight into what is going on in the model that was not available before.

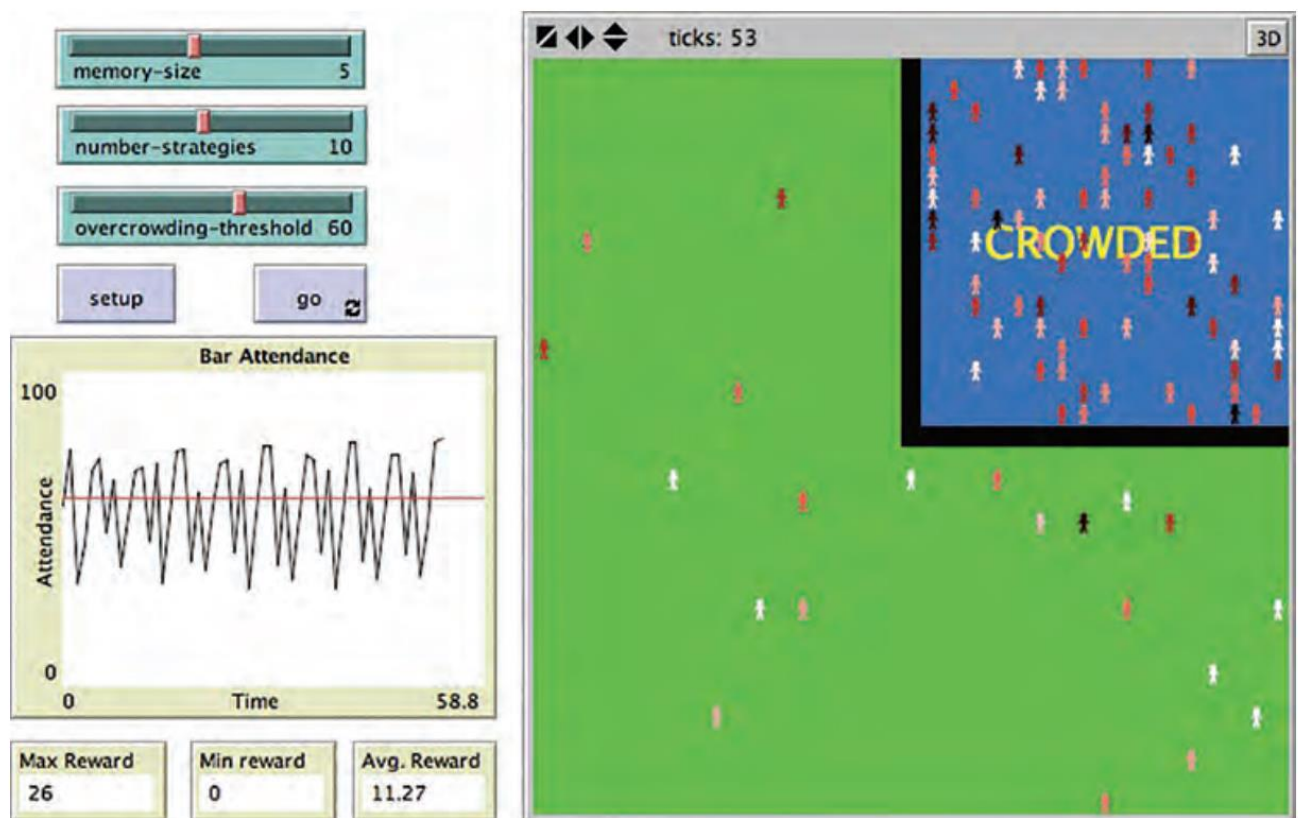


Figure 3.24
El Farol model: second extension.

Lesson 108 : Third Extension: Histogram reward vales

[Click here to download .pptx](#)

Third Extension: Histogram Reward Values

The **second extension** provides more data about how the agents in the model are doing, but this data has been aggregated so it is not obvious how many agents occupy each reward level. For example, are the agents distributed uniformly across reward levels or are many agents at the minimum value and many agents at the maximum value with few agents in between? To find out an answer to this question, we are going to add another plot to the El Farol model. This plot will be a histogram of the distribution of rewards. This is similar to the plot we created in chapter 2 for the Simple Economy model, but in this model, we will need to use the plot widget's "Plot update commands" feature.

To do this we first need to create a new plot in the Interface tab. In the configuration for this plot, we can give the plot the name "Reward Distribution," and change the Mode of the pen to "Bar" by clicking on the pencil in the first row of the Plot pens table. We tell the pen to display a histogram of the distribution by putting the following code in "Pen update commands."


```
histogram [ reward ] of turtles
```

This tells the pen to display a histogram of the REWARDS of the turtles. If we ran the model in this state, we would see a histogram drawn. However, the axes of the plot would not update correctly. Thus, we must enter the following under “Plot setup commands.”

```
set-plot-y-range 0 1  
set-plot-x-range 0 (max [ reward ] of turtles + 1)
```

These commands are run every time the plot is updated. They set the X and the Y range

of the plot window to reflect the current values that we are plotting. By setting the Y-range to 1, we are letting NetLogo automatically decide how to increase this range if it needs to in order to show all the data. (See figure 3.25.)

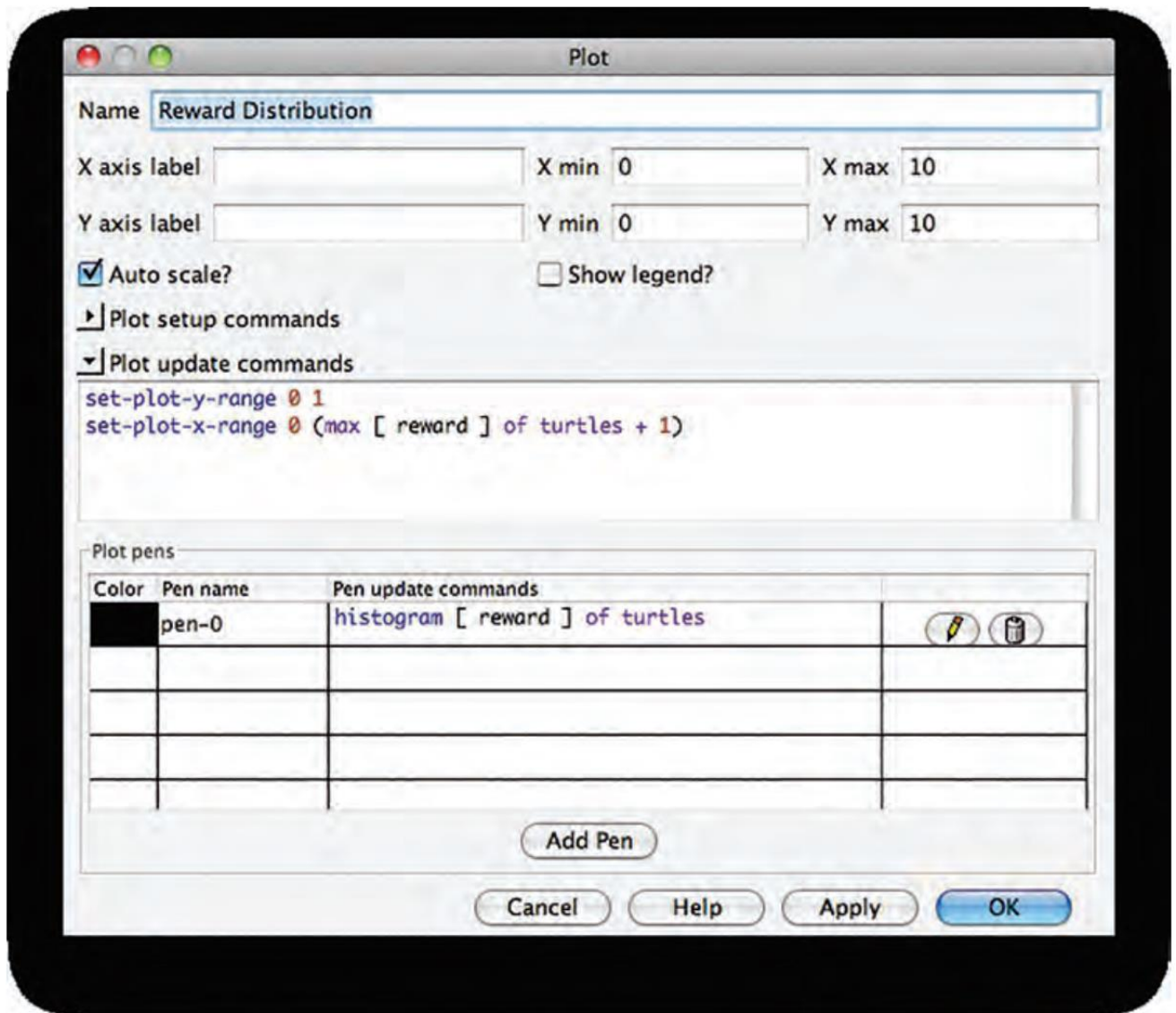


Figure 3.25

El Farol model: reward plot properties.

As you run the model over time, you will see it start out similar to a normal distribution,

but the distribution will quickly change with a few groups of individuals maintaining large rewards and many more having a lower reward. This lends credence to the hypothesis that there are a few agents that achieve a high reward level, but then there is a large gap between these high achieving agents, and the majority of agents, which achieve more average reward levels. (See figure 3.26.)

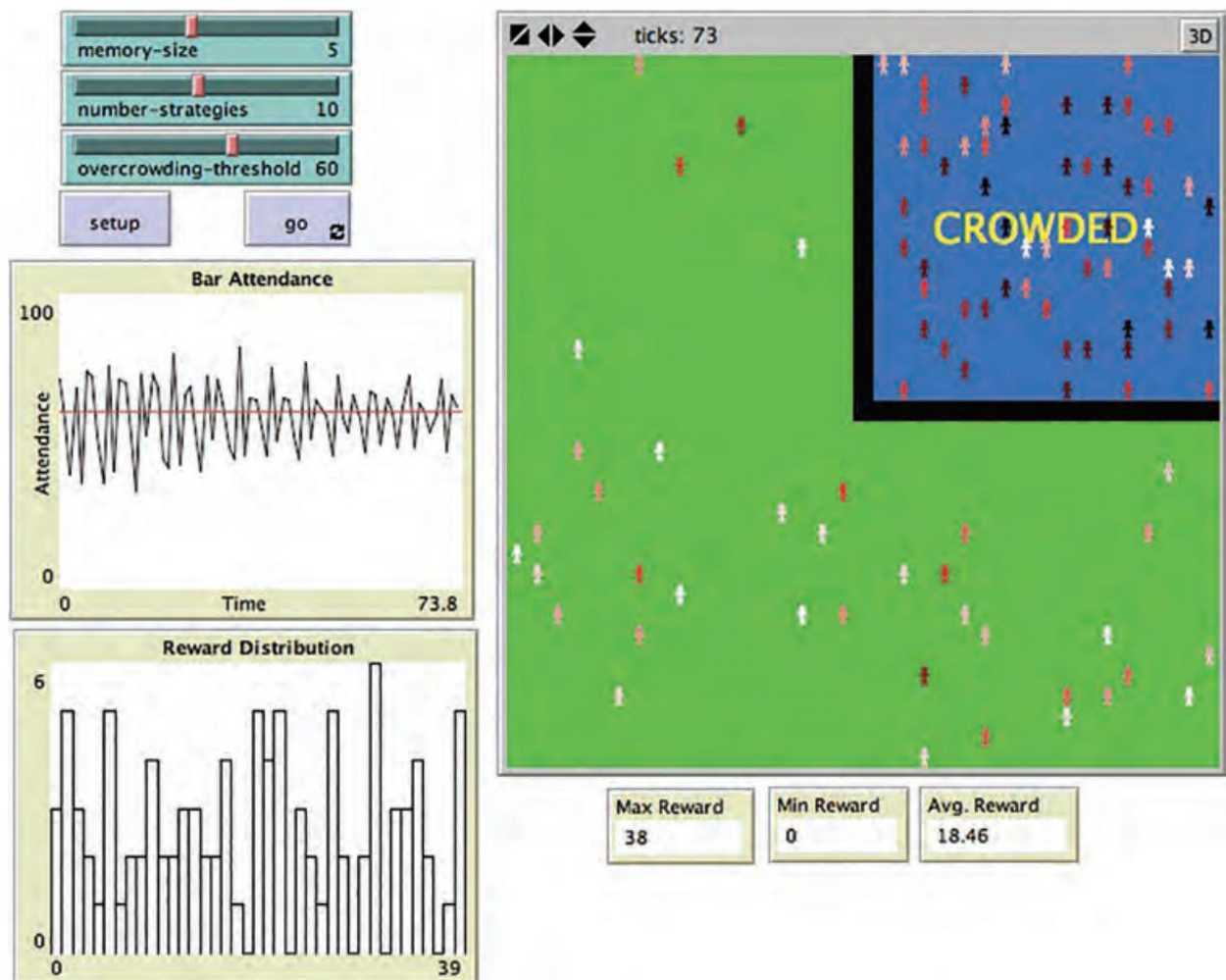


Figure 3.26
El Farol model: third extension.

Lesson 109 : Advanced Modelling Applications

[Click here to download .pptx](#)

Advanced Modeling Applications

The El Farol model has been investigated in many different ways. Partially because it is an exciting model in that it combines both ABM and machine learning (Rand, 2006 ; Rand & Stonedahl, 2007).

Machine learning

Machine learning enables the creation of powerful agent-based models where the agents not only change their actions over time, but also their strategies, i.e., the agents change the way they decide to take actions.

Minority Game

The El Farol model has also been idealized into the Minority Game (Challet, Marsili & Zhang, 2004), which has been studied by physicists and economists because it provides insights into complex systems. For instance, both El Farol and the Minority Game can be seen as a crude approximation for financial markets. In some sense, if everyone else in a financial market is selling, you want to be buying, and if everyone else is buying then you want to sell; being in the minority is usually a good way to make money.

Economic Model

The El Farol model is also one example of an economic model. Another early agentbased economic model was the Artificial Stock Market developed by a group of Santa Fe Institute researchers (Arthur et al., 1997). This model attempted to simulate the stock market and allowed researchers to investigate how investors affect phenomena like booms and busts in the market.

Economic Models In NetLogo

There are several other economic models in the NetLogo models library, such as the Oil Cartel model and the Root Beer Supply Chain (see figure 3.27).

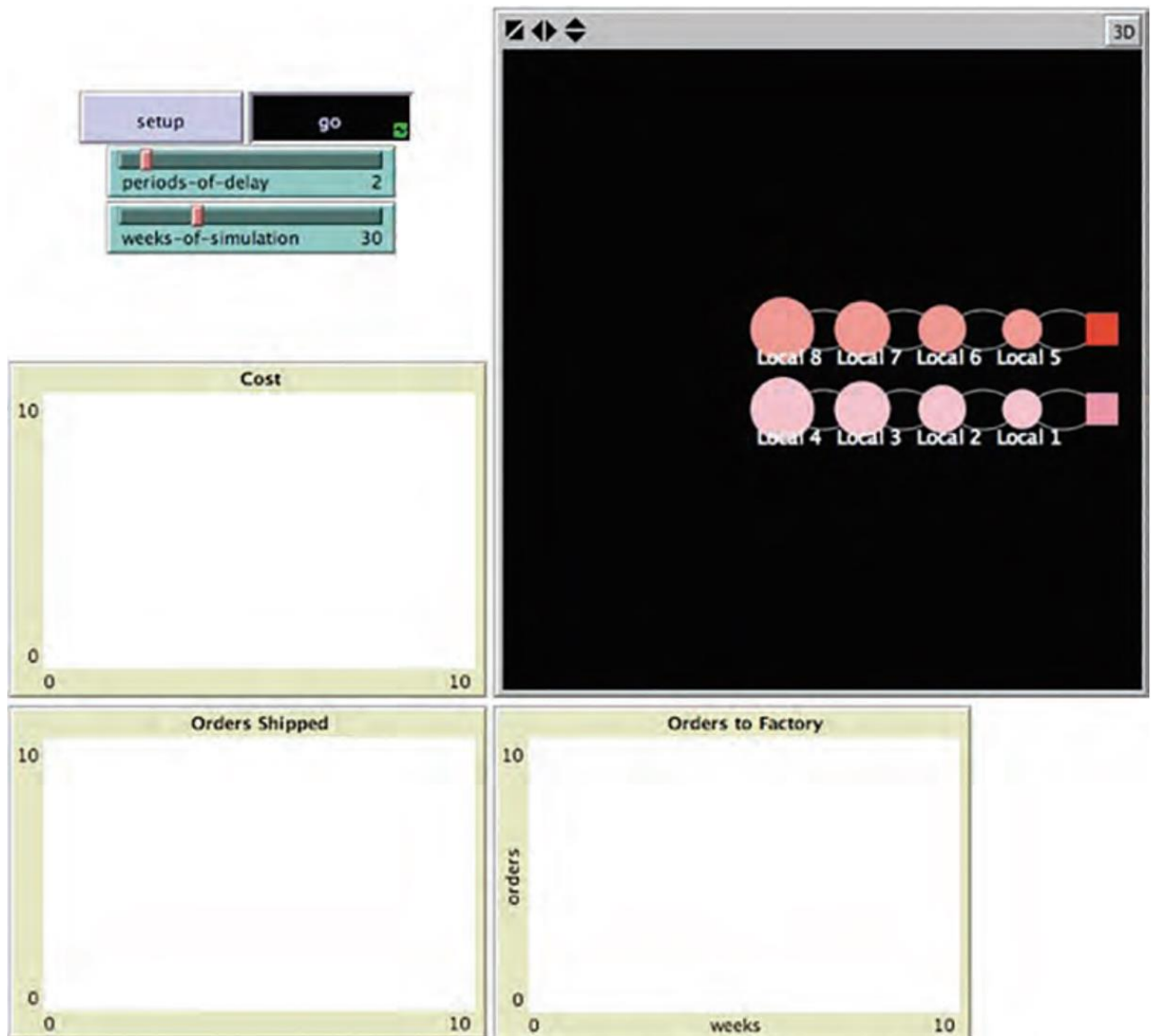


Figure 3.27

NetLogo HubNet Root Beer Game model. (Wilensky & Stroup 2003). <http://ccl.northwestern.edu/netlogo/models/HubNetRootBeerGame>.

Both of these models are interesting because they make use of the HubNet facility that is also provided with NetLogo.

HubNet

HubNet (Wilensky & Stroup, 1999c) is a NetLogo feature that enables Participatory Simulations, which is a simulation method where humans control agents in an agent-based model (see chapter 8 for more information on participatory simulations). This gives humans the ability to mix with nonhuman agents and to gain a deeper understanding of how their decisions affect and are constrained by a complex system.

After extending four NetLogo models, you should have the tools to extend many other NetLogo models.

Tools To Extend NetLogo Models

In the Fire model extensions and in the DLA extensions, we have seen that it is often useful to extend a model by replacing deterministic rules by probabilistic rules. Conversely, as we saw in modeling wind and long-distance transmission in the Fire model, it is sometimes useful to replace a probabilistic rule with an agent-based mechanism that generates the probabilities. And with each change to the model, we need to consider whether we also need to design a new metric for measuring model performance.

Lesson 110 : Creating Agent-based Model

[Click here to download .pptx](#)

First Step of Devising a Question

- This module is intended to take you from the first step of devising a question or area you want to explore, all the way through designing, and building your model, to refining your question and revising your model, to analyzing your results and answering your question.

Sequence of Steps

This sequence is presented here in linear order, but in reality these steps fold back on each other and are part of an iterative exploration and refinement of the model and motivating question.

Three Main Sections

To facilitate this process, this module is broken into three main sections:

- (1) Designing your model will take you through the process of determining what elements to include in your model,
- (2) Building your model will demonstrate how to take a conceptual model and create a computational object, and
- (3) Examining your model will address how to run your model, create some results, and analyze those results to provide a useful answer to your motivating question.

Model Of An Ecological System

Throughout this module, we will be designing, building, and examining a simple model of an ecological system.

The basic question that we will be addressing is: “How do the population levels of two habitat-sharing animal species change over time?”

Wolf Sheep Simple model

For our purposes in this module, we will call this model the Wolf Sheep Simple model. Though we will discuss this model in the context of two biological species, the model could be generalized to other situations such as companies competing for consumers, electoral parties competing for votes, or viruses evolving in a computer system. More important, the components that we will be developing in this model are basic components utilized in most ABMs.

Lesson 111 : Designing your Model

[Click here to download .pptx](#)

Designing Your Model

There are many ways of designing an agent-based model.¹ Which you choose will depend on many factors including the type of phenomenon to be modeled, your level of knowledge of the content domain, your comfort with NetLogo coding and your personal modeling style.

Major Categories of Modeling

We consider two major categories of modeling: *phenomena-based modeling* and *exploratory modeling*.

PHENOMENA-BASED MODELING

Reference Pattern

In *phenomena-based modeling*, you begin with a known target phenomenon. Typically, that phenomenon has a characteristic pattern, known as a *reference pattern*. Examples of reference patterns might include common housing segregation patterns in cities, spiral-shaped galaxies in space, leaf arrangement patterns on plants, or oscillating population levels in interacting species.

Goal Of Phenomena-Based Modeling

The goal of *phenomena-based modeling* is to create a model that will somehow capture the reference pattern. In ABM, this translates to finding a set of agents, and rules for those agents, that will generate the known reference pattern.

Once you have generated the reference pattern you have a candidate explanatory mechanism for that pattern and may also vary the model parameters to see if other patterns emerge, and perhaps try to find those patterns in data sets or by conducting experiments. You can also use

phenomena-based modeling with other forms of modeling, such as equation-based modeling. In equation-based modeling, this would mean writing equations that will give rise to the reference pattern.

EXPLORATORY MODELING

The second core modeling form is exploratory modeling. This form is perhaps less common in equational contexts than it is in ABM. In exploratory modeling with ABM,

you create a set of agents, define their behavior, and explore the patterns that emerge.

We then refine our model in the direction of perceived similarities with these phenomena

and converge toward an explanatory model of some phenomenon.

Formulate Research Question

Another distinction in modeling methodology is to what degree we specify a question to be answered by a model.

- At one end of the spectrum, we formulate a specific research question (or set of questions) such as “ How does a colony of ants forage for food? ” or “ How does a flock of geese fly in a V-shape? ”
- At the other end, we may only begin with a sense of wanting to model ants or bird behavior, but without a clear question to be answered.
- As we explore the model design space, we will gradually refine our question to one that can be addressed by a specific model.

Designing The Conceptual Model

Yet a third dimension is the degree to which the process of designing the conceptual model is combined with coding your model.

In some cases, it is advisable to work out the entire conceptual model design in advance of any coding of the model. This is referred to as *top-down* design.

Top-Down Design

In a top-down design, the model designer will have worked out the types of agents in the model, the environment they reside in, and their rules of interaction before writing a single line of code.

Bottom-Up Design

In other cases, the conceptual model design and the coding of the model will coevolve, each influencing the evolution of the other. This is often referred to as *bottom-up* design. In bottom-up design, you choose a domain or phenomenon of interest with or without specifying a formal question. Using this approach, you would then start writing code relevant to that domain, building the conceptual model from the bottom

up, accumulating the necessary mechanisms, properties and entities, and perhaps formulating some formal research questions along the way. For example, in a bottom-up design, you might start with a question about how an economic market would evolve, code some behaviors of buyers and sellers and, in so doing, realize you'll need to add brokers as agents in the model.

Design Principle

We will emphasize the top-down approach.

- The top-down design process starts by choosing a phenomenon or situation that you want to model or coming up with a question that you want to answer, and then designing agents and rules of behavior that model the elements of the situation. You then refine that *conceptual model* and continue to revise it until it is at a fine enough level of detail that you can see how to write the code for the model.

ABM Design Principle

Throughout the design process there is one major principle that we will use. We call this the *ABM design principle*: Start simple and build toward the question you want to answer.

Components Of Design Principle

There are two main components of this principle.

First Component: The first is to begin with the simplest set of agents and rules of behavior that can be used to explore the system you want to model. This part of the principle is illustrated by a quote from Albert Einstein, “The supreme goal of all theory is to make the irreducible basic elements as simple and as few as possible without having to surrender the adequate representation of a single datum of experience” (1933). Or in another phrase he is reputed to have said: “Everything should be made as simple as possible, but not simpler.” In the case of ABM, this means making your model as simple as possible given that it must provide you with a stepping-stone toward your final destination.

Second Component: Second, always have your question in mind, which means not adding anything to your model that does not help you in answering your question. The statistician George Box provides a quote that illustrates this point, “All models are wrong, but some models are useful” (1979). What Box meant was that all models are by necessity incomplete because they simplify aspects of the world. However, some of them are useful because they are designed to answer particular questions and the simplifications in the model do not interfere with obtaining that answer.

Usefulness of ABM Design Principle

This core ABM design principle is useful in several ways.

- First, it reminds us to examine every candidate model agent and agent-rule and eliminate it if progress can be made without it. It is not uncommon for novice modelers to build a model in which certain components have no effect whatsoever. By starting small and slowly adding elements to your model, you can make sure that these extraneous components never get developed. By examining each additional component as to whether it is needed to answer the research question you are pursuing, you reduce the temptation to, paraphrasing William of Occam, “multiply entities unnecessarily.” In so doing, you reduce the chance of introducing ambiguities, redundancies, and inconsistencies into your model.
- Another virtue of the ABM design principle is that, by keeping the model simple, you make it both more understandable and easier to verify.
 - **Verification** is the process of ensuring that a computational model faithfully implements its target conceptual model.
 - **Simpler Conceptual Model:** A simpler conceptual model leads to a simpler model implementation, which makes it easier to verify the model. Starting simple and building up also facilitates the process of just-in-time results.

Identifying a Question: To apply our principle to the context of the Wolf Sheep Simple model, we need to start our design by reflecting on two habitat-sharing animal species and identifying simple agents and behaviors for our model. We will start by identifying a question we want to explore, which is required by the ABM design principle (top-down version). After that we will discuss what the agents in our models are and how they can act. Then we move on to the environment and its characteristics. As part of this process, we need to discuss what happens in an individual time step of the model. Finally, we discuss what measures we will be using to answer our question.

Lesson 112 : Choosing Your Questions

[click here to download .pptx](#)

Choosing Your Questions

Choosing a question may seem to be a separate issue from model design.

Natural Progression

After all, the natural progression seems to be:

- first choose a question, and
- second build a model to answer that question.

Sometimes, that may indeed be the procedure we follow, but in many instances we will need to refine our questions when we start to think about it in an agentbased way.

Wolf Sheep Simple model

Our original question for the Wolf Sheep Simple model was:

- “ How do the population levels of two species change over time when they coexist in a shared habitat? ”

We will now evaluate whether this question is one that is amenable to ABM and refine our question within the ABM paradigm.

ABM For Making Sense Of Systems

Agent-based modeling is particularly useful for making sense of systems that have a number of interacting entities, and therefore have unpredictable results. As we discussed

in early modules of the course, there are certain problems and questions that are more amenable to ABM solutions.

Consider A Different Modeling Method

If our primary question of interest violates our guidelines, it may be an indication that we should consider a different modeling method.

Assumption

For example, we might be interested in examining the dynamics of two very large populations under the assumption that the species are homogenous and well-mixed (no spatial component or heterogeneous properties) and that the population level of each species is simply dependent on the population level of the other species.

EBM vs. ABM

If that is the case then we could have used an equation-based model instead of an agent-based model since EBM's work well for large homogeneous groups, and as we mentioned in previous chapters, there is a classic EBM for this situation known as the Lotka-Volterra differential equations (Lotka, 1925 ; Volterra, 1926).

Conceptualize The Agents

ABM will be more useful to us if we are thinking of the agents as heterogeneous with spatial locations. This affects how we conceptualize the agents. One aspect of the animals that is likely to be relevant to our question is how they make use of their resources. Animals make use of food resources by converting them to energy hence we will want to make sure that our agents have different amounts of energy and different locations in the world.

Agent's Interaction With Environment

A third guideline is to consider whether the aggregate results are dependent on the interactions of the agents and on the interaction of the agents with their environment. For example, if one species is consuming another then the results will be dependent on agent interaction. Predator-prey interactions are usually set in rich environments. Keeping the ABM design principle in mind, we start with the simplest environment — we enrich the environment a little by going beyond just predators and prey and including resources from the environment that the lowest level prey species consume.

Time Dependent Processes

Yet another guideline is that agentbased modeling is most useful for modeling time dependent processes. In the Wolf Sheep Simple model, our core interest lies in examining how population levels change over time. We might therefore refine our question to focus on conditions that lead to the two species coexisting together for some time. In this way our guidelines help us evaluate whether our question is well suited to ABM and, if so, to focus our question and conceptual model.

Lesson 113 : A Concrete Example

[click here to download .pptx](#)

A Concrete Example

Now that we have identified our research question in detail it can be useful to consider a particular context for this research question.

Phenomena-based Agent-based Models

Earlier, we discussed reference patterns as a source of phenomena-based agent-based models. Sometimes that reference pattern is the original inspiration for the model.

Refined Research Question

Other times, as now, we have refined our research question enough that we seek out a reference pattern that will help us test whether our model is a valid answer to the question.

Predator-prey Relations

In the case of the predator-prey relations, there is a famous case of cohabiting small predator-prey populations in a small geographic area. This is the case of fluctuating wolf and moose populations in Isle Royale, Michigan. The wolves and moose of Isle Royale have been studied for more than five decades. This research represents the longest continuous study of any predator-prey system in the world. . . . Isle Royale is a remote wilderness island, isolated by the frigid waters of Lake Superior, and home to populations of wolves and moose. As predator and prey, their lives and deaths are linked in a drama that is timeless and historic. Their lives are historic because we have been documenting their lives for more than five decades.

This research project is the longest continuous study of any predator-prey system in the world. (From the Wolves & Moose of Isle Royale Project Website, <http://isleroyalewolf.org/>) Figure 4.2 shows the wolf and moose populations in Isle Royale from 1959 through 2009.

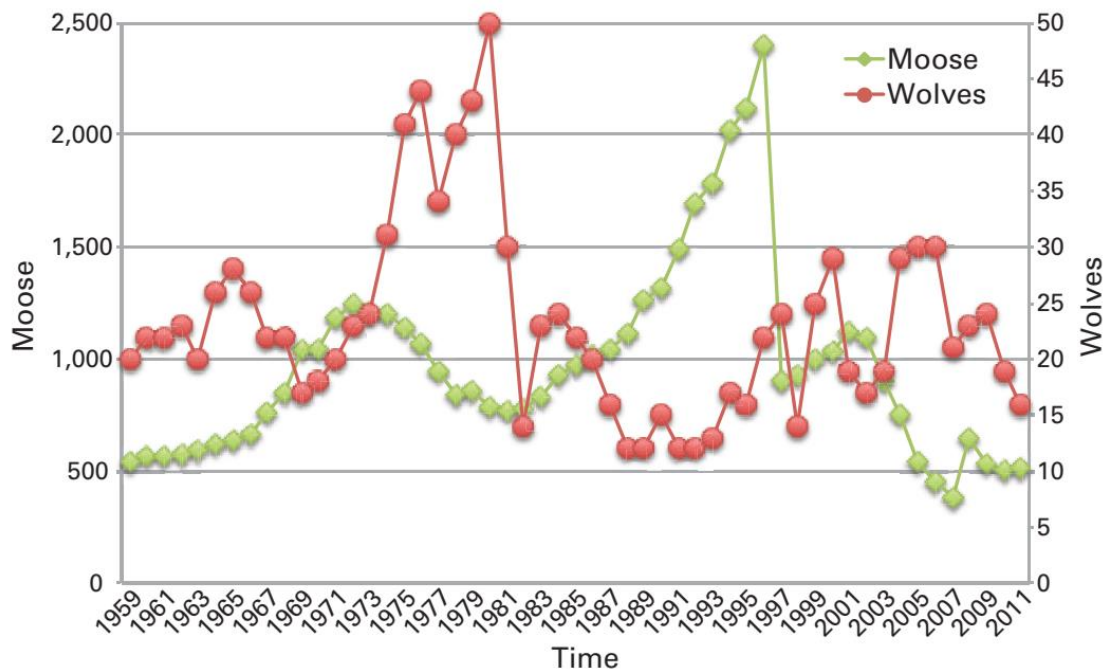


Figure 4.2

Five decades of fluctuating wolf and moose populations at Isle Royale. Note that when the wolf population peaks, the moose population is at a low point and, similarly, when the moose population peaks, the wolf population is at a low point.

This graph can serve as a reference pattern for our model. Our completed model will have to be able to generate a graph “similar” to this one to be a possible explanatory model of these phenomena. In general, this process is called *validation*.

As we can see in the data from Isle Royale, the wolf and moose populations in Isle Royale have been sustaining themselves for more than fifty years without either species going extinct. The populations also exhibit a rough oscillation, with moose at a low when wolves peak and vice versa. This data can serve as a reference pattern for our phenomenabased modeling. It allows us to further refine our research question to this: “ Can we find model parameters for two species that will sustain *oscillating* positive population levels in a limited geographic area when one species is a predator of the other and the second species consumes resources from the environment? ” In the models in this chapter, instead of modeling wolf and moose, we will model

wolf and sheep. The wolf and moose data set is well established, but our goal in this chapter is not to match this particular data, but to introduce you to classic examples of predator-prey modeling and to try to reproduce the oscillating sustained pattern of population levels.

Lesson 114 : Choosing Your Agents

[click here to download .pptx](#)

Choosing Your Agents

Now that we have identified and contextualized our driving research question, we can begin to design the components that will help us answer it.

What Are The Agents In The Model?

The first question we should ask ourselves is: What are the agents in the model? When designing our agents, we want to choose those components of our model that are autonomous and have properties, states, and behaviors that could possibly have bearing on our question.

Agent Overload

But we must be careful to avoid agent overload. Depending on the perspective one takes, almost any model component could be considered an agent. However, a model that is designed with an excess of agent types can quickly become unmanageable.

Entities Relevant to Research Question

When choosing what are to be the agents in a model, it is important to concentrate on those autonomous entities which are most relevant to our research question. A related issue is the “granularity” of the agent. Every entity is composed of multiple smaller entities.

What is The Right Level of Entity to Choose?

What is the right level of entity to choose? Should our agent be molecules or atoms? Body organs or cells? Some agents can be treated as mass properties. If we want to model a field of grass, we might not want to model every blade of grass, but instead choose “clumps” of grass as our agents.

Granularity Of Each Agent in temporal scale

It is important that the granularity of each agent be at roughly the same level. For instance in the temporal scale, if you are modeling the sheep actions at the level of days of activity, but the grass minute by minute, that can be difficult to reconcile.

Granularity of Each Agent In Physical Scale

In the physical scale, if there is more grass than the sheep can consume then you will not see many interesting behaviors since grass will not serve as a limiting condition.

Proto-agents

If we suspect that, in the future, some model entities might need to become full agents, we can choose to design them as proto-agents. By the term proto-agent, we mean agents that do not have individual properties, states, or behaviors but instead inherit some or all of their characteristics from a global agent type. For example, in the Wolf Sheep Simple model we might desire to have a human hunter who interacts with the other two species. This hunter might simply eliminate a part of the populations every now and then.

Create a Simpler Proto-agent

There is no need to build the hunter as a full agent at the start; instead we can create a simpler proto-agent that has the ability to kill off a random percentage of the population. Eventually, if needed, this hunter could become a full agent and have full properties and behavior just like any other agent.

Three Agent Types

Given the preceding discussion, we start our Wolf Sheep Simple model design by choosing three agent types.

We model the predators, which we will call wolves, and the prey, which we will call sheep, and the resources the sheep consume, grass. We could have added many other agents to this model. For example, we could model the hunter described above or the precipitation levels or soil nutrition. However, by choosing just wolves, sheep, and grass, we stick to the ABM design principle.

Lesson 115 : Choosing Agent Properties

[click here to download .pptx](#)

Choosing Agent Properties

Agents have properties that distinguish them from other agents. It is important to determine these properties in advance so we can conceptualize the agent and design the agents' interaction with each other and with the environment.

Wolf Sheep Simple model - Three Properties

In the Wolf Sheep Simple model we give the sheep and wolves three properties each:

- (1) an energy level, which tracks the energy level of the agent,
- (2) a location, which is where in the geographic area the agent is, and
- (3) a heading, which indicates the direction the agent is currently moving or would be moving.

Energy Property

The energy property is not merely describing temporary energy (such as whether an animal is fresh or fatigued). Rather, “energy” incorporates some notion of the amount of “vitality” in a creature, abstracting away the messy details of metabolism, calorie storage, or starvation, and condensing it all into a single measure.

Adding Additional Properties

We could add additional properties and some of them might be useful for future extensions. For instance, we could add a movement speed and allow different agents to move at different speeds, or an offense/defense capability that affects the ability of the individual to predate or resist predation.

Lesson 116 : Choosing Agent Behavior Choosing Parameters of the Model

[click here to download .pptx](#)

Choosing Agent Behavior

In addition to designing the structure of the agents, it is important to determine what kind of behavior the agents can exhibit. These behaviors are necessary to describe how agents interact with each other and the environment.

Wolf Sheep Simple Model

In the Wolf Sheep Simple model, sheep and wolves share many common behaviors.

They both have the ability to turn randomly, move forward, reproduce and die. However, sheep and wolves differ in that sheep have the ability to consume grass and wolves have the ability to consume sheep. This differentiates the two species/agent types from each other. Of course, once again there are many other behaviors we could prescribe for these agents. For instance, we could give sheep the ability to huddle in herds to defend against wolf attacks, or the ability to fight back. The wolves could have the ability to move at different speeds from a walk to a run or to chase sheep. Wolves and sheep also engage in a number of behaviors, such as sleeping, digesting food, and seeking shelter during a thunderstorm. However, again the behaviors we have described (moving, reproducing, eating, and dying) are reasonable choices for a simple model that can address our research question. For the grass clump agents, we give one simple behavior, the ability to grow.

Lesson 117 : Choosing Parameters of Model

[click here to download .pptx](#)

Choosing Parameters of the Model

We could decide to write one set of completely specified rules to control the behavior of all of these agents and their environmental interactions during a time step, but it makes more sense to create some parameters that enable us to control the model, so that we can easily examine different conditions. A next step is to define what attributes of the model we will be able to control through parameters.

Several Possible Parameters Of Interest

There are several possible parameters of interest in the Wolf Sheep Simple model.

- For instance, we will want to be able to control the number of initial sheep and wolves. This will enable us to see how different values of the initial population levels affect the final population levels.
- Another factor to control is how much energy it costs an agent to move. Using this parameter, we can make the landscape more or less difficult to traverse, and thus simulate different types of terrain.

Parameters For Controlling The Energy

Related to the cost of movement is the energy that each species gains from food. Thus, we choose to have parameters for controlling the energy gained from grass and the energy gained from sheep.

Finally, since the sheep consume grass, in order to sustain the population over time we will want the grass to regrow. So we will need a parameter for the grass regrowth rate.

Homogenous Parameters

There are many other parameters that we could have included in this model. For instance, the parameters we chose are homogenous across the model. In other words, one sheep will gain the same from grass as any other sheep.

Heterogeneous Parameters

However, we could make this model more heterogeneous by drawing the energy gain for each sheep from a normal distribution, and have two model parameters that control the mean and variance of the energy gain.

Add Parameters to Control Aspects

We could also add parameters to control aspects that we are currently planning to specify as constant values in the code. For example, we did not create a parameter to control the speed of the agents.

Lesson 118 : Summary of the Wolf Sheep Simple Model Design

[click here to download .pptx](#)

Summary of the Wolf Sheep Simple Model Design

Now that we have gone through the major design steps, we can create a summary document that describes our model design. The Wolf Sheep Simple model can be described in the following way:

Driving Question Under what conditions do two species sustain oscillating positive population levels in a limited geographic area when one species is a predator of the other and the second species consumes limited but regenerating resources from the environment?

Agent

Agent Types Sheep, Wolves, Grass

Agent Properties Energy, Location, Heading (wolf and sheep), Grass-amount (grass)

Agent Behaviors Move, Die, Reproduce (wolf and sheep), Eat-sheep (wolf only), Eat-grass (sheep only), Regrow (grass)

Parameters Number of Sheep, Number of Wolves, Move Cost, Energy Gain From Grass, Energy Gain From Sheep, Grass Regrowth Rate

Time Step:

1. Sheep and Wolves Move
2. Sheep and Wolves Die
3. Sheep and Wolves Eat
4. Sheep and Wolves Reproduce
5. Grass Regrows

Measures Sheep Population versus Time, Wolf Population versus Time It is quite useful while designing a model to write notes to yourself, as we have done in this section. You will find these notes invaluable after you have left the model for a while, since you will be able to go back and recall why you made certain decisions and alternative choices that you considered. Also, for the purposes of explaining your model to others, it is very helpful to have such documentation about the model. Finally, we recommend that you date your notes as you create them so that you can track your model design process.

Set Of Questions

For instance, if you are using the top-down design process that we have just discussed, then you might look over the following set of questions and write down answers to them as a provisional guide for how to build your model:

1. What part of your phenomenon would you like to build a model of?
2. What are the principal types of agents involved in this phenomenon?
3. In what kind of environment do these agents operate? Are there environmental agents?
4. What properties do these agents have (describe by agent type)?
5. What actions (or behaviors) can these agents take (describe by agent type)?
6. How do these agents interact with this environment or each other?
7. If you had to define the phenomenon as discrete time steps, what events would occur in each time step,

and in what order?

8. What do you hope to observe from this model?

Bottom-up Approach

A more bottom-up approach would not start with these questions. Instead, you might know only that you want to build some kind of ecological model and could begin by creating some sheep and have them spread out in the world.

Next, you might think of and start implementing some behaviors for the sheep such as moving, taking steps, eating, reproducing and dying.

Implementing Some Behaviors

In order for the sheep to eat, you might decide to add grass to the model.

Having sheep eat grass provokes the question, “What eats sheep?” Therefore, you modify the model to include wolves, as predators for the sheep. This process allows the wolf sheep model to be designed and even implemented without first considering the final goal of the model.

Lesson 119 : Examining a Model

[click here to download .pptx](#)

Examining a Model

We have found one set of parameters that exhibits the behavior of our reference pattern.

We note that these particular values for the parameters do not correspond to any real predator and prey populations. We did not calibrate our model from real-world data, so the parameter values themselves are not important. But, discovering that there exist model parameters that exhibit our reference pattern gives us insight into the natural phenomenon we were trying to model.

Parameters of two Species

Now that we have built our model, and we have found a set of parameters that allow the wolf and sheep populations to coexist and to oscillate, we have partially answered our research question: What parameters of two species will sustain oscillating positive population levels in a limited geographic area when one species is a predator of the other and the second species consumes limited but regrowing resources from the environment? We now know that it is possible for the rules we set up to produce the target reference pattern and therefore they could be a possible generative explanation for that pattern.

Observing the Behavior Of Model Once

However, just observing the behavior of this model once does not provide us with a robust answer. First, because many components of our model are stochastic in nature, there is no guarantee that the model when run again with the exact same parameters will exhibit the same behavior. Second, we have found one set of parameters that allow wolves and sheep to exist, but are there other parameter settings? A more general answer would allow us to make statements about ranges of parameters, and relationships between parameters, that allow both the wolves and the sheep to survive.

Variety of Parameter Settings

However, if we run the model many times with a variety of parameter settings, this will create a lot of data. Thus, to give us even a simple answer to our question, we really need to examine multiple runs across multiple different parameter settings and summarize this data in a useful way.

Lesson 120 : Multiple runs

[click here to download .pptx](#)

Multiple Runs

Whenever you have a model that has stochastic components, it is important to run the model several times so that you can be certain you have correctly characterized the behavior of the model.

Anomalous Behavior

If the model is run only once, you might happen to see anomalous behavior that is not what the model usually produces. For instance, in the Wolf Sheep Simple model it might be possible to run the model and arrive at a state in which there is simply one wolf and one sheep, and the sheep produces a second sheep often enough to keep the wolf fed, but not often enough to produce three sheep. However, due to the way the wolves and the sheep wander around the landscape, such an outcome is extremely unlikely, and it would not be typical of the model.

Run the Model Multiple Times

Thus, it is important to run the model multiple times so that you can characterize the normal/average behavior of the model and not the aberrant behavior of the model. On the other hand, there are times when the anomalous/aberrant model behavior is what you are interested in investigating, in which case you will need to run the model many times in order to find it and characterize it.

BehaviorSpace Tool

Most ABM platforms provide a way to do this. NetLogo provides you with the BehaviorSpace tool (Wilensky & Shargel, 2002) that enables you to run a model for several iterations with the same (or different) parameter settings and collect the results.

Number of Time Steps

One additional consideration when performing multiple runs is how many time steps to run the model for. Since we want to be able to compare the results across different random number seeds, it is useful to hold the number of time steps we run the model constant.

Lesson 121 : Predator-Prey Models: Additional Context

[click here to download .pptx](#)

Predator-Prey Models: Additional Context

One of the first uses of agent-based modeling was for modeling of ecosystems, where it was often called individual-based modeling (see the appendix for a brief discussion of the historical role of individual-based modeling). Modeling of ecological systems has been of interest to biologists and environmentalists for quite some time. If we can better understand how ecologies operate and what the effects of successful ecosystems on the global environment are, then we may be able to better intervene to assist in the sustenance of these systems. For more information on this topic, refer to Grimm and Railsback (2005) .

Predator-prey Interaction

One of the first attempts to study ecologies and population dynamics in a concrete way was the work of Lotka (1925) and Volterra (1926) . They developed a system of equations to describe a two species predator-prey interaction.

Lotka-Volterra Equations

These simple equations showed that you could meaningfully model ecological systems with just a few parameters. And once you have a model of a system you can start to explore options for perturbing the system into favorable states. The general result of the Lotka-Volterra equations is that predators and prey populations move in cycles. This is seen in figure 4.11.

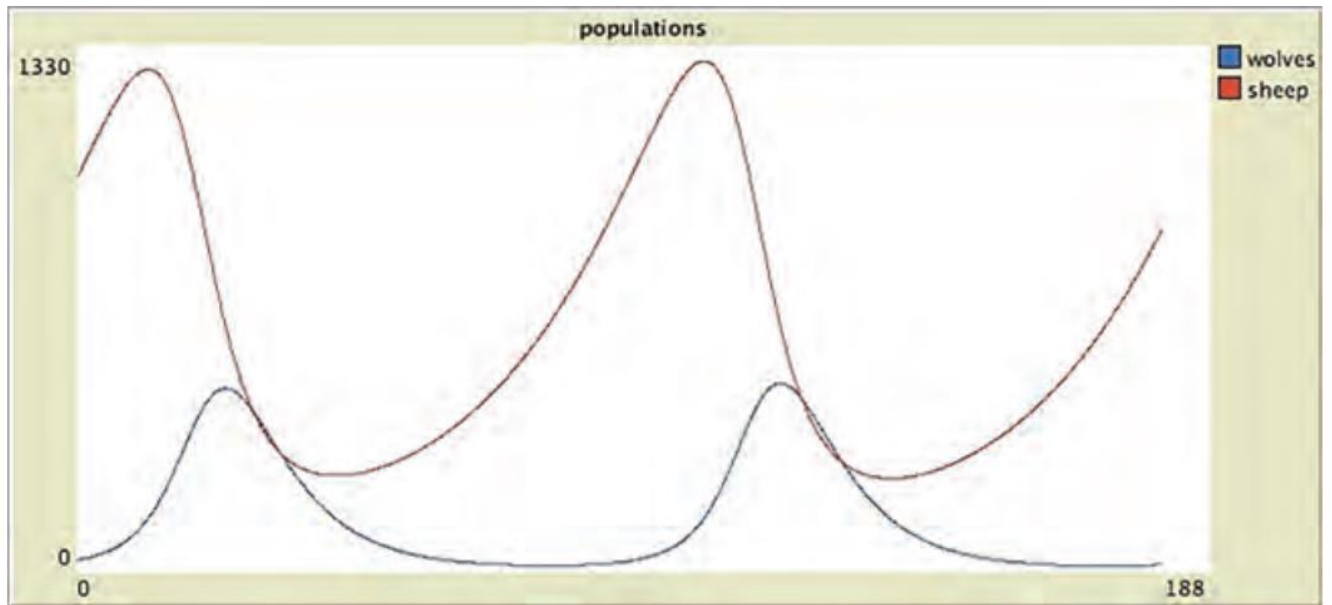


Figure 4.11

Lotka-Volterra relationship (Wolf Sheep Predation System Dynamics model) (Wilensky, 2005).

Depending on the parameter settings, if the two populations are equal at the start, then the predators will begin consuming the prey; this will result in a decrease of the prey population.

Prey and Predator Population

Eventually, the prey will die off to the point where predators cannot find any more prey to eat. This will in turn cause predators to die off because they have no prey to feed on. At this point the prey will be able to reproduce before the predators eat them, and the prey population will increase. As the prey population increases it will become easier for the predator population to find them and the predator population will increase as well.

This, in turn, results in the prey population's decreasing trend, and the whole process begins again.

Lotka-Volterra Equations

The Lotka-Volterra equations have been a standard way of describing the fluctuations in predator and prey populations. Because they are differential equations (see figure 4.12), they represent a continuous model of population change.

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

Figure 4.12

Classic Lotka Volterra differential equations. x is the number of prey, y is the number of predators, and α , β , γ , and δ are parameters describing the interactions of the two species.

But populations are not continuous; they are discrete. Sometimes a continuous approximation of a discrete process is fine, but in this case it may be an oversimplification. The general problem is that as the population of a species becomes very small, standard differential equation models do not allow the population to actually go to 0, which means that there is always a positive probability that the population will rebound. However, this is not what happens in the real world. There is no rebounding from 0.1 prey. If a species goes extinct, it goes extinct, and there is no probability that it will rebound.

“ Nano-sheep Problem ”

This phenomenon is so common in differential equation-based modeling that it is sometimes referred to as the “ nano-sheep problem ” in specific reference to the wolf-sheep/predator-prey model (Wilson, 1998). The problem is that nano-sheep (i.e., a millionth of a sheep) do not exist: Sheep either exist or do not exist, and therefore modeling them with a continuous distribution can be problematic. To rectify the nano-sheep problem, biologists have created agent-based (or to use the term biologists favor, individual-based) models of predator-prey relationships. Under certain conditions these models have been shown to replicate the Lotka-Volterra results, but without the nano-sheep problem. They also predict that this system is in fact highly susceptible to extinctions, something that the Lotka-Volterra equations fail to capture (Wilson et al., 1993 ; Wilson, 1998).

Gause Work

In 1934, Gause showed that indeed for isolated predator and prey (with no other competing species) the agent-based model was more accurate in its predictions — indeed, extinctions happen much more frequently than the Lotka-Volterra equations predict.

Lesson 122 : Advanced Modeling Applications

[click here to download .pptx](#)

Advanced Modeling Applications

More broadly, the modeling of environmental and ecological systems has a long and rich history, representing some of the first applications of an agent-based modeling paradigm (DeAngelis & Gross, 1992). It also continues to be an exciting area for current research.

Modeling of Food Webs

One use of agent-based models in ecological systems is the modeling of food webs (Yoon et al., 2004). This combines ABM with another new complex systems methodology, network analysis (Schmitz & Booth, 1997).

Structure of Ecological Interactions

By understanding both the structure of ecological interactions via network analysis and the process of animal interactions via ABM, researchers gain a deeper insight into overall ecological systems (See, e.g., figure 4.13).

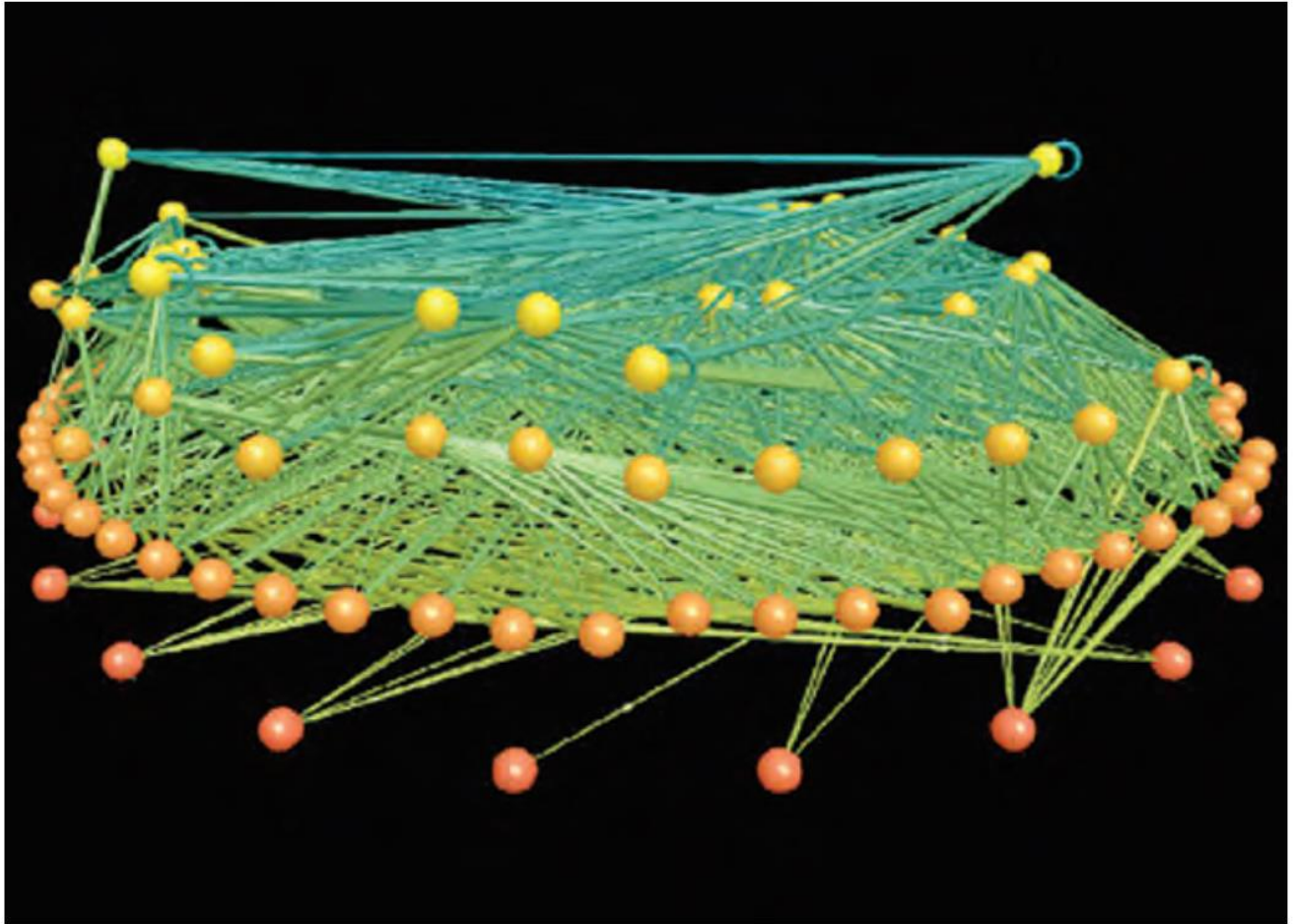


Figure 4.13
Little Rock Lake food web (foodwebs.org, 2006).

ABM and Ecological Modeling

Another particularly interesting application of ABM and ecological modeling has been in doing prescriptive design of engineered systems to ameliorate human interventions in the environment.

Modeling Of Fish Populations

For instance, Weber and his colleagues have done significant modeling of fish populations, and then examined the effect of various fish ladders near dams on salmon populations (Weber et al., 2006).

Ecological models can also be underpinnings of models of evolution. Agent-based modeling has been frequently used to model evolution of organisms (Aktipis, 2004; Gluckmann & Bryson, 2011; Hillis, 1991; Wilensky & Novak, 2010).

ABM Approach as Natural Selection

Evolution lends itself well to the ABM approach as natural selection and other evolutionary mechanisms can be thought of as computational algorithms. Models can be used to try to understand adaptation and speciation in the historical record. Figure 4.14 shows two examples of such models of evolution from the NetLogo models library.

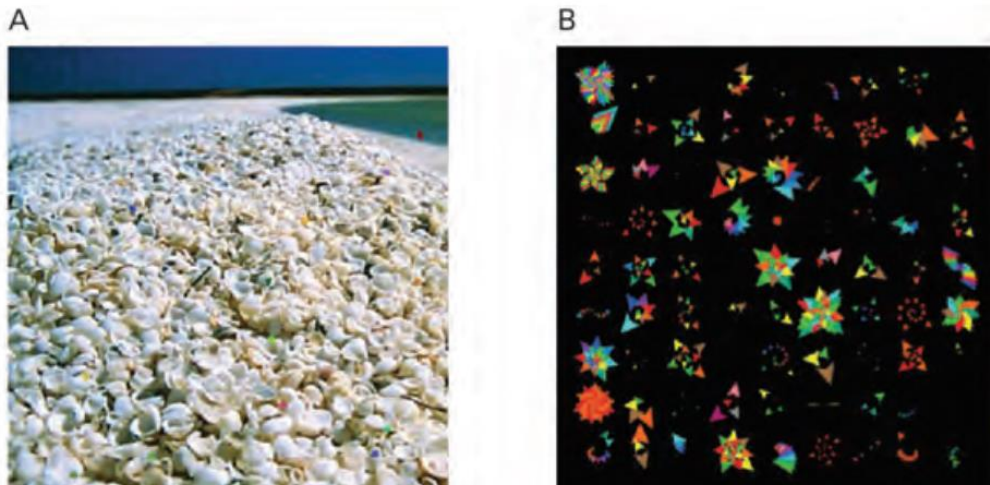


Figure 4.14

Agent-based models of evolution. (A) Evolution of camouflage of insects on a landscape. Different landscapes lead to the bugs evolving colorations that camouflage them in that landscape. (<http://ccl.northwestern.edu/netlogo/models/BugHuntCamouflage>.) (B) Evolution of computational biomorphs. Artificial flowers evolve through mating and blending their characteristics. (<http://ccl.northwestern.edu/netlogo/models/SunflowerBiomorphs>.)

Lesson 123 : The Components of Agent-Based Modeling

[click here to download .pptx](#)

The Components of Agent-Based Modeling

Components of ABM

The main components of any ABM are *agents*, *environment*, and *interactions*. Agents are the basic ontological units of the model, while the environment is the world in which the agent lives.

Distinction Between Agents and Environment

The distinction between agents and environment can be fluid, as the environment can sometimes be modeled as agents. Interactions can occur either between agents or between agents and the environment.

Agent Actions

Agent actions can also occur internally, directly affecting only the agent's internal state. Such is the case when an agent is trying to determine which action to take, as when the residents in the Segregation model are deciding whether or not they are unhappy. The environment is not merely passive; it can also act autonomously. For instance, in the models we discussed in chapter 4, the grass regrows on its own.

Additional Components

To this list of three basic components, we will also add two additional components.

Observer/User Interface

The first is called the Observer/User Interface. The observer is an agent itself,¹ but one that has access to all of the other agents and the environment. The observer asks agents to carry out specific tasks.

User Interface

Users of agent-based models can interact with the agents by way of the User Interface, which enables the user to tell the observer what the model should do. The second component, the Schedule, is what the observer uses to tell the agents when to act.

Schedule

The Schedule often involves user interaction as well. In NetLogo models, the interface typically includes SETUP and GO buttons. The user presses SETUP and then GO to schedule these events to occur.

We will now describe each of these five components in further depth. Throughout this chapter, we will use a variety of models from the NetLogo models library to illustrate our examples. We start by examining the Traffic Basic model, a simple model of traffic flow (found in the Social Science section of the library). An early version of this model was designed by two high school students (Resnick, 1996; Wilensky & Resnick, 1999) to explore how traffic jams form. The students thought that they would have to include traffic accidents, radar traps, or some other form of traffic diversion to create a traffic jam, but they built an initial model without any such hindrances. To their surprise, traffic jams still formed despite the lack of impediments. This happens because as cars speed up and approach the cars in front of them, they eventually have to slow down, causing the cars behind them to slow down creating a ripple effect backward. Eventually, the car at the front of the jam will be able to move again, but by that time, there are many cars behind that car that cannot move, causing the traffic jam to move backward even as the traffic jam moves forward.² This simple model (figure 5.1) again illustrates that emergent phenomena often do not correspond to our intuitions.

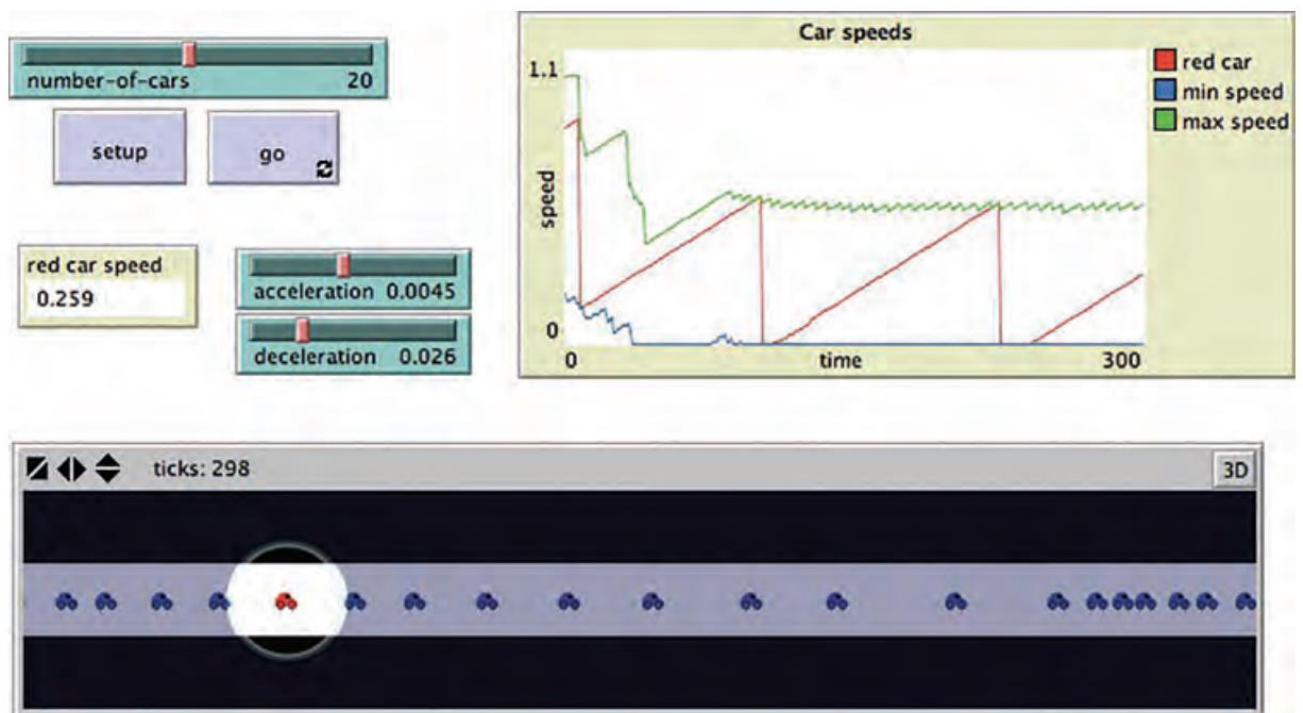


Figure 5.1
Traffic Basic model (Wilensky, 1997b).

Lesson 124 : Agents

[click here to download .pptx](#)

Agents

Agents are the basic units of agent-based modeling. As such, it is important to choose the design of your agents carefully.

Properties and Actions

The two main aspects that define agents are the *properties* that they have and the *actions* (sometimes called behaviors or methods) that they can execute. *Agent properties* are the internal and external state of the agents — their data and description. *Agent behaviors or actions* are what the agents can do. Besides these two main agent attributes, there are also several issues that are related to agent design.

“Grain-size”

First is the issue of agent “grain-size”: which is most effective for the chosen model? For example, if you are modeling a political system, do you want to model the individual actors or, instead, the political institutions or even each nation’s government as a single entity?

Agent Cognition

A second factor to consider is agent cognition. How much capability do the agents have to observe the world around them and make a decision? Do they act in a stimulus-response fashion? Or do they plan out their actions?

Proto-agents and Meta-agents

Finally, we discuss some special types of agents:

- **proto-agents**, which are not fully specified agents; and
- **meta-agents**, composed of other agents.

Lesson 125 : Properties & Behaviors

[click here to download .pptx](#)

Properties

Agent properties describe an agent’s current state, the items that you see when you inspect an agent. Earlier, we briefly described how to use a patch monitor to see the current state of a patch, but you can also use monitors to inspect turtles and other types of agents as well. In this example, we are going to explore agent properties using a turtle monitor, but in NetLogo, you can monitor links and patches as well.

If you inspect one of the cars in Traffic Basic, you see (under the graphical image of the agent’s local environment) the list of properties described in figure 5.2. This list contains two sets of properties. The first set is a standard set of properties of every turtle created in NetLogo: WHO, COLOR, HEADING, XCOR, YCOR, SHAPE, LABEL, LABEL-COLOR, BREED, HIDDEN?, SIZE, PEN-SIZE, and PEN-MODE. Patches and links also have a default set of properties. For patches, these are PXCOR, PYCOR, PCOLOR, PLABEL, and PLABEL-COLOR, while for links, these are END1, END2, COLOR, LABEL, LABEL-COLOR, HIDDEN?, BREED, THICKNESS, SHAPE, and TIE-MODE.

NetLogo

In NetLogo, turtles and links have COLOR as a property, while patches have PCOLOR as a property. Likewise, turtles have XCOR and YCOR as properties, while patches have PXCOR and PYCOR. To make things simpler, turtles can directly access the underlying properties of their current patch. For instance, for a turtle to set its color to the color of the underlying patch (effectively making it invisible), the turtle can just execute SET COLOR PCOLOR. If the patch property had the same name as the turtle property, this would not work, since there would be a confusion over which COLOR the code was describing. Only turtles can access patch properties directly, since a turtle can only be on one and exactly one patch. Links

can span multiple patches and are always connected to multiple turtles, so you have to specify which patch you are referring to if you want to use a patch property in a link procedure. Similarly, you have to specify which link you are referring to when you want to use a link property in a turtle or patch procedure.

Link Properties

Moreover, patches cannot directly access turtle or link properties, because a patch could have 0, 1 or many links or turtles on it. Therefore, you must specify which turtle or link you are referring to when accessing their properties.

In the inspector window, the default properties of the agent appear first. These are

followed by the properties that the model author has specifically added to the agents for the model (e.g., in figure 5.2, SPEED, SPEED-LIMIT, SPEED-MIN). These authordefined properties should be described in the Info Tab associated with the model as well as in the comments in the Code Tab.



Figure 5.2
Agent properties for the Traffic Basic model.

Agent Properties for the Traffic Basic Model

For this model, SPEED describes the current speed of the car, SPEED-LIMIT is the maximum speed of the car and SPEED-MIN, its minimum speed.

When the Traffic Basic model starts up, all of these properties are set in the SETUPCARS procedure:

Lesson 126 : Collections of Agents

[click here to download .pptx](#)

Collections of Agents

Types of Agents

Agents are typically divided into three main types:

mobile agents that can move about the landscape;

stationary agents that cannot move at all; and

connecting agents that link two or more other agents.

NetLogo

In NetLogo, turtles are mobile agents, patches are stationary agents, and links are connecting agents. From a geometric standpoint, turtles are generally treated as shapeless area-less points, although they may be visualized with various shapes and sizes). Thus, even if a turtle appears to be large enough to be on several patches at the same time, it is only contained by the patch at the turtle 's center (given by XCOR and YCOR). Patches are sometimes used to represent a passive environment and are acted upon by the mobile agents; other times, they can take actions and perform operations.

Patches and Turtles

A primary difference between patches and turtles is that patches cannot move. Patches also take up a defined space/area in the world; thus, a single patch can contain multiple turtle agents on it.

Links

Links are also unable to move themselves. They connect two turtles that are the “ end nodes ” of the link, but the visual representations of the links do move when their end nodes move.

Links are often used to represent the relationship between turtles. They can also represent the environment — e.g., by defining transportation routes that agents can move along, or by representing friendship/communication channels. Despite these differences, all three share the ability to behave, that is, to take actions and perform operations on themselves.

Breeds of Agents

Besides these three predefined agent-types, modelers can also create their own types of agent called breeds. The breed of an agent designates the category or class to which the agent belongs. In the Traffic Basic model, all of the agents are of the same type, so it is not necessary to distinguish between different agent breeds. Instead, we used agents of the “ turtles ” breed, the default breed in NetLogo. Different breeds of agents are required if different agents have different properties or actions.

Wolf and sheep

we had two breeds of agents, the “ wolf ” the “ sheep ” breeds. Even though these two breeds had the same set of properties, it was useful to create two breeds because

each had different characteristic actions — wolves eat sheep, while sheep eat grass. We defined the breeds at the beginning of the model:

```
breed [sheep a-sheep]
breed [wolves wolf]
```

At the same time, we can define the properties that breeds have:

```
turtles-own [energy]
```

In this case, both the wolves and the sheep have the same property of “ energy, ” but we could also give them separate properties. For instance, we could give the wolves a “ fangstrength ” property and the sheep a “ wooliness ” property. The fang-strength could be used to determine how successful a wolf is at killing sheep. Wooliness, by contrast, could be an indication of how well the sheep was able to fend off the wolves ’ attack, if we assume that the wolves sometimes get a mouthful of wool instead of flesh. If this were the case, we would add two additional lines:

```
sheep-own [ wooliness ]
wolves-own [ fang-strength ]
```

These properties are in addition to the properties that all agents have, so that the sheep would have “ wooliness ” and “ energy ” just as the wolves have “ fang-strength ” and “ energy. ”

Sets of Agents Breeds are particular collections of agents where the collections are defined by the kinds of properties and actions of their agents. We can also define collections of agents in other ways. NetLogo uses the term *agentset* to designate an unordered collection of agents, and we will also use this term/definition throughout this textbook. Usually we construct agentsets either by collecting agents that have something in common (e.g., agent location or other properties) or by randomly selecting a subset of another agentset.

In the Traffic Basic model, we could create a set of all the agents that have a speed over 0.5. In NetLogo, this is usually done with the `WITH` primitive. Here is an example of how we could create such an agentset that we could insert into the `GO` procedure in this model:

```
let fast-cars turtles with [speed > 0.5]
```

However, we usually want to ask these turtles to do something specific. For instance,

we can ask all of the turtles that have a high speed to set their size bigger so they are easier to see.

```
let fast-cars turtles with [speed > 0.5]
ask fast-cars [
  set size 2.0
]
```

The “let” statement above collects the fast cars into an agentset. If we don’t plan to “talk to” this agentset again, we can do without the `LET` and instead construct the agentset “on the fly” and ask the turtles directly:

```
ask turtles with [speed > 0.5] [
  set size 2.0
]
```

If you insert this code into the model, you will soon see that all of the cars are big. This is because we never told the turtles to set their size to small again, once their speed has dropped below 0.5. To do that we would need to add some more code. One way to accomplish this would be to use another `ask`.

```

ask turtles with [speed > 0.5] [
    set size 2.0
]
ask turtles with [speed <= 0.5] [
    set size 1.0
]

```

Another way to accomplish the same goal, without creating agentsets, is to ask all the turtles to do something, but choose what actions they take based on their properties. In the previous example, all of the faster turtles took their actions before any of the slower turtles took their actions. In the following example, all of the agents are asked with the same ASK command, so the order in which the turtles take actions will be random. In this case the result of running the code will be the same, but depending on what actions the agents take (for instance, if they were to change their speed, instead of their size), the results could be different.

```

ask turtles [
    ifelse speed > 0.5 [
set size 2.0
]
[
    set size 1.0
]
]

```

Another method for creating agentsets is on the basis of their location. The Traffic Basic model does this in the GO procedure:

```

let car-ahead one-of turtles-on patch-ahead 1
    ifelse car-ahead != nobody
        [ slow-down-car ]

```

TURTLES-ON PATCH-AHEAD 1 creates an agentset of all of the cars in the patch in front of the current car. If there is at least one such car, it then selects one of these cars at random, using the ONE-OF primitive, and sets the speed of the current car to a little less than the speed of that car ahead. There are many other ways to access a collection of agents based on their location, such as using NEIGHBORS, TURTLES-AT, TURTLES-HERE, and IN-RADIUS.

Agentsets and Lists Throughout our examples, we have used both agentsets and lists, sometimes explicitly and sometimes implicitly. This may be a good time to stop and clarify what they are, how they work, how they differ, and under what circumstances we would use one over the other. Agentsets and lists are both variables that can contain one or many other variables. We can specify that a variable is an agentset or a list when we create them, by using their respective constructor reporters. If we want to create an empty list, we can write

```
let a-list []
```

If we want, we can then add numbers, strings, and even turtles to the list.

```
;; we put items at the start of the list
set a-list fput 1 a-list
set a-list fput "and" a-list
set alist fput turtle 0 a-list
show a-list
;; prints [(turtle 0) "and" 1]
```

In contrast to lists, which can hold any type of item, an agentset can hold only agents.

Moreover, it can hold only agents of the same type, such as turtles, patches, or links.

NetLogo has special reporters for empty agentsets, no-turtles, no-patches, and no-links.

```
;; this creates an empty agentset of turtles
let an-agentset no-turtles
```

no-turtles is an empty turtle agentset (i.e., an agentset containing no turtles), so by setting an-variable to no-turtles, we specify that this variable is of the type agentset. Now that we have an empty agentset, we can then add turtles to it by using the turtle-set reporter:

```
;; this adds turtle 0 to the agentset
set an-agentset (turtle-set turtle 0)
;; this adds turtle 1 and turtle 2 to the agentset
set an-agentset (turtle-set an-agentset turtle 1 turtle 2)
show an-agentset
;; prints (agentset, 3 turtles)
```

We can only put agents (turtle-breeds, patches and links) in agentsets, but we can put

anything we want, including agents, in lists. There are two important properties that set agentsets and lists apart from each other.

First, we can “ask” agentsets to do things, but we cannot ask lists to do things. So for instance, when we use

```
ask turtles [] ;; do stuff
```

what we are really doing is first creating an agentset of all turtles, and then asking all those turtles, in a random order, to do whatever we put in the brackets. If we try this with lists, we will simply get an error.

Second, agentsets are unordered. This means that whenever we invoke them, they will output a list of agents in a random order. Notice how showing a list shows both what is inside the list and the order in which it appears, but showing an agentset shows only what is inside the agentset.

So when we want to interact with turtles, when would we use one or the other? Unless

we have a very good reason to, we always use agentsets. The primary reason is that we usually want our turtles to do things in a random order. That is because there could be threats to model validity if the agents always execute in the same order. For instance, in the Wolf Sheep Predation model, some sheep would have an unfair advantage if they are always first to check if there is grass on their patches.

But sometimes we do want to determine the order in which turtles do things. For instance, it could be that we want some turtles to have an advantage. In that case we would need to create an ordered list, and order them by whatever parameter we think determines their advantage. If turtles have different information, we might, for example, want the turtles with more information to take their turns later.

```
turtles-own [information] ;; information determines how late they get their turn
;; (later is better because then they will know what everyone else did
;; before making their decision)

;; create an empty list
let a-list []
;; sort-on reports a list containing turtles sorted by the parameter specified
;; in the brackets
set a-list sort-on [information] turtles
```

a-list now contains all turtles sorted by information in ascending order. But, as we just discussed, we cannot ask lists of agents to do things. Rather, we must iterate through the lists, and ask each agent in the list to do what we ask. For this we can use the foreach command:

```
foreach a-list [
  ask ? [ ] ;; do stuff here
]
```

The “ ? ” is a special variable that takes on the value of each element of the list. So this code will iterate through each turtle in the list, and ask each in that particular order, to do what we specify in the brackets.

Similarly we can create lists of patches. Suppose we want to label the patches with numbers in left-to-right, top-to-bottom order. We can create a sorted list of patches and then label them using the code below:

```
;; patches are labeled with numbers in left-to-right,
;; top-to-bottom order
let n 0
foreach sort patches [
  ask ? [
    set plabel n
    set n n + 1
  ]
]
```

Agentsets and Computation Before we end our discussion of agentsets, it should be noted that once you ask an agentset to perform an action, all the agents collected at that moment (and only those agents) will perform the action. If one agent ’ s (agent A) action causes another agent (agent B) in the agentset to no longer satisfy the collection criteria, B will still perform the action. Likewise, if A ’ s action causes agent C, which is not in the agentset, to meet the collection criteria, C will still not perform the action.

Agent-set Ordering model

As an illustration, let ’ s consider this code snippet from the Agentset Ordering model in the chapter 5 subfolder of the IABM Textbook folder in the models library:

```

to setup
  clear-all
  create-turtles 100 [
    set size random-float 2.0
    forward 10
  ]
end

to go
  ask turtles [
    set color blue
  ]
  ask turtles with [ size < 1.0 ] [
    ask one-of turtles with [ size > 1.0 ] [
      set size size - 0.5
    ]
    set color red
  ]
  print count turtles with [ color = red ]
  print count turtles with [ size < 1.0 ]
end

```

The bolded code has the potential to change the set of agents with $\text{size} < 1.0$ by making some agents smaller, thereby changing the agentset defined in the first ASK.

Agent-set Efficiency model

```

;; GO-1 sets red patch labels to a small random number and
;; green patch labels to a larger random number
to go-1
  if any? patches with [ pcolor = red] [
    ask patches with [ pcolor = red] [
      set plabel random 5 ;; red patches are labeled 0-4
    ]
  ]
  if any? patches with [ pcolor = green] [
    ask patches with [ pcolor = green] [
      set plabel 5 + random 5 ;; green patches are labeled 5-9
    ]
  ]
  tick
end

```

This code computes each of the two agentsets PATCHES WITH [PCOLOR = RED] and PATCHES WITH [PCOLOR = GREEN] twice.

;; GO-2 has the same behavior as go-1 above. But it is more efficient as it computes each of the patch agentsets only once.

```

to go-2
  let red-patches patches with [ pcolor = red ]
  let green-patches patches with [ pcolor = green ]
  if any? red-patches [
    ask red-patches [
      set plabel random 5
    ]
  ]
  if any? green-patches [
    ask green-patches [
      set plabel 5 + random 5
    ]
  ]
  tick
end

```

The procedure GO-2 below has the same behavior as GO-1, but it is more efficient in that it computes each of those agentsets only once.

A more pernicious issue arises if we change just one line of GO-1 and GO-2:


```
;; GO-3 shows what happens if patch colors are changed "on the fly"
to go-3
  if any? patches with [ pcolor = red] [
    ask patches with [ pcolor = red] [
      set pcolor green
    ]
  ]
  if any? patches with [ pcolor = green] [
    ask patches with [ pcolor = green] [
      set pcolor red
    ]
  ]
  tick
end
```

If you run this code, you might expect to get a picture that looks like figure 5.4; that is, you expect red patches to turn green and vice versa.

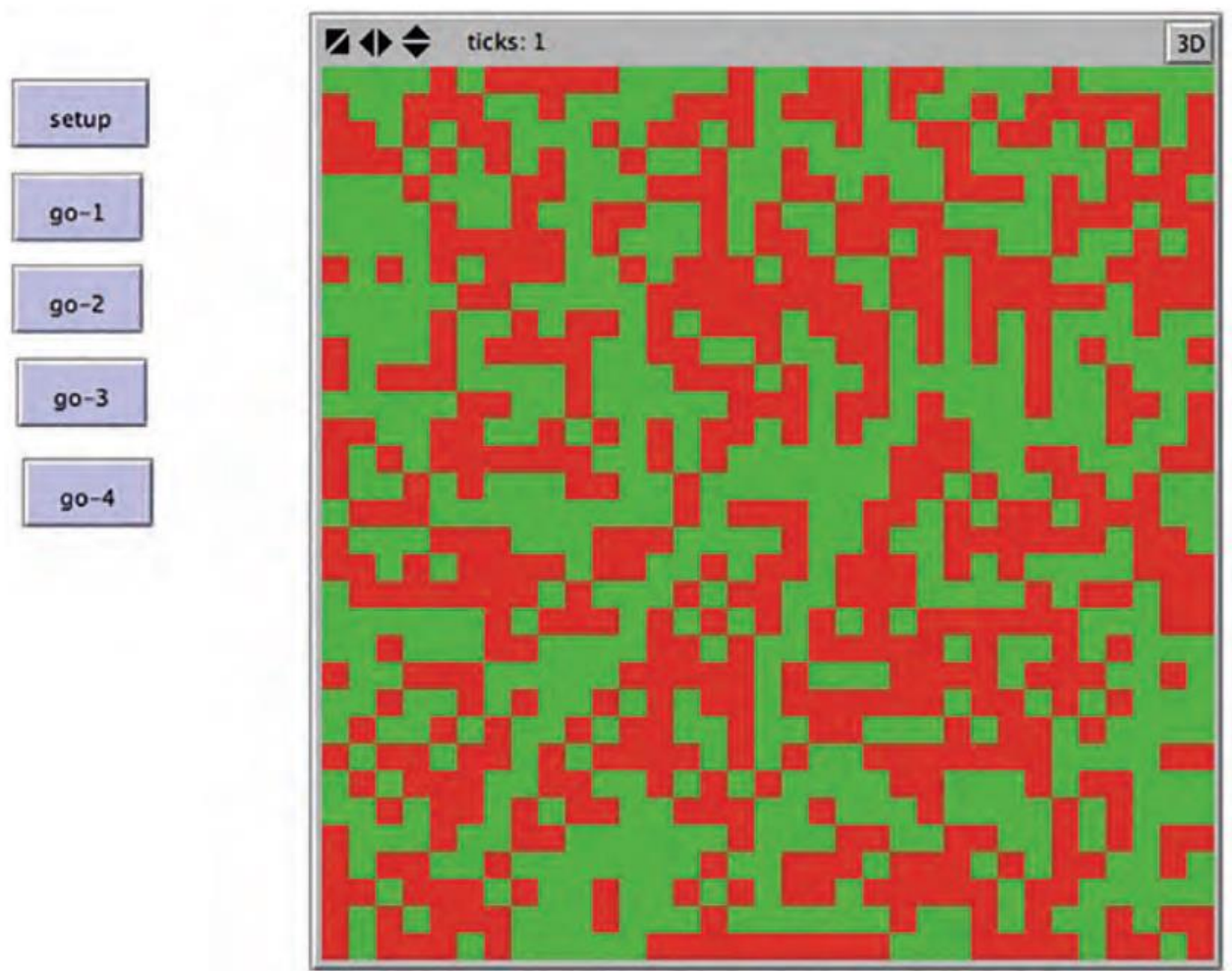


Figure 5.4

Constructing agentsets at the outset results in expected behavior.

In fact, this is not what happens. Instead, this code will result in a picture like figure 5.5, where all the patches are red. Why does this unexpected behavior happen? This is another example of the ordering issue we just discussed above. The first “if statement” turns the patches green, so by the time the second “if statement” is executed, all the patches are green and therefore then turn red.

By computing the agentsets ahead of time, as in GO-4, you are not only using more efficient code but are also ensuring that the green-patches agentsets you ask to execute the instructions are the same green-patches agentsets as at the start of the procedure and not the set of green patches that result from the first “if-statement,” resulting in the expected picture of figure 5.4.

;; GO-4 shows what happens if you keep track, at the outset, of which patches
;; are red and which are green

to go-4

```
    let red-patches patches with [pcolor = red]
    let green-patches patches with [pcolor = green]
    if any? red-patches [
      ask red-patches [
        set pcolor green
      ]
    ]
    if any? green-patches [
      ask green-patches [
        set pcolor red
      ]
    ]
  ]
```

tick

end

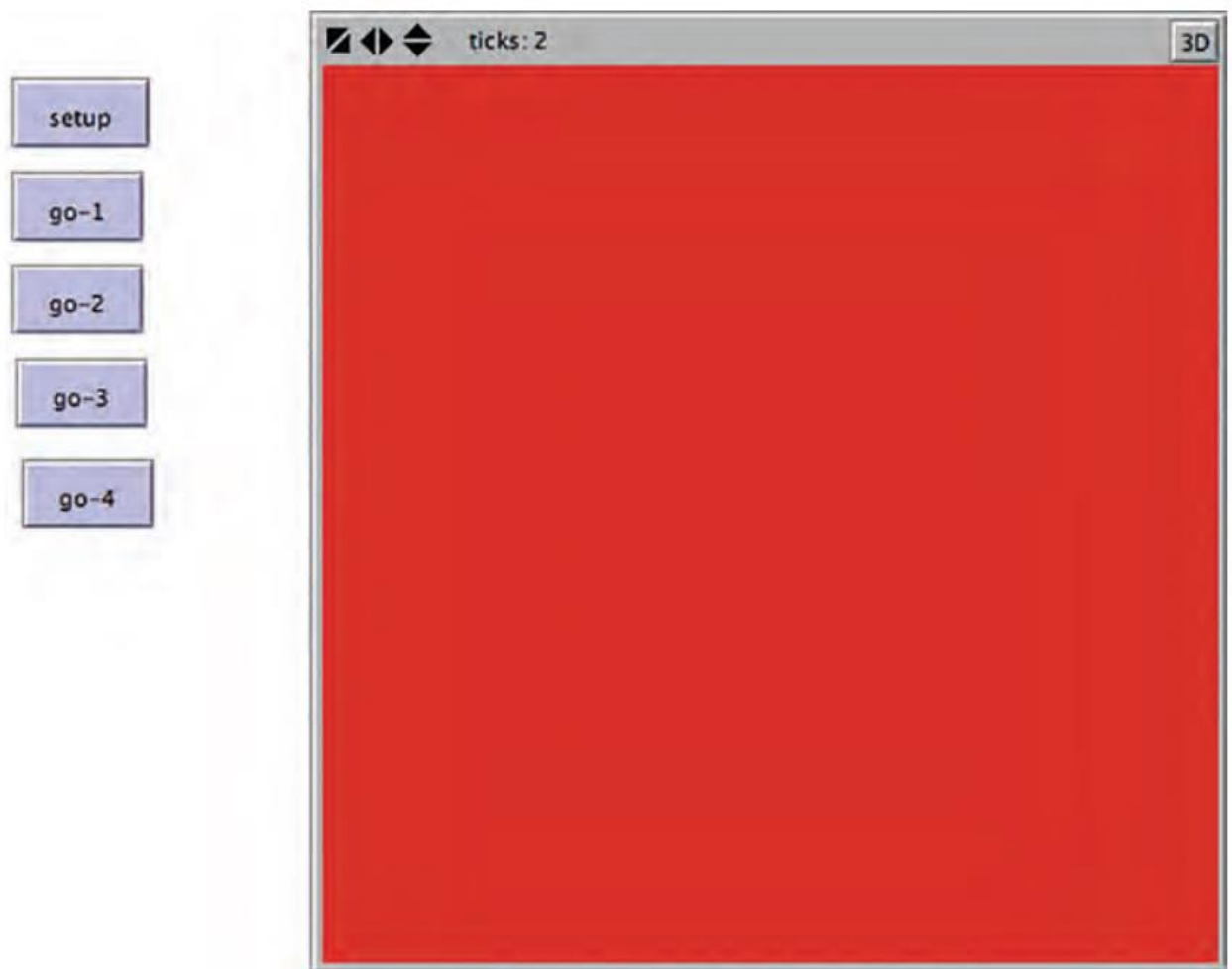


Figure 5.5
Constructing agentsets after the model state has changed, results in unintended outcome.

Lesson 127 : The Granularity of an Agent

[click here to download .pptx](#)

The Granularity of an Agent

One of the first considerations when designing an agent-based model is at what granularity should you create your agent — at what level of complexity is the agent you are modeling.

Level of Complexity for the Agents

So how do you choose the level of complexity for the agents in your model? The guideline is to choose agents such that they represent the fundamental level of interaction that pertains to your question about the phenomenon.

Tumor Model

For instance, if you look at the Tumor model (shown in figure 5.6) in the Biology section of the NetLogo models library, the agents are cells within the human body, since the research question the model examines is how tumor cells spread.

NetLogo Tumor model

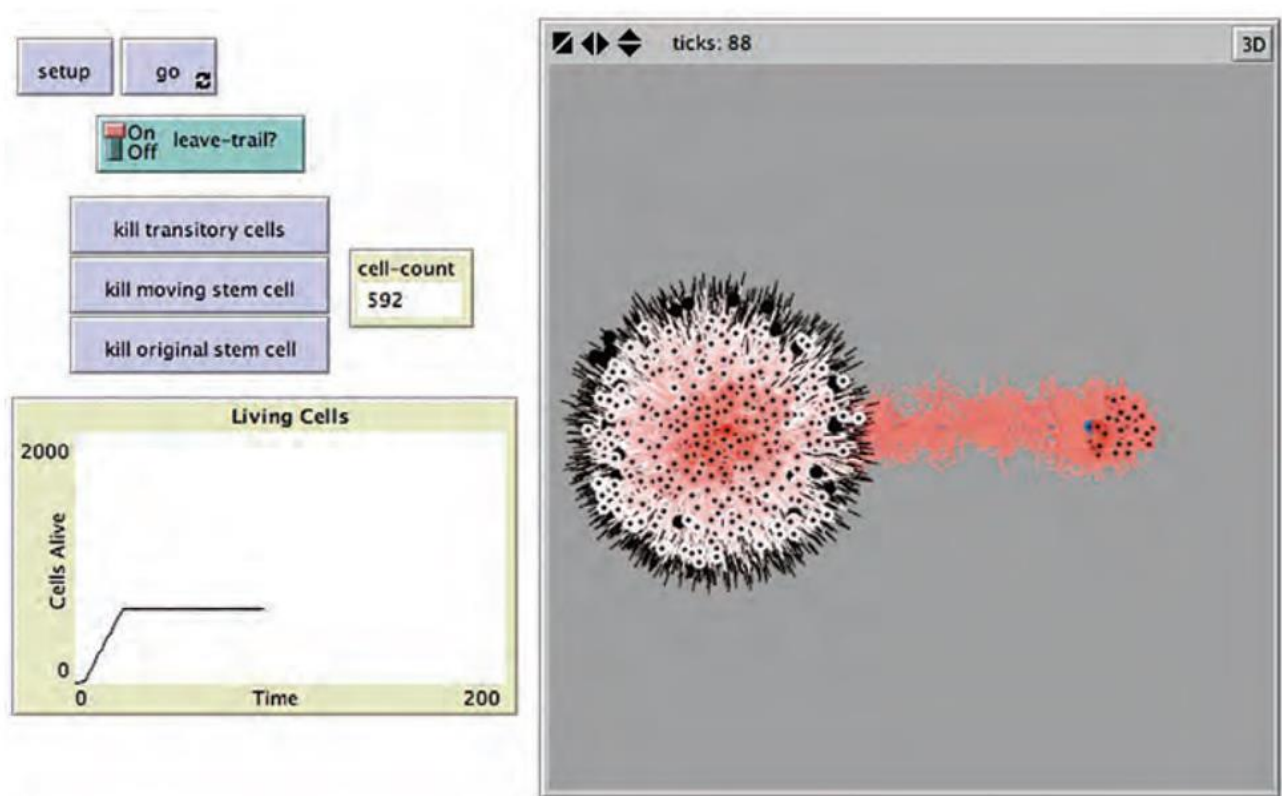


Figure 5.6

NetLogo Tumor model. <http://ccl.northwestern.edu/netlogo/models/Tumor> (Wilensky, 1998b).

However, even though the AIDS model in the Biology section of the NetLogo models library (see figure 5.7) is also concerned with disease, the basic agents are humans instead of cells.

NetLogo AIDS model

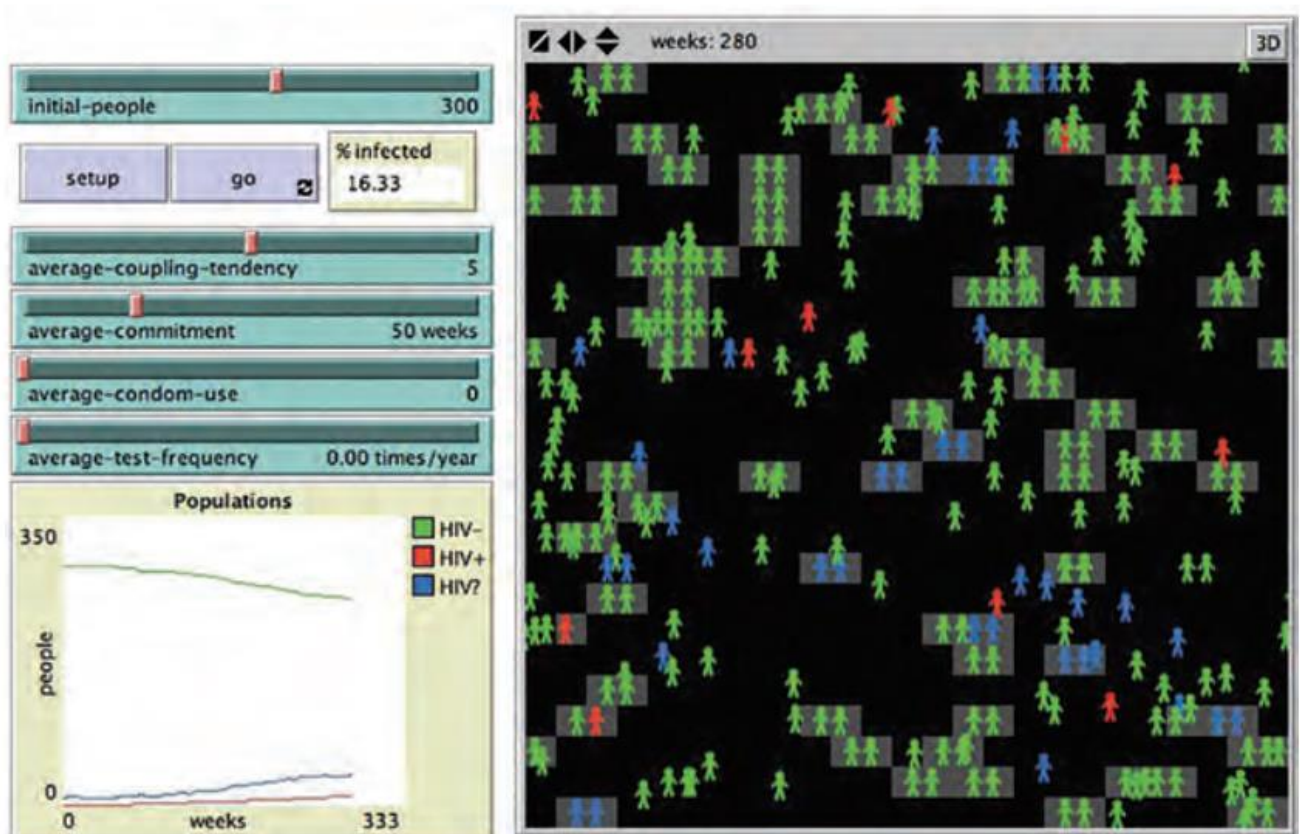


Figure 5.7

NetLogo AIDS model. <http://ccl.northwestern.edu/netlogo/models/AIDS> (Wilensky, 1997a).

Question Of Concern

This is because the AIDS model is more concerned with how the disease spreads between humans instead of how it spreads within the human body. In the Tumor model, the question of concern is how cells interact to create cancer.

Thus, the most important interactions happen at the cell level, and this makes cells a good choice for agents. In the AIDS model, the question of concern is how humans interact to spread disease, with humans as the agents being the most appropriate choice.

Choices Of Granularity

These two models highlight the relationship between the question being investigated, the interactions being modeled, and the resulting choices of granularity for the agents.

Level Of Complexity

Both individual cells and groups of cells can be valid choices for the level of complexity of these agents, and deciding between them will depend on details of the question you are asking as well as considerations of computational complexity.

Lesson 128 : Agent Cognition

[click here to download .pptx](#)

Agent Cognition

As we have described, agents have different properties and behaviors. Still, how do agents examine their properties and the world around them to decide what actions to take? This question is resolved by a decision-making process called agent cognition.

Types Of Agent Cognition

We will discuss several types of agent cognition:

- *reflexive agents*,
- *utility-based agents*,
- *goal-based agents*, and
- *adaptive agents* (Russell & Norvig, 1995).

Often, these types of cognition are thought of in increasing order of complexity, with reflexive agents being the simplest, and adaptive agents the most complex. In reality, though this is not a strict hierarchy of complexity.

Instead, these are descriptive terms that help us talk about agent cognition and can be mixed and matched; for instance, it is possible to have a utility-based adaptive agent.

Reflexive

Reflexive agents are built around very simple rules. They use if-then rules to react to inputs and take actions (Russell & Norvig, 1995). For instance, the cars in the Traffic Basic model are reflexive agents. If you look at the code that controls their actions, it looks like this:

```
let car-ahead one-of turtles-on patch-ahead 1 ;;choose a car on the patch ahead
ifelse car-ahead != nobody [ ;; if there is a car ahead
    slow-down-car car-ahead ;; set car speed to be slower than car ahead
]
[ ;; otherwise, speed up
    speed-up-car ;; increase speed variable by the acceleration
]
;; ...
fd speed
```


Put in words, this code says that if there are cars ahead, then slow down below the speed of the car in front; if there are not any cars ahead, then speed up. This is a reflexive action based on the state of the program, hence the name *reflexive agent*. This is the most basic form of agent cognition (and often a good starting point), but it is possible to make the agent cognition more sophisticated.

Utility-based Form of Agent Cognition

For instance, we could elaborate this model by giving the cars gas tanks and fuel efficiencies based on the speed they are going. We could then have the cars change their speeds in order to improve their fuel efficiencies. As a result, the agents might have to speed up and slow down at different times than they currently do to minimize their gas usage while still not causing accidents. Giving the agents this type of decision-making process would give them a *utility-based* form of agent cognition in which they attempt to maximize a utility function — namely, their fuel efficiency (Russell & Norvig, 1995).

Implementation Of Model Of Agent Cognition

To implement this model of agent cognition, we need to start by replacing our SPEED-UP-CAR procedure with a new procedure that accounts for the car 's fuel efficiency.

```
;; choose a car on the patch ahead
let car-ahead one-of turtles-on patch-ahead 1
ifelse car-ahead != nobody [ ;; if there is a car ahead
    slow-down-car car-ahead ;; set car speed to be slower than car ahead
]
[ ;; otherwise, adjust speed to find ideal fuel efficiency
  adjust-speed-for-efficiency ]
```

We want the ADJUST-SPEED-FOR-EFFICIENCY procedure to maintain the same speed if the car is at the maximally fuel efficient speed. If it is not at its most efficient speed, the logic in this procedure should have the car speed up if it is moving too slow and slow down if the car is moving too fast. Additionally, the car would still need to slow down if it was about to crash into the car in front of it. As such, the true utility function includes an exception that gives a utility of 0 to any action that results in a crash. As a result, we can leave the first part of the code that slows the car before it crashes and just add ADJUST-SPEED-FOR-EFFICIENCY when there is no car directly ahead, as illustrated in the code below from the Traffic Basic Utility model, of the NetLogo models library.


```

;; car procedure
to adjust-speed-for-efficiency
  if (speed != efficient-speed) [ ;; if car is at efficient speed, do nothing
    if (speed + acceleration < efficient-speed) [
      ;; if accelerating will still put you below the efficient speed then accelerate
      set speed speed + acceleration
    ]
    ;;
    ;; if decelerating will still put you above the efficient speed then decelerate
    if (speed - deceleration > efficient-speed) [
      set speed speed - deceleration
    ]
  ]
end

```

Utility Functions

In the language of utility functions, each car agent is minimizing a function f ,

defined by:

$$f(v) = |v - v^*|$$

where v is the current velocity of the car and v^* is the most efficient velocity. The constraint that is described in the code is that the v cannot be adjusted arbitrarily, but instead, can only be changed by a limited increment every time step. By trying different values for EFFICIENT-SPEED, you can obtain quite different traffic patterns from the original model. For low values of EFFICIENT-SPEED, the system can achieve a free-flow state (with no jams) on a consistent basis.

Goal-based Agents

The third type of agent cognition we will discuss is goal-based cognition. Imagine that each car in the Traffic Grid model (see figure 5.8) from the social sciences section of the NetLogo models library has a home and a place of work, with the goal of moving from home to work in a reasonable amount of time. Now, not only do the agents have to be able to speed up and slow down, but they also have to be able to turn left and right. In this version of the model, the cars are **goal-based agents**, since they have a goal (getting to and from work) that they are using to dictate their actions.

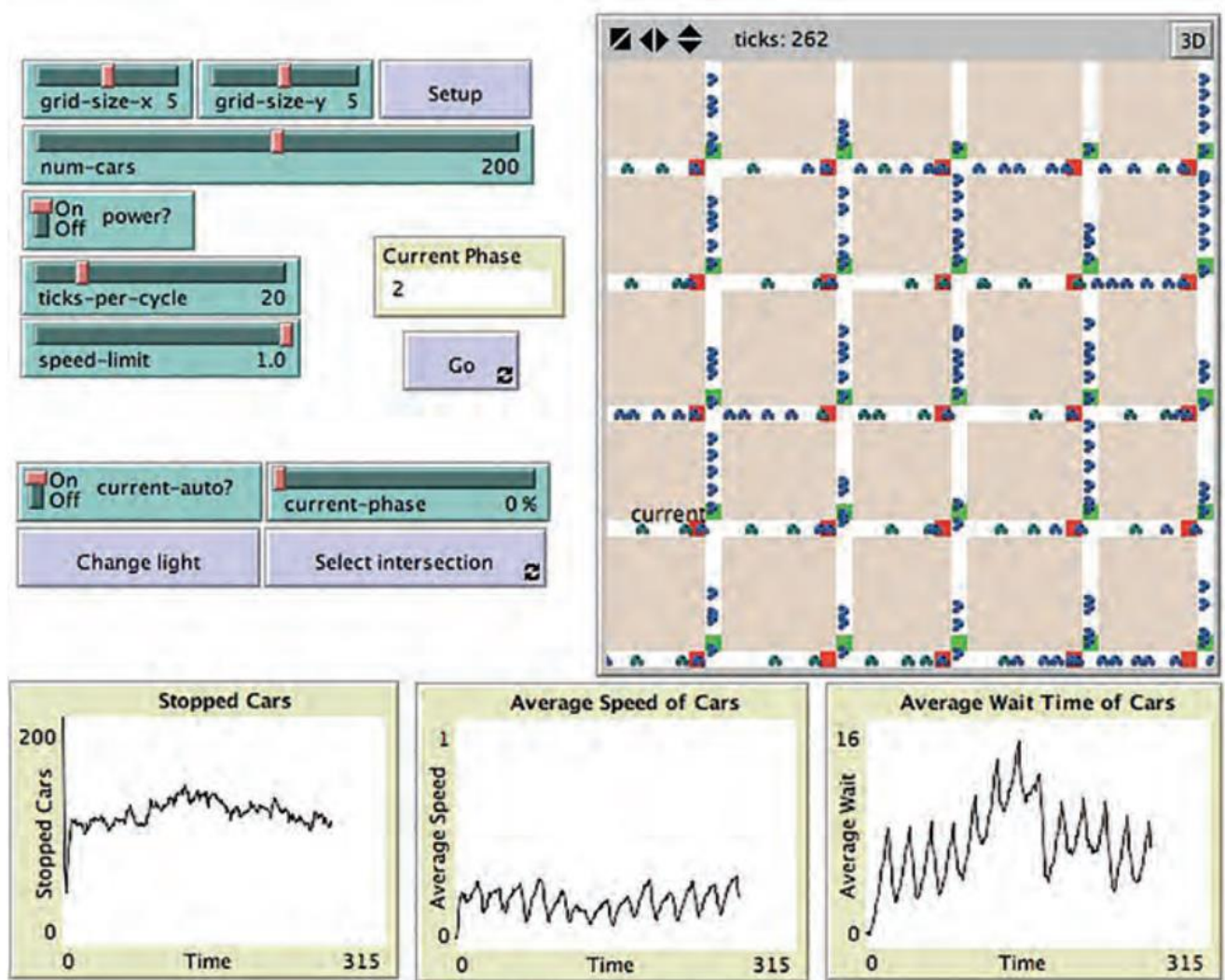


Figure 5.8

Traffic Grid model. <http://ccl.northwestern.edu/netlogo/models/TrafficGrid> (Wilensky, 2002b).

Implementing The Goal-based Version of the Model

To implement the goal-based version of the model, we start by defining the procedure that the cars will use to navigate between their two desired destinations. Instead of just moving straight all the time, as with the cars in the original model, the new model should have the car at each step deciding which of its neighboring road patches is the closest to its destination, subsequently proceeding in that direction. We do this in the procedure called NEXT-PATCH. First, each car checks if it has arrived at its goal, and if so, switches its goal:

```

;; if I am going home and I am on the patch that is my home
;; I turn around and head towards work (my goal is set to "work")
    if goal = house and patch-here = house [
        set goal work
    ]
;; if I am going to work and I am on the patch that is my work
;; I turn around and head towards home (my goal is set to "home")
    if goal = work and patch-here = work [
        set goal house
    ]

```

The code above doesn't quite work. That's because the house and work are off-road and the cars remain on the road, so the condition "patch-here = house" or "patch-here = work" will never be satisfied. As we placed the house and work adjacent to the road, we can fix this by having the car check if it is right next to its goal.

```

;; if I am going home and I am next to the patch that is my home
;; I turn around and head towards work (my goal is set to "work")
if goal = house and (member? patch-here [neighbors4] of house) [
    stay set goal work
]
;; if I am going to work and I am next to the patch that is my work
;; I turn around and head towards home (my goal is set to "home")
    if goal = work and (member? patch-here [neighbors4] of work) [
        stay set goal house
    ]

```

Having established a goal, each car then chooses an adjacent patch to move to that is the closest to its goal. It does this by choosing candidate patches to move to (adjacent patches on the road) and then selecting the candidate closest to its goal.

The resultant NEXT-PATCH procedure is:

```

;; establish goal of driver
to-report next-patch
;; if I am going home and I am next to the patch that is my home
;; I turn around and head towards work (my goal is set to "work")
if goal = house and (member? patch-here [neighbors4] of house) [
    stay set goal work
]
;; if I am going to work and I am next to the patch that is my work
;; I turn around and head towards home (my goal is set to "home")
if goal = work and (member? patch-here [neighbors4] of work) [
    stay set goal house
]
;; CHOICES is an agentset of the candidate patches which the car can
;; move to (white patches are roads, green and red patches are lights)
let choices neighbors with [pcolor = white or pcolor = red or pcolor = green]
    ;; choose the patch closest to the goal, this is the patch the car will move to
    let choice min-one-of choices [distance [goal] of myself]
;; report the chosen patch
report choice
end

```

Adaptive Agent

One of the powerful advantages of agent-based modeling is that agents can change not only their decisions but also their strategies (Holland, 1996).

Adaptive Agent: An agent that can change its strategy based on prior experience is an *adaptive agent*. Unlike conventional agents, which will also do the same thing when presented with the same circumstance, adaptive agents can make different decisions if given the same set of inputs.

Traffic Basic model

In the Traffic Basic model, cars do not always take the same action; they operate differently based on the cars around them by either slowing down or speeding up. However, regardless of what has happened to the cars in the past (i.e., whether they got stuck in traffic jams or not) they will continue to take the same actions in the same conditions in the future. To be truly adaptive, agents need to be able to change not only their actions in time, but also their strategies.

They must be able to change how they act because they have encountered a similar situation in the past and can react differently this time based on their past experience.

In other words, the agents learn from their past experience and change their behavior in the future to account for this learning.

Example Of Adaptive Cognition

For instance, the agent could have observed that in the past that, when there was a car five patches ahead of it, it braked too quickly, even though it could have waited longer to brake without hitting the other car.

In the future, it could change its rule for braking to brake only when a car is four patches ahead. This agent is now an **adaptive agent** because it not only modifies its **actions**, but also, its **strategies**.

Lesson 129 : Other Kinds of Agents

[click here to download .pptx](#)

Other Kinds of Agents

We have already discussed breeds and various types of agents, but there are two other special kinds of agents that deserve at least a brief mention.

- The first type are **meta-agents**, *agents made up of other agents*;
- The second is **proto-agents**, *placeholder agents that allow you to define interactions for your fully defined agents with other entities that have not been fully developed*.

Meta-Agents Many things that we would consider agents in reality are actually composed of other agents. In reality, all “agents” are composed of other agents; to cite the oft-told parable, “It’s turtles all the way down.” In other words, there is always a lower level of detail that you can use to describe an agent in your model, at least until you reach the most basic level so far described by physics. For instance, if we have selected a human to be the agent, he or she is actually composed of many subagents, with these subagents in turn being different depending on how you view the human. You could view the subagents of a human as the systems of the body — e.g., the immune and the respiratory systems — or you could view the subagents of a human as psychological aspects like the intellect and emotion. Moreover, these subagents are not the most basic level; in our example, the systems of the body are made up of organs, tissues, and cells.

Meta-agents and Sub-agents

At each of these levels, we are describing the relationship between **meta-agents** (*agents composed of other agents*) and **subagents** (*agents which compose other agents*). Still, agents can also be both meta-agents and subagents at the same time. For instance, organs are composed of cells and are meta-agents. However, they also play a compositional role in the systems of the human body and are thus subagents.

We can use meta-agents in our ABMs by defining the subagents that make up our agents and providing them with their own actions and properties.

Meta-agent Appears to be a Single Agent

From the perspective of another meta-agent, a meta-agent appears to be a single agent. If a person meets another person, they do not (usually) directly interface with the heart or the lungs (unless the meeting takes place at the surgical table). Instead, they interface with the person as a whole.

Modeling Agents and Their Interactions

When we think of modeling agents and their interactions, we have to determine what level of granularity we want to describe the agent behaviors. However, we always have the option of refining our model by converting agents into meta-agents that describe the agents that constitute other agents. Sometimes, it can be useful to represent the agents in our models not as autonomous individuals, but instead, as meta-agents composed of other agents. NetLogo doesn't include explicit language support for these meta-agents, though there are commands for locking together the movement of several agents (e.g., the TIE command) that may be useful in some circumstances. However, there is nothing to prevent you from designing models that have groups of agents representing single agents.

Proto-Agents

To truly be an agent within the agent-based modeling framework, **an entity must have its own properties or actions.** Sometimes, though, it can be desirable to create agents that, rather than having their own properties or behaviors, are instead placeholders for future agents. We call these agents **proto-agents**, and their primary purpose is to enable us to specify how other agents would interact with them if they

were fleshed out into full agents.

Example of Proto-agent

For instance, if you were creating a model of residential location decision making, you would have residents as agents; however, since where a resident lives is greatly influenced by places of employment and services (e.g., grocery stores and restaurants) you might also want to include these “service centers” as agents as well. The same level of detail necessary for the residents, who are the focal agents of concern, may not be necessary for the service centers. Instead, they might be rendered as placeholders to represent where residents might potentially find jobs and transact business. However, as you continue to refine the model, you might give the service centers additional decision-making abilities. For instance, they might have a more elaborate model of market demand and decide where to locate using their own properties and beliefs about the future growth of the world. Keeping these agents as proto-agents early on means that when you add in the more richly detailed versions to the model, you do not have to go back and revise your resident agents. Instead, you can use the interaction events that they had already been using to interface with the service center proto-agents.

Lesson 130 : Environments

[click here to download .pptx](#)

Environments

Another early and critical decision is how to design the *environment* of the agent-based model. The environment consists of the conditions and habitats surrounding the agents as they act and interact within the model. The environment can affect agent decisions, and, in turn, can be affected by agent decisions.

Environment Can Affect Agent

For instance, in the Ants model from chapter 1, the ants leave pheromone in the environment that changes the environment, and in turn changes the behavior of the ants. There are many different kinds of environments that are common in ABMs. In this section, we will discuss a few of the most common types of environments.

Implementation of an Environment

Before we discuss the types of environments, it is important to mention that the environment itself can be implemented in a variety of ways. First, the environment can be composed of agents such that each individual piece of the environment can have a full set of properties and actions.

NetLogo

In NetLogo, this is the default view of the environment — the environment is represented by the agentset of patches. This allows different parts of the environment to have different properties and act differently based on their local interactions.

Lesson 131 : Spatial Environments

[Click here to download .pptx](#)

Spatial Environments

Spatial environments in agent-based models generally have two variants:

- discrete spaces and
- Continuous spaces.

Continuous Spaces: In a mathematical representation, in continuous spaces between any pair of points, there exists another point,

Discrete Spaces: While in discrete spaces, though each point has a neighboring point, there do exist pairs of points without other points between them, so that each point is separated from every other point.

ABM Implementation

However, when implemented in an ABM, all continuous spaces must be implemented as approximations, so that continuous spaces are represented as discrete spaces where the spaces between the points are very small. It should be noted that both discrete and continuous spaces can be either finite or infinite.

Discrete Spaces: The most common discrete spaces used in ABM are *lattice graphs* (also sometimes referred to as *mesh graphs* or *grid graphs*), which are environments where every location in the environment is connected to other locations in a regular grid.

Toroidal Square Lattice

For instance, every location in a toroidal square lattice has a neighboring location up, down, to the left, and to the right. As mentioned, the most common representation of the environment in NetLogo is patches, which are located on a 2D lattice underlying the world of the ABM (see figure 5.9 for a colorful pattern of patches whose code is simply `ASK PATCHES [ SET PCOLOR PXCOR * PYCOR ]`). This uniform connectivity makes them different from network-based environments.

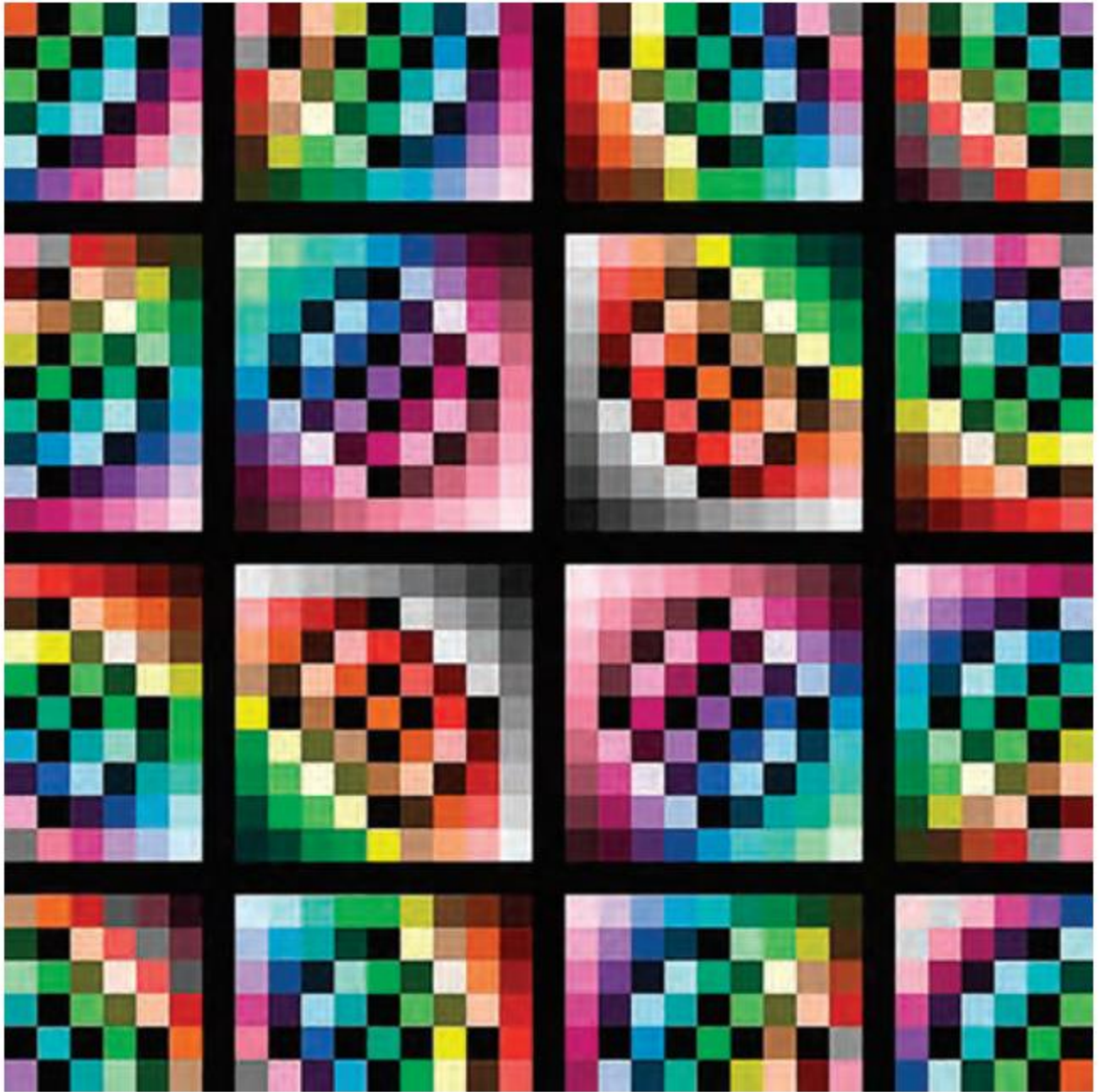


Figure 5.9
Patches displaying a range of colors.

The two most common types of lattices are:

- Square Lattices
- Hexagonal Lattices

Square Lattices The square lattice is the most common type of ABM environment. A square lattice is one composed of many little squares, akin to the grid paper used in mathematics classrooms.

There are two classical types of neighborhoods on a square lattice:

- The *von Neumann neighborhood*, consisting of four neighbors located in the cardinal directions (see figure 5.10a); and
- The *Moore neighborhood*, comprising the 8 adjacent cells (See figure 5.10b).

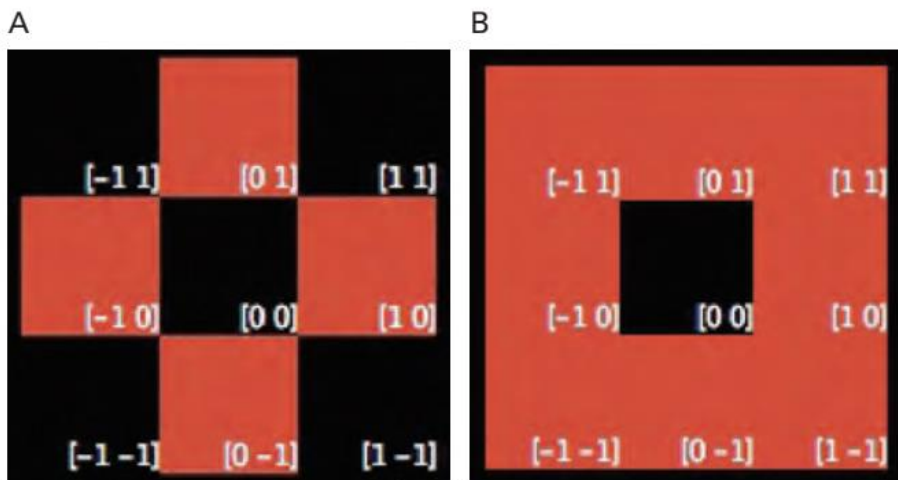


Figure 5.10

(a) Von Neumann Neighborhood. (b) Moore Neighborhood. The black cell in the center is the focal cell, and the red cells comprise its neighborhood.

Von Neumann Neighborhood

A von Neumann neighborhood (named after John von Neumann, a pioneer of cellular automata theory among other things) of radius 1 is a lattice where each cell has four neighbors: up, down, left, and right.

Moore Neighborhood

A Moore neighborhood (named after Edward F. Moore, another pioneer of cellular automata theory) of radius 1 is a lattice in which each cell has eight neighbors in the eight directions that touch either a side or a corner: up, down, left, right, up-left, up-right, down-left, and down-right. In general, a Moore neighborhood gives you a better approximation to movement in a plane, and since many ABMs model phenomenon where planar movement is common, it is often the preferred modeling choice for discrete motion.

Hex Lattices: A hex lattice has some advantages over square lattices. The center of a cell in a square grid is farther from the centers of some of the adjacent cells (the diagonally adjacent ones) than other such

cells. However, in a hex lattice, the distance between the center of a cell and all adjacent cells is the same. Moreover, hexagons are the polygons with the most edges that tile the plane, and for some applications, this makes them the best polygons to use. Both of these differences (equidistance between centers and the number of edges) mean that hex lattices more closely approximate a continuous plane than square lattices. But because square lattices match more closely a Cartesian coordinate system, a square lattice is a simpler structure to work with; even when a hexagonal lattice would be superior, many ABMs and ABM toolkits nevertheless employ square lattices. However, with a little effort, any modern ABM environment can simulate a hexagonal lattice in a square lattice environment. For instance, you can see a hex lattice environment in the NetLogo Code examples, Hex Cells example, and Hex Turtles example (see figures 5.11 and 5.12).

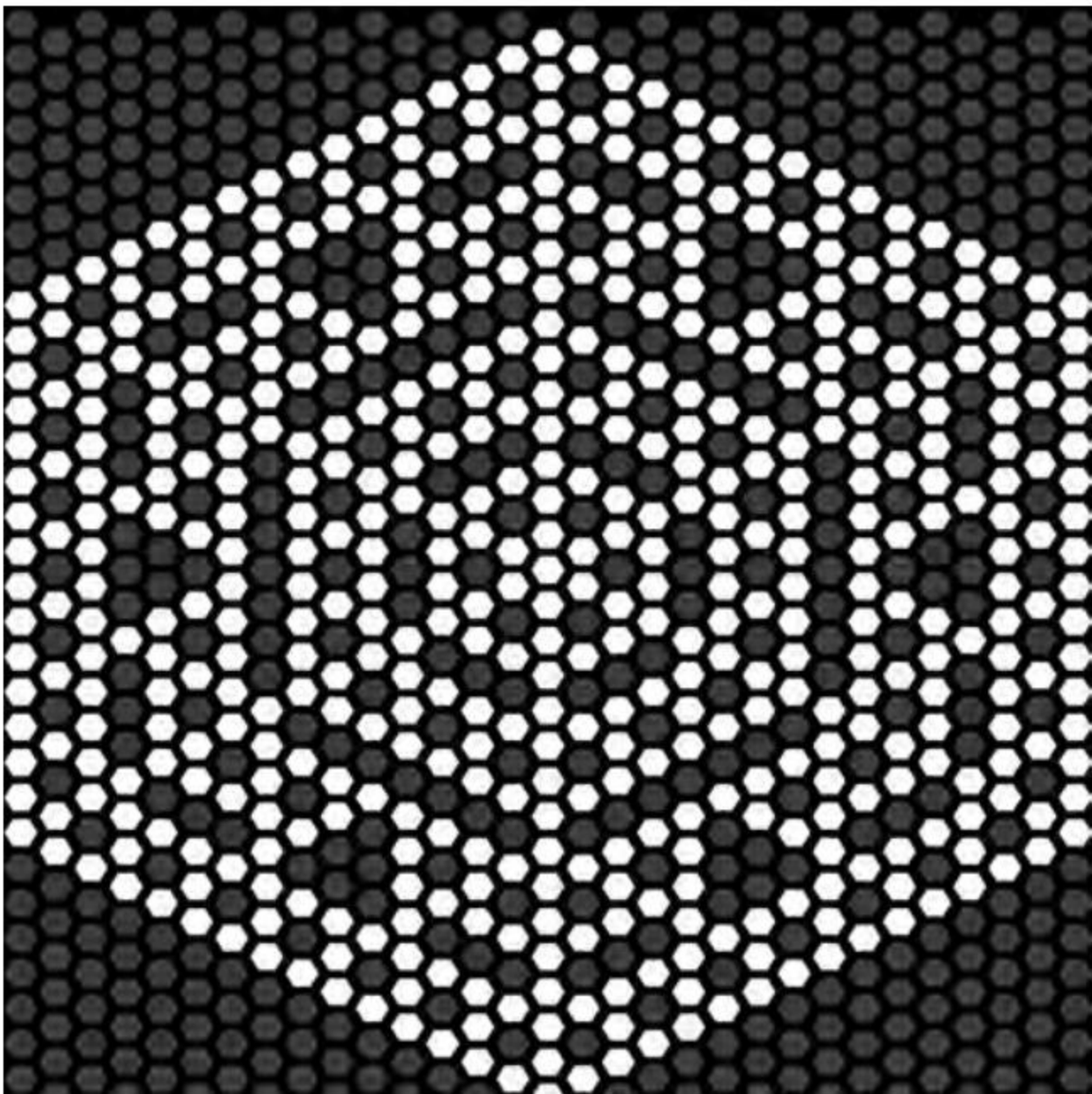


Figure 5.11

Hex Cells example from the NetLogo models library.

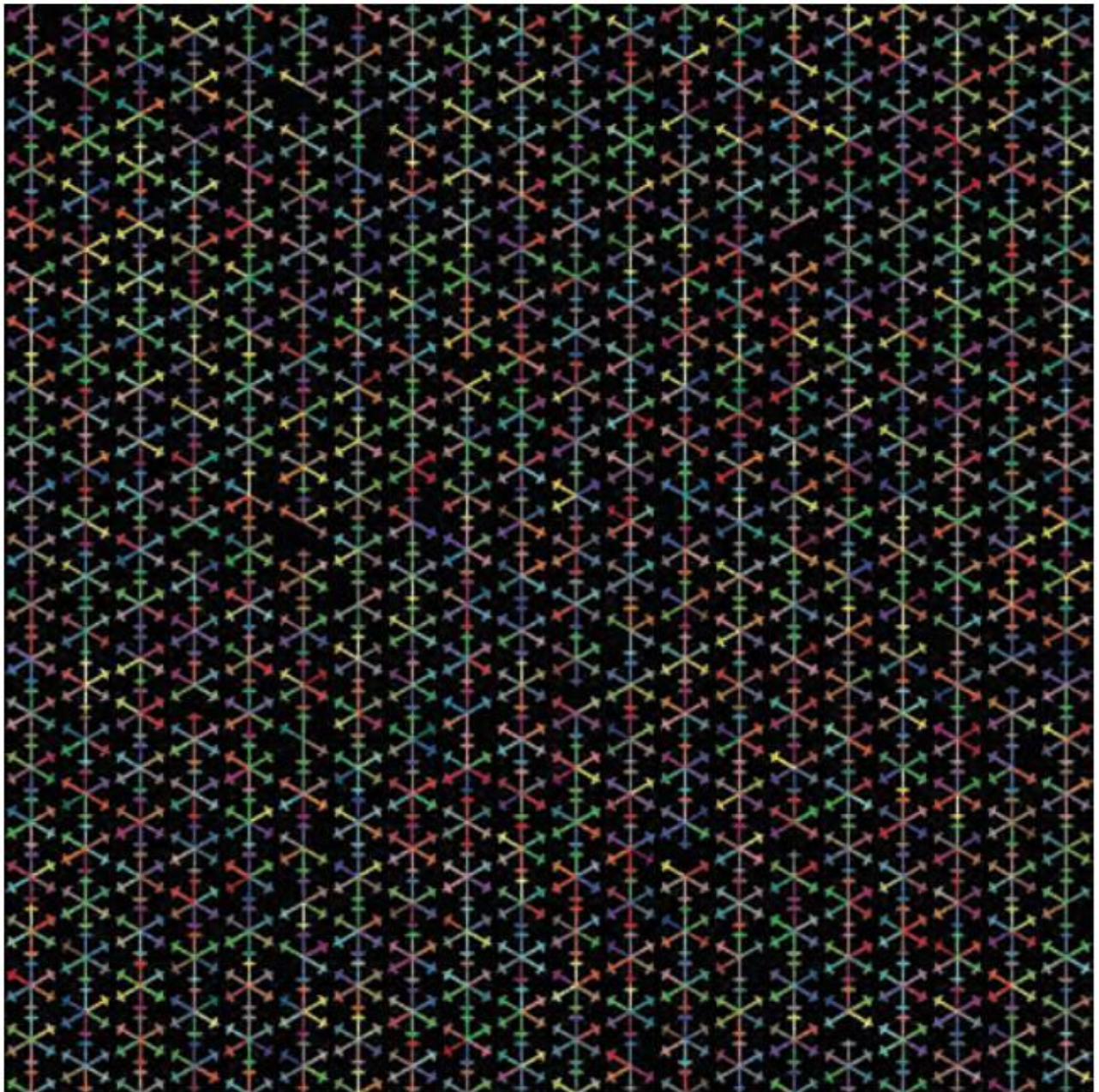


Figure 5.12

Hex Turtles example from the NetLogo models library.

In the Hex Cells example, each patch in the world maintains a set of six neighbors, with the agents located on each patch having a hexagonal shape. In other words, the world is still rectangular, but we have defined a new set of neighbors for each patch. In the Hex Turtles example, the turtles have an arrow shape and start along headings that are evenly divisible by 60 degrees; when they move they only move along these same 60-degree angles.

Continuous Spaces

- In a continuous space, there is no notion of a cell or a discrete region within the space. Instead, agents within the space are located at points in the space. These points can be as small as the resolution of the number representation allows.
- In a continuous space, agents can move smoothly through the space, passing over any points in between their origin and destination, whereas in a discrete space, the agent moves directly from the center of one cell to another.
- Because computers are discrete machines, it is impossible to exactly represent a continuous space. We can, however, represent it at a level of resolution that is very fine.
- In other words, all ABMs that use continuous spaces are actually using a very detailed discrete space, while the resolution is usually high enough that to suffice for most purposes.

Boundary Conditions One other factor that comes into play when working with spatial environments is how to deal with boundaries, an issue for Hex and Square lattices as much as for continuous spaces. If an agent reaches a border on the far left side of the world and wants to go farther left, what happens?

Three Standard Approaches

There are three standard approaches to this question, referred to as *topologies* of the environment:

- (1) it reappears on the far right side of the lattice (toroidal topology);
- (2) it cannot go any farther left (bounded topology); or
- (3) it can keep going left forever (infinite plane topology).

Toroidal Topology

A *toroidal* topology is one in which all of the edges are connected to another edge in a regular manner. In a rectangular lattice, the left side of the world is connected to the right side, while the top of the world is connected to the bottom. Thus, when agents move off the world in one direction, they reappear at the opposite side. This is sometimes called *wrapping*, because the agents wrap off one side of the world and onto another.

In general, using a toroidal topology means that the modeler can ignore boundary conditions, which usually makes model development easier. If the world is nontoroidal, then modelers have to develop special rules to handle what to do when an agent encounters a boundary in the world. In a spatial model, this conflict is one of whether the agent should turn around simply take a step backward. Indeed, the most commonly used environment for an ABM is a square toroidal lattice.

Bounded Topology

A *bounded* topology is one in which agents are not allowed to move beyond the edges of the world. This topology is a more realistic representation of some environments. For example, if you are modeling

agricultural practices, it is unrealistic for a farmer to be able to keep driving the tractor east and then end up back on the west side of the field. Using a torus environment may affect the amount of fuel required to plow the fields, so a bounded topology might be a better choice depending on the questions the model is trying to address. It is also possible to have some of the limits of the world be bounded while others are wrapping. For instance, in the Traffic Basic model, where the cars only drive from left to right, the top and bottom of the world are bounded.

Infinite-plane Topology

Finally, an *infinite-plane* topology is one where there are no bounds. In other words, agents can keep moving and moving in any direction forever. In practice, this is done by starting with a smaller world such that whenever agents move beyond the edges of the world, the world is expanded. At times, infinite planes can be useful if the agents truly need to move in a much larger world. While some ABM toolkits provide built-in support for infinite plane topologies, NetLogo does not. However, it is possible to work around this limitation by giving each turtle a separate pair of x and y coordinates from the built-in ones. Then, when an agent moves off the side of the world, we can hide the turtle and keep updating this additional set of coordinates until the agent moves back onto the view. In most cases, however, a toroidal or bounded topology will be the more appropriate (and simpler) choice to implement.

Lesson 132 : Network-Based Environments

[click here to download .pptx](#)

Network-Based Environments

Link: A *link* is defined by the two ends it connects, which are frequently referred to as *nodes*. However, we will use the network/node/link vocabulary throughout this module.

In NetLogo, links are their own agent-type.

Lattice networks

In fact, lattice graphs can also be called *lattice networks*, with the property that each position in the network looks exactly like every other position in the network. However, ABM environments usually do not implement lattice environments as networks for both conceptual and efficiency reasons. Additionally, using patches as the default topology allows for either discrete or continuous representations of the space, whereas a network is always discrete.

Random Networks

Network-based environments have been found useful in studying a wide variety of phenomena, such as the spread of disease or rumors, the formation of social groups, the structure of organizations, or even the structure of proteins. There are several network topologies that are commonly used in ABM. Besides the regular networks described earlier, the three most common network topologies are *random*, *scale-free*, and *small-world*.

In *random networks*, each individual is randomly connected to other individuals. These networks are created by randomly adding links between agents in the system. For example, if you had a model of agents moving around a large room and connected every agent in the room to another agent based on which agent had the next largest last two digits of their social security number, you would probably create a random network. We show one simple methods for creating a random network. This code is also in the Random Network model in the chapter 5 subfolder of the IABM Textbook folder of the NetLogo models library

Code for creating a random network


```
;; Create nodes.
to setup
  clear-all
  create-turtles num-nodes [
    set shape "circle"
    setxy random-xcor random-ycor
  ]
  reset-ticks
end

;; Ask each node to create a link with a random other node.
;; Results in "just under" NUM-NODES links because no new link is created
;; if a node tries to connect to a node with which it is already linked.
to wire1
  ask links [ die ]
  ask turtles [
    create-link-with one-of other turtles
  ]
end

;; Pick a random node and ask it to create a link with a random other node.
;; Results in "just under" NUM-NODES links because no new link is created
;; if a node tries to connect to a node with which it is already linked.
to wire2
  ask links [ die ]
  repeat num-nodes [
    ask one-of turtles [
      create-link-with one-of other turtles
    ]
  ]
end

;; This is the classic Erdős-Rényi random network.
;; It uses WHILE to ensure we get NUM-LINKS links.
to wire3
  ask links [ die ]
  if num-links > max-links [ set num-links max-links ]
  while [ count links < num-links ] [
```

Scale-free networks have the property that any subnetwork of the global network has the same properties as the global network. A common way to create this type of network is by adding new nodes and links to the system so that extant nodes with a large number of links are more likely to receive new connections (Barabási, 2002). This technique is sometimes called *preferential attachment*, since nodes with more connections are attached to preferentially. This method for network creation tends to produce networks with central nodes that have many radiating links; because of the resemblance to a bicycle wheel, this network structure is sometimes also called hub-and-spoke. Many real-world networks such as the Internet, electricity grids, and airline routes have similar properties to a scale-free network.

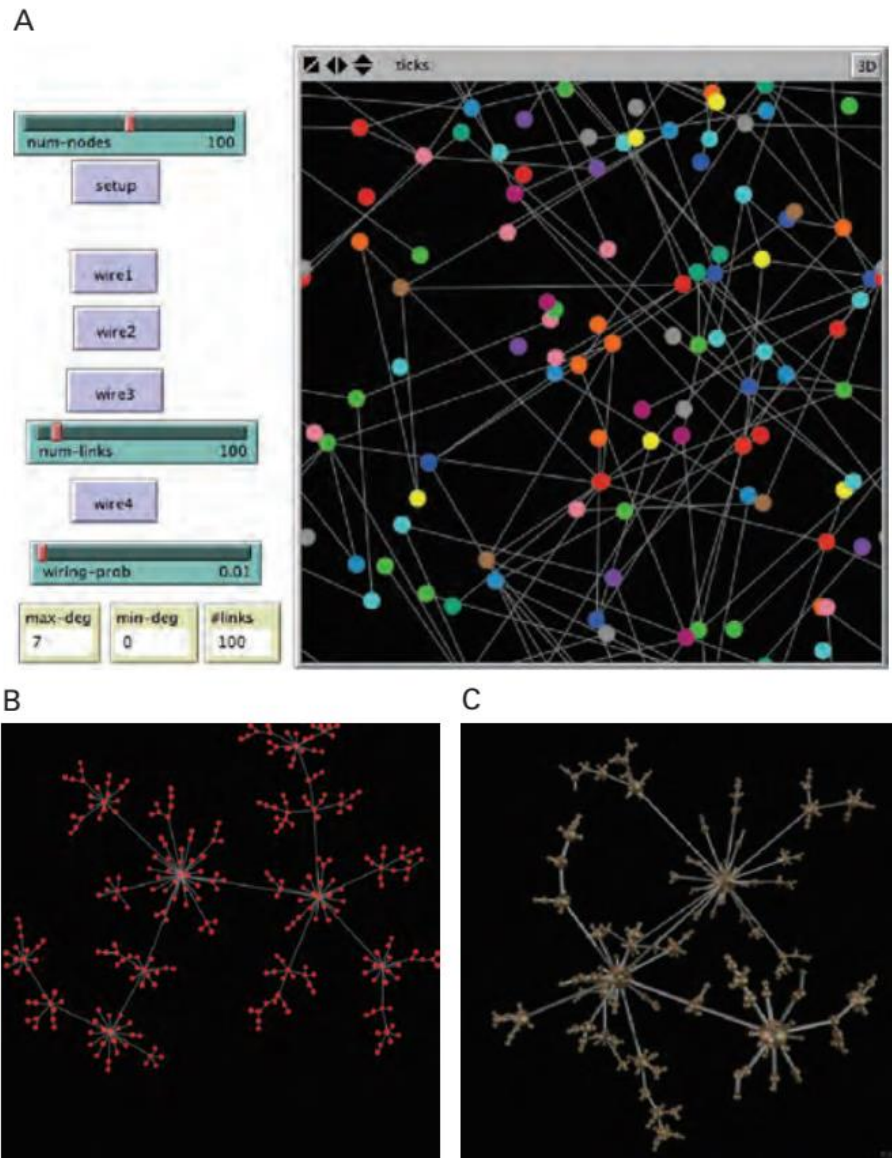


Figure 5.15

(A) Random Network model showing classic Erdős-Rényi random network. (B, C) Scale-Free Network from Preferential Attachment model. <http://ccl.northwestern.edu/netlogo/models/PreferentialAttachment> (Wilensky, 2001).

Small-world Network

The final standard network topology is known as a *small-world network*. *Small-world* networks are made up of dense clusters of highly interconnected nodes joined by a few long distance links between them. Due to these long distance links, it does not take many links for information to travel between any two random nodes in the network. Small-world networks are sometimes created by starting with regular networks, like the 2D lattices described before, then randomly rewiring some of the connections to create large jumps between occasional agents.

Ways to Characterize Networks

There are many ways to characterize networks. Two commonly used ways are

- Average Path Length
- Clustering Coefficient

Average Path Length

The *average path length* is the average of all the pairwise distances between nodes in a network. In other words, we measure the distance between every pair of nodes in the network and then average the results. Average path length characterizes how far the nodes are from each other in a network.

Clustering Coefficient of a Network

The *clustering coefficient* of a network is the average fraction of a node's immediate neighbors who are also neighbors of the node's other neighbors. In other words, it is a measure of the fraction of my friends who are also friends with each other. In networks with a high average clustering coefficient, any two neighboring nodes tend to share many of their neighbors in common, while in networks with a low average clustering coefficient there is generally little overlap between groups of surrounding neighbors.

NetLogo also includes a special extension, the network extension, for creating, analyzing, and working with networks.

Lesson133 : SpecialEnvironments

[click here to download .pptx](#)

Special Environments

The two methods we have demonstrated for defining the environment, two-dimensional (2D) grid-based (i.e., lattice) and network-based, are both instances of “ interaction topologies. ” Interaction topologies describe the paths along which agents can communicate and interact in a model. Besides the two interaction topologies we have looked at so far, there are several other standard topologies to consider. Two of the most interesting topologies involve the use of 3D worlds and Geographic Information Systems (GIS). 3D worlds allow agents to move in a third dimension as well as the two dimensions in traditional ABMs. GIS formats enable the importation of layers of real-world geographical data into ABMs. We will discuss each of these in turn.

3D Worlds

3D environments enable model developers to explore complex systems which are irreducibly bound up with a third dimension as well as sometimes increasing the apparent physical realism to their models.

There is a version of NetLogo called NetLogo 3D (it is a separate application in the NetLogo folder) that allows modelers to explore ABMs in three dimensions. There are many implementations of classic ABMs that have been developed for this environment in the NetLogo 3D models library (*Wilensky, 2000*). For instance, there is a three-dimensional version of the Percolation model (see figure 5.16).

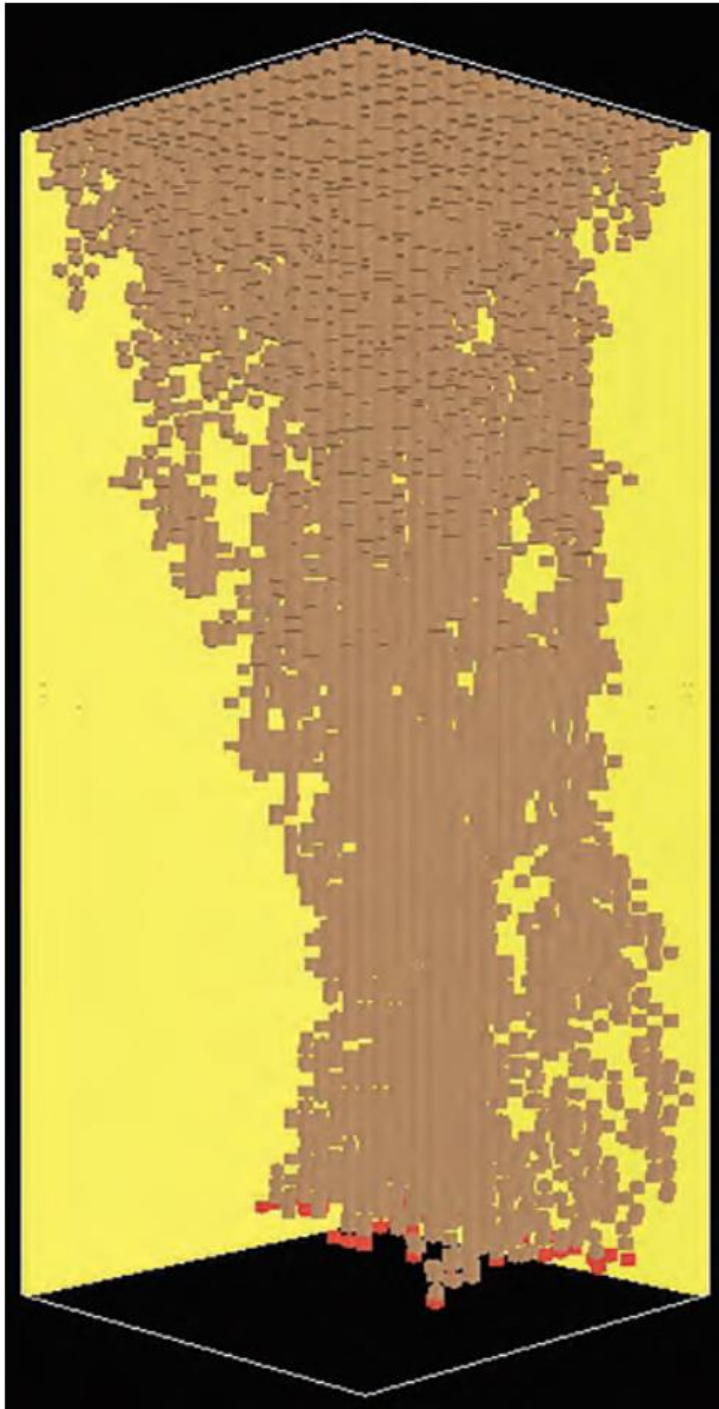


Figure 5.16

Percolation 3D. <http://ccl.northwestern.edu/netlogo/models/Percolation3D> (Wilensky & Rand, 2006).

Working With 3D Worlds

Working with 3D worlds is not much different from working with 2D worlds. Although, using a 3D world does require additional data and commands. For instance, agents now have a Z-coordinate, and new commands are needed to manipulate this new degree of freedom.

In 3D, the orientation of an agent can no longer be described by its heading alone, we must also use *pitch* and *roll* to describe the orientation of the agent. If you think about an agent as an airplane, then the *pitch* of the agent is how far from horizontal the nose of the airplane is pointing. For instance, if the nose of the airplane is pointing straight up then the pitch is 90 degrees (see figure 5.17).

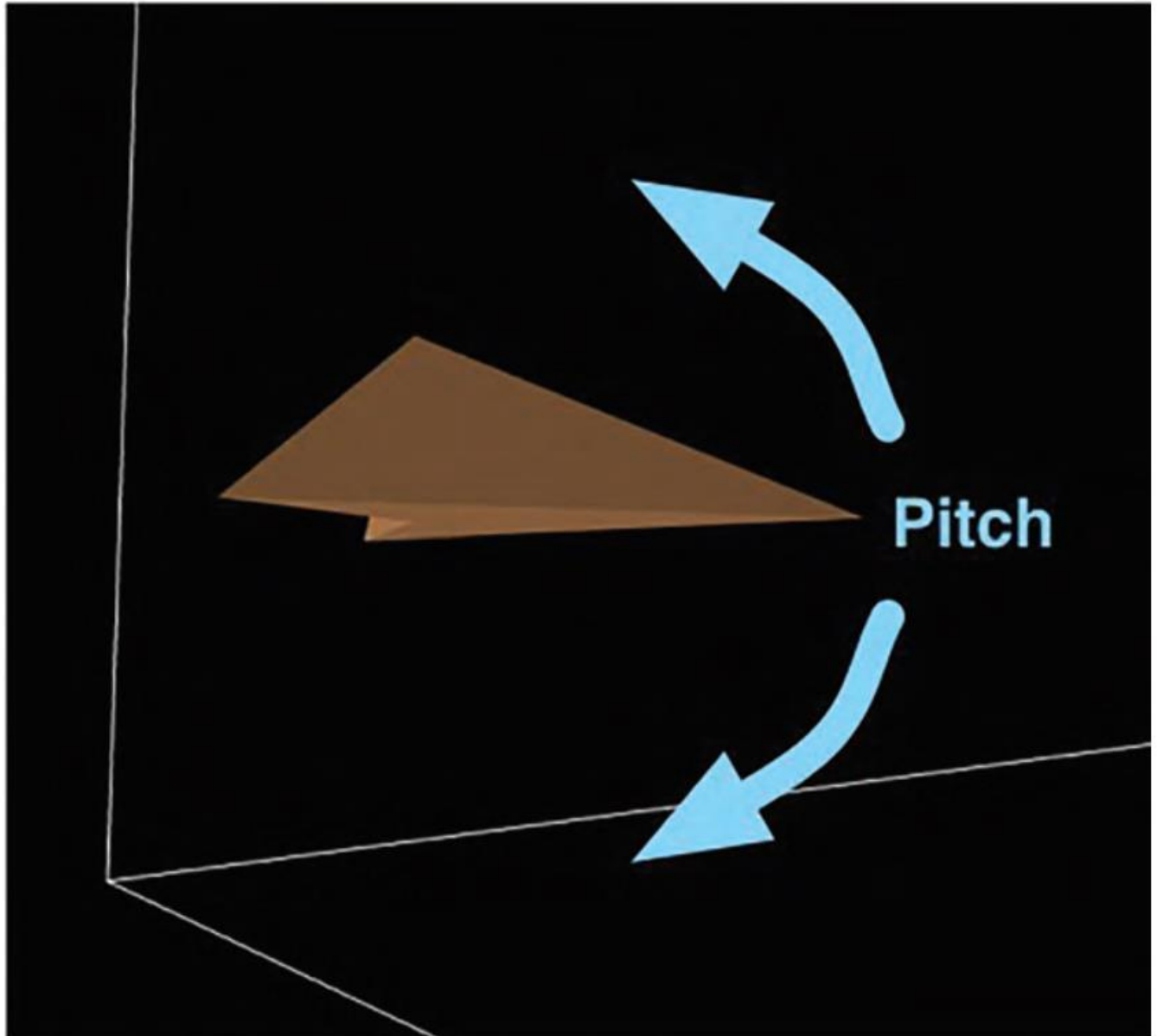


Figure 5.17
Pitch.

Using the airplane metaphor the *roll* of an agent is how far the wings are from horizontal. For instance, if the wings are pointing up and down then the roll of the agent is 90 degrees (see figure 5.18). In many 3D systems, heading has a new name, which is *yaw*, but in NetLogo we still use `HEADING` regardless of whether you are in 2D or 3D NetLogo to keep things consistent.

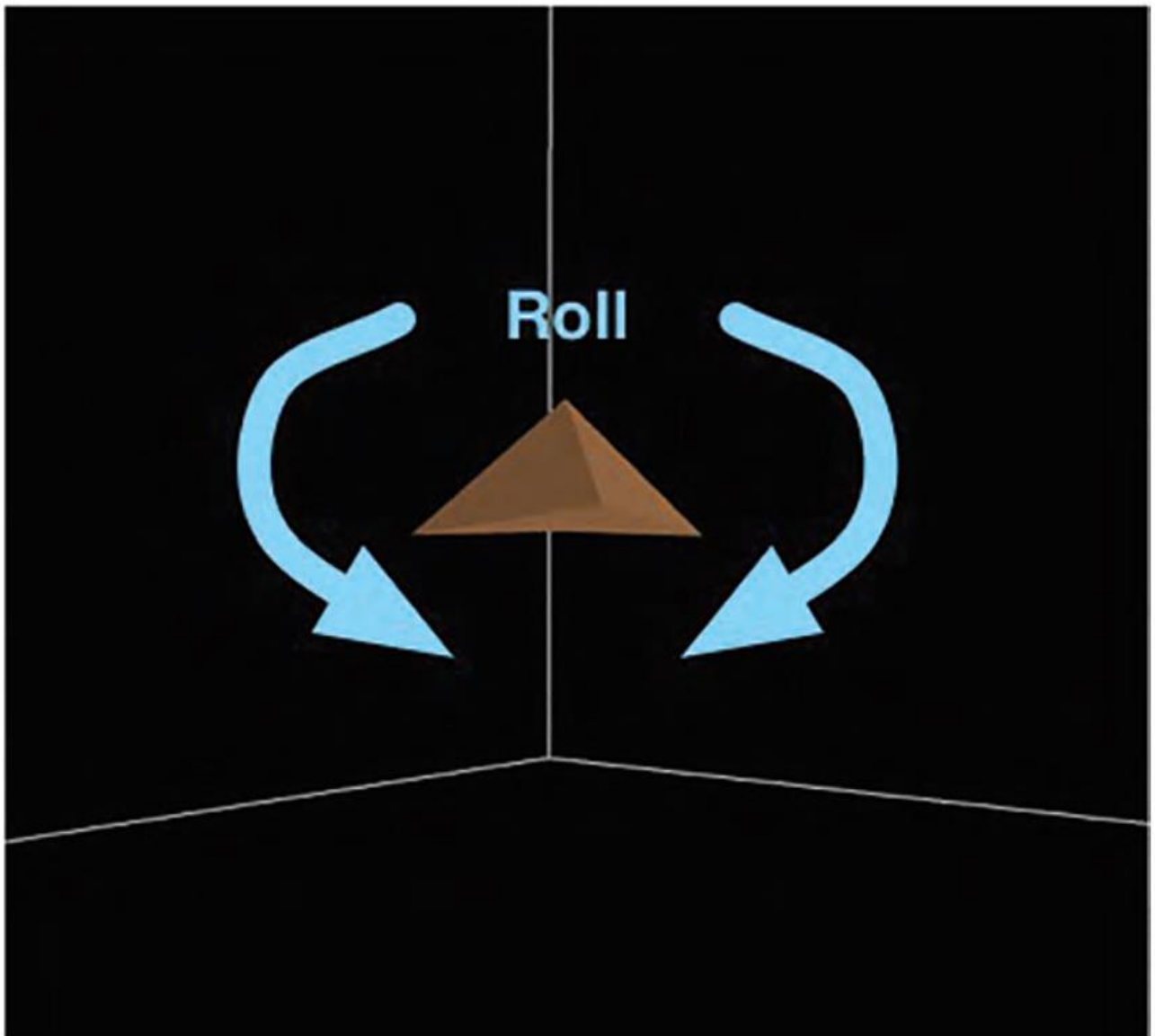


Figure 5.18
Roll.

To see the difference between writing 3D models and 2D models, let us examine the code in each model. The code for the `PERCOLATE` procedure in 2D Percolation looks like this:

```

to percolate
  ask current-row with [pcolor = red] [
    ;; oil percolates to the two patches southwest and southeast
    ask patches at-points [[-1 -1] [1 -1]]
      [ if (pcolor = brown) and (random-float 100 < porosity)
        [ set pcolor red ] ]
    set pcolor black
    set total-oil total-oil + 1
  ]
  ;; advance to the next row
  set current-row patch-set [patch-at 0 -1] of current-row
end

```

Notice that in the 2D version, the code has each patch look at patches that are below, left, and right of the patch. This will change in the 3D version. In 3D Percolation (Wilensky & Rand, 2006), PERCOLATE looks like this:

```

to percolate
  ask current-row with [pcolor = red] [
    ;; oil percolates to the four patches one row down in the z-coordinate and
    ;; southwest, southeast, northeast, and northwest
    ask patches at-points [[-1 -1 -1] [1 -1 -1] [1 1 -1] [-1 1 -1]]
      [ if (pcolor = black) and (random-float 100 < porosity)
        [ set pcolor red ] ]
    set pcolor brown
    set total-oil total-oil + 1
  ]
  ;; advance to the next row
  set current-row patch-set [patch-at 0 0 -1] current-row
end

```

The difference between 3D percolate and 2D

The only difference between the 3D percolate and its 2D counterpart is that instead of percolating a line of patches at a time, we percolate a square of patches. As a result, each patch must ask four patches below it (in a cross shape) to percolate, not just two as it does in the 2D version.

GIS-Based

Geographic Information Systems (GIS) are environments that record large amounts of data that are related to physical locations in the world.

GISs are widely used by environmental scientists, urban planners, park managers, transportation engineers, and many others, and help to organize data and make decisions about any large area of land.

Using GIS, we can index all the information about a particular subject or phenomenon by its location in the physical world. Moreover, GIS researchers have developed analysis tools that enable them to quickly examine the patterns of this data and its spatial distribution.

GIS Tools And Techniques

As a result, GIS tools and techniques allow for a more in-depth exploration of the pattern of a complex system.

Agents moving on a GIS terrain may be constrained to interact on that terrain. Thus, GIS can serve as an interaction topology.

The 3D model, Flocking 3D

Where does ABM fit in to this picture? GIS can provide an environment for an ABM

to operate in. Since ABM encompasses a rich model of process of a complex system, it is a natural match for GIS, which has a rich model of pattern. By allowing ABM to examine and manipulate GIS data, we can build models that have a richer description of the complex system we are examining. GIS thus enables modelers to construct more realistic and elaborate models of complex phenomena.

Lesson 134 : interactions

[click here to download .pptx](#)

Interactions

Now that we have discussed both agents and the environments in which they exist, we will look at how agents and environments interact.

There are five basic classes of interactions that exist in ABMs:

- *agent-self*
- *environment-self*
- *agent-agent*
- *environment-environment*
- *agent-environment*

We will discuss each in turn along with some examples of these common interactions.

Agent-Self Interactions Agents do not always need to interact with other agents or the environment.

In fact, a lot of agent interaction is done within the agent.

For instance, most of the examples of advanced cognition that we discussed in the Agent section involve the agent interacting with itself.

The agent considers its current state and decides what to do. One classic type of self-agent interaction is birth.

Birth events are a typical event in ABMs

In these, one agent creates another agent. Below is the birth routine:

```
;; check to see if this agent has enough energy to reproduce
to reproduce
  if energy > 200 [
    set energy energy - 100 ;; reproduction costs energy to the parent
    hatch 1 [ set energy 100 ] ;; which is transferred to the offspring
  ]
end
```

As you can see, the agent considers its own state and, based on this state, decides whether or not to give birth to a new agent. It then manipulates its state, lowering its energy and creating the new agent. This is the typical way of having agents reproduce.

Environment-Self Interactions Environment-self interactions are when areas of the environment alter or change themselves. For instance, they could change their internal state variables as a result of calculations. The classic example of an environment-self interaction is when the grass regrows:

```
;; regrow the grass
to regrow-grass
  ask patches [
    set grass-amount grass-amount + grass-regrowth-rate
    if grass > 10 [
      set grass 10
    ]
  ]
  recolor-grass
end
```

Each patch is asked to examine its own state and increment the amount of grass it has, but if it has too much grass then it is set back to the maximum value it can contain.

Agent-Agent Interactions Interactions between two or more agents are usually the most important type of action within agent-based models. We saw a canonical example of agent-agent interactions in the Wolf Sheep Predation model when the wolves consume the sheep:

```
;; wolves eat sheep
to eat-sheep
  if any? sheep-here [ ;; if there are sheep here then eat one
    let target one-of sheep-here
    ask target [
      die
    ]
    ;; increase the energy by the parameter setting
    set energy energy + energy-gain-from-sheep
  ]
end
```

In this case, one agent is consuming another agent and taking its resources, whereby the wolf always eats the sheep. However, it is also possible to add competition or flight to this model, where the wolf gets a chance of eating the sheep and the sheep gets a chance to flee. Competition is another example of agent-agent interaction.

Communication

A final example of agent-agent interaction is communication. Agents can share information about their own state as well as that of the world around them. This type of interaction allows agents to gain information to which they might not have direct access.

Environment-Environment Interactions: Interactions between different parts of the environment are probably the least commonly used type of interaction in agent-based models. However, there are some common uses of environment-environment interactions: one of these is diffusion. In the Ants model discussed in chapter 1, the ants place a pheromone in the

environment, which is then diffused throughout the world via an environment-environment interaction. This interaction is contained in the following piece of code in the main GO procedure:

```
diffuse chemical (diffusion-rate / 100)
ask patches
  [ set chemical chemical * (100 - evaporation-rate) / 100
  ;; slowly evaporate chemical
  recolor-patch ]
```

Agent-Environment Interactions: Agent-environment interactions occur when the agent manipulates or examines part of the world in which it exists, or when the environment in some way alters or observes the agent. A common type of agent-environment interaction involves agents observing the environment. The Ants model demonstrates this kind of interaction when the ants examine the environment to look for food and sense pheromone:

```
to look-for-food ;; turtle procedure
  if food > 0
  [ set color orange + 1      ;; pick up food
    set food food - 1        ;; and reduce the food source
    rt 180                   ;; and turn around
  stop ]
  ;; face in the direction where the chemical smell is strongest
  if (chemical >= 0.05) and (chemical < 2)
  [ uphill-chemical ]
end
```

In the Ants model, the patches contain food and chemical, so the first part of this code checks to see if there is any food in the current patch. If there is food, the ant then picks up the food and turns around back to the nest, and the procedure stops.

Otherwise, the ant checks to see if there is chemical, wherein it follows the chemical in that direction.

Agent Movement

Another common type of agent-environment interaction is agent movement. In some ways, movement is simply an agent-self interaction, since it only alters the current agent's state.

In the Ants model the ants move around by “wiggling”.

Lesson 135 : Observer/User Interface

[click here to download .pptx](#)

Observer/User Interface

Now that we have talked about the agents, the environments, and the interactions that occur between agents and environmental attributes, we may discuss who controls the running of the model.

The *observer* is a high-level agent that is responsible for ensuring that the model runs and proceeds according to the steps developed by the model author.

The observer issues commands to agents and the environment, telling them to manipulate their data or to take certain actions. Most of the control that model developers have with an ABM is mediated through the observer. However, the observer is a special agent. It does not have many properties, though it can access global properties like any agent or patch can.

Properties Specific to the observer

The only properties that one could consider to be specific to the *observer* are those relating to the perspective from which the modeled world is viewed. For instance, in NetLogo the view may be centered on a specific agent, or focusing a highlight on a certain agent, using the FOLLOW, WATCH, or RIDE commands (See figure 5.21).

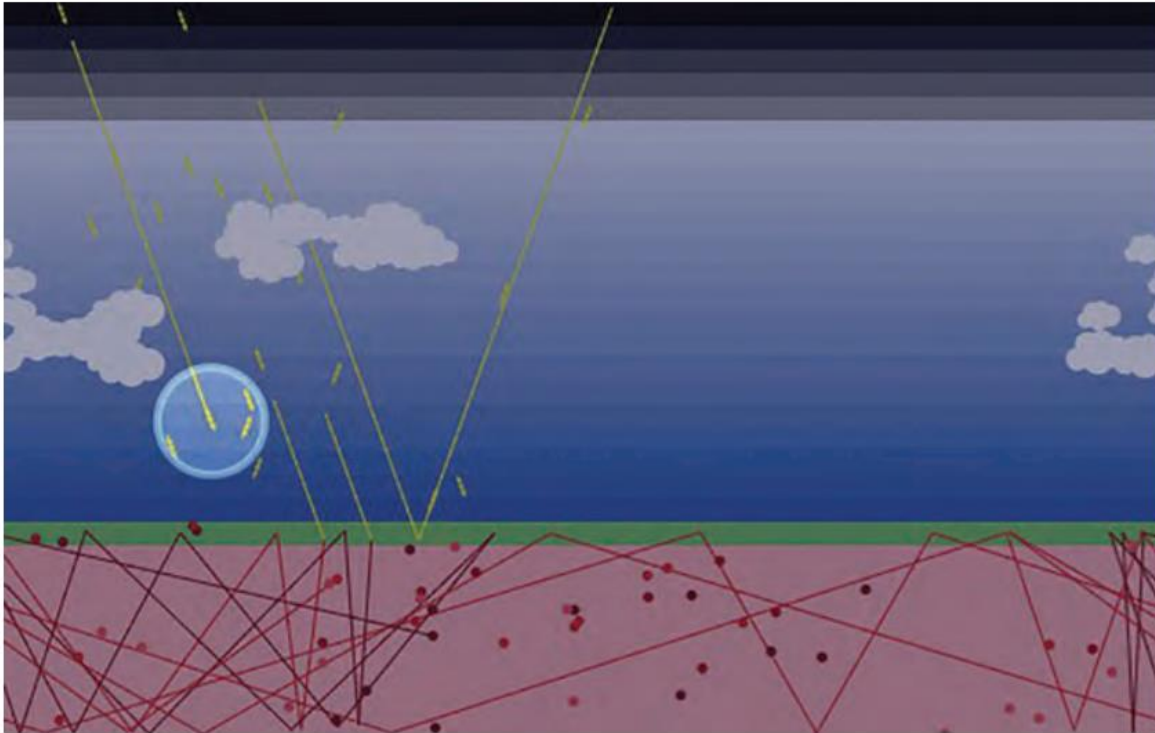


Figure 5.21

Climate Change model (with a sun ray turtle being watched, which is shown by the transparent “halo” around it). <http://ccl.northwestern.edu/netlogo/models/ClimateChange> (Tinker & Wilensky, 2007).

User Input and Model Output

We have already made use of many of the standard ways to interface with an ABM when we extended models in chapter 3 and when we built our first

model in chapter 4, but it is worth recapping some of them herein. ABMs require a control interface or a parameter group that allows the user to setup different parameters and settings for the ABM. The most common control mechanism is a *button*, which executes one or more commands within the model; if it is a *forever button* it will continue to execute those commands until the user presses the button again.

Command Center

A second way that the user can request that actions be performed within the ABM is via the command center (along with the mini – command centers within agent monitors). The command center is a very useful feature of NetLogo, as it allows the user to interactively test out commands, and manipulate agents and the environment.

Sliders, Switches & Input Boxes

Sliders enable the model user to select a particular value from a range of numerical values. For instance, a slider could range from 0 to 50 (by increments of 0,1) or 1 to 1,000 by increments of 1. In the Code tab the value of a slider is accessed as if it were a global variable. *Switches* enable the user to turn various elements of a model off or on. In the Code tab, they are also accessed as global variables, but they are Boolean variables. *Choosers* enable a model user to select a choice from a predefined drop-down menu that the modeler has created. Again these are accessed as global variables in the Code tab, but they are variables that have strings as their values, these strings being the various choices in the chooser. Finally, *input boxes* are more free-form, allowing the user to input text that the model can use.

Monitors , Plots & Notes

As for output controls, *monitors* display the value of a global variable or calculation updated several times a second. They have no history but show the user the current state of the system. *Plots* provide traditional 2D graphs enabling the user to observe the change of an output variable over time. *Output boxes* enable the modeler to create freeform text-based output to send to the user. Finally, *notes* enable the modeler to place text information on the **Interface tab** (for example, to give a model user directions on how to use the model). Unlike in monitors, the text in notes is unchanging (unless you manually edit them).

Visualization

Visualization is the part of model design concerned with how to present the data contained in the model in a visual way. Creating cognitively efficient and aesthetic visualizations can make it much easier for model authors and users to understand the model. Though there is a long history of work on how to present data in static images (Bertin, 1967; Tufte, 1983, 1996), there is much less work on how to represent data in real-time dynamic situations. However, attempts have been made to take current static guidelines and apply them to dynamic visualizations (Kornhauser, Rand & Wilensky, 2007). In general, there are three guidelines that should be kept in mind whenever designing the visualization of an ABM: simplify, explain, and emphasize.

Guidelines for Designing Visualization

Simplify the visualization Make the visualization as simple as possible so that anything that does not present additional usable information (or that is irrelevant to the current point being explained) has been eliminated from the visualization. This prevents the model user from being distracted by unnecessary “ graph clutter ” (Tufte, 1983).

Explain the components If there is an aspect of the visualization that is not immediately obvious then there should be some quick way to determine what that visualization is illustrating, such as a legend or description. Without clear and direct descriptions of what is going on the model user may misinterpret what the model author is attempting to portray. If a model is to be useful it is necessary that anyone viewing the model can easily understand what it is saying.

Emphasize the main point Model visualizations are themselves simplifications of all the possible data that a model could present to the model user. Therefore, a model visualization should emphasize the main points and interactions that the model author wants to explore, and, in turn, to communicate to end users. By exaggerating certain aspects of the visualization they can draw attention to these key results.

NetLogo Ethnocentrism model

Every NetLogo agent has a shape and color, and by selecting appropriate shapes and colors we can highlight some agents while backgrounding others. For instance, to simplify visualization if agents in our model are truly homogenous, like the ants, then we might make all the agents the same color, as in the Ants model. To explain the visualization we might change their shape to indicate something about their properties. For instance, in the Ethnocentrism model based on Hammond and Axelrod 's model (2003) agents employing the same strategies have the same shape, even if they are different colors.

Lesson 136 : Schedule

[click here to download .pptx](#)

Schedule

The *schedule* is a description of the order in which the model operates. Different ABM toolkits can have more or less explicit representations of the schedule. In NetLogo, there is no single identifiable object that can be identified as “ the schedule. ” Rather, the schedule is the order of events that occur within the model, which depends on the sequence of buttons that the user pushes, and the code/procedures that those buttons run. We will first discuss the common SETUP/GO idiom which is employed in almost all agent-based models, and then move on to discuss some of the subtler issues concerning scheduling in ABMs.

SETUP and GO First of all, there is usually an initialization procedure that creates the agents, initializes the environment, and readies the user interface. In NetLogo this procedure is usually called SETUP and it executes whenever a user presses the SETUP button on a NetLogo model. The SETUP routine usually starts by clearing away all the agents and data related to the previous run of the model. Then it examines how the user has manipulated the various variables controlled by the user interface creating new agents and data to reflect the new run of the model. For instance, in Traffic Basic the SETUP procedure looks like this:

```
to setup
  clear-all
  ask patches [ setup-road ]
  setup-cars
  watch sample-car
end
```

The other main part of the schedule is what is often called the main loop, or in NetLogo, the GO procedure. The GO procedure describes what happens in one time unit (or tick) of the model. Usually that involves the agents being told what to do, the environment changing if necessary, and the user interface updating to reflect what has happened. In Traffic Basic the GO procedure looks like this:


```

to go
;; if there is a car right ahead of you, slow down to a speed below its speed
  ask turtles [
    let car-ahead one-of turtles-on patch-ahead 1
    ifelse car-ahead != nobody
      [ slow-down-car car-ahead]
      ;; otherwise, speed up
      [ speed-up-car ]
  ;; don't slow down below speed minimum or speed up beyond speed limit
    if speed < speed-min [ set speed speed-min ]
    if speed > speed-limit [ set speed speed-limit ]
    fd speed ]
  tick
end

```

Asynchronous vs. Synchronous Updates

If a model uses an *Asynchronous* update schedule, this means that when agents change their state, that state is immediately seen by other agents.

In a *Synchronous* update schedule changes made to an agent are not seen by other agents until the next clock tick — that is, all agents update simultaneously.

Sequential vs. Parallel Actions

Within the realm of asynchronous updating, agents can act either sequentially or in parallel.

Sequential actions involve only one agent acting at a time while *Parallel* actions are those in which all agents act independently. In NetLogo (versions 4.0 and later), sequential action is the standard behavior for agents.

Moreover, for the agents to act truly in parallel, you would need parallel hardware so that the actions of each agent would be carried out by a separate processor.

However, there is an intermediate solution. *Simulated concurrency* uses one processor to simulate many agents acting in parallel.

Lesson 137 : Analyzing Agent-Based Models

[click here to download ppt](#)

[click here to download video](#)

Types of Measurements

Heretofore, we have examined ABMs, modified them, built them from scratch, and analyzed their behavior. In this module, we will learn to employ ABM to produce new and interesting results about the domain that we are investigating. What kinds of results can ABMs produce? There are many different ways of examining and analyzing ABM data.

Choosing just one of these techniques can be limiting; therefore, it is important to know the advantages and disadvantages of a variety of tools and techniques. It is often useful to consider your analysis methods *before* building the ABM, to enable you to design output that is conducive to your analysis.

Modeling the Spread of Disease

If someone catches a cold and is coughing up a storm, he might infect others. Those that he comes into contact with — his friends, co-workers, and even strangers — may catch the cold. If a cold virus infects someone, that person might spread that disease to five other people (six now infected) before they recover. In turn, those five other people might spread the cold to five more people each (thirty-one are now infected), and those twenty-five people might spread the cold to five additional people (a hundred and fifty-six people are now infected). In fact, the rate of infection initially rises exponentially.

However, since this infection count grows so quickly, any population will eventually reach the limit of the number of people who can be infected. For instance, imagine that the 156 people mentioned above all work for the same company of 200 individuals. It is impossible for the remaining 125 people to each infect five new people, and thus the number of infected people will tail off because there is no one left to infect. As we have described it so far, this simple model assumes that each person infects the same number of people, which is manifestly not the case in real contexts. As a person moves through

their workspace, it might be the case that, they happen to not see many people in one day, whereas another individual might see many people. Also, our initial description assumes that if one person infects five people and another person infects five, there will not be any overlap. In reality, there is likely to be substantial overlap. Thus, the spread of disease in a workplace is not as straightforward as our initial description suggests. Suppose that we are interested in understanding the spread of disease, and we want to build an ABM of such a spread. How should we go about doing it?

First, we need some agents that keep track of whether they are infected with a cold or not. Additionally, these agents need a location in space and the ability to move. Finally, we need the ability to initialize the model by infecting a group of individuals. That is exactly how the NetLogo model we will be discussing in this module behaves (See figure 6.1). Individuals move around randomly on a landscape and infect other individuals whenever they come into contact with them.

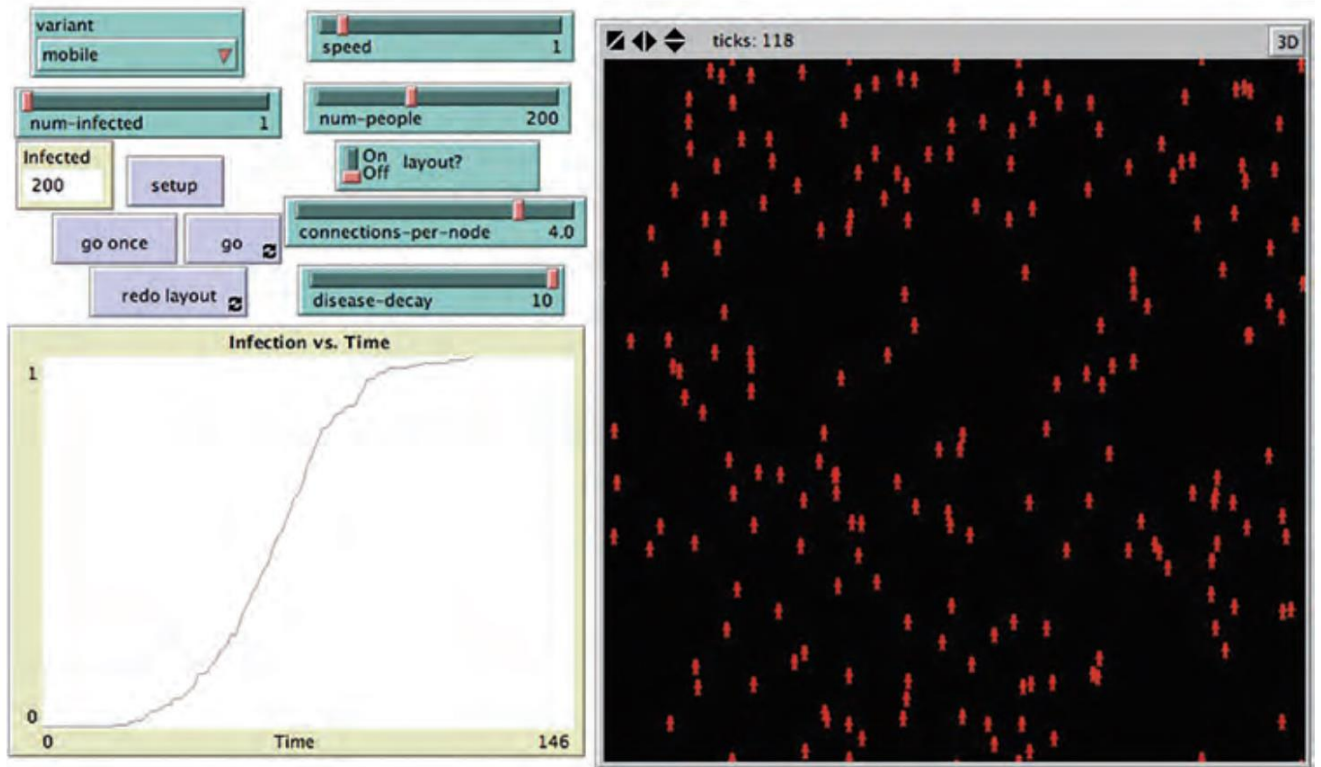


Figure 6.1
Spread of Disease model.

Though this model is simple, it exhibits interesting and complex behavior. For instance, what happens if we increase the number of people in the model? Does the disease spread quicker throughout the population, or does it take a longer time because there are more people? Let us run the model at population sizes of 50, 100, 150, and 200, and examine the results. We will keep the size of the world constant, so that, as we increase the number of individuals, we are also increasing the population density. Along the way we will write down at what time the entire population becomes infected (see table 6.1).

Table 6.1
Infection Data

Population	50	100	150	200
Time to 100% Infection	419	188	169	127

Based on these results, we conclude that as the population density increases, the time to full infection dramatically decreases.

Let us suppose that we show this data to a friend of ours and she does not believe it. She believes that the time to 100 percent infection should grow linearly with the population. She looks over the code and determines that it seems to match the description (a process called verification, which we will discuss in the next chapter). After that, she runs the model and collects the same data that we did (a form of replication). Her data is in table 6.2.

Table 6.2
Your Friend's Data

Population	50	100	150	200
Time to 100% Infection	305	263	118	126

These results do not support our friend's prediction that the time to 100 percent infection will grow as the population increases, but, on the other hand, they are quite different from the results that were originally collected. In fact, the time to 100 percent infection for 150 and 200 increases in our friend's data, seemingly contradicting our original results. Moreover, if we run the model several more times you might again get different results. We need some methods for determining if there are trends in the data.

The data is inconsistent because most ABM models employ randomness in their algorithms — i.e., the code makes use of a random number generator. How the agents move around on the landscape is not specifically determined, but instead is the result of several calls to the random number generator at each time step that determine the actions of any particular agent. Moreover, these random decisions occur at least once for each agent in the population for each time step. Clearly, then, one set of runs is not enough to characterize the behavior of this model. Suppose, then, that we collect additional data for a set of population densities for ten different model runs as in table 6.3. Though most of these runs look more like our original results than our friend's results, it might be difficult to see clear trends, and to analyze the results overall. Thus, to describe these patterns of behavior it makes sense to turn to some statistics.

Table 6.3
Raw Data

Population	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Run 8	Run 9	Run 10
50	419	365	305	318	323	337	432	380	430	359
100	188	263	256	205	206	205	201	181	202	231
150	169	118	163	146	143	167	137	121	140	140
200	127	126	113	111	133	129	109	101	105	133

Lesson 138 : Types of Measurements

[click here to download ppt](#)

[click here to download video](#)

Here, we will understand about the different types of measurements in an ABM. Statistical Analysis of ABM:

- Moving beyond Raw Data
- Statistical results are the most common way of looking at any kind of scientific data
- The general methodology behind descriptive statistics is to provide numerical measures These measures summarize a large data set
- They also describe the data set in such a way that it is not necessary to examine every single value.

Summary Statistics

Population	Mean	Std. Dev.
50	366.8	47.39385802
100	213.8	27.40154091
150	144.4	17.65219533
200	118.7	12.12939497

-

Experiment name

Vary variables as follows:

```
[ "connections-per-node" 4.1 ]
[ "speed" 1 ]
[ "num-people" [50 50 200] ]
[ "num-infected" 1 ]
[ "infect-environment?" false ]
```

Either list values to use, for example:
 ["my-slider" 1 2 7 8]
 or specify start, increment, and end, for example:
 ["my-slider" [0 1 10]] (note additional brackets)
 to go from 0, 1 at a time, to 10.
 You may also vary max-pxcor, min-pxcor, max-pycor, min-pycor, random-seed.

Repetitions

run each combination this many times

Measure runs using these reporters:

```
ticks
```

one reporter per line; you may not split a reporter across multiple lines

☐ Measure runs at every tick
 If unchecked, runs are measured only when they are over

Setup commands:

Go commands:

BehaviorSpace Data Imported into a Spreadsheet

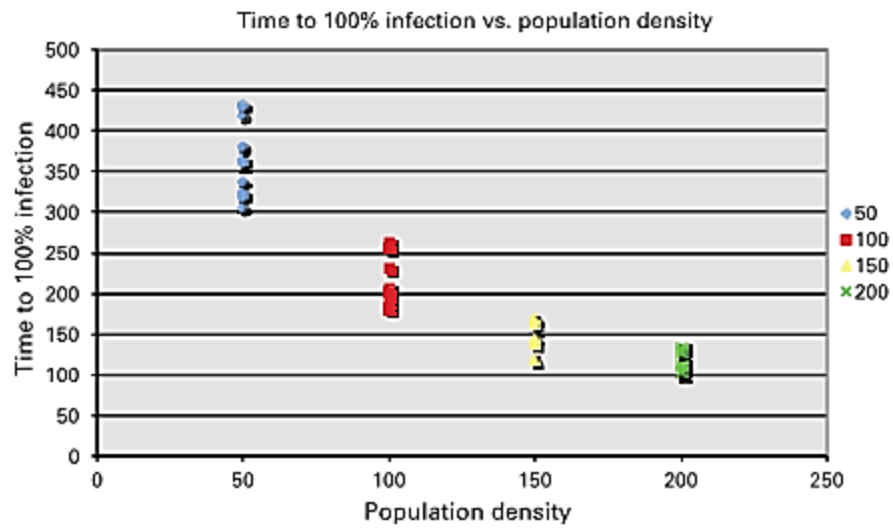
BehaviorSpace Table data

population-density

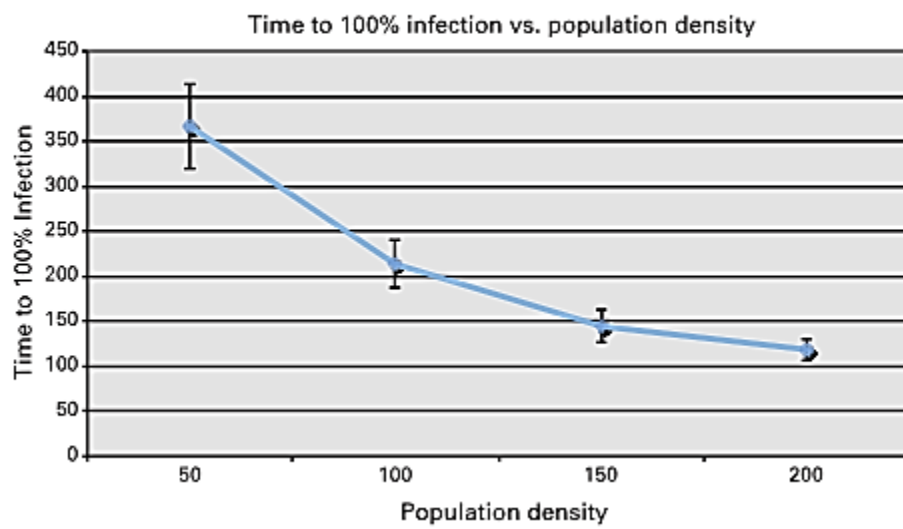
DATE

TIME

[run number]	network?	layout?	connections-per-node	speed	num-people	num-infected	infect-environment?	[tick]	ticks
1	FALSE	FALSE	4.1	1	50	1	FALSE	299	299
2	FALSE	FALSE	4.1	1	50	1	FALSE	432	432
3	FALSE	FALSE	4.1	1	50	1	FALSE	444	444
4	FALSE	FALSE	4.1	1	50	1	FALSE	400	400
5	FALSE	FALSE	4.1	1	50	1	FALSE	467	467
6	FALSE	FALSE	4.1	1	50	1	FALSE	397	397
7	FALSE	FALSE	4.1	1	50	1	FALSE	337	337
-	---	---	---	-	--	-	---	---	---



- Figure 2.4
-



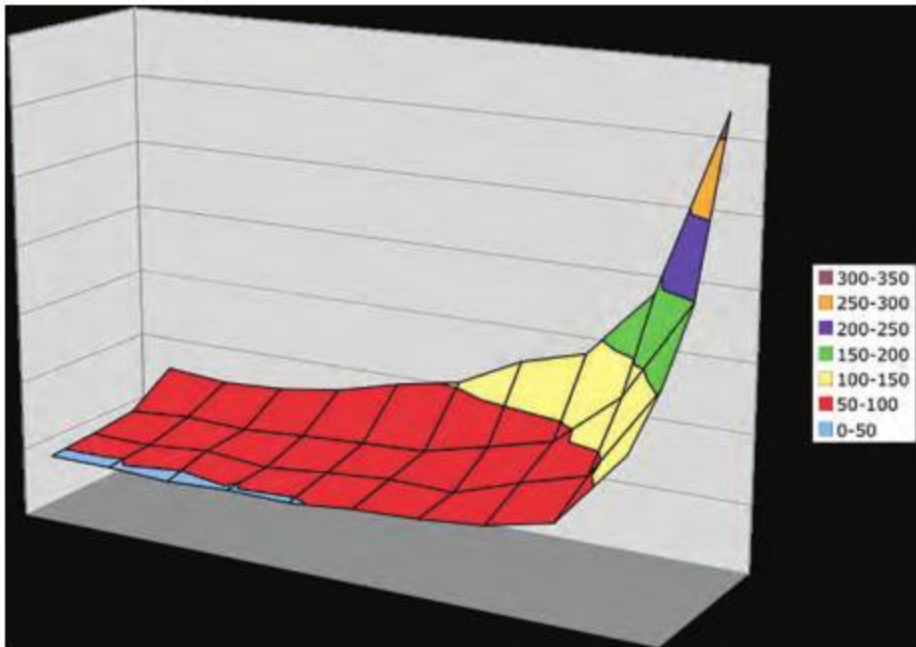


Figure 6.9

3D Chart of NUM-PEOPLE and DISEASE-DECAY versus time to 100 percent infection.

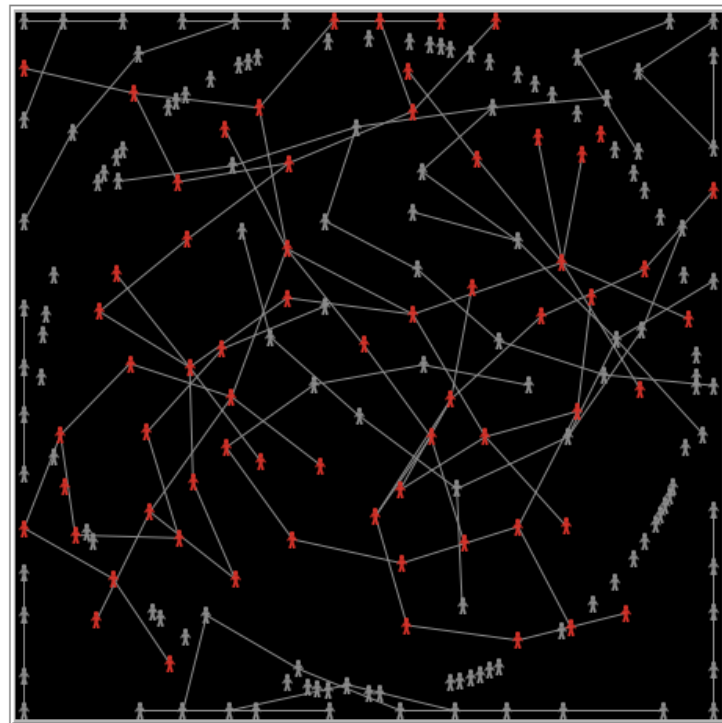
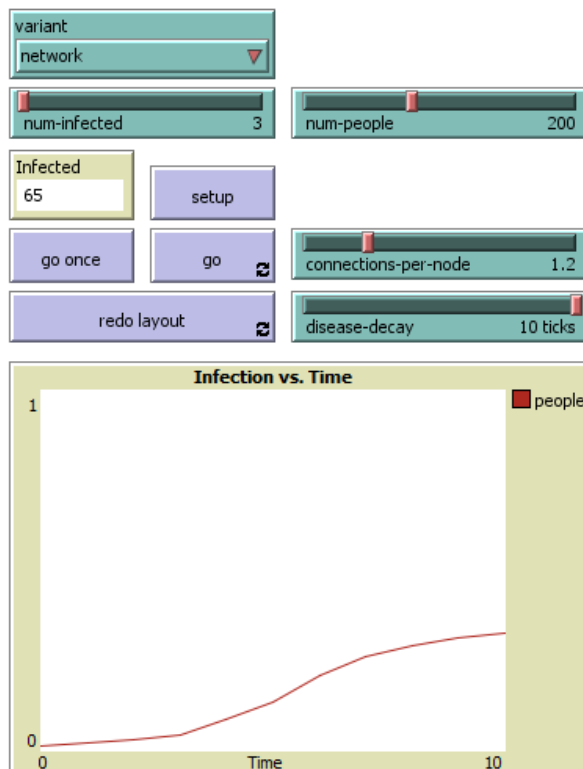
- We have learnt about the types of measurements in the analysis of an ABM.
- Credits: Uri Wilensky book.

Lesson 139 : Modeling the Spread of Disease

[click here to download .pptx](#)

Here we will Model the spread of disease in detail.

- If a cold virus infects someone, that person might spread that disease to five other people (six now infected) before they recover.
- In fact, the rate of infection initially rises exponentially.
- Suppose that we are interested in understanding the spread of disease, and we want to build an ABM of such a spread. How should we go about doing it?
- First, we need some agents that keep track of whether they are infected with a cold or not
- We need the ability to initialize the model by infecting a group of individuals
- Individuals move around randomly on a landscape and infect other individuals whenever they come into contact with them.



- We conclude that as the population density increases, the time to full infection dramatically decreases
- In the beginning, when the first person becomes infected, if there are not many other people around, the person has no one to infect, and thus the infection rate increases slowly.
- However, if there are many people around then there will be plenty of infection opportunities
- Moreover, at the end of the run, when there are only one or two uninfected agents, they will be more likely to run into someone with an infection if the population count is high.
- This is true despite the fact that the total number of people that need to be infected increases.

- To describe these patterns of behavior it makes sense to turn to some statistics
- Credits : Uri Wilensky

Lesson 140 : Statistical Analysis of ABM: Moving beyond Raw Data

[click here to download .pptx](#)

Here we will learn about the general methodology behind descriptive statistics that is to provide numerical measures that summarize a large data set and describe the data set in such a way that it is not necessary to examine every single value.

- Suppose we are interested in determining whether a coin is fair. (i.e., it is as likely to turn up heads when flipped as it is to turn up tails) then we can conduct a series of experiments where we flip the coin and observe the results.
- It is much easier to look at means and standard deviations than it is to examine large series of data
- (e.g., for HHHHTTHTTT, the observed probability is 0.5, and the expected outcome for ten trials is to observe five heads with a standard deviation of 1.58).
- To apply this technique to our Spread of Disease model in more depth, we can create summary statistics

Summary Statistics		
Population	Mean	Std. Dev.
50	366.8	47.39385802
100	213.8	27.40154091
150	144.4	17.65219533
200	118.7	12.12939497

- We see that the mean time to 100 percent infection declines as the population density increases.
- Another interesting result is that as the population density goes up, the standard deviation goes down.
- This means that the data is less varied.
- These results seem to confirm our original hypothesis that as population density increases the mean time to infection declines.
- Within ABM, statistical analysis is a common method of confirming or rejecting hypotheses
- ABMs create large amounts of data (the Spread of Disease model is just a small example), and if we can summarize that data we can examine large amounts of output in an efficient manner.
- Most ABM toolkits give you the basic ability to carry out simple statistical analysis within the package itself

- (e.g., in NetLogo there are MEAN and STANDARD-DEVIATION primitives). Thus, while the model is running, the ABM itself can generate summary statistics.
- The NetLogo R extension can be used to conduct analyses with the R statistical package
- Credits : Uri Wilensky

Lesson 141 : The Necessity of Multiple Runs within ABM

- When you are trying to collect statistical results from an ABM you should run the model multiple times and collect different results at different points.
- Here we will understand how ABM toolkits will provide you with a way to collect the data from these runs automatically.
- In NetLogo there is a tool called BehaviorSpace. ABM toolkits are often full-featured programming languages, allowing you to write your own tools for creating experiments to produce the data sets you want to analyze.
- These tools will automatically run the model multiple times with multiple different settings and collect the results in some easy to use format like the CSV files mentioned earlier.
- Let us call this experiment “ population density”

Experiment

Experiment name: population-density

Vary variables as follows (note brackets and quotation marks):

- [["variant" "mobile"]
- [["connections-per-node" 4.1]
- [["num-people" [50 50 200]]
- [["num-infected" 1]

Either list values to use, for example:
["my-slider" 1 2 7 8]
or specify start, increment, and end, for example:
["my-slider" [0 1 10]] (note additional brackets)
to go from 0, 1 at a time, to 10.
You may also vary max-pcor, min-pcor, max-pycor, min-pycor, random-seed.

Repetitions: 10
run each combination this many times

☒ Run combinations in sequential order
For example, having ["var" 1 2 3] with 2 repetitions, the experiments "var" values will be:
sequential order: 1, 1, 2, 2, 3, 3
alternating order: 1, 2, 3, 1, 2, 3

Measure runs using these reporters:

ticks

one reporter per line; you may not split a reporter across multiple lines

☐ Measure runs at every step
if unchecked, runs are measured only when they are over

Setup commands: setup

Go commands: go

Stop condition: the run stops if this reporter becomes true

Final commands: run at the end of each run

Time limit: 0
stop after this many steps (0 = no limit)

OK Cancel

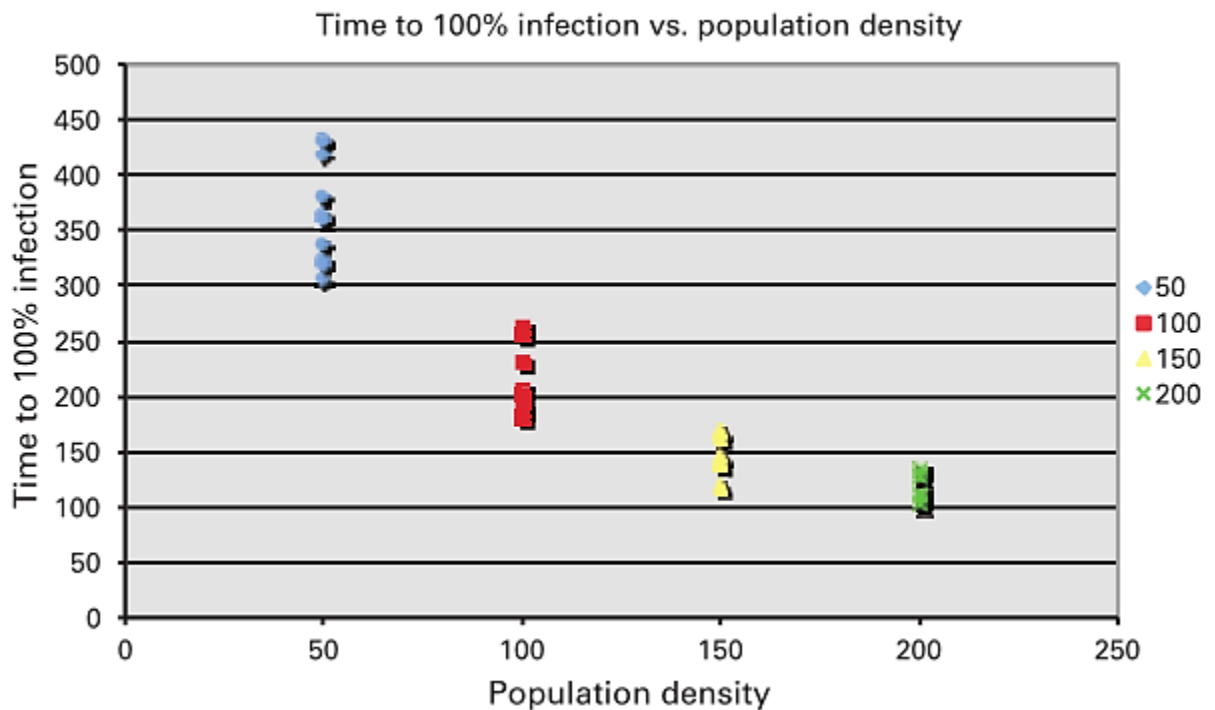
- The SETUP and GO commands allow you to specify any additional NetLogo code that you need to make the model start and go.
- “ Stop condition ” allows you to specify special stop conditions for each run, and
- “ Final commands ” allows you to insert any commands that you want executed between runs of the model
- In general in ABM, it is important to carry out multiple runs of your experiments so that you can determine if some result is truly a pattern or just a one-time occurrence.
- One common way is to start by manually running your model multiple times, but to get a better sense of the results it is usually much easier to use a batch experiment tool.
- We have illustrated the BehaviorSpace tool, which is the batch experiment tool for NetLogo.
- Credits : Uri Wilensky

Lesson 142 : Using Graphs to Examine Results in ABM

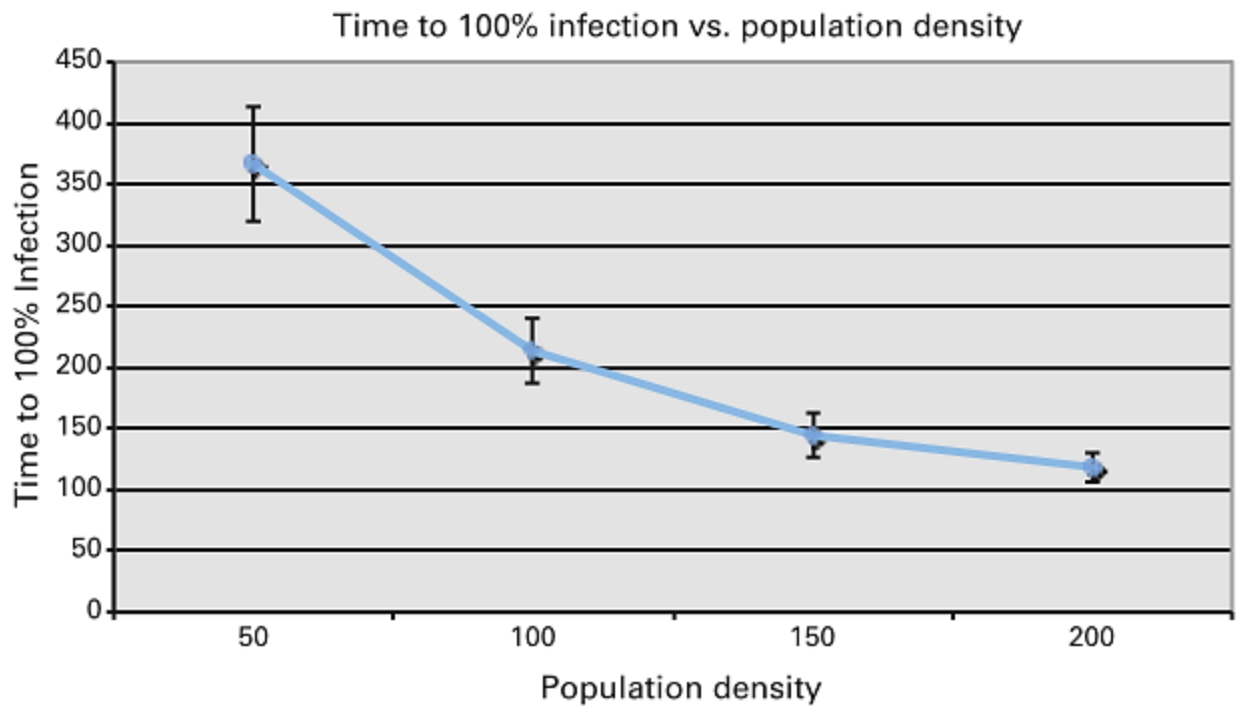
[click here to download .pptx](#)

In NetLogo creating simple graphs is easy to do and will often suffice for simple data analysis.

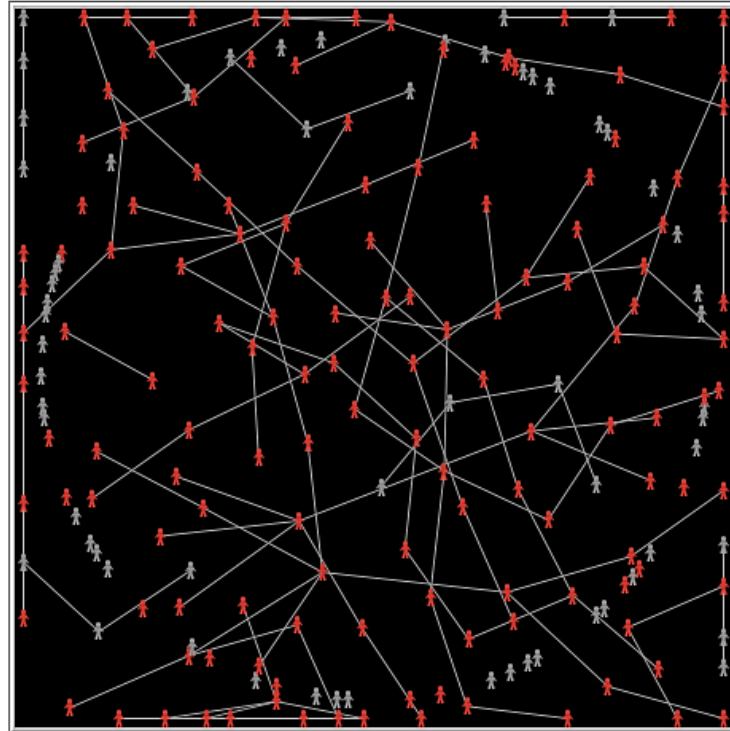
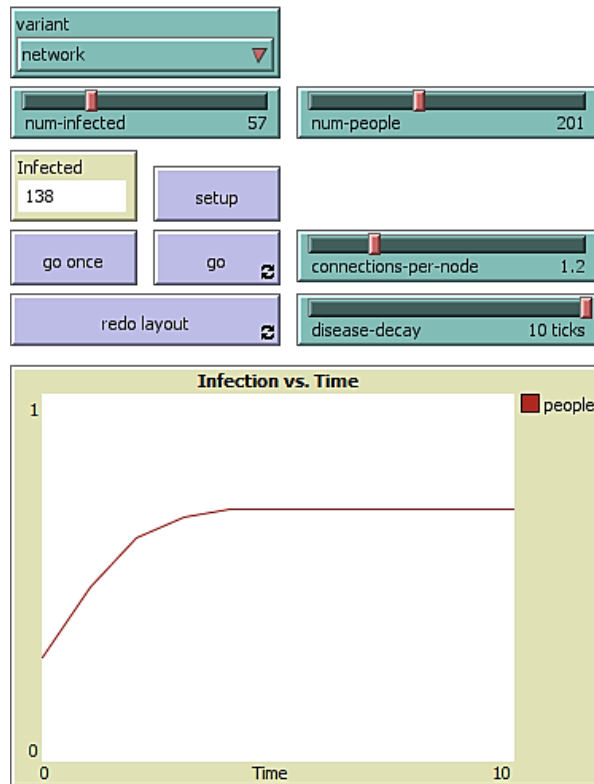
But with complex data sets, designing a useful and immediately informative graph can be challenging and is the subject of an extensive body of literature.



- We can quickly see how the data is distributed and how the data changes with population density



-
- This new figure might not be easier to understand than previous figure, but if there were one hundred data points in previous figure rather than ten data points, then a figure like this one might be very helpful.
- Many ABM toolkits include capabilities for continually updating graphs and charts during the running of a model and thus enable you to see the progress of the model temporally
- For instance, in the Spread of Disease model there is a graph that illustrates the change in the fraction of infected agents as time proceeds

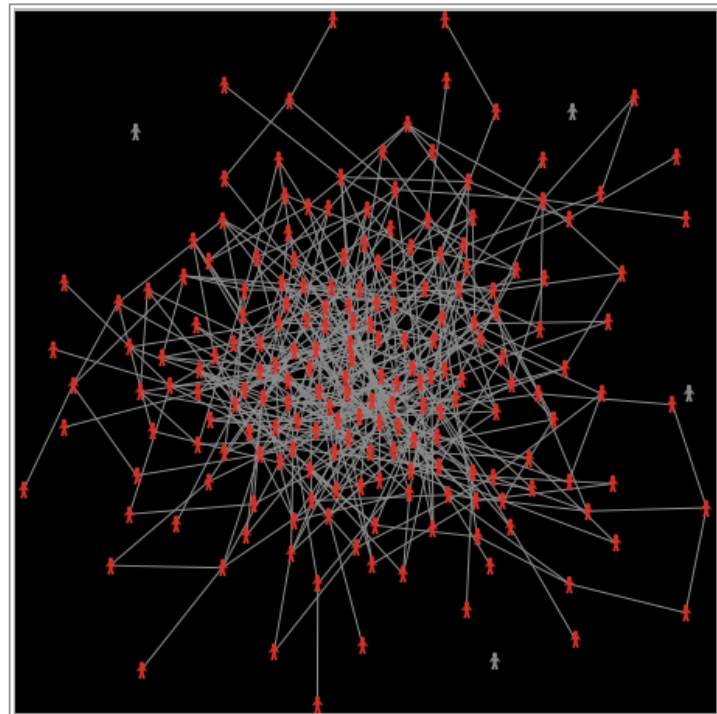
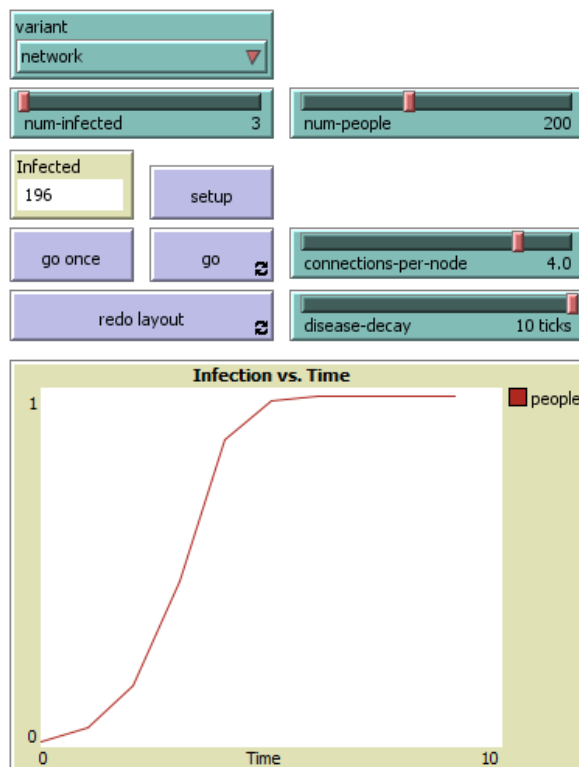


-
- This is one example of how we can use time series to help understand the behavior of a model.
- Summary: We have seen the use of graphs (plots) in simulation software
- Credits: Uri Wilensky

Lesson 143 : Analyzing Networks within ABM

Here, we understand the analysis of “networks within ABM.”

- Interactions do not occur in physical space. But rather they can also occur across social networks.
- E.g. some diseases spread only through certain kinds of social networks. If we set the chooser to “network” we can explore this further.
- The property which determines how many connections.
- Each individual has to other individual in the network.
- Known as “connections-per-node”.

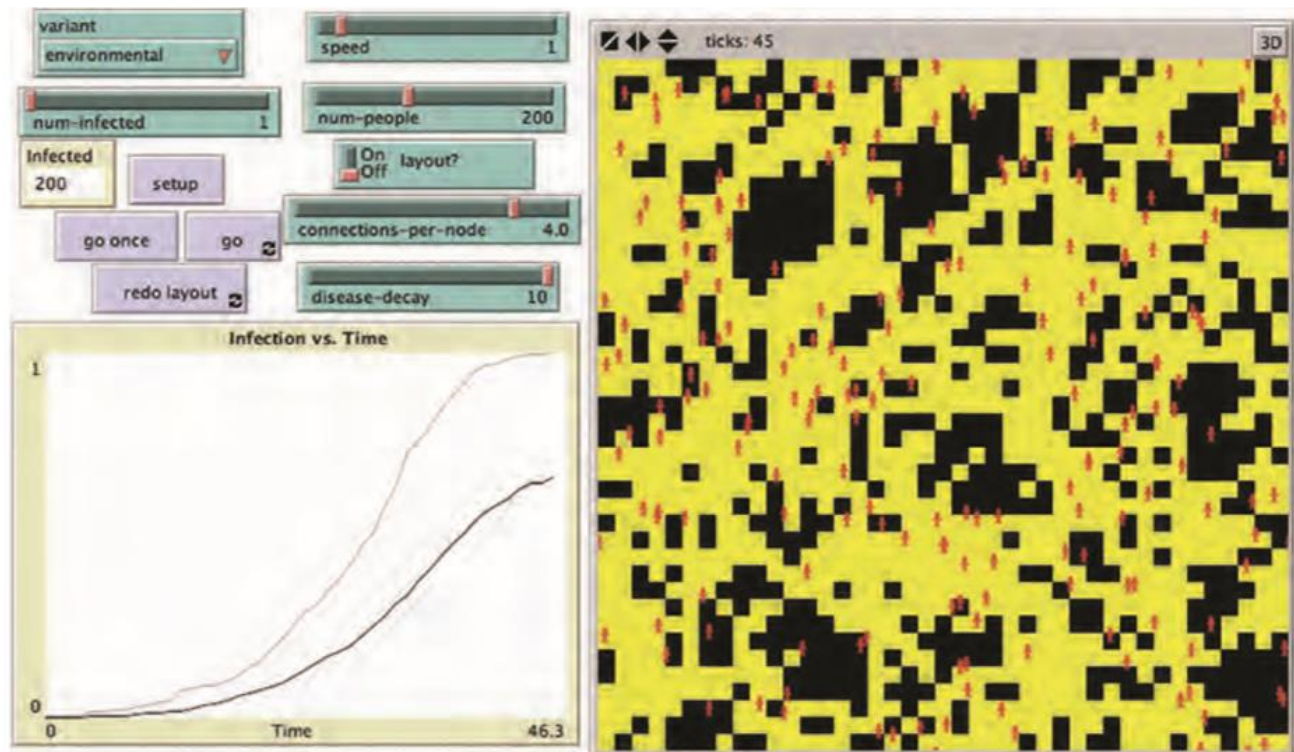


- A well known property of random graph is the average number of individuals infected.
- Grows substantially
- The connections-per-node exceeds 1.0
- Forms a giant component in the network.
- The property of average path length, which measure the distance between any two nodes in the network.
- Affect the spread of disease in the network.
- Another property is clustering co-efficient.
- Different properties and tools are there to measure and analyze a wide variety of metrics
- Associated with social network analysis (SNA).
- To summarize, each of these network properties can be analyzed as to their effect on the spread of disease.
- Reports and toolkits like UCInet can be used for further examination.
- Credits: Uri Wilensky book.

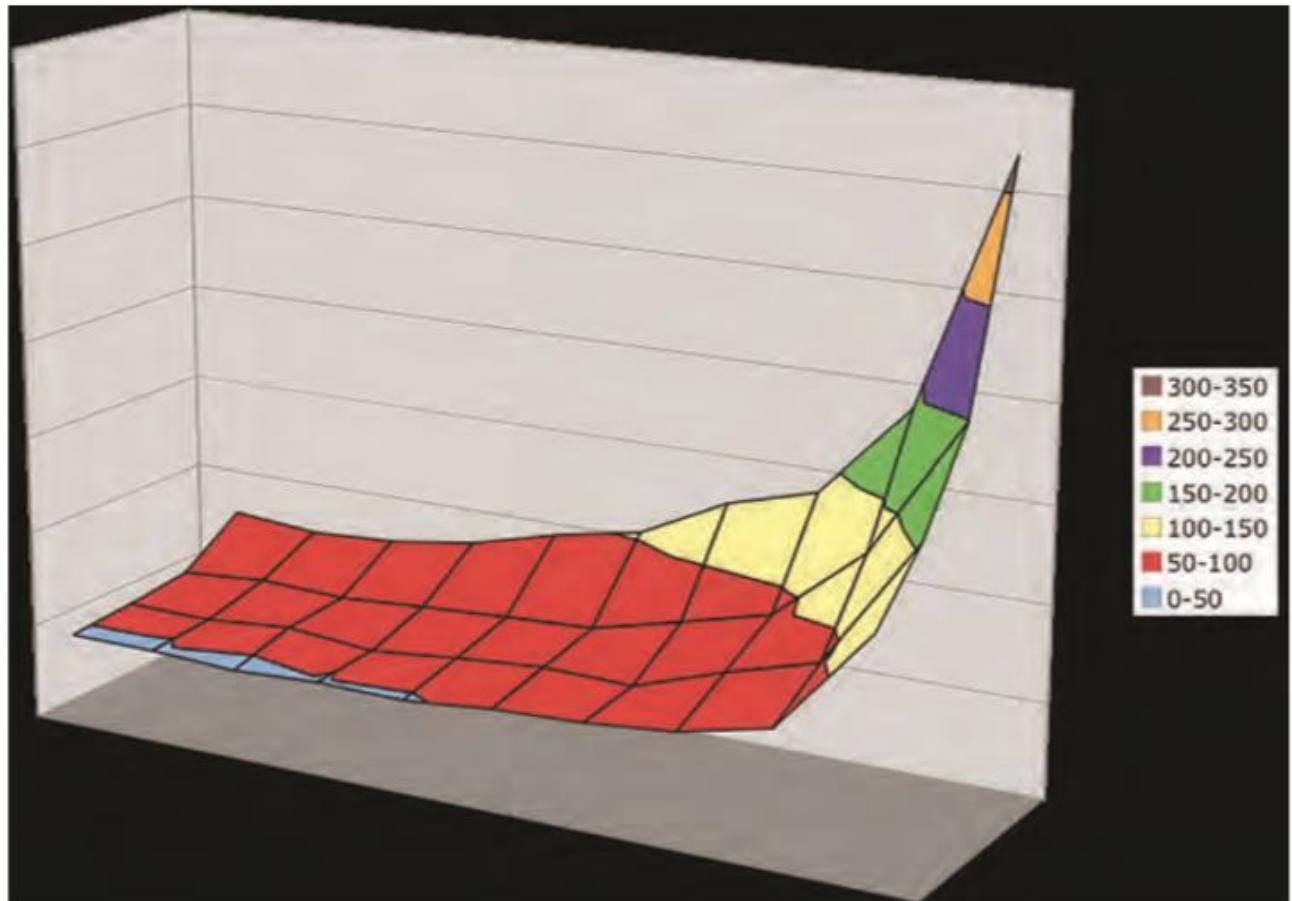
Lesson 144 : Environmental Data and ABM

In this section we are going to see agent-to-environment and environment-to-agent transmission.

- The key areas to understand are:
 - Spread of disease model for examining.
 - Environmental interaction effect.



-
- The path below any agent will become yellow.
- Change the rate if DISEASE-DECAY 0 to 10 at a single time intervals
- A long DIESEASE-DECAY might have negligible over having a small DISEASE-DECAY.
- Investigating using behaviour space, we notice that a powerful aspect of ABMs is that it also shows us the pattern of infection.



-
- Leaving long stringy patterns of environmental infection.
- To summarize, we have seen the working of environmental data and ABMs.
- Credits: Uri Wilensky book

Lesson 145 : Correctness of a Model

[click here to download .pptx](#)

- Here, we understand the about the “correctness of a model”. Output relating the concerned issue must be accurate.
- Model accuracy is evaluated through validation, verification and replication.
- Implemented model corresponds to, and explains, some phenomenon in the real world.
- Model verification: determining whether an implemented model corresponds to the target conceptual model.
- Make sure that the model has been implemented correctly.
- Model replication is the implementation by one researcher or group of researchers of a conceptual model previously implemented by someone else.
- Set of results from a model that corresponds to the real world is not sufficient.
- Multiple runs are often needed to confirm that a model is accurate.
- Verification, validation, and replication collectively underpin the correctness, and thus utility, of a model.
- Credits: Uri Wilensky book.

Lesson 146 : Verification

[click here to download .pptx](#)

- Here, we understand the role of “Verification” in Correctness of a Model.
- In larger models, the code can be difficult to understand as it evolves over time.
- Verification ensures the elimination of “bugs” from the code.
- The process of debugging becomes more difficult for complex models.
- Our goal, however, is to keep the process easy and simple.
- Build the model simply to begin with – it will be easier to verify. Expand the complexity of the model as necessary. This incremental approach makes it easy to verify the additional components.
- Even if all the components are verified, it is still possible that the system is not. Complications may arise from the interaction between model components.
- In this section we have examined the issue of verification of a model in the context of a simple ABM.
- Credits: Uri Wilensky book

Lesson 147 : Communication

[click here to download .pptx](#)

Here, we understand need of “communication” for the correctness of a model.

- Sometimes a team of people builds a model. Other team members actually implement the model. Therefore, verification becomes critical.
- Communication is critical to ensure that the implemented model is correct.
- It is essential.
- E.g. In the voting model.
- Political scientists differentiate between Moore and Van Neumann neighborhoods.
- Small world network and hexagonal versus a rectangular grid.
- In an ideal situation, the model author and the implementer is the same person which averts the sort of communication errors.
- But when it is not, then there is often room for human error and misunderstanding.
- In the past it was difficult to be expert model implementer and model author.
- However, low-threshold ABM languages, such as NetLogo is narrowing the gap between author and implementer.
- We have seen how communication plays an important role in the correctness of a model.
- How communication can fill the gap between the implementer and the author of the model.
- Credits: Uri Wilensky book.

Lesson 148 : Describing Conceptual Models

[click here to download ppt](#)

Here, we will understand how “Describing Conceptual Models” plays a role in analyzing correctness of a model.

- Implementing the voting model, we may realize subsequently that we never understand the idea completely. This could happen if we talked with a political scientist.
- Describing how we plan to implement the model is a specific type of document.
- We and the political scientist should have the same “conceptual model” in our mind.
- Describe the model in more formal terms. This includes a pictorial description of the model using flowcharts.
- Subsequently, we can convert the flowcharts into pseudo-code
- The goal of pseudo-code is to serve as a midway point between natural language and programming language.
- Pseudocode:

```
Voters have votes = {0, 1}
```

```
For each voter:
```

```
    Set vote either 0 or 1, chosen with equal probability
```

```
Loop until election
```

```
    For each voter
```

```
        If majority of neighbors' votes = 1 and vote = 0 then set vote 1
```

```
        Else If majority of neighbors' votes = 0 and vote = 1 then set vote 0
```

```
        If vote = 1: set color blue
```

```
        Else: color = green
```

```
    Display count of voters with vote = 1
```

```
    Display count of voters with vote = 0
```

```
End loop
```

-
- Other methods are UML, choosing a language similar to pseudo-code and NetLogo.
- We have learnt about processes involved in describing conceptual model which include flowcharts, pseudo-code, UML, and NetLogo.
- Credits: Uri Wilensky book

Lesson 149 : Verification Testing

[click here to download here](#)

Here, We will discuss and understand how we can use “Verification Testing” for finding out about the correctness of a model.

- When implementing the design into code we need to follow ABM core design principles.

```
patches-own
[
  vote ;; my vote (0 or 1)
  total ;; sum of votes around me
]

to setup
  clear-all
  ask patches [
    if (random 2 = 0) ;; half a chance of this
    [ set vote 1 ]
  ]
  ask patches [
    if (random 2 = 0) ;; half a chance of this
    [ set vote 0 ]
  ]
  ask patches [
    recolor-patch
  ]
end

to recolor-patch ;; patch procedure
  ifelse vote = 0
  [ set pcolor green ]
  [ set pcolor blue ]
end
```

-

```
to check-setup
  let diff abs ( count patches with [ vote = 0 ] - count patches
with [ vote = 1 ] )
  if diff > .1 * count patches [
    print "Warning: Difference in initial voters is greater than 10%."
  ]
end
```

-

```

to setup
  clear-all
  ask patches [
    set vote random 2    ;; 0 or 1, with equal probability
  ]
  ask patches [
    recolor-patch
  ]
  check-setup
end

```

-
- This verification technique is a form of unit testing. We can modify the code without disrupting previous code.

```

to go
  ask patches [
    set total (sum [vote] of neighbors)
  ]
  ;; this is equivalent to count neighbors with [vote = 1]
  ;; use two ask patches blocks so all patches compute "total"
  ;; before any patches change their votes
  ask patches [
    ifelse vote = 0 and total >= 4 [
      set vote 1
    ]
    [if vote = 1 and total <= 4 ] [
      [set vote 0
    ]
    recolor-patch
  ]
  tick
end

```

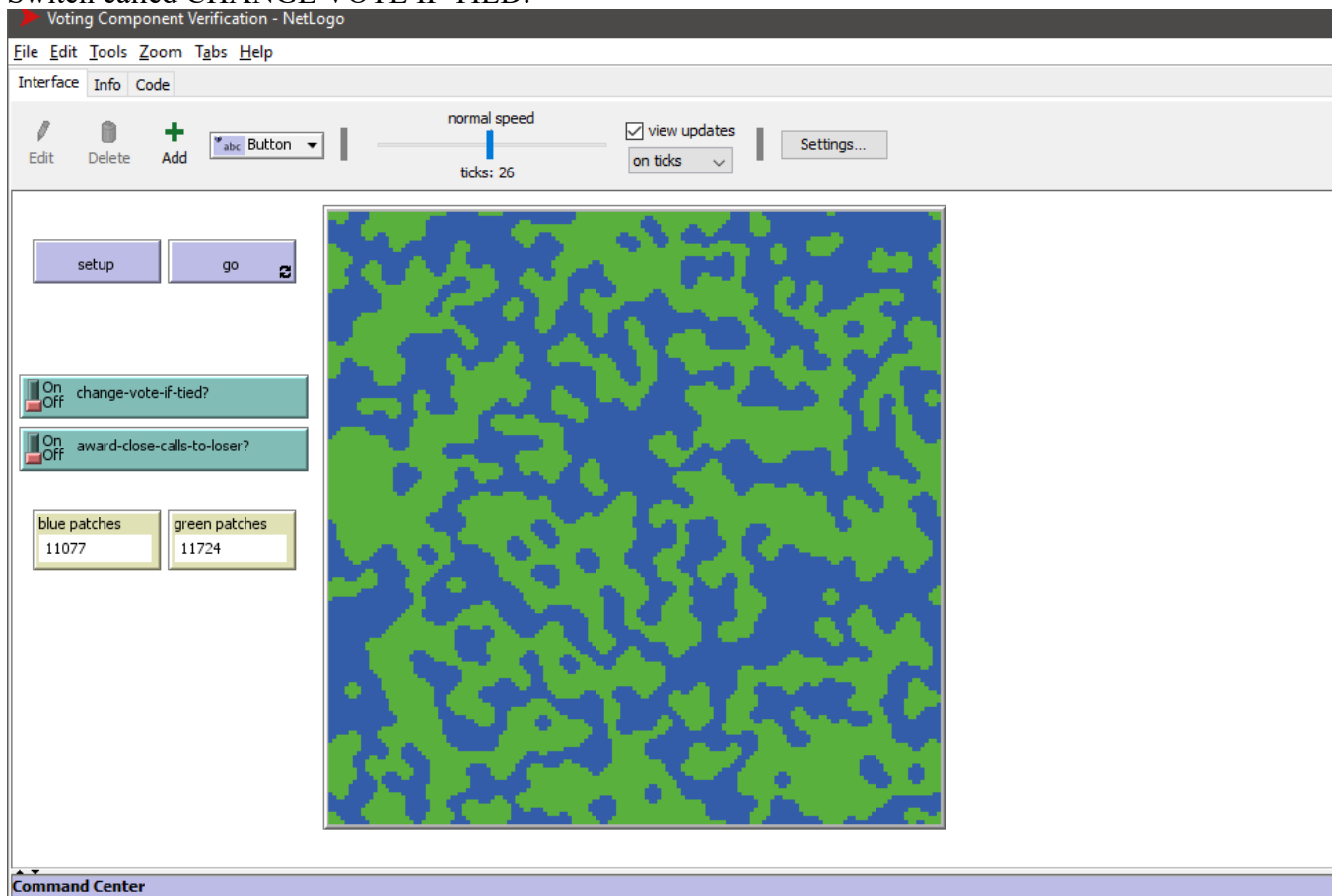
-
- We have understood how an incremental approach makes it easy to implement the model correctly.
- We are then able to write our code into NetLogo for verification purpose.
- Credits: Uri Wilensky book

Lesson 150 : Beyond Verification

[click here to download ppt](#)

Here, we will learn about going beyond verification using agent-based modeling.

- Sometimes results are not produced to what the implementers and authors hypothesized.
- Results of modeling can therefore end up causing further confusions for the scientists.
- Jagged edges can occur due to ties in votes.
- Design the model that neighbors do not change their votes if tied.
- Switch called CHANGE VOTE IF TIED.




```

patches-own [
  vote    ;; my vote (0 or 1)
  total   ;; sum of votes around me
]

to setup
  clear-all
  ask patches [
    set vote random 2 ;; set vote to either 0 or 1
    recolor-patch
  ]
  reset-ticks
  check-setup
end

to go
  ;; keep track of whether any patch has changed their vote
  let any-votes-changed? false
  ask patches [
    set total (sum [ vote ] of neighbors)
  ]
  ;; use two ask patches blocks so all patches compute "total"
  ;; before any patches change their votes
  ask patches [
    let previous-vote vote
    if total < 3 [ set vote 0 ] ;; if majority of your neighbors vote 0, set your vote to 0
    if total = 3 [
      ifelse award-close-calls-to-loser?

```

```

        [ set vote 1 ]
        [ set vote 0 ]
    ]
    if total = 4 and change-vote-if-tied? [
        set vote (1 - vote) ;; invert the vote
    ]
    if total = 5 [
        ifelse award-close-calls-to-loser?
            [ set vote 0 ]
            [ set vote 1 ]
    ]
    if total > 5 [ set vote 1 ] ;; if majority of your neighbors vote 1, set your vote to 1
    if vote != previous-vote [ set any-votes-changed? true ]
    recolor-patch
]
;; if the votes have stabilized, we stop the simulation
if not any-votes-changed? [ stop ]
tick
end

to recolor-patch ;; patch procedure
    ifelse vote = 0
        [ set pcolor green ]
        [ set pcolor blue ]
    end
end

;; This procedure checks to see if the SETUP procedure sets up the model with roughly
;; equal numbers of blue and green patches
to check-setup
    ;; count the difference between the number of green and the number of blue patches
    let diff abs (count patches with [ vote = 0 ] - count patches with [ vote = 1 ])
    if diff > .1 * count patches [
        print "Warning: Difference in initial voters is greater than 10%."
    ]
end

```

- ☐ ; Copyright 2008 Uri Wilensky.
- Switch AWARD CLOSE CALLS TO LOSER?
- With both switches on, we have a different outcome.

```

to go
  ask patches
    [ set total (sum [vote] of neighbors) ]
  ;; use two ask patches blocks so all patches compute "total"
  ;; before any patches change their votes
  ask patches
    [ if total > 5 [ set vote 1 ]
      if total < 3 [ set vote 0 ]
      if total = 4
        [ if change-vote-if-tied?
          [ set vote (1 - vote) ] ] ;; switch vote
      if total = 5
        [ ifelse award-close-calls-to-loser?
          [ set vote 0 ]
          [ set vote 1 ] ]
      if total = 3
        [ ifelse award-close-calls-to-loser?
          [ set vote 1 ]
          [ set vote 0 ] ]
      recolor-patch ]
  tick
end

```

-
- We have seen the details of how to go beyond verification.
- Credits: Uri Wilensky book

Lesson 151 : Sensitivity Analysis and Robustness

[click here to download ppt](#)

Here, we will discuss sensitivity analysis and robustness.

- Creating the parameter to test the hypothesis of initial balance in one direction.
- Using BehaviorSpace, we can run the experiment varying from 25 to 75 percent increment.
- Two conditions:
- If no voter switch vote in the last step the model will stop.
- The model will stop after one hundred times the steps have executed.

```
patches-own
[
  vote    ;; my vote (0 or 1)
  total   ;; sum of votes around me
]

to setup
  clear-all
  ask patches [
    ifelse random 100 < initial-green-pct
      [ set vote 0 ]
      [ set vote 1 ]
    recolor-patch
  ]
  reset-ticks
  check-setup
end

to go
  ;; keep track of whether any patch has changed their vote
  let any-votes-changed? false
  ask patches [
    set total (sum [ vote ] of neighbors)
  ]
  ;; use two ask patches blocks so all patches compute "total"
  ;; before any patches change their votes
```

```

]
;; use two ask patches blocks so all patches compute "total"
;; before any patches change their votes
ask patches [
  let previous-vote vote
  if total < 3 [ set vote 0 ] ;; if majority of your neighbors vote 0, set your vote to 0
  if total = 3 [
    ifelse award-close-calls-to-loser?
      [ set vote 1 ]
      [ set vote 0 ]
  ]
  if total = 4 and change-vote-if-tied? [
    set vote (1 - vote) ;; invert the vote
  ]
  if total = 5 [
    ifelse award-close-calls-to-loser?
      [ set vote 0 ]
      [ set vote 1 ]
  ]
  if total > 5 [ set vote 1 ] ;; if majority of your neighbors vote 1, set your vote to 1
  if vote != previous-vote [ set any-votes-changed? true ]
  recolor-patch
]

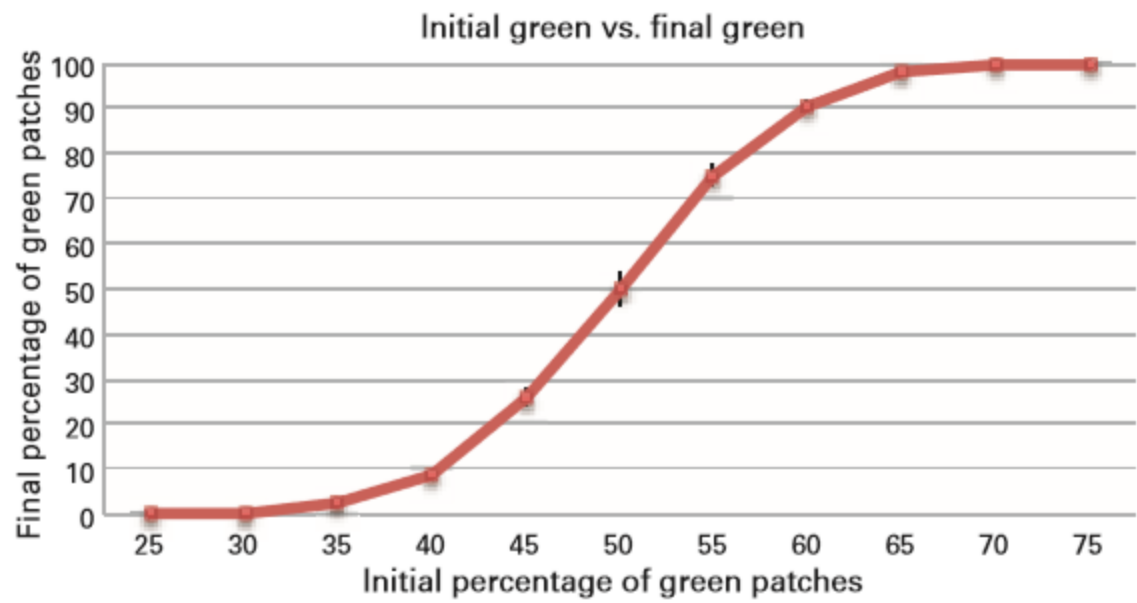
;; if the votes have stabilized, we stop the simulation
if not any-votes-changed? [ stop ]
tick
end

to recolor-patch ;; patch procedure
  ifelse vote = 0
    [ set pcolor green ]
    [ set pcolor blue ]
end

;; This procedure checks to see if the SETUP procedure sets up the model with
;; roughly expected numbers, given the value of the initial-green-pct slider
to check-setup
  let expected-green (count patches * initial-green-pct / 100)
  let diff-green (count patches with [ vote = 0 ]) - expected-green
  if diff-green > (.1 * expected-green) [
    print "Initial number of green voters is more than expected."
  ]
  if diff-green < (- .1 * expected-green) [
    print "Initial number of green voters is less than expected."
  ]
end

; Copyright 2008 Uri Wilensky.
; See Info tab for full copyright and license.

```



-
- Past research methodologies for sensitivity analysis include Active Nonlinear Testing.
- We have seen the details of Sensitivity Analysis and Robustness
- Credits : Uri Wilensky book

Lesson 152 : Verification Benefits and Issues

[click here to download ppt](#)

Here, we are going to examine benefits and issues related to verification.

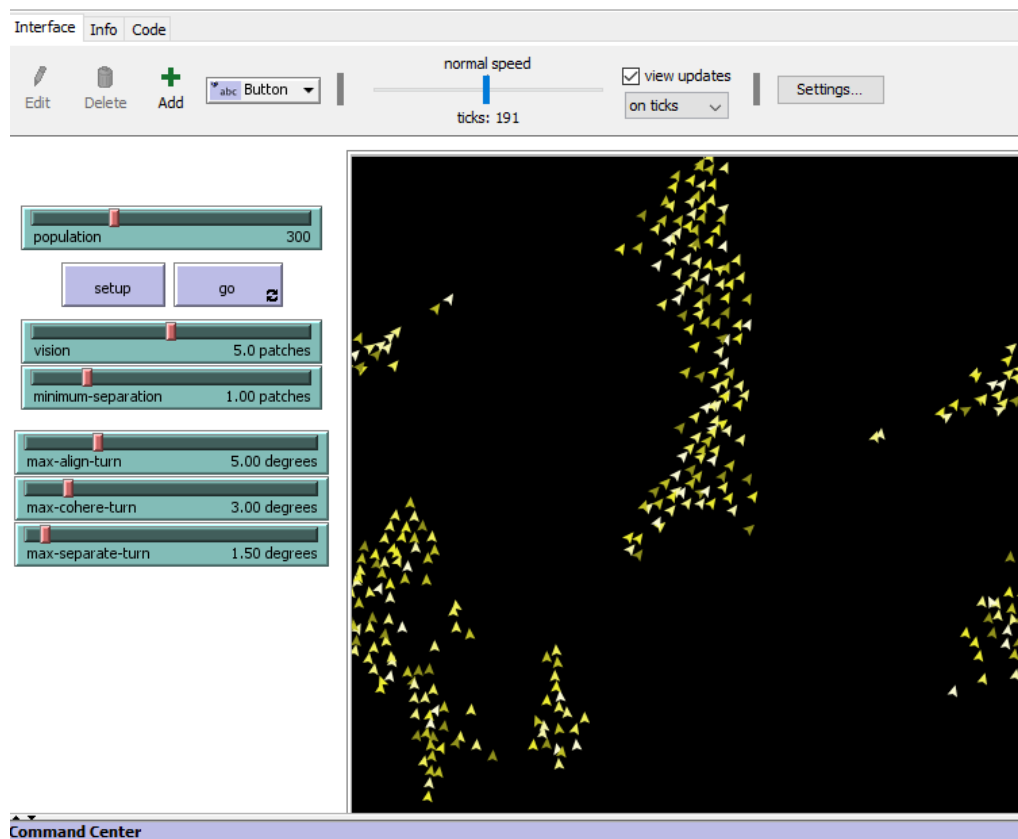
- We need to develop an understanding of the cause of unexpected outcomes and the impact of small changes.
- Implemented model needs to correspond well with the conceptual model.
- This all depends on the model's low level rules as well as an understanding of the mechanisms involved. A bug in the code can produce surprising results.
- Understanding the operation of the model is therefore quite essential. It is also important to note that the verification process is not binary.
- We have discussed verification benefits and issues.
- Credits: Uri Wilensky book

Lesson 153 : Validation

[click here to download ppt](#)

We will discuss the validation of agent based modeling.

- Validation involves corresponding between implemented model and reality.
- The various topics related to validation include:
- Two axis.
- Macrovalidation.
- Face validation.
- Flocking model.
- A classical agent based model.




```

turtles-own [
  flockmates      ;; agentset of nearby turtles
  nearest-neighbor ;; closest one of our flockmates
]

to setup
  clear-all
  create-turtles population
  [ set color yellow - 2 + random 7 ;; random shades look nice
    set size 1.5 ;; easier to see
    setxy random-xcor random-ycor
    set flockmates no-turtles ]
  reset-ticks
end

to go
  ask turtles [ flock ]
  ;; the following line is used to make the turtles
  ;; animate more smoothly.
  repeat 5 [ ask turtles [ fd 0.2 ] display ]
  ;; for greater efficiency, at the expense of smooth
  ;; animation, substitute the following line instead:
  ;; ask turtles [ fd 1 ]
  tick
end

```

```

to flock ;; turtle procedure
  find-flockmates
  if any? flockmates
  [ find-nearest-neighbor
    ifelse distance nearest-neighbor < minimum-separation
    [ separate ]
    [ align
      cohere ] ]
end

to find-flockmates ;; turtle procedure
  set flockmates other turtles in-radius vision
end

to find-nearest-neighbor ;; turtle procedure
  set nearest-neighbor min-one-of flockmates [distance myself]
end

;;; SEPARATE

to separate ;; turtle procedure
  turn-away ([heading] of nearest-neighbor) max-separate-turn
end

;;; ALIGN

to align ;; turtle procedure
  turn-towards average-flockmate-heading max-align-turn
end

```

```

;;; ALIGN

to align ;; turtle procedure
  turn-towards average-flockmate-heading max-align-turn
end

to-report average-flockmate-heading ;; turtle procedure
  ;; We can't just average the heading variables here.
  ;; For example, the average of 1 and 359 should be 0,
  ;; not 180. So we have to use trigonometry.
  let x-component sum [dx] of flockmates
  let y-component sum [dy] of flockmates
  ifelse x-component = 0 and y-component = 0
    [ report heading ]
    [ report atan x-component y-component ]
end

;;; COHERE

to cohere ;; turtle procedure
  turn-towards average-heading-towards-flockmates max-cohere-turn
end

to-report average-heading-towards-flockmates ;; turtle procedure
  ;; "towards myself" gives us the heading from the other turtle
  ;; to me, but we want the heading from me to the other turtle,
  ;; so we add 180
  let x-component mean [sin (towards myself + 180)] of flockmates
  let y-component mean [cos (towards myself + 180)] of flockmates
  ifelse x-component = 0 and y-component = 0
    [ report heading ]
    [ report atan x-component y-component ]
end

;;; HELPER PROCEDURES

to turn-towards [new-heading max-turn] ;; turtle procedure
  turn-at-most (subtract-headings new-heading heading) max-turn
end

to turn-away [new-heading max-turn] ;; turtle procedure
  turn-at-most (subtract-headings heading new-heading) max-turn
end

;; turn right by "turn" degrees (or left if "turn" is negative),
;; but never turn more than "max-turn" degrees
to turn-at-most [turn max-turn] ;; turtle procedure
  ifelse abs turn > max-turn
    [ ifelse turn > 0
      [ rt max-turn ]
      [ lt max-turn ] ]
    [ rt turn ]
end

; Copyright 1998 Uri Wilensky.
; See Info tab for full copyright and license.

```

- Here, we have discussed the validation of agent based models.
- Credits: Uri Wilensky book.

Lesson 154 : Macrovalidation vs. Microvalidation

[click here to download ppt](#)

Here, we will understand the difference between “macrovalidation and microvalidation“ and examine their role in the validation of a model.

- ABMs are built from agents hence we can directly compare them with those that exist in the real world.
- For instance, we can ask whether the agents have properties similar to real birds in flocking model.
- There are some limitation e.g. real birds can fly in three dimensions but our bird agents move in two dimension only. However, these limitations does not make our model invalid.
- We can also build the flocking model in three dimensions and examine the resultant flocks to see if they are relevantly different from the 2D flocks.
- The other major avenue of validation is to investigate the relationship between the global properties of the model and the flocking patterns of real birds, a process called macrovalidation.
- By showing that our model corresponds to the macro-level phenomenon, we further validate that our model is descriptive of real world systems.
- Macrovalidation tells you if you have captured the important parts of the system, whereas microvalidation tells you if you 've captured the important parts of the agent's individual behavior.

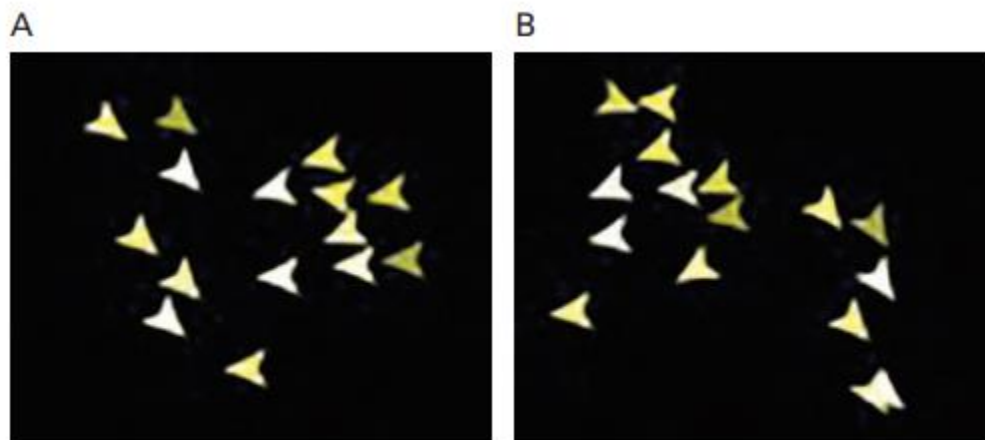


Figure 7.9
Two mini-flocks (A) before and (B) after a collision.

-
- We now understand how macrovalidation and microvalidation compares to each other.
- Credits: Uri Wilensky book.

Lesson 155 : Face Validation vs. Empirical Validation

[click here to download ppt](#)

Here we will discuss how “face validation” is different from “empirical validation”.

- Face validation is the process of showing that the mechanisms and properties of the model look like mechanisms and properties of the real world.
- Empirical validation is making sure that the model generates data. It corresponds to similar patterns of data in the real world.
- Face validity can exist at both micro and macro levels.
- Determining whether the birds in the model correspond to real birds is face microvalidity.
- While determining if the flocks correspond to the appearance of real flocks is face macrovalidity.
- Empirical validation sets a higher bar. Data produced by the model must correspond to empirical data derived from the real-world.
- Measures and numerical data generated both by the model and the actual phenomenon.
- Inputs and outputs in “ the real world ” are often poorly defined. It is hard to isolate and measure parameters from a real-world phenomenon.
- The process of finding the parameters and initial conditions that cause the model to match with real-world data is called calibration.
- We are now able to understand how these four types of validation (micro-face, macro-face, micro-empirical, and macro-empirical) characterize the majority of validation efforts carried out.
- Credits: Uri Wilensky book

Lesson 156 : Validation Benefits and Questions

[click here to download ppt](#)

Here we will understand why it is important to validate a model.

- A valid model can be useful for extracting general principles about the world.
- Changing the mechanisms and parameters can often help predict what might occur in the real world.
- A model is not either valid or invalid:
- A model can be said to be more or less valid based upon how closely it has been compared to the real process it is modeling.
- A model is never inherently valid.
- Its validity comes from the context of what it is being used for.
- Something in the model corresponds to something in reality.
- The user compare his observation's with real world and use it as the basis of his validation.
- Here we have understood the benefits of validation and question it arises.
- Credits: Uri Wilensky book.

Lesson 157 : Replication

[click here to download ppt](#)

Here, we will understand about the “Replication”.

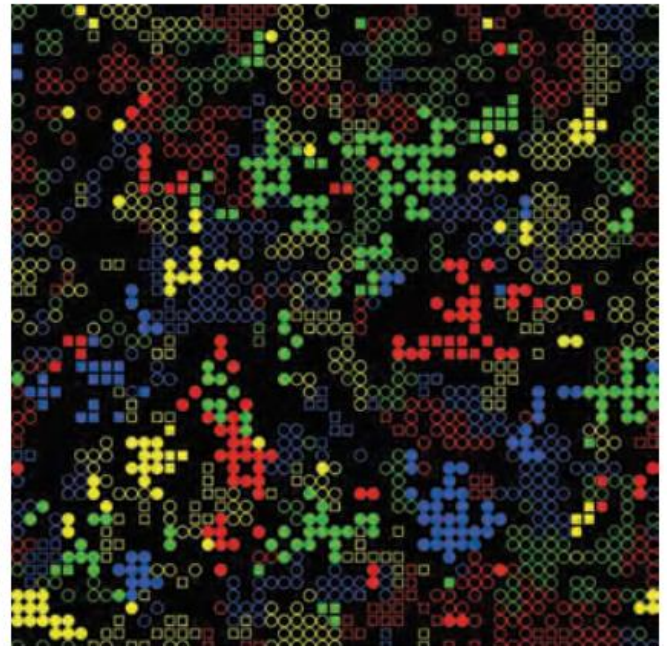
Replication

- Model replication is the implementation by one researcher or group of researchers of a conceptual model previously implemented by someone else.
- Scientists must publish the details of how the experiment was conducted.
- Then subsequent teams of scientists carry out the experiment themselves to ascertain.
- Replicating a physical experiment strengthens original results.
- Comparing both the experimental setup and ensuing results.
- Replicating a computational model serves this same purpose.
- Replicating a computational model increases our confidence.
- New implementation of the model has yielded the same results as the original.

A



B



- This model includes a mechanism for inheritance of strategies.
- The model suggests that “ethnocentric” behavior can evolve under a wide variety of conditions.
- Even when there are no native “ethnocentrics”.
- Here we understand about the basic concepts of replication and how it is useful for the correctness of a model.
- Credits: Uri Wilensky book

Lesson 158 : Replication of Computational Models: Dimensions and Standards

[click here to download ppt](#)

Here, we will understand about the “Replication of Computational Models: Dimensions and Standards”.

Replication

- Replication refers to the creation of a new implementation of a conceptual model based on the previous implementation.

Dimensions

- An original model and replicated model can differ across at least six dimensions.
- Time
- Hardware
- Languages
- Toolkits
- Algorithms
- Authors

Successful Replication

- A successful replication is one in which the replicators are able to establish that the replicated model creates outputs sufficiently similar to the outputs of the original.

Replication Standard

- The criterion by which the replication 's success is judged is called the replication standard.
- Different replication standards exist for the level of similarity between model outputs.

Categories of Replication Standard

- There are three categories of replication standard.
- Numerical identity
- Distributional equivalence.
- Relational alignment.

Data

- ABMs usually produce large amounts of data.
- much of which is usually irrelevant.
- Data that are central to the conceptual model should be measured and tested during replication.

Summary

- Here, we understand about dimensions of conceptual model like time, hardware, toolkit, language, algorithms and author.
- Then we discussed about replication standard and its categories.
- Credits: Uri Wilensky book

Lesson 159 : [click here to download ppt](#)

Overview

- We are going to see the benefits of replication in this section.

Benefits of Replication

- Advances scientific knowledge.
- Model verification.
- Model validation.
- Shared understanding.
- Shared understanding.
- Creation of sets of terms, idioms and best practices.
- Communicate about model.
- Model verification.
- Distinct implementations. producing same results.
- Conceptual model grows by capturing confidence.
- Correction made if there is any difference found in implemented model and replication.
- Model validation.
- Correspondence between the outputs.
- Validation of a model on the basis of output closer to real-world.
- Reevaluating the original mapping.
- Researchers investing and getting interested in it through replication
- Describing the modeling process through developing a language.
- Culture of replication fosters.
- "mean" and "standard deviation" applying them to tests, overtime, replication of ABM experiment to understand "time step" and "shuffled lists".

Summary

- Here, we understood the benefits of replication in modeling.
- Credits: Uri Wilensky book.

Lesson 160 : Recommendations For Model Replicators

[click here to download ppt](#)

Overview

- Here, we will learn about recommendations for model replicators.

Recommendations for model replicators

- RS is to produce the level of precision to establish hypothesis regularity.
- Examples : “numerical identity”, “distributional equivalence” and “relational alignment”.
- How detailed the description of the conceptual model is in the original paper or to communicate with authors or original model.
- Beneficial to delay the contact with original author until after first attempt to recreate the original model.
- Differences between public conceptual model and an implementation can be interesting and resulting in new discoveries.
- Becoming familiar with the toolkit in which original model was written.
- Better understanding of original model.
- Replicator understands the subtler working.
- Deliberately implementing a strategy.
- Obtain the source code of original model.
- Effective for illuminating discrepancies in the two model implementations.
- Groupthink
- Unconsciously adopting some of the practices of the original model developer.

Lesson 161 : Recommendations for Model Authors

[click here to download ppt](#)

Overview

- Here, we will understand the recommendations for the model authors.

Recommendations for the model authors

- Research paper should be well specified.
- Complete source code for the model may not sufficient.
- Model authors must make their source code publicly available or at least the “pseudo code”.
- To what extent the model developer presents a sensitivity analysis.
- Small modification to the original model can effect results drastically.
- These sensitive differences should be published by the authors.
- Original author may not be the original implementer.
- Implementation will not be veridical.
- Model authors implement their own models using “ low-threshold ” and toolkits.

- Authors to examine their conceptual models through the lens of a potential model replicator.

Summary

- We have understood the recommendations for model authors in modelling.
- Credits: Uri Wilensky book

Table 7.1

Details to be included in published replications. For each category, sample options to choose from are listed.

Categories of Replication Standards:

Numerical Identity, Distributional Equivalence, Relational Alignment

Focal Measures:

Identify particular measures used to meet goal

Level of Communication:

None, Brief email contact, Rich discussion and personal meetings

Familiarity with Language/Toolkit of Original Model:

None, Surface understanding, Have built other models in this language/toolkit

Examination of Source Code:

None, Referred to for particular questions, Studied in-depth

Exposure to Original Implemented Model:

None, Reran original experiments, Ran experiments other than original ones

Exploration of Parameter Space:

Examined results from original paper, Examined other areas of the parameter space

Summary

- We have understood the recommendation for model replicators in modeling.
- Credits: Uri Wilensky book.

Lesson 162 : Complex Adaptive Systems

[click here to download ppt](#)

Overview

- Here, we will understand the basics about complex adaptive systems (CAS).

Complex Adaptive Systems

- Complexity
- Adaptation
- Systems

Ludwig von Bertalanffy

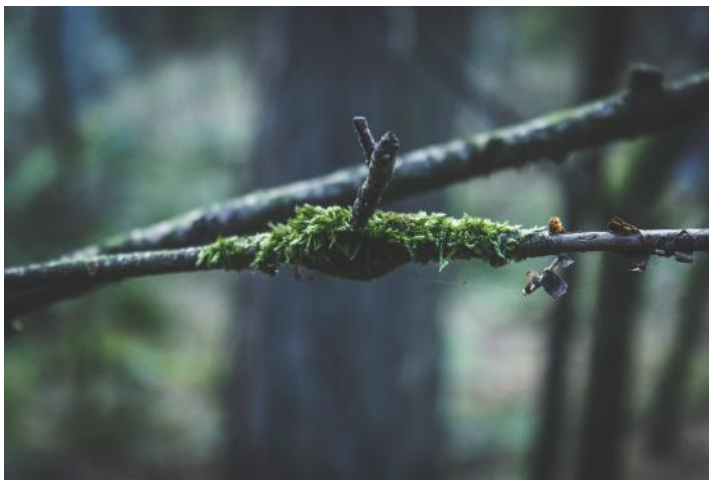
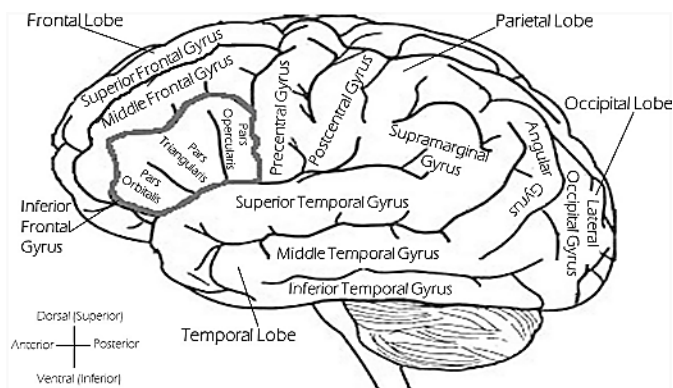


- Father of Systems Theory
- Wrote “General Systems Theory”, published in US following WW2.

- The systems approach is an old concept.
- The approach stands on the assumption that breaking down of a complex concept into simple easy to understand units helps in better understanding of the complexity.
- Ludwig von Bertalanffy first proposed it as 'General System Theory'

- A system is a delineated part of the universe which is distinguished from the rest by an imaginary boundary. (NECSI)
- Properties of the system, the properties of the universe excluding the system which affect the system and the interactions / relationships.
- An adaptive system (or a complex adaptive system, CAS) is a system that changes its behavior in response to its environment.
- The adaptive change that occurs is often relevant to achieving a goal or objective. (NECSI)
- Complexity is:
- ...the (minimal) length of a description of the system.
- ...the (minimal) amount of time it takes to create the system.
- Here the length of a description is measured in units of information.

Examples



Summary

- We have learnt about the basic ideas of Complex Adaptive Systems.
- Credits: Uri Wilensky Book.

Lesson 163 : History

[click here to download ppt](#)

Overview

- Here, we will understand about the historical perspective of CAS

History

- Started with the concepts of General Systems Theory
- Biologists focused primarily on individual cells
- Social Scientists focused on connections
- John Holland and cas/CAS

Summary

- We have learnt about the historical perspective of CAS.
- Credits: Niazi & Hussain book.

Lesson 164 : Complexity

Overview

- Here, we will understand about the basic ideas of complexity

Complexity

- Disambiguation from Computational Complexity
- Nonlinearity
- Large number of variables
- Chaos theory
- The term "chaos" is popularly used to refer to disorder or confusion.
- In science, chaos is an important conceptual paradox that has a precise mathematical meaning:
- A chaotic system is a deterministic system that is difficult to predict.
- A deterministic system is defined as one whose state at one time completely determines its state for all future times.
- So what does it mean for a chaotic system to be difficult to predict?
- Butterfly effect

Summary

- We have learnt about the basics of Complexity in CAS.
- Credits: Niazi and Hussain book.

Lesson 165 : Adaptation

[click here to download ppt](#)

Overview

- Here, we will understand about adaptation in CAS.

Adaptation

- Adaptive changes in interactions
- Who changes
- What changes
- When changes
- How changes

Summary

- We have learnt about the adaptation in CAS.
- Credits: Niazi and Hussain book.

Lesson 166 : Systems Approach

Overview

- Here, we will understand about the Systems approach.

Systems Approach

- Concept of a System - boundaries
- Many systems can be correlated
- Example: Life/Humans/Society
- Whole vs. parts
- Reductionist vs. Wholistic approach

Summary

- We have learnt about the systems perspective of things.
- Credits: Niazi and Hussain book.

Lesson 167 : Modeling of CAS

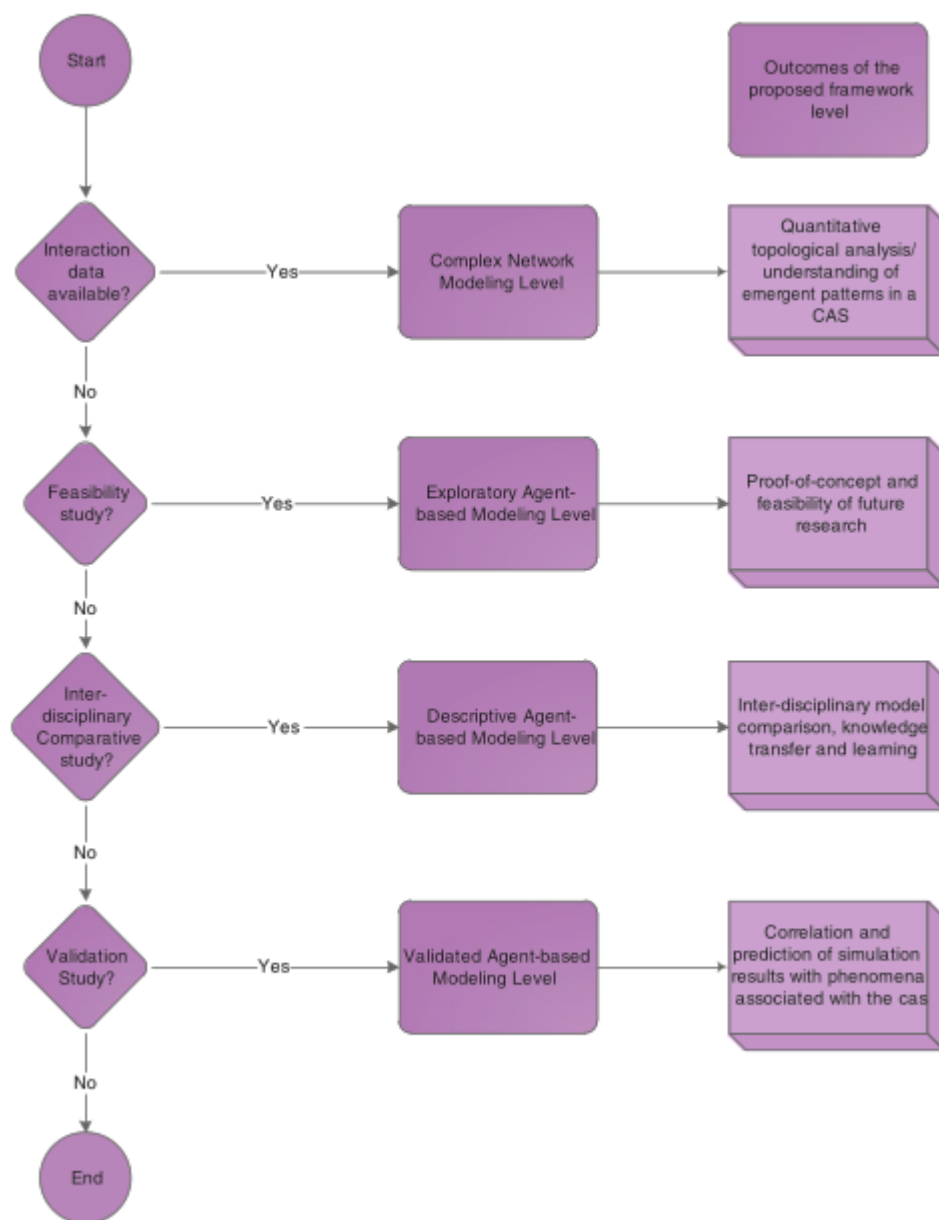
[click here to download ppt](#)

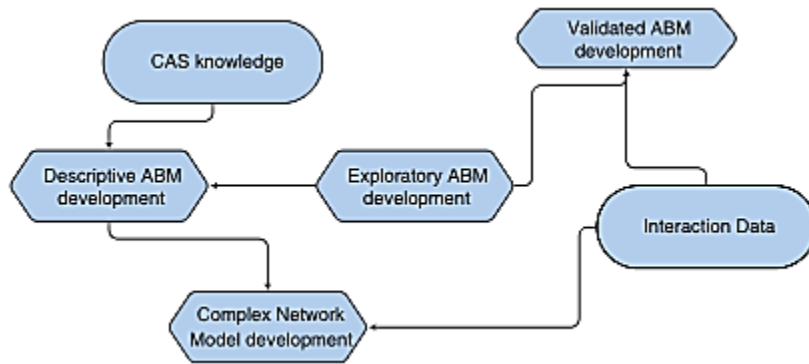
Overview

- Here, we will understand about the problems encountered in modeling of CAS.

Modeling of CAS

- Cognitive Agent-based (CABC) Framework





Summary

- We have learnt about the modeling of CAS using CAB framework
- Credits: Niazi and Hussain book.

Lesson 168 : Agent-based Approach

[click here to download ppt](#)

Overview

- Here, we will understand about the agent-based approach to modeling

Agent-based Approach

- Agent – something that acts
- There is a concept of agent in domains as distinct as:
 - AI/CS/Robotics
 - Sociology (Individual)
 - Biology
 - Ecology
 - Etc.
- CABC framework has 3 levels for Agent-based modeling
- Exploratory Agent-based modeling (EABM)
- Descriptive Agent-based Modeling (DREAM)
- Validated Agent-based Modeling (VOMAS-based Modeling or VABM)
- Exploratory ABM
- DREAM
- Virtual Overlay Multiagent System (VOMAS)
- Validated Agent-based Modeling

Summary

- We have learnt about various approaches for modeling using Agents and the CABC framework.
- Credits: Niazi and Hussain book.

Lesson 169 : Complex Network-based Modeling

[click here to download ppt](#)

Overview

- Here, we will understand about complex networks/graphs and modeling using the CABC Level 1

Complex Network-based Modeling

- What is a graph?
- Graph/Network
- Same entity different terms in different domains
- Network = weighted graph
- Different aspects of modeling using networks
- Labels
- arcs/lines
- Directed/Undirected
- Centrality-based
- Ego
- Sentiment graphs
- Community Detection
- Key benefits
- Very well-defined
- Lot of free tools
- Lot of community support
- Well-defined mathematical models
- Cons
- Difficult to figure out
- Difficulty in graph mining – lack of expertise in the community
- Missing data can be very difficult
- Requires complete data
- Complexity in deciding data boundaries

Summary

- We have learnt about complex-network modeling using the CABC framework
- Credits: Niazi and Hussain book.

Lesson 170 : Cognitive Agent-based Computing Framework

[click here to download ppt](#)

Overview

- Here, we will understand how to put modeling and simulation to use employing the CABC framework.

Cognitive Agent-based Computing Framework

- Data and Models
- How much data is needed
- Why choose Agent-based Modeling?
- Why choose Complex network modeling?
- When to choose EABM?
- When to choose DREAM?
- When to choose VOMAS?

Summary

- We have learnt about putting the CABC framework to work.
- Credits: Niazi and Hussain book.

Lesson 171 : Useful Statistical Models

[click here to download ppt](#)

Overview

- Here, we will learn about the useful statistical models.

Useful Statistical Model

- The models that are useful in the case of limited data are:
- Queueing systems.
- Inventory and supply-chain systems.
- Reliability and maintainability.
- Limited data.
- Other distributions.

Queueing Systems

- Simulation solved the waiting line problems.
- Interval and service time patterns were given in queueing examples.
- In these examples service and arrival time was probabilistic.
- But it is possible to have constant arrival time and constant service time.
- “Arrivals” can occur due to many reasons like machine break downs.
- Exponential distribution used to simulate the random exponential distribution of services.
- For special cases truncated normal distribution can be utilized.

Inventory and supply-chain systems

- Three random variables
- Number of units demanded per order or per time period.
- Time between demands.
- Lead time.
- Simple inventory systems have constant demand and lead time constant or zero.
- Mathematical tractability based demand and lead time in inventory theory text could be invalid.
- Variety of demand patterns are satisfied by geometric, Poisson and negative binomial distributions.

Reliability and Maintainability

- Failure time is modeled with many distributions.
- When failure occurs randomly model distribution is considered as exponential.
- Modeling standby redundancy required for gamma distribution.

Limited Data

- Three distributions.
- Uniform distribution
- Triangular distribution
- Beta distribution
- Uniform distribution is just a special case of beta distribution

Summary

- Here, we discussed about useful statistical model, queuing systems, Inventory and supply-chain systems, Limited data, Reliability, and maintainability.
- Credits: Jerry Banks book.

Lesson 172 : Bernoulli Distribution

[click here to download ppt](#)

Overview

- Here, we will learn about the Bernoulli distribution.

Bernoulli Distribution

- Bernoulli trail is one of the simplest experiment you can do in statistic and probability which has one out of two possible outcome.
- Bernoulli distribution is a discrete probability distribution for Bernoulli trail which has one out of two possible outcomes success or fail.
- Let us consider an example which consists of n trails and each of which can be a success or a failure.
- $X_j = 1$ if the jth experiment is success
- $X_j = 0$ if its failure.
- N trails and each trail has only two out comes success or failure.

$$p(X_1, X_2, \dots, X_n) = p_1(X_1) \cdot p_2(X_2) \cdot \dots \cdot p_n(X_n)$$

$$p_j(x_j) = p(x_j) = \begin{cases} p, & x_j = 1, j = 1, 2, \dots, n \\ 1 - p = q, & x_j = 0, j = 1, 2, \dots, n \\ 0, & \text{otherwise} \end{cases}$$

- Success here happens with probability (p) and failure at (p – 1).
- X is a discrete random variable. It takes value 1 in case of success and 0 in case of failure.

- Below is the expected value of random variable X is calculated a

$$E(X_j) = 0 \cdot q + 1 \cdot p = p$$

- Below is the variance of the random variable X is calculated as:

$$V(X_j) = \left[(0^2 \cdot q) + (1^2 \cdot p) \right] - p^2 = p(1 - p)$$

Summary

- We have understand about the Bernoulli distribution.
- Credits: Jerry Banks book.

Lesson 173 : Binomial Distribution

[click here to download ppt](#)

Overview

- Here, we will learn about the binomial distribution.

Binomial Distribution

- Binomial distribution is the discrete probability distribution with parameters n and p of the number of successes in the sequence of n independent experiments.
- Each n and p with its own boolean-value.
- Random variable.
- Bernoulli trial's random variable X has binomial distribution:

$$p(x) = \begin{cases} \binom{n}{x} p^x q^{n-x}, & x = 0, 1, 2, \dots, n \\ 0, & \text{otherwise} \end{cases}$$

- This equation is for calculating the probability of a specific outcome with all successes.

$$P(\overbrace{SSS \dots SS}^{x \text{ of these}} \overbrace{FF \dots FF}^{n-x \text{ of these}}) = p^x q^{n-x}$$

- S = success.
- In first X trials.
- $n-x = n - x = f$ (failures).

$$\binom{n}{x} = \frac{n!}{x!(n-x)!}$$

- $q = 1 - p$
- This equation shows the required number of successes and failures.

$$p(x) = \begin{cases} \binom{n}{x} p^x q^{n-x}, & x = 0, 1, 2, \dots, n \\ 0, & \text{otherwise} \end{cases}$$

- Again this equation can be used to calculate mean and variance of binomial distribution.
- X = sum of independent Bernoulli random variables. Then:

$$X = X_1 + X_2 + \dots + X_n$$

- Expected value $E(X)$

$$E(X) = p + p + \dots + p = np$$

- n = total number of experiments.
- p = probability of each experiment getting successful result.
- X = binomially random variable.
- Variance $V(X)$

$$V(X) = pq + pq + \dots + pq = npq$$

- n = total number of experiments.
- p = probability of each experiment getting successful result.
- X = binomially random variable.
- q = probability of failure in each experiment.

Summary

- We have understand about the Binomial distribution.
- Credits: Jerry Banks book.

Lesson 174 : Poisson Distribution

[click here to download ppt](#)

Overview

- Here, we will learn about the Poisson distribution.

Poisson Distribution

- A discrete frequency distribution that gives the probability of independent events occurring in a fixed interval of time is known as the Poisson distribution.

$$p(x) = \begin{cases} \frac{e^{-\alpha} \alpha^x}{x!}, & x = 0, 1, \dots \\ 0, & \text{otherwise} \end{cases}$$

- $X = 0, 1, 2, 3, \dots$
- $e = e = 2.71828$
- $a = \alpha =$ mean number of successes in the given time interval or region of space.
- $a > 0, \alpha > 0$

$$E(X) = \alpha = V(X)$$

- This is the important property of Poisson distribution that is mean and variance is equal to α .

$$F(x) = \sum_{i=0}^x \frac{e^{-\alpha} \alpha^i}{i!}$$

- This is the cumulative distribution function.

Summary

- We have understand the Poisson distribution.
- Credits: Jerry Banks book.

Lesson 175 : Uniform Distribution

[click here to download ppt](#)

Overview

- Here, We will understand “Uniform Distribution” also known as “rectangular distribution”.

Uniform Distribution

- The uniform distribution is a continuous distribution.
- It takes values within a specified range.
- Probability density function for a uniform distribution always take a random number on [0,1].

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{if } a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

- Consider a random variable X on the interval [a, b] distributed uniformly. The, equation is given by:
- $X = a + (b - a)R$
- For the derivative of function.
- First, compute the cdf of variable X .
- Second, set $F_X = R$ on the range of X .
- Third, solve the equation F_X in terms of R for X .
- **Step 1:**
 - The cdf for the equation is :

$$F(x) = \begin{cases} 0 & : x < 0 \\ x & : 0 \leq x < 1 \\ 1 & : x \geq 1. \end{cases}$$

- If X takes only discrete values 0 and 1 then:

$$F(x) = \begin{cases} 0 & : x < 0 \\ 1/2 & : 0 \leq x < 1 \\ 1 & : x \geq 1. \end{cases}$$

- **Step 2:**

- Set

$$F(x) = (X - a)(X - b)R$$

- Step 3:
- Solve for X in terms of R which proves.

$$X = a + (b - a)R$$

Summary

- Here, we understood about the fundamentals of Uniform Distribution.
- Credits, Discrete Event System Simulation, Jerry Banks.

Lesson 176 : Exponential Distribution

[click here to download ppt](#)

Overview

- Here, we will understand About “Exponential Distribution”.

Exponential Distribution

- The Exponential Distribution is a continuous valued probability distribution.
- Takes positive real values and λ which controls the shape of the distribution.
- The probability density function

$$f(x; \lambda) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0, \\ 0 & x < 0. \end{cases}$$

- This can also be defined as right-continuous Heaviside step function.

$$f(x; \lambda) = \lambda e^{-\lambda x} H(x)$$

- Here λ is called the rate parameter.
- The cumulative distribution function

$$F(x; \lambda) = \begin{cases} 1 - e^{-\lambda x} & x \geq 0, \\ 0 & x < 0. \end{cases}$$

- This can also be defined as heaviside step function

$$F(x; \lambda) = (1 - e^{-\lambda x}) H(x)$$

- Here λ is the number of occurrences per time unit.

The inverse transform technique is used when the form of cdf is so simple that its f^{-1} can be computed easily.

This technique for exponential distribution consist of four steps defined next.

- **Step 1:**
- Compute the cdf of random variable X given

$$F(x) = 1 - e^{-\lambda x}, x \geq 0.$$

- **Step 2**
- Set $F(X) = R$ on the range of X which becomes as.

$$1 - e^{-\lambda X} = R \text{ on the range } x \geq 0.$$

- **Step 3**
- Solve the equation $F(X) = R$ for X in terms of R as follows:

$$1 - e^{-\lambda x} = R$$

$$e^{-\lambda x} = 1 - R$$

$$-\lambda X = \ln(1 - R)$$

$$X = -\frac{1}{\lambda} \ln(1 - R)$$

- Here, X is called random-variate generator.
- **Step 4**
- **Generate uniform random numbers to compute desired random variates.**

$$X_i = F^{-1}(R_i)$$

- For exponential case.

$$X_i = -\frac{1}{\lambda} \ln(1 - R_i)$$

- If $i=1,2,3 \dots$ $i = 1, 2, 3 \dots$ replace $1 - R_i^{1 - R_i}$ with $R_i^{R_i}$ as follows:

$$X_i = -\frac{1}{\lambda} \ln R_i$$

- Which proves that both $R_i^{R_i}$ and $1 - R_i^{1 - R_i}$ are uniformly distributed on $[0, 1]$.

Summary

- We understand about “Exponential Distribution”
- Credits, Discrete Event System Simulation, Jerry Banks.

Lesson 177 : Triangular Distribution

[click here to download ppt](#)

Overview

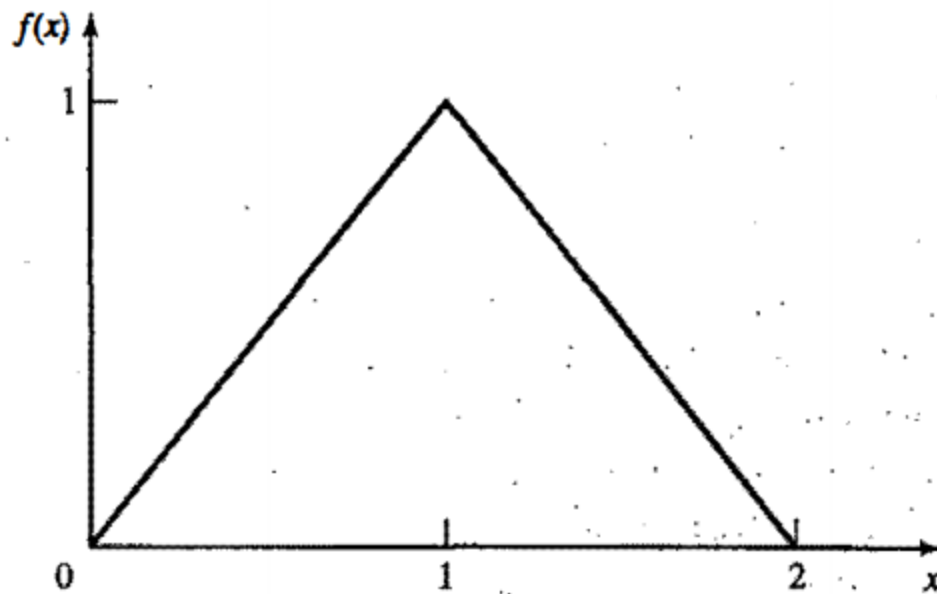
- Here, we will discuss about “Triangular Distribution”.

Triangular Distribution

- A triangular distribution is a continuous probability distribution with a probability density function shaped like a triangle defined by
- The minimum value a
- The maximum value b
- The peak value c
- Where $a < b$ and $a \leq c \leq b$
- Probability density function with variable x

$$f(x) = \begin{cases} x, & 0 \leq x \leq 1 \\ 2-x, & 1 < x \leq 2 \\ 0, & \text{otherwise} \end{cases}$$

- Where endpoints are $0, 2$ and mode at 1 .



- **Triangular Distribution**
- Its cumulative distribution function is given as :

$$F(x) = \begin{cases} 0, & x \leq 0 \\ \frac{x^2}{2}, & 0 < x \leq 1 \\ 1 - \frac{(2-x)^2}{2}, & 1 < x \leq 2 \\ 1, & x > 2 \end{cases}$$

- For $0 \leq X \leq 1$, $0 \leq X \leq 1$:

$$R = \frac{x^2}{2}$$

- Which implies that $x=2R \quad x = \sqrt{2R}$
- For $1 \leq X \leq 2$, $1 \leq X \leq 2$:

$$R = 1 - \frac{(2 - X)^2}{2}$$

- Which implies that $x=2-2(1-R) \quad x = 2 - \sqrt{2(1-R)}$
- Thus X is generated as:

$$X = \begin{cases} \sqrt{2R}, & 0 \leq R \leq \frac{1}{2} \\ 2 - \sqrt{2(1-R)}, & \frac{1}{2} < R \leq 1 \end{cases}$$

Summary

- Here, we have understood the basics of Triangular Distribution, its pdf, and cdf.

Lesson 178 : CHARACTERISTICS OF QUEUEING SYSTEMS

[click here to download ppt](#)

Overview

- Here, we will learn about characteristics of queueing systems.

CHARACTERISTICS OF QUEUEING SYSTEMS

- Customers and servers are the key elements of a queueing system.
- A Customer is anything that visits a facility and requires some service.
- Servers the service provider.
- Not every time the customer visits the service facility.

Table 6.1 Examples of Queueing Systems

<i>System</i>	<i>Customers</i>	<i>Server(s)</i>
Reception desk	People	Receptionist
Repair facility	Machines	Repairperson
Garage	Trucks	Mechanic
Tool crib	Mechanics	Tool-crib clerk
Hospital	Patients	Nurses
Warehouse	Pallets	Fork-lift Truck
Airport	Airplanes	Runway
Production line	Cases	Case-packer
Warehouse	Orders	Order-picker
Road network	Cars	Traffic light
Grocery	Shoppers	Checkout station
Laundry	Dirty linen	Washing machines/dryers
Job shop	Jobs	Machines/workers
Lumberyard	Trucks	Overhead crane
Sawmill	Logs	Saws
Computer	Jobs	CPU, disk, CDs
Telephone	Calls	Exchange
Ticket office	Football fans	Clerk
Mass transit	Riders	Buses, trains

Summary

- We have learnt about the characteristics of the Queueing systems.
- Credits: Jerry Banks books.

Lesson 179 : The Calling Population

[click here to download ppt](#)

Overview

- Here, we will learn about the calling population.

The Calling Population

- Calling population is the population of potential customers.
- Finite or infinite.
- Example of the bank having five machines and each machine is a customer and worker is its server.
- Infinite calling population when the system has a large population of potential.
- Examples are banks and restaurants.
- Arrival rate is the main difference between the finite and infinite population models.
- Infinite population system is not affected by the arrival rate.
- The Finite system depends on the no of customers waiting and being served.

Summary

- We have learnt about the calling population and what are effects of finite and infinite population.
- Credits: Jerry Banks book.

Lesson 180 : System Capacity

[click here to download ppt](#)

Overview

- Here, we will learn about the system capacity.

System Capacity

- Every queueing system has some capacity.
- They have different number of customers entering the system.
- There is a limit for every queueing system to hold the number of customers.
- If the entering system finds the capacity full it will return to the calling population.
- Some systems have unlimited capacity
- Like tickets sold for a concert to students and many students can wait to purchase them.
- In limited systems, there is a distinction in arrival time and effective arrival time.

Summary

- We have learnt about the system capacity.
- Credits: Jerry Banks book.

Lesson 181 : The Arrival Process

[click here to download ppt](#)

Overview

- Here, we will learn about the arrival process.

Arrival Process

- For infinite population models the arrival process characterized in terms of interarrival times of successive customers.
- Probability distribution is used to characterize the interarrival times when arrival is occurred in random time.
- Poisson arrival process is the most important for random arrivals.
- If A_n = inter-arrival
- Between $n-1$ (customer) and n (customer) then
- A_n is exponentially distributed with mean $1/\lambda$ time units.
- The arrival rate is λ customers per time unit.
- So the number of arrivals in a time interval of length t has Poisson distribution with mean λt customers.
- In restaurants and banks the Poisson arrival process has been employed successfully.
- Scheduled arrival is the second arrival class.
- Inter-arrival could be either constant or constant plus or minus a small random amount to represent early or late arrivals.
- Third situation occurs when one customer is always present in the queue.
- So we do not have an idle server.
- For finite population the arrival process is different.
- We define the customer as pending when he is out of the queue system and a member of potential calling population.
- **Arrival Process**

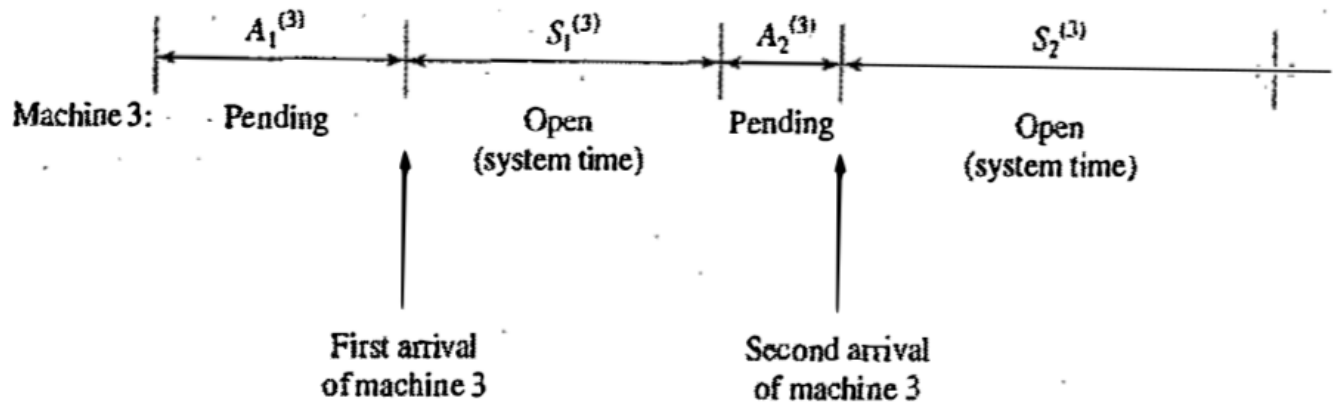


Figure 6.2 Arrival process for a finite-population model.

- Important application of finite population model is machine repair problem.

Summary

- We have learnt about the finite and infinite arrival processes.
- Credits: Jerry Banks book.

Lesson 182 : QUEUE BEHAVIOR AND QUEUE DISCIPLINE

[click here to download ppt](#)

Overview

- Here, we will learn about the Queue behavior and Queue discipline.

Queue Behavior

- The actions of the customer while waiting in the queue system for his service is known as the queue behaviour.
- The customers can baulk, renege or jockey.

Queue Discipline

- The logical ordering of the customers in a queue and determining that which customer should come for service when the server is free.
- FIFO, LIFO, SIRO, SPT and PR are some common queue disciplines.

Summary

- We have learnt about the Queue behavior and Queue discipline.
- Credits: Jerry Banks book.

Lesson 183 : Service Times and the Service Mechanism

[click here to download ppt](#)

Overview

- Here, we will learn about the Service Times and the Service Mechanism.

Service Times and the Service Mechanism

- Service times of successive arrivals are denoted by $S_1 S_2 \dots$
- Their duration can be random or constant.
- This latter case is characterized as a sequence of random or identically distributed variables.
- The exponential, Weibull, gamma, lognormal and truncated normal distributions are used for service times.
- Sometimes the services are identically distributed for all same customers. Same type customers can have different service time.
- Number of service centers and interconnecting queues are present in a queue system.
- Parallel working servers for each service centre.
- Server = c
- Single server ($c = 1$)
- Multiple servers ($1 < c < \infty$)
- Unlimited servers ($c = \infty$)

Service Times and the Service Mechanism

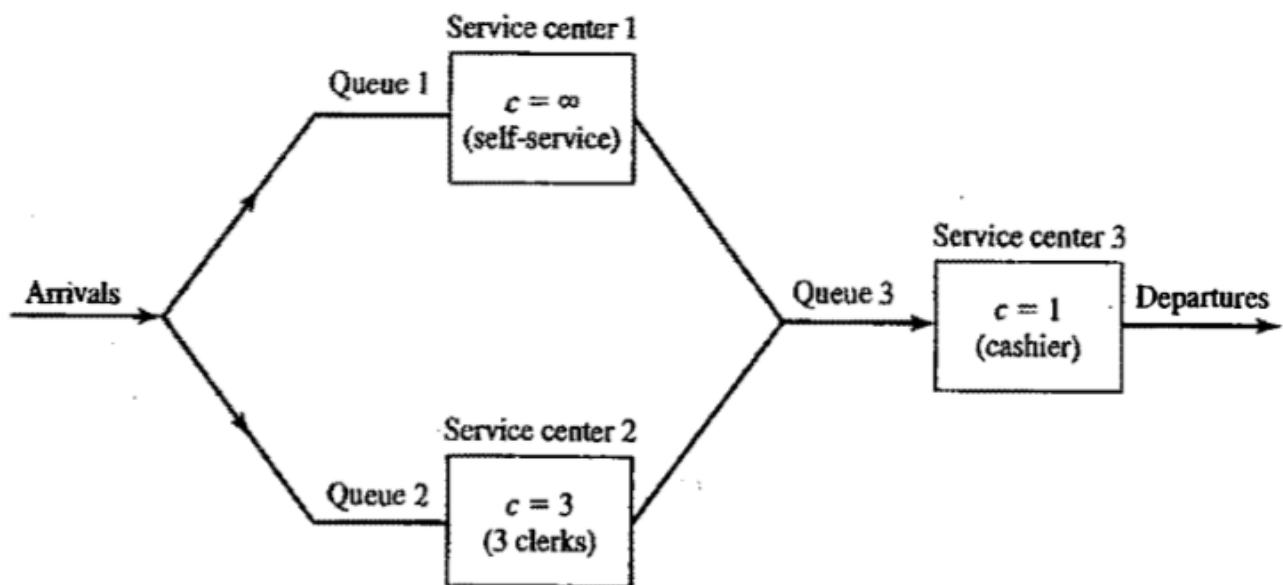


Figure 6.3 Discount warehouse with three service centers.

Service Times and the Service Mechanism

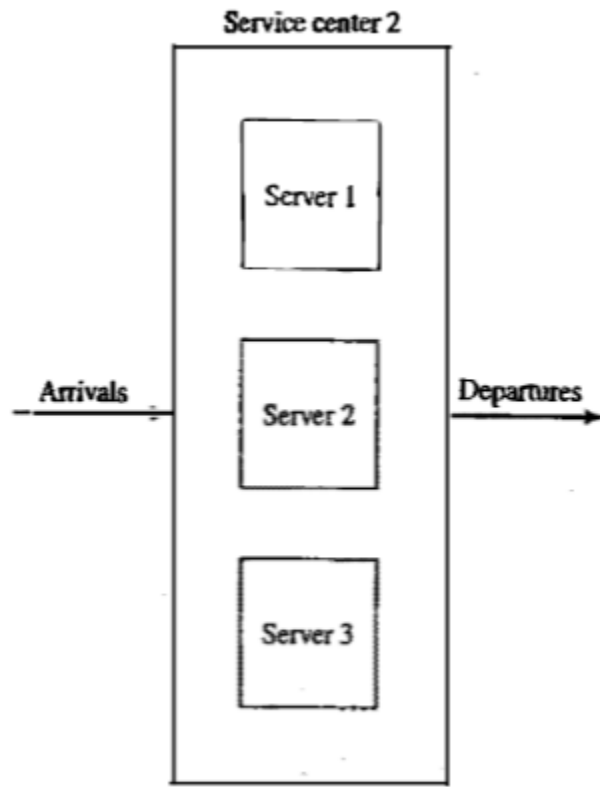


Figure 6.4 Service center 2, with $c = 3$ parallel servers.

Service Times and the Service Mechanism

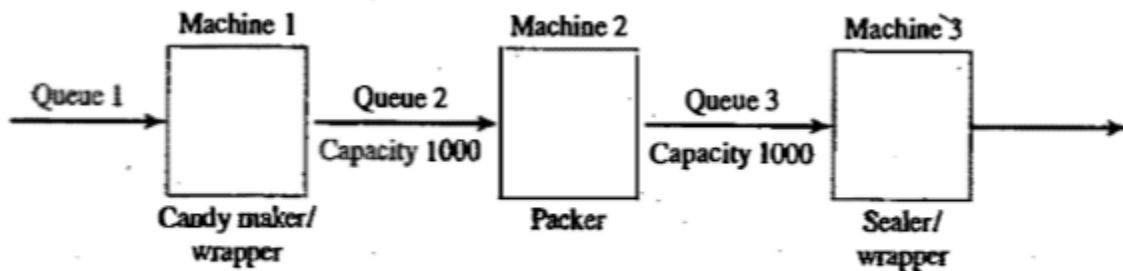


Figure 6.5 Candy-production line.

Summary

- We have learnt about the Service times and the Service mechanism.
- Credits: Jerry Banks book.

Lesson 184 : Queueing Notation

[click here to download ppt](#)

Overview

- Here, we will learn about the Queueing notation.

Queueing Notation

- Proposed by Kendall [1953]
- For parallel server systems.
- A/B/c/N/K based on this format.
- “A” represents the inter-arrival-time distribution.
- “B” represents the service-time distribution.
- “C” represents the number of parallel servers.
- “N” represents the system capacity.
- “K” represents the size of the calling population.
- Common symbols for A and B include M (exponential or Markov)
- D (constant or deterministic).
- Ek (Erlang of order k).
- PH (phase-type).
- H (hyperexponential).
- G (arbitrary or general)
- GI (general independent).

Queueing Notatio

Table 6.2 Queueing Notation for Parallel Server Systems

P_n	Steady-state probability of having n customers in system
$P_n(t)$	Probability of n customers in system at time t
λ	Arrival rate
λ_e	Effective arrival rate
μ	Service rate of one server
ρ	Server utilization
A_n	Interarrival time between customers $n - 1$ and n
S_n	Service time of the n th arriving customer
W_n	Total time spent in system by the n th arriving customer
W_n^q	Total time spent in the waiting line by customer n
$L(t)$	The number of customers in system at time t
$L_q(t)$	The number of customers in queue at time t
L	Long-run time-average number of customers in system
L_q	Long-run time-average number of customers in queue
w	Long-run average time spent in system per customer
w_q	Long-run average time spent in queue per customer

Lesson 185 : Long-run Measures Of Performance Of Queueing Systems

[click here to download ppt](#)

Overview

- Here, we will learn about the long-run measures of performance of queueing systems.

LONG-RUN MEASURES OF PERFORMANCE

- The primary long-run measures of performance of queueing systems are:
- Long-run time-average number of customers in the system (L).
- In the queue (LQ).
- Long-run average time spent in the system (w).
- In the queue (wQ) per customer.
- Server utilization.
- A Proportion of time that a server is busy (p).
- Waiting line and mechanism is a system.
- System can also be sub-system of queue system.
- “queue” means waiting line alone.
- Other measurements include long-run proportion of customer who is delayed longer than t^* time units.
- Customers turned away due to capacity constraints.
- Waiting line containing more than k_0 customers.
- G/G/c/N/K is defined.
- Their performance relationships and simulation run.
- The two types of estimators are:
- An ordinary sample average.
- A time-integrated sample average.

Summary

- We have learnt about the long-run measures of performance of queueing systems.
- Credits: Jerry Banks book.

Lesson 186 : Properties Of Random Number

[click here to download ppt](#)

Overview

- Here, we are going to learn about properties of random numbers.

Properties of Random Numbers

- Uniformity and independence are two important statistical properties.
- Each sample must be drawn from continuous uniform distribution between 0 and 1.

$$f(x) = \begin{cases} 1, & 0 \leq x \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

- The density function and the expected value of R_i^{Ri} is:

$$E(R) = \int_0^1 x dx = \frac{x^2}{2} \Big|_0^1 = \frac{1}{2}$$

- And the variance is:

$$V(R) = \int_0^1 x^2 dx - [E(R)]^2 = \frac{x^3}{3} \Big|_0^1 - \left(\frac{1}{2}\right)^2 = \frac{1}{3} - \frac{1}{4} = \frac{1}{12}$$

- Generated by a digital computer.
- Individual computation is expensive so computationally efficient method should be used.
- Routine should be portable for different computers and programming languages.
- Routine having a long cycle.
- Replicable random numbers.
- Random numbers should closely approximate the ideal statistical properties.
- Techniques to generate random is easy to develop.
- Techniques that produce sequences that are independent and uniformly distributed are difficult.

Summary

- We have learnt about the properties of the random numbers.
- Credits: Jerry Banks book.

Lesson 187 : Linear Congruential Method

[click here to download ppt](#)

Overview

- Here, we will learn about the linear congruential method.

Linear Congruential Method

- Widely used technique.
- Yields sequences with longer periods.
- Produces the sequence of integers.
- Between zero and $m - 1$.

$$X_{i+1} = (aX_i + c) \bmod m, \quad i = 0, 1, 2, \dots$$

- X_0 = seed.
- a = multiplier.
- c = increment ($c \neq 0$).
- m = modulus.
- This is called mixed congruential method.
- When $c = 0$ then known as multiplicative congruential.
- Interval $[0,1]$ is divided into n classes than expected observation for each interval is N/n .
- N is the total number of observations.
- Observing a value in a particular interval is independent.

Summary

- We have learnt about the linear congruential method.
- Credits: Jerry Banks book.

Lesson 188 : Generation Of Pseudo-random Numbers

[click here to download ppt](#)

Overview

- Here, we will learn about the generation of pseudo-random numbers.

Generation Of Pseudo-random Numbers

- False number generation.
- Removes the potential of true randomness.
- Set of random numbers can be replicated.
- Producing numbers between 0 and 1 that simulates the properties of uniform distribution and independence.
- Some errors or departures are:
- Not uniformly distributed random numbers.
- Not too high or too low variance.
- Generated numbers might not be continuous valued but are discrete valued.
- **Dependence:** Autocorrelation between numbers.
- Numbers successively higher or lower than adjacent numbers.
- Several numbers above the mean followed by the several numbers below the mean.
- Departures from uniformity and independence for a particular generation scheme often can be detected by tests.
- If detected then dropped in the favour of an acceptable generator.

Summary

- We have learnt about the generation of pseudo-random numbers.
- Credits: Jerry Banks book.

Lesson 189 : Tests for Random numbers

[click here to download ppt](#)

Overview

- Here, we will understand about the Tests for Random numbers.

Tests for Random numbers

- Two categories:
- 1. Uniformity.

$$H_0 : R_i \sim U[0, 1]$$

$$H_1 : R_i \neq U[0, 1]$$

- H_0 denotes numbers distributed uniformly on the interval $[0, 1]$.
- 2. Independence.

$$H_0 : R_i \sim \text{independently}$$

$$H_1 : R_i \neq \text{independently}$$

- H_0 denotes the independent numbers.
- A level of significance α must be stated for each test.
- Where α is probability of rejecting the null hypothesis.

$$\alpha = P(\text{reject } H_0 \mid H_0 \text{ true})$$

- Software not specifically developed for simulation comes with random number generator pre-installed

Additional Tests:

- Good's serial test, the median-spectrum test, the run's test and a variance heterogeneity test.

Summary

- Here, we understood about the tests for random numbers.
- Credits: Jerry banks book.

Lesson 190 : Random Variate Generation

[click here to download ppt](#)

Overview

- Here, we will understand about the “Random Variate Generation”.

Random Variate

- A random variate is a generated variable using uniformly distributed pseudorandom numbers.
- A random variate can be uniformly or nonuniformly distributed.
- Modeling activities is unpredictable or indeterminate.
- Random Variate Generations explain and illustrate some techniques for generating random
- Sometimes there is no availability of random-variate-generation libraries.
- It is up to the modeller to create an acceptable routine.
- This topic includes:
 - inverse-transform technique
 - Acceptance-rejection technique
- Special properties
- The composition method

Summary

- Here, we understood about Random Variate Generation.
- Credits: Jerry banks book.

Lesson 191 : Inverse Transform Technique

[click here to download ppt](#)

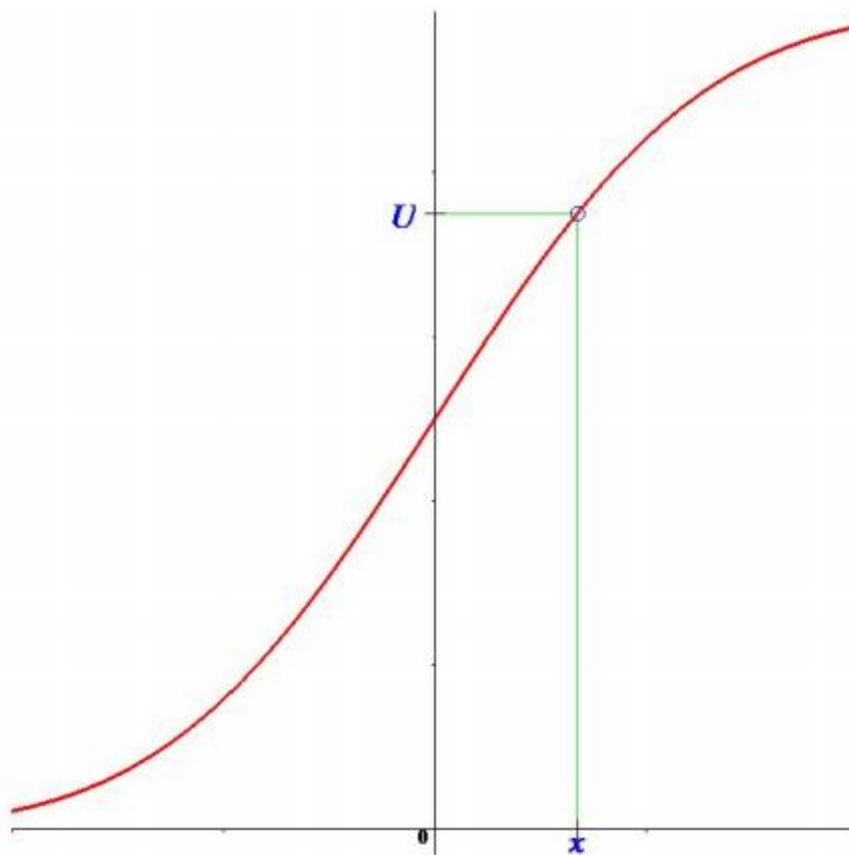
Overview

- Here, we will understand about the “Inverse Transform Technique”.

Inverse Transform Technique

- The inverse-transform technique can be used to sample from the exponential.
- The uniform.
- The Weibull.
- The triangular and empirical distributions.
- The basic principle is to find the inverse function of F^F , $F^{-1}F^{-1}$ such that $FF^{-1}=I$ $FF^{-1}=I$
- Where $F^{-1}F^{-1}$ denotes the solution of the equation $r=F_x^r = F(x)$ in terms of y .

Inverse Transform Technique



Inverse Transform Technique

- This technique explains:
 - Exponential Distribution
 - Uniform Distribution
 - Weibull Distribution
 - Triangular Distribution
 - Empirical Continuous Distribution
 - Continuous Distribution
 - Discrete Distribution

Summary

- Here, we understood about Inverse Transform Technique.
- Credits: Jerry banks book.

Lesson 192 : Acceptance Rejection Technique

Overview

- Here, We will understand “Acceptance Rejection Technique”.

Acceptance Rejection Technique

- Acceptance Rejection Technique is a method used to generate random variate using some steps explained next.
- Step 1. Generate a Random Number R .
- Step2a. If $R \geq 1/4$ Accept R .
- Step2b. If $R \leq 1/4$ Reject R .
- If another uniform random variate on $[1/4, 1]$ is needed, repeat the procedure.
- The random variate generated using above methods is indeed uniformly distributed over $[1/4, 1]$.
- Take any $1/4 \leq a < b \leq 1$ then

$$P(a < R \leq b | 1/4 \leq R \leq 1) = \frac{P(a < R \leq b)}{P(1/4 \leq R \leq 1)} = \frac{b - a}{3/4}$$

- Proves that probability for uniform distribution is on $[1/4, 1]$.
- The efficiency of an acceptance-rejection technique rest on reducing the total of rejections.
- Which depends on computer being used.
- The skills of programmer.
- Rejected numbers.
- The Distributions which operates on acceptance rejection technique are:
 - Poisson Distribution
 - Non-stationary Poisson process
 - Gamma Distribution.

Summary

- Here, we understood about the Acceptance Rejection Technique.
- Credits, Jerry Banks book.

Lesson 193 : Other topics in Randomness

[click here to download ppt](#)

- **Overview**
- Here, we will discuss about “Other topics in Randomness”.

Special Properties

- Special Properties are variate generation techniques based on a particular feature of probability distributions.

Direct Transformation

- Direct transformation is used for the normal and lognormal distributions.
- The cdf is given by:

$$\Phi(x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt, \quad -\infty < x < \infty$$

- Box and Muller made this method and is easy to program in scientific programming languages.

Convolution Method

- The probability distribution of a sum of two or more independent random variables is called convolution method.
- This method uses two random variables.
- Sum them and generate a new random variable.
- This technique can be applied on Erlang and Binomial variates.

More Special Properties

- The other relationship among distributions include relation between gamma and beta distribution.
- We can generate beta variates, using two gamma variates with acceptance-rejection technique.

Summary

- Here, We understood about the Special Properties, Direct transformation, convolution method and other special properties.
- Credits: Jerry banks book.

Lesson 194 : Data Collection

[click here to download ppt](#)

Overview

- Here, we will discuss about “Data Collection”.

Data Collection

- The relation between construction of the model and the collection of needed input data is constant.
- The required data elements changes with complexity of the model.
- Time taking nature of data collection.
- Which type of data is to be collected is dictated by the objectives of study.

Summary

- Here, We understood about the Data Collection.
- Credits: Jerry banks book.

Lesson 195 : Identifying the Distribution with Data

[click here to download ppt](#)

Overview

- Here, we will understand About “Identifying the Distribution with Data”.

Identifying the Distribution with Data

- Data distributions are graphical methods.
- These methods are used for selecting input distribution families.
- The specific distribution is often specified by estimating its parameters.

Summary

- We understood about Data Distribution and how data parameters are selected.
- Credits, Jerry Banks book.

Lesson 196 : Histograms

[click here to download ppt](#)

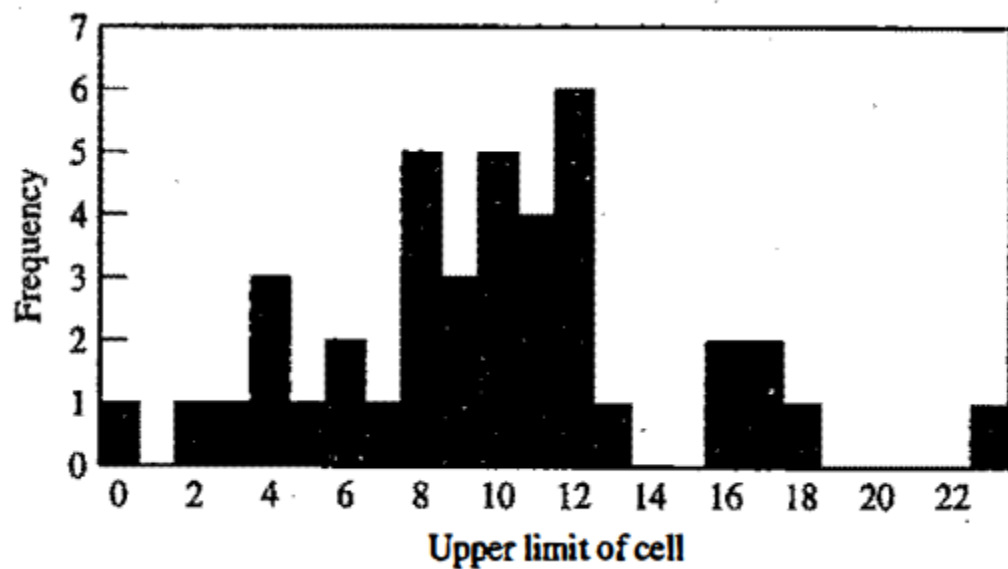
Overview

- Here, we will understand about the “Histograms”.

Histograms

- Histograms is a powerful technique for identifying how a distribution is shaped.
- Histogram is also known as frequency distribution.
- There are some methods for constructing a histogram.
 1. Divide the data ranges into intervals of equal width.
 2. Label the horizontal axis to follow the nominated intervals.
 3. Find the frequency of occurrences inside each intervals.
 4. Label the vertical axis to plot total occurrences for every interval.
 5. Plot the frequencies on the vertical axis.
- The number of class intervals is decided by the number of observations.
- Amount of scatter in the data.

Histograms



Lesson 197 : Selecting the Family of Distributions

Overview

- Here, we will understand about the “Selecting the Family of Distributions”.

Selecting the Family of Distributions

- The selection of a family of distributions is based on the context for what it is being investigated.
- The shape of the histogram.
- The Distributions which are frequently encountered and easy to analyze includes exponential, normal and poisson distribution.
- Whereas difficult to analyze includes the beta, gamma and Weibull distributions.
- Out of many, Here are some examples of distributions which were created on the physical basis.
 -
 - Binomial
 - Negative Binomial
 - Poisson
 - Normal
 - Lognormal
- An input Model is an estimate of reality.
- There is no “true” distribution for any stochastic input process.

Summary

- Here, we understood about the Family of distributions, its types and input model.
- Credits: Jerry banks book.

Lesson 198 : Quantile-Quantile Plot

Overview

- Here, we will understand about the “Quantile-Quantile Plot”.

Quantile-Quantile Plot

- A Quantile-Quantile is a graphical method for comparison of two probability distributions by plotting their quantiles against each other.
- If $F_Y = P(X \leq \gamma) = F(\gamma) = P(X \leq \gamma) = q$ for $0 < q < 1$
- Here x is random variable with cdf F .
- Q -quantile of X is value of x .
- We write $\gamma = F^{-1}(q)$ when F has an inverse.
- If $x_1, x_2, \dots, x_n$ and $y_1, y_2, \dots, y_n$ and $y_1 \leq y_2 \leq \dots \leq y_n$
- Where $y_1 \leq y_2 \leq \dots \leq y_n$
- Let j denote the ranking of order
- $j=1$ for smallest
- $j=n$ for largest
- Then
- $y_j \approx F^{-1}\left(\frac{j-1/2}{n}\right)$

Quantile-Quantile Plot

Lesson 199 : Parameter Estimation

Overview

- Here, we will learn about the parameter estimation.

Parameter Estimation

- After selecting the family of distribution, the next step is to estimate the parameters of the distribution.
- Many useful distribution estimators are described here.
- Software packages integrated into simulation languages are available.

Summary

- We have learnt about the parameter distribution.
- Credits: Jerry Banks book.

Lesson 200 : Preliminary Statistics: Sample Mean and Sample Variance

[Click here to download ppt](#)

Overview

- Here, we will learn about the preliminary statistics: sample mean and sample variance.

Preliminary Statistics

- To estimate the parameters of a hypothesized distribution.
- When discrete and continuous raw data is available then the following equation is used:

$$\bar{X} = \frac{\sum_{i=1}^n X_i}{n}$$

- $\bar{X}$ = sample mean.
- $x_1, x_2, \dots, x_n$ = observation in a sample of size n.
- The sample variance s^2 is defined as:

$$S^2 = \frac{\sum_{i=1}^n X_i^2 - n\bar{X}^2}{n-1}$$

- When discrete or continuous raw data are available following modified equations are used:

$$\bar{X} = \frac{\sum_{j=1}^k f_j X_j}{n}$$

- Where $\bar{X}$ is the sample mean.
- And the sample variance is :

$$S^2 = \frac{\sum_{j=1}^k f_j X_j^2 - n\bar{X}^2}{n-1}$$

- Here k is the number of distinct values of X.
- f_j is the observed frequency of the value X_j of X.
- When data is discrete or continuous and have been placed in class intervals then the sample mean and the sample variance are approximated from the following equations:

$$\bar{X} = \frac{\sum_{j=1}^c f_j m_j}{n}$$

- For the sample variance the following equation is used.

$$s^2 = \frac{\sum_{j=1}^c f_j m_j^2 - n \bar{X}^2}{n-1}$$

- f_j observed frequency of j^{th} class interval.
- m_j midpoint of j^{th} interval.
- c = number of intervals.

Summary

- We have learnt about the preliminary statistics: sample mean and sample variance.
- Credits: Jerry Banks book.

Lesson 201 : Evolution of Computer Systems Architectures

[click here to download ppt](#)

Overview

- **Here, we will learn about the evolution of computer systems architectures**

Evolution of Computer Systems Architectures

- ENIAC was the first computer system with no operating system and was large in size. They used vacuum tubes.
- Early computer doesn't have a good operating system, databases.
- They doesn't have any high-level programming language for simplifying program.

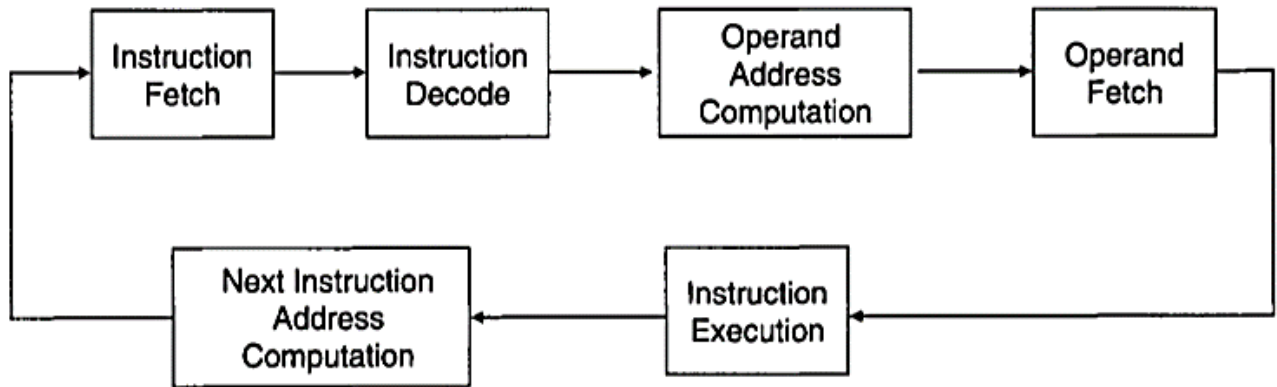
Systems Architecture

- Central Processing Unit.
- Graphics Processing Unit.
- Temporary memory.
- Permanent Memory.
- Input unit.
- Output unit.
- Control element.
- Disk Drives.

CPU Architectures

- Central processing unit.
- CU fetch functions.
- ALU performs functions.
- Registers to store the operands.
- L1, L2, L3 Cache.
- CPU Execution cycle.

Instruction architectures



Instructions and Memory Architectures

- Reduced instruction set computer (RISC).
- Highly optimized instructions.
- Complex instruction set computer (CISC).
- Memory address register.
- Memory data register.

Secondary Storage and Peripheral Device Architecture

- I/O devices control secondary storage.
- Magnetic tape drives.
- Magnetic disk drives.
- Compact optical disk drives.
- Disk jukeboxes.
- High density data storage.

Network Architecture

- Share information and processing capacity in real life.
- Another path to send or receive data.
- Central switching element. Central storage repository.
- Connected using intelligent interface units.

Summary

- We have learnt about the evolution of computer systems architectures.
- Credits: Paul Fortier book.

Lesson 202 : Evolution of Database Systems

[click here to download ppt](#)

Overview

- Here, we will learn about the evolution of database systems.

Evolution of Database Systems

- Powerful tools to manage information.
- Early computers lag in online data storage.
- System architects have to rely on external storage.
- Initial storage was constructed using simple direct addressing.
- File system used coarse semantic means.
- Information Centralization.
- Security.
- Integrity.
- File systems added finer-grained definitions of stored information.
- Relational database system was introduced.
- Database must be entered at a specific entry point.
- Must traverse the path leading to the specific item.
- An issue for network is its legacy.
- Database system's demise began with development.
- Publication of Codd's relational database model.
- Information can be formed into tables called relations.
- Have to use calculus and algebra to operate.
- Concurrent access was not present in early network-based databases.
- The serializability theory and concurrency control led to further improvements in database technology.
- "ACID" properties.
- Object-oriented database was developed.
- For application developers to store data and data types efficiently.
- But did not possess some fundamental concepts.
- Object relational databases.
- Embraced by the U.S and international vendors.
- Next change will be in the area of transactions and transaction processing.
- ACID properties may be altered.

Summary

- We have learnt about the evolution of database systems.
- Credits: Paul Fortier book.

Lesson 203 : Evolution of Operating Systems

Overview

- **Here, we will learn about the evolution of operating systems.**

Evolution of Operating Systems

- Operating system is computer's software, firmware, and hardware.
- Interacts at low-level.
- Manage sharing of computer's resources.
- The most privileged of software elements on the system.
- Requires hardware.
- Early coders load the software in a location then make hardware to start processing.
- No automated means to switch between different jobs.
- Monitor or batch operating system provided the solution to transition between processing jobs.

Services of Operating System

- Process and Hardware management.
- Memory management.
- I/O management.
- Inter-process synchronization and communications.
- Resource allocation.
- Protection of system and user resources.

Evolution of Operating Systems

- interrupt service routine checks everything and determines what to perform next.
- Efficient use of computing facilities.
- Convenient interface to user by hiding the details of machine.
- Transparent use of computer resources.
- Improved architectures and instruction execution schemes.
- Improve the performance of the processor.
- Not to depend on a single processor
- Add entire new processors.
- Distributed processors.
- Processors operating system is the single global operating system.
- In one case the entire operating system can be replicated on each site.
- Each processor has a specific function.
- New concepts are still being examined.
- Client/server processing.
- Multiprocessing operating system forum.

Lesson 204 : Evolution of Computer Networks

Overview

- Here, we will learn about the evolution of computer networks

Evolution of Computer Networks

- The term network can mean many things.
- In computer it refers to combination of interconnected resources.
- ARPANET was an early computer network.
- A loose coalition of devices that share information.
- All this evolved into internet.
- ARPANET provided early research into communication protocols.
- Greater availability and access to a wider range of devices.
- As more applications were added.
- Information access and manipulation took on greater emphasis.
- As the networks improve new applications arose.
- Remote job execution was added with information exchange.
- But still real-time was not provided.
- They had to develop more advance protocols and network operating systems.
- It was time consuming.
- Local area network was introduced in 1970s.
- LAN became more widely available.
- LAN designs have been produced to fit an extremely wide spectrum of user requirements.
- LAN is widely used in all aspects of society.
- LAN had rapid growth in 1990s and early 2000s.
- LAN can connect a device from anywhere.
- LAN is used to link factory robots.
- LAN provide sensor data, control data and feedback.
- Walt Disney World used LAN for different services.
- Large banks also use LAN for interconnection of their local sites.
- Wireless technology has improved.
- Began in 1970s with Aloha net as foundation.
- Wireless phone network development has opened the door for computer networks.
- More development will be made to provide more applications.

Summary

- We have learnt about the evolution of computer networks.
- Credits: Paul Fortier book.

Lesson 205 : Need for Performance Evaluation

Overview

- **Here, we will learn about the need for performance evaluation**

Need for Performance Evaluation

- Selecting a specific computer architecture for an application.
- An operating system.
- A database system.
- A wide area or local area network system.
- If not used properly then decrease productivity.
- Operating system can increase the concurrency access.
- Or can block access.
- A database can provide efficiently sharing information among many applications.
- Or blocking of information by dropping data availably.
- LAN provides means for streamline information processing and eliminate redundancies.
- But it may also deter users from logging on because of link or protocol problems.
- We need to select the computer system mapped to the needs.
- A computer system can be very simple with simple processor etc.
- A computer can be highly elaborate with multiple processors etc.
- A purchaser must decide according to motherboard its using.
- While purchasing the system all this must be kept in mind.

Summary

- We have learnt about the Need for performance evaluation.
- Credits: Paul Fortier book.

Lesson 206 : Role of Performance Evaluation in Computer Engineering

Overview

- Here, we will learn about role of performance evaluation in computer engineering.

Role of Performance Evaluation in Computer Engineering

- Client/server architectures.
- These systems are from Government, industry and academia.
- There is constant demand for improved reliability and availability.
- Present systems provide more features.
- Sharing of resources on global scale.
- Ability to bring computing power to the user.
- The optimal design or selection of computer system is therefore very important.

Summary

- We have learnt about the role of performance evaluation in computer engineering.
- Credits: Paul Fortier book.

Lesson 207 : Overview of Performance Evaluation Methods

Overview

- Here, we will learn about the overview of performance evaluation methods.

Overview of Performance Evaluation Methods

- Models provide a tool to define a system.
- Define its problems in a concise fashion.
- This provide an environment to study a system before its existence.
- Developed on the base of theoretical laws and principles.
- Modeling is better if:
- Physical laws are available.
- Pictorial representation is possible.
- Input/output elements are manageable.
- But we don't have clear physical laws.
- Faithful model of a system must be constructed.
- Model a slice of the real-world system.
- Which element is important to understand in faithful system.
- We can say its intuition.

Lesson 208 : Performance Metrics and Evaluation Criteria

Overview

- Here, we will learn about performance metrics and evaluation criteria.

Performance Metrics and Evaluation Criteria

- Performance metrics and evaluation criteria are required for systems.
- User must follow a methodology.
- The motivations.
- The environment.
- Technological boundaries.
- Must understand the usage of the system.
- Needs and uses must be defined.
- Must compile a wish list of all potential uses.
- User must know processing requirements.
- Communication transfer.
- Management requirements.
- Define or know why we need something at first priority.
- Checking in which environment the system fits.
- Checking in which technological aspect it fits.
- Connecting to diverse set of company assets.
- Link planned new resources.
- Need large volume of data for system requirements.
- How this data assist in the selection?
- Collection of data can be divided in quantitative and qualitative.
- Specific performance measures can be derived from one set of data.
- Subjective measures can be derived from another.

Summary

- We have learnt about the performance metrics and evaluation criteria.
- Credits: Paul Fortier book.

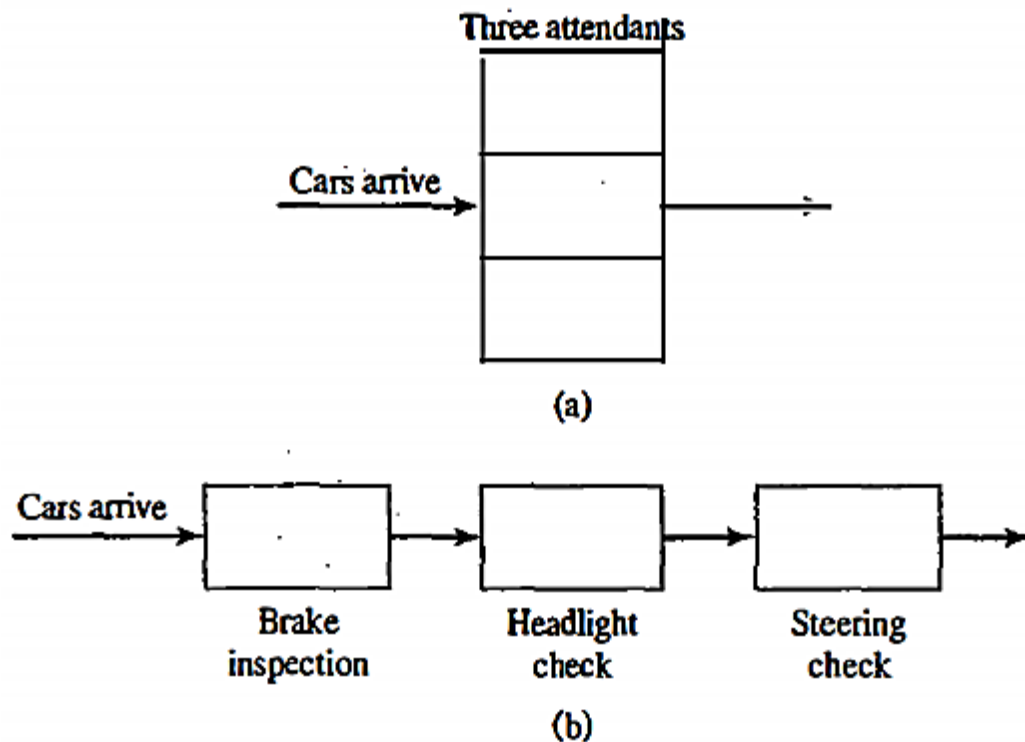
Lesson 209 : Comparison of Two System Designs

Overview

- Here, we will understand about the “Comparison of Two System Designs”.

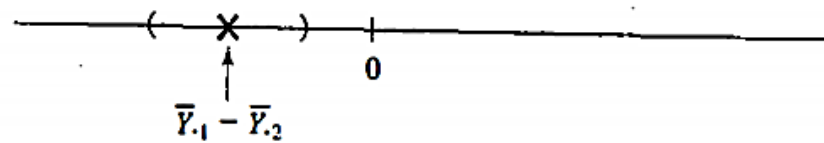
Comparison of Two System Designs

- A simulation analyst can compare two possible configurations.
- Using supply chain inventory system there ordering policies will be compared.
- Replications is used to analyze the output data.
- The mean performance measure for system ‘I’ will be denoted by $\theta_i, i=1,2, \theta_i = (i = 1,2)$.
- The goal of the simulation experiments is to obtain point.
- Interval estimates of the difference in mean performance.
- Namely $\theta_1 - \theta_2 (\theta_1 - \theta_2)$.
- Inverse Transform Technique
- Replications is used to analyze the output data.
- The mean performance measure for system ‘I’ will be denoted by $\theta_i, i=1,2, \theta_i = (i = 1,2)$.
- Vehicle Safety Inspection:

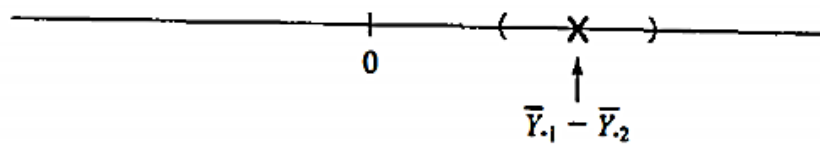


- In comparing two systems, number of replications R_i to be made and run length $TE_i T_E^{(i)}$ for each model $i=1,2$ ($i = 1,2$) is decided.

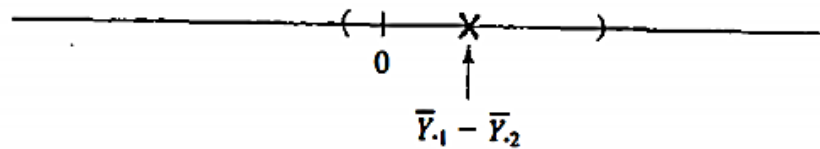
System	Replication				Sample Mean	Sample Variance
	1	2	...	R_i		
1	Y_{11}	Y_{21}	...	Y_{R1}	$\bar{Y}_1$	S_1^2
2	Y_{12}	Y_{22}	...	Y_{R2}	$\bar{Y}_2$	S_2^2



(a)



(b)



(c)

Summary

- Here, we understood about Comparing two system designs.
- Credits: Jerry banks book.

Lesson 210 : Independent Sampling with Equal Variances

Overview

- Here, We will understand “Independent Sampling with Equal Variances”.

Independent Sampling with Equal Variances

- Independent Sampling refers to different and independent random numbers streams to simulate the two systems.
- **Acceptance-Rejection**
- The observations of simulated system 1, $Y_{r1}, r=1, \dots, R_1 \{Y_{r1}, r=1, \dots, R_1\}$ are all statistically independent of simulated system 2, $Y_{r2}, r=1, \dots, R_2 \{Y_{r2}, r=1, \dots, R_2\}$.
- Y_i is given by:

$$V(\bar{Y}_i) = \frac{V(Y_{ri})}{R_i} = \frac{\sigma_i^2}{R_i}, \quad i = 1, 2$$

- Where $Y1\bar{Y}_1$ and $Y2\bar{Y}_2$ are statistically different as:

$$\begin{aligned} V(\bar{Y}_1 - \bar{Y}_2) &= V(\bar{Y}_1) + V(\bar{Y}_2) \\ &= \frac{\sigma_1^2}{R_1} + \frac{\sigma_2^2}{R_2} \end{aligned}$$

- It implies two variances are equal (but unknown in value) $\sigma_1^2 = \sigma_2^2 = \sigma^2$.
- If two-sample-t confidence interval approach is used. The point estimate of the mean difference is: $Y1\bar{Y}_1 - Y2\bar{Y}_2$
- Where $Y1\bar{Y}_1$ is given as:

$$S_i^2 = \frac{1}{R_i - 1} \sum_{r=1}^{R_i} (Y_{ri} - \bar{Y}_i)^2$$

$$= \frac{1}{R_i - 1} \left(\sum_{r=1}^{R_i} Y_{ri}^2 - R_i \bar{Y}_i^2 \right)$$

- Here, S_i^2 is an estimator of variance σ_i^2 . Assuming $\sigma_1^2 = \sigma_2^2 = \sigma^2$ a pool estimate of σ^2 is obtained as:

$$S_p^2 = \frac{(R_1 - 1)S_1^2 + (R_2 - 1)S_2^2}{R_1 + R_2 - 2}$$

- Where $v = R_1 + R_2 - 2$
- The c.i. $\theta_1 - \theta_2$ for with the standard error is computed by:

$$\text{s.e.}(\bar{Y}_1 - \bar{Y}_2) = S_p \sqrt{\frac{1}{R_1} + \frac{1}{R_2}}$$

- The standard error is an estimate of the standard deviation of the point given as: $\sigma \sqrt{1/R_1 + 1/R_2}$

Summary

- Here, we understood about the Independent Sampling.
- Credits, Jerry Banks book.

Lesson 211 : Independent Sampling with Unequal Variances

Overview

- Here, we will discuss about “Independent Sampling with Unequal Variances”.

Independent Sampling with Unequal Variances

- If the assumption of equal variances cannot safely be made, an approximate $100(1-\alpha)\%$ $100(1-\alpha)\%$ for $\theta_1 - \theta_2$ $\theta_1 - \theta_2$ can be computed as:

$$s.e.(\bar{Y}_1 - \bar{Y}_2) = \sqrt{\frac{S_1^2}{R_1} + \frac{S_2^2}{R_2}}$$

- Degrees of freedom, v can be as:

$$v = \frac{(S_1^2 / R_1 + S_2^2 / R_2)^2}{[(S_1^2 / R_1)^2 / (R_1 - 1)] + [(S_2^2 / R_2)^2 / (R_2 - 1)]}$$

- A minimum number of replications $R_1 \geq 6$ $R_1 \geq 6$ and $R_2 \geq 6$ $R_2 \geq 6$ is recommended for this procedure.

Summary

- Here, We understood about Independent Sampling with Unequal Variances.
- Credits: Jerry banks book.

Lesson 212 : Common Random Numbers

Overview

- Here, we will understand About “Common Random Numbers”.

Common Random Numbers

- CRN means, for each replication, the same random numbers are used to simulate both systems.
- Therefore, R1 and R2 must be equal, say $R_1 = R_2 = R$.
- The purpose of CRN is to induce positive correlation between y_1 and y_2 .
- A variance reduction in point estimator of mean difference is given by:

$$\begin{aligned} V(\bar{Y}_1 - \bar{Y}_2) &= V(\bar{Y}_1) + V(\bar{Y}_2) - 2\text{cov}(\bar{Y}_1, \bar{Y}_2) \\ &= \frac{\sigma_1^2}{R} + \frac{\sigma_2^2}{R} - \frac{2\rho_{12}\sigma_1\sigma_2}{R} \end{aligned}$$

- Compare variance of $Y_1 - Y_2$, call it V_{cm} given as :

$$V_{\text{CRN}} = V_{\text{IND}} - \frac{2\rho_{12}\sigma_1\sigma_2}{R}$$

- Which implies:

$$V_{\text{CRN}} < V_{\text{IND}}$$

- To compute a 100(1 - α)% c.i compute the differences:

$$D_r = Y_{r1} - Y_{r2}$$

- Compute Sample mean difference:

$$\bar{D} = \frac{1}{R} \sum_{r=1}^R D_r$$

- The sample variance of the difference $\{D_r\}$ is computed as:

$$S_D^2 = \frac{1}{R-1} \sum_{r=1}^R (D_r - \bar{D})^2$$

$$= \frac{1}{R-1} \left(\sum_{r=1}^R D_r^2 - R\bar{D}^2 \right)$$

- Where standard error of $Y_1 - Y_2$ $\bar{Y}_1 - \bar{Y}_2$ is estimated by:

$$\text{s.e.}(\bar{D}) = \text{s.e.}(\bar{Y}_1 - \bar{Y}_2) = \frac{S_D}{\sqrt{R}}$$

- Implementation of common random number:
 1. Dedicate a random-number stream to a specific purpose.
 2. For system with external arrivals, dedicate one random stream to one of arrival and their attributes.
 3. For systems having an entity performing given activities in a cyclic fashion, assign a random-number stream to this entity.
 4. If synchronization is not possible use independent streams of random numbers for this subset.

Summary

- We understand about Common Random Numbers.
- Credits, Jerry Banks book.

Lesson 213 : Confidence Intervals with Specified Precision

Overview

- Here, we will understand about the “Confidence Intervals with Specified Precision”.

Confidence Intervals with Specified Precision

- Confidence intervals for the difference between two systems' performance can be achieved in a parallel way.
- If the goal is to obtain error in our estimate $\theta_1 - \theta_2$ to be less than $\pm \epsilon$ then number of replications R should be:

$$H = t_{\alpha/2, \nu} \text{s.e.}(\bar{Y}_1 - \bar{Y}_2) \leq \epsilon$$

- Using $R \geq 2R_0 \geq 2$ we obtain an initial estimate.
- We solve total number of replication $R \geq R_0^R \geq R_0$ to achieve half-length criterion.
- We make replications $R \geq R_0^R \geq R_0$ for each system to compute confidence interval and half-length criterion.

Summary

- Here we understand about Confidence Intervals with Specified Precision.
- Credits: Uri Wilensky book

Lesson 214 : Comparison of Several System Designs

Overview

- Here, we will understand about the “COMPARISON OF SEVERAL SYSTEM DESIGNS”.

COMPARISON OF SEVERAL SYSTEM DESIGNS

- To compare “K” alternative system designs.
- Many different statistical procedures have been developed.
- These are used to analyze simulation data and to draw statistically sound inferences about parameters θ_i .
- These procedures are classified as:
 - Fixed-sample size.
 - Sequential sampling.
- **Fixed Sample Size**

A predetermined sample size is used to draw inferences via hypothesis tests or confidence intervals.

Example:

- - The interval estimation of a mean performance measure.

Advantages:

- - An easily estimated or known cost in terms of computer time before running the experiments.
 - Can be used when :
 - Computer time is limited.
 - A pilot study is being conducted.

Disadvantages

- - A strong Conclusion could be impossible.
 - A hypothesis test may lead to a failure to reject the null hypothesis.
- **Sequential Sampling**
 - In sequential sampling more and more data are collected until an estimator with a pre-specified precision is achieved.
 - In two stage procedure, an initial sample estimates additional observations needed to draw conclusions with a specified precision.

- Goal of simulation analyst
- Estimation of each parameter, θ_i .
- Comparison of each performance measure.
- All pair wise comparisons.
- selection of the best θ_i .

Summary

- Here, we understood about the “COMPARISON OF SERIAL SYSTEM DESIGNS”.
- Credits: jerry banks book.

Lesson 215 : Bonferroni Approach to Multiple Comparisons

Overview

- Here, we will understand about the “Bonferroni Approach to Multiple Comparisons”.

Bonferroni Approach to Multiple Comparisons

- If C confidence intervals are computed and that the i th interval has confidence coefficient $1 - \alpha_i$.
- The Bonferroni inequality states that:

$$P(\text{all statements } S_i \text{ are true, } i = 1, \dots, C) \geq 1 - \sum_{j=1}^C \alpha_j = 1 - \alpha_E$$

- Where

$$\alpha_E = \sum_{j=1}^C \alpha_j$$
 is called overall error probability.

- This is equivalent to:

$$P(\text{one or more of the } C \text{ confidence intervals does not contain the parameter being estimated}) \leq \alpha_E$$

- Here, α_E provides the upper bound.
- A major advantage of Bonferroni approach is that runs with independent sampling and or common random numbers.
- The major disadvantage of the Bonferroni approach in making a large number of comparisons is the increased width of each individual interval.
- The width of a c.i. is a measure of the precision of the estimate.
- 3 possible ways of using the Bonferroni Inequality when comparing K alternative system designs:

1. Construct a $100(1 - \alpha_i)\%$ c.i. for parameter θ_i , where number of intervals is $C = K$.

2. Compare all designs to one specific design that is:

Construct a $100(1 - \alpha_i)\%$ c.i. for $\theta_i - \theta_1$.

Here, the number of intervals is $C = K - 1$.

3. Compare all designs to each other. That is :

- - Construct a $100(1 - \alpha_i)\%$ c.i. for $\theta_i - \theta_j$.
 - With K designs, the number of intervals is computed as $C = K (K - 1) / 2$.

Summary

- Here, we understood about Bonferroni Approach.
- Credits: Jerry banks book.

Lesson 216 : Bonferroni Approach to Selecting the Best

Overview

- Here, We will understand “Bonferroni Approach to Selecting the Best”.

Bonferroni Approach to Selecting the Best

- Gross Level
- If there are K system designs, and the i TH design has expected performance θ_i .
- Then, we're interested in best system.
- Where “best” means maximum expected performance.
- Refined
- At a refined level, we are also interesting in knowing how much better the best is relative to alternatives.
- It helps us in choosing a inferior system not deficient by much.
- Difference**

$$\theta_i - \max_{j \neq i} \theta_j$$

- If system design i is the best, then $\theta_i - \max_{j \neq i} \theta_j$ is equal to the difference in performance between the best and the second best.

$$\theta_i - \max_{j \neq i} \theta_j$$

- If system design i is not the best then, $\theta_i - \max_{j \neq i} \theta_j$ is equal to the difference between system i and the best.
- i^* denote index of the best system.

$$\theta_i - \max_{j \neq i} \theta_j$$

- Smaller the difference $\theta_i - \max_{j \neq i} \theta_j$ is, the more certain we find the best system.
- If the difference between system i^* and the other is significant, then we try to achieve i^* with high probability.
- The best system probability should be at least

$$1 - \alpha \text{ whenever } \theta_{i^*} - \max_{j \neq i^*} \theta_j \geq \epsilon$$

- We select best of one of near best if multiple system are within ϵ of the best.
- Achievement**
- This procedure achieves the desired probability and also forms $100(1-\alpha)\%$

$$\theta_i - \max_{j \neq i} \theta_j$$

$100(1 - \alpha)\%$ confidence intervals for

Lesson 217 : Bonferroni Approach to Screening

Overview

- Here, we will discuss about “Bonferroni Approach to Screening”.

Bonferroni Approach to Screening

- When there exist multiple systems and two stage procedure is not possible.
- Then, divide set of systems into the best.
- This is achieved by using screening or subset selection procedure.
- Independent sampling with unequal variances
- This promises that stayed subset contains best system.
- With probability $\geq 1 - \alpha \geq 1 - \alpha$.
- When the data is distributed normally, by independent sampling or CRN is used.
- This subset may contain all K systems, only one or some.
- It depends on the number of replications and sample variances.
- Procedure
- Specify correct probability selection and sample size.
- Make R replications of the system.
- Calculate the sample and sample variance of the difference.
- If bigger is better then retain selected subset system.
- If smaller is better then retain selected subset system.
- All design not retained can be eliminated from further considerations.

Summary

- Here, We understood about Bonferroni Approach to Screening.
- Credits: Jerry banks book.

Lesson 218 : Metamodeling

Overview

- Here, we will understand About “Metamodeling”.

Metamodeling

- Metamodeling is the process of generating metamodels.
- Where metamodel refers to model of a model.
- There is a simulation output variable Y.
- It is related to k independent variables.
- Dependent variable Y is random variable.
- Independent variable is design variable.

- Relation between independent and dependent variable is represented by simulation model.
- Metamodel simplifies this relationship by simpler mathematical function.
- This relationship is represented by as function $Y=f(x)$ $Y = f(x)$.
- Most of the time relationship is unknown.
- Function is created using unknown parameter.
- Regression analysis estimates this parameter.

Summary

- We understood about Metamodeling.
- Credits, Jerry Banks book.

Lesson 219 : Simple Linear Regression

Overview

- Here, we will understand about the “Simple Linear Regression”.

Simple Linear Regression

- The relationship is between independent variable x and a dependent variable y.
- Linear relationship.
- Value of y for given x is:

$$E(Y | x) = \beta_0 + \beta_1 x$$

- β_0 intercept on y axis.
- β_1 is the slope.
- Y can be described as:

$$Y = \beta_0 + \beta_1 x + \epsilon$$

- ϵ is random error with mean zero.
- This model is commonly called a simple linear regression model.
- n pair of observations $y_1, x_1, y_2, x_2 \dots y_n, x_n$ $(y_1, x_1), (y_2, x_2) \dots (y_n, x_n)$
- β_0 and β_1 estimates the observation.
- Sum of squares of the deviation and regression line is minimized.
- The individual observations:

$$Y_i = \beta_0 + \beta_1 x_i + \epsilon_i, i = 1, 2, \dots, n$$

- $\epsilon_1, \epsilon_2, \dots, \epsilon_1, \epsilon_2, \dots$ are uncorrelated random variables.
- ϵ_i is given by:

$$\epsilon_i = Y_i - \beta_0 - \beta_1 x_i$$

- It shows how ϵ_i is related to x_i, Y_i and $E(Y_i | x_i)$.
- The sum of squares of the deviation is :

$$L = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (Y_i - \beta_0 - \beta_1 x_i)^2$$

- Where L is called least-squares function.
- Rewriting Y_i as:

$$Y_i = \beta'_0 + \beta_1 (x_i - \bar{x}) + \epsilon_i$$

- Where:

$$\beta'_0 = \beta_0 + \beta_1 \bar{x} \text{ and } \bar{x} = \sum_{i=1}^n x_i / n.$$

- It is often called as Transformed linear regression model.

$$L = \sum_{i=1}^n [Y_i - \beta'_0 - \beta_1 (x_i - \bar{x})]^2$$

- Taking partial derivatives and setting each to zero.

$$n\hat{\beta}'_0 = \sum_{i=1}^n Y_i$$

$$\hat{\beta}_1 \sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n Y_i (x_i - \bar{x})$$

- These are called normal equations.

Summary

- Here we understand about Simple Linear Regression.
- Credits: Uri Wilensky book

Lesson 220 : Testing for Significance of Regression

Overview

- Here, we will understand about the “Testing for Significance of Regression”.

Testing for Significance of Regression

- Testing for the significance of regression provides another means for assessing the adequacy of the model.
- Error component ε_i is normally distributed.
- Testing for Significance of Regression
- The adequacy of the model is checked by residual analysis.
- Developed by variance properties of β_0 and β_1 .
- **Alternative hypothesis:**

$$H_0 : \beta_1 = 0$$

$$H_1 : \beta_1 \neq 0$$

- H_0 indicates no relationship between x and Y.
- x is of little value in explaining the variability in Y.
- Illustration Best Estimator:

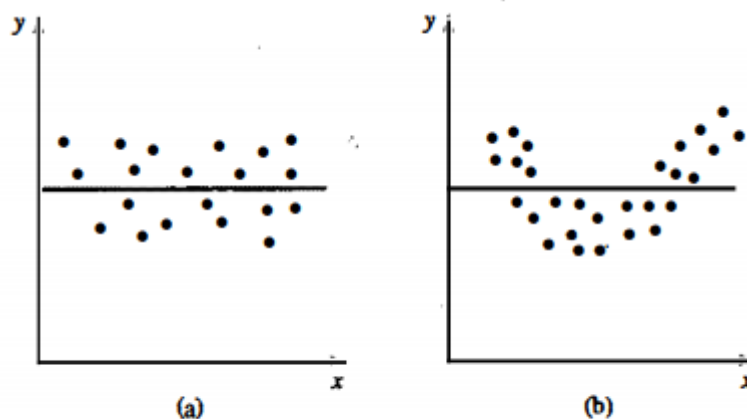


Figure 12.5 Failure to reject $H_0 : \beta_1 = 0$.

- Illustration Non-linear relationship:

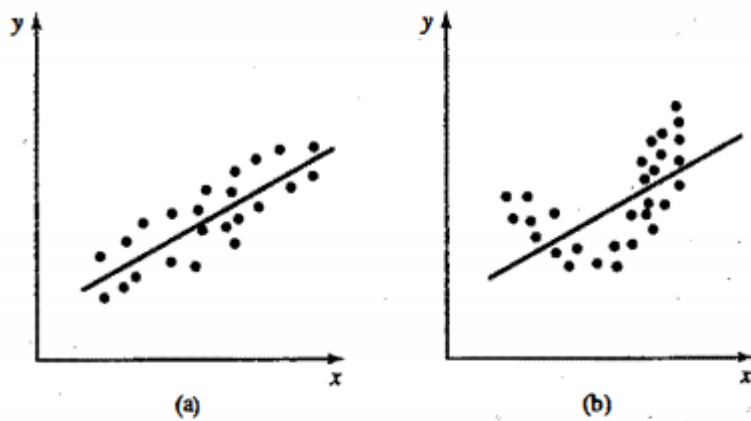


Figure 12.6 $H_0 : \beta_1 = 0$ is rejected.

- Statistic for significance of regression:

$$t_0 = \frac{\hat{\beta}_1}{\sqrt{MS_E / S_{xx}}}$$

- MS_E is mean squared error given by:

$$MS_E = \sum_{i=1}^n \frac{e_i^2}{n-2}$$

- Testing for Significance of Regression
- The direct method calculates $\sum_{i=1}^n e_i^2$.

$$\sum_{i=1}^n e_i^2 = S_{yy} - \hat{\beta}_1 S_{xy}$$

- Where S_{yy} is corrected sum of squares Y given by:

$$S_{yy} = \sum_{i=1}^n Y_i^2 - \frac{\left(\sum_{i=1}^n Y_i\right)^2}{n}$$

Summary

- Here, we understood about the “Testing for Significance of Regression”.

Lesson 221 : What Does 'Optimization via Simulation' Mean?

Overview

- Here, we will understand about the “Optimization via Simulation”.

Optimization via Simulation

- Optimization is a key tool.
- Well developed algorithms for classes of problems.
- Linear Programming is Most famous.
- Much of optimization deals with performance of design.
- In discrete event simulation, result is a random variable.
- Let $x_1, x_2, \dots, x_n$ be m controllable design variables.
- Let $Y(x_1, x_2, \dots, x_n)$ be observed simulation output performance.
- $x_1, x_2, \dots, x_n$ might denote number of Automated Guided Vehicles.
- $Y(x_1, x_2, \dots, x_n)$ could be total Material Handling System acquisition and operation cost.
- $Y(x_1, x_2, \dots, x_n)$ with respect to $x_1, x_2, \dots, x_n$.
- Y is random variable, so cannot optimize actual value of Y .
- Definition of optimization.

maximize or minimize $E(Y(x_1, x_2, \dots, x_m))$

- $E(Y(x_1, x_2, \dots, x_n))$ is long-run average cost of operating the MHS.
- x_1 AVGs, x_2 load per average, and routing algorithm x_3 .
- Formulation of performance measure.

$$Y'(x_1, x_2, x_3) = \begin{cases} 1, & \text{if } Y(x_1, x_2, x_3) \leq D \\ 0, & \text{otherwise} \end{cases}$$

1. Combine all of the performance measures into a single measure.

- - The most common being cost.

2. Optimize with respect to one key performance measure.

- - Evaluate with secondary performance measures.

3. Optimize with respect to one key performance measure.

- - Consider those meeting certain constraints on performance.
- Expected cost and cycle time.

Summary

- Here, we understood about Optimization via Simulation.
- Credits: Jerry banks book.

Lesson 222 : Difficulties in Optimization via Simulation

Overview

- Here, We will understand “Why is Optimization via Simulation Difficult?”

Difficulties in Optimization via Simulation

- If design variables are numerous optimization can be difficult.
- A diverse collection of design variable types is known as the structure of performance function
- The performance can not be evaluated , but can be estimated.
- No conclusion with estimates.
- It can frustrate optimization algorithms that try to move in improving directions.
- It can be eliminated using many replications.
- Or using long runs.
- At each design point that the performance estimate has essentially no variance.
- It causes other designs to be explored.
- Standard sampling variability force optimizations are as following.
- Guarantee a prespecified probability of correct selection.
- Guarantee asymptotic convergence.
- Optimal for deterministic counterpart.
- Robust heuristics.
- Robust heuristics are the most common algorithms.
- It is found in commercial optimization via simulation software.

Summary

- Here, we understood about Bonferroni Approach to Selecting the Best.
- Credits, Jerry Banks book.

Lesson 223 : Using Robust Heuristics

Overview

- Here, we will discuss about “Using Robust Heuristics”.

Using Robust Heuristics

- It means a procedure that does not depend on strong problem structure.
- Can be applied on decision variables and sampling variability.
- Generic algorithms and tabu search are examples.
- There are k possible solutions to the optimization via simulation problem.
- On each iteration a GA operates on a "population" of p solutions.
- Basic Genetic Algo
- Set the iteration counter $j = 0$.
- Ruin simulation experiments.
- Select population p from $P(j)$.
- Recombine Solutions via $P(j + 1)$.
- Set $j = j + 1$ and repeat.
- The G.A can be terminated after some number of iterations.
- At termination X^* with smallest $Y(x)$ is chosen as best.
- **Basic Tabu Search**
- Set the iteration counter $j = 0$. select initial x^* in X
- Find solution x' which minimizes $Y(x)$.
- If $Y(x') < Y(x^*)$ then $x^* = x'$.
- Update the list of Tabu and repeat from step 2.
- Tabu search can be terminated when some number of iterations has passed without changing x^* .
- T.S is fundamentally a discrete-decision-variable optimizer.

$$p(0) = \{x_1(0) \dots x_p(0)\}$$

Summary

- Here, we have understood concepts about Robust Heuristics
- Credits: Jerry banks book.

Lesson 224 : Random Search: An Illustration

Overview

- Here, we will understand About “An Illustration: Random Search”.

Random Search: An Illustration

- Random search provides guaranteed asymptotic converges in certain conditions.
- Convergence can be slow and memory consumptive.
- random search is not a Robust Heuristic.
- Random search algorithm requires finite number of possible designs.
- Rule out problem with continuous decision variables.
- Algorithms designed for continuous variable problem is there.
- On each iteration, we compare a current solution to a randomly chosen competitor.
- If competitor is better it becomes current good solution.
- **Step 1.**
 - Initialize random variables $C_i = 0$.
 - Initial solution i^0 and set $C_{i^0} = 1$.
 - C_i count number of times i is visited.
- **Step 2.**
 - Chose another solution except i .
 - Such as each solution has equal chance of being selected.
- **Step 3.**
 - Run two simultaneous experiments.
 - Use i^0 and i to obtain $Y(i^0)$ and $Y(i)$
 - Set $i^0 = i$
- **Step 4.**
 - Set $C_{i^0} = C_i + 1$
 - If not done go to step 2.
 - If done, Then select i^* such that $C(i^*)$ is largest count.

Summary

- We understood about Random search.
- Credits, Jerry Banks book.

Lesson 225 : Introduction to performance evaluation

Overview

- Here, we will see the introduction to performance evaluation.

Introduction to performance evaluation

- Understanding all elements of the computer system.
- All aspects are important while understanding the issues.
- Realizing the requirements.
- Computer system must function correctly.
- Performing the intended function efficiently.
- Describing the computer system's design.
- Construction.
- Fielding.
- Life-cycle maintenance.

Summary

- We have learnt about the introduction to the performance evaluation.
- Credits: Paul Fortier book.

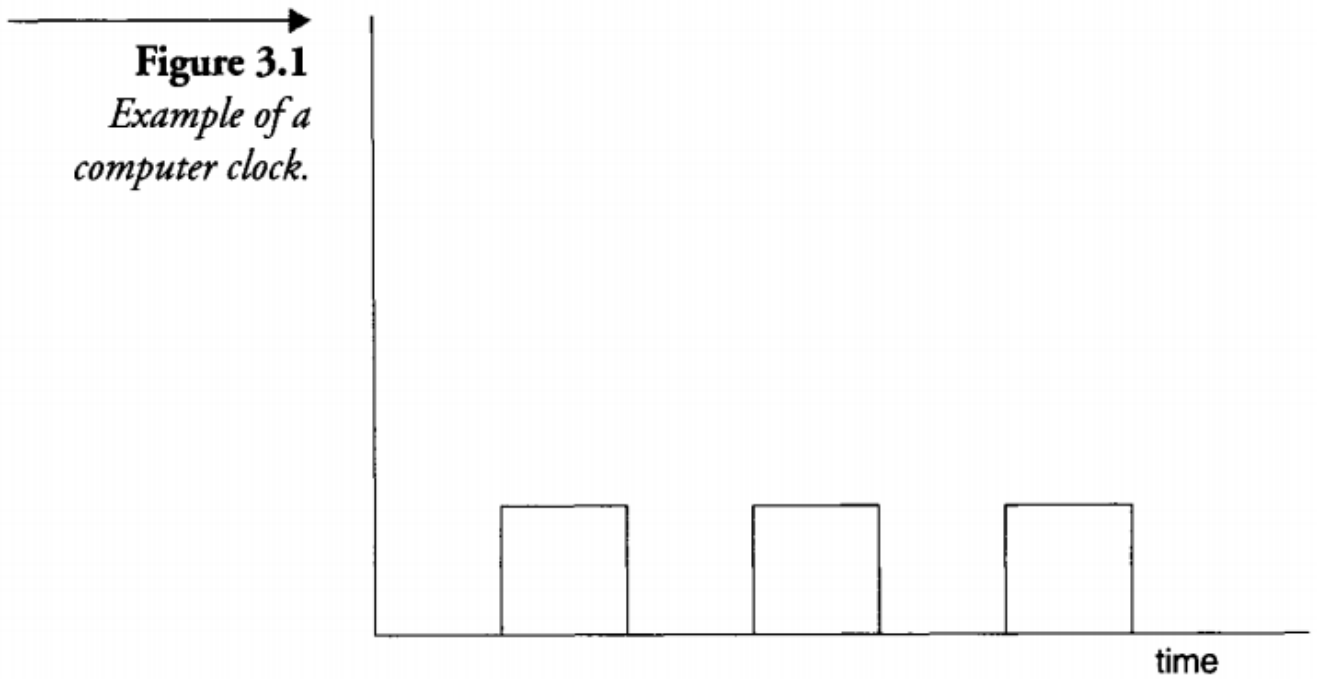
Lesson 226 : Events

Overview

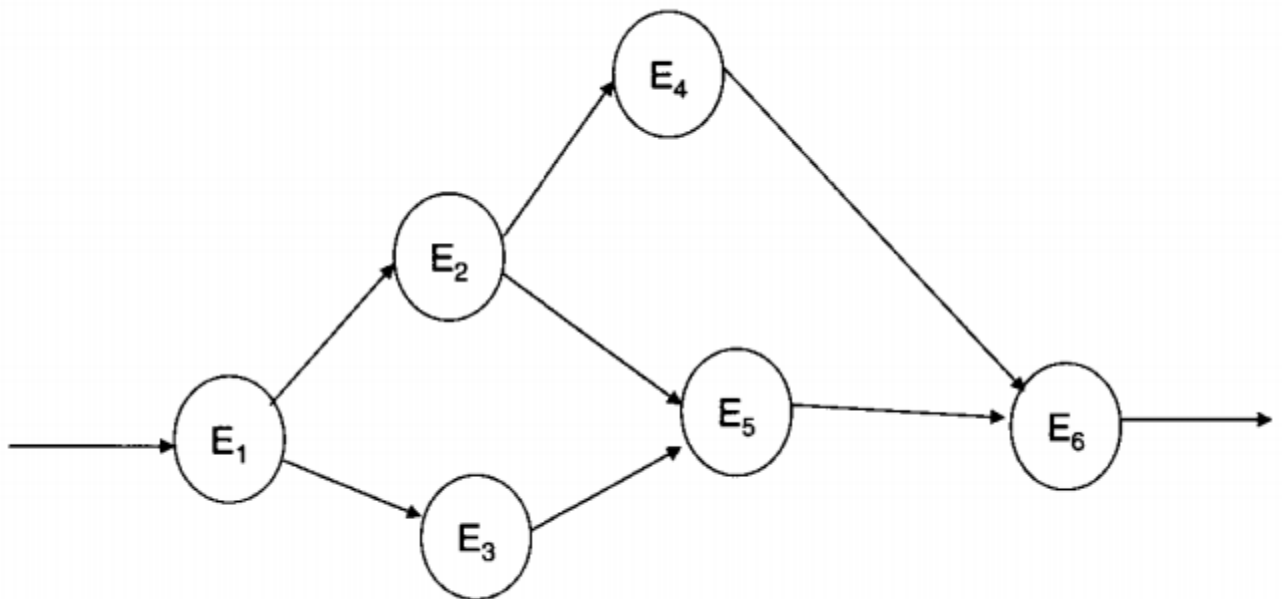
- **Here, we will understand about the “Events”.**

Events

- Time is an important measure.
- Useful if we can measure under computer evaluation.
- An event describes an entity of interest in our system.
- E.g. Clock cycle.



- All events must be controlled.
- Each action is designed to synchronize with other actions.
- Small actions makes larger events.
- System analyst must have understanding of the events.
- And the relationship among different events.



Lesson 227 : Measurements (sampling)

Overview

- Here, we will learn about the measurements (sampling).

Measurements (sampling)

- Events represent all of the real actions that occur in the computer system.
- Must know all the possible conditions.
- Measurements (sampling)
- State is an important concept in any computer system.

$$S = \{E_1(\text{value}), E_2(\text{value}), E_3(\text{value}), \dots, E_n(\text{value})\}$$

- Each event must have valid components.
- Primary types of measures are:
- Count a number of items over a given time period.
- Measures all state variables.
- Measures the fraction of time the system is within a state.
- Valid state must reach.
- Many ways are there to measure system events and decision depends on many factors.
- In hardware monitoring system analyst has ability to add instrumentation.
- Can also use integral test hardware.
- The hardware monitoring be designed as an integral component of the system.
- Must determine and define all aspects.
- Software monitoring requires designers to system hardware elements.
- As well as low level software elements.
- Need to have access to low level operating system calls.
- Hybrid monitoring uses the concepts of both software and hardware monitoring.
- Synchronization is required.

Summary

- We have learnt about the Measurements (sampling).
- Credits: Paul Fortier book.

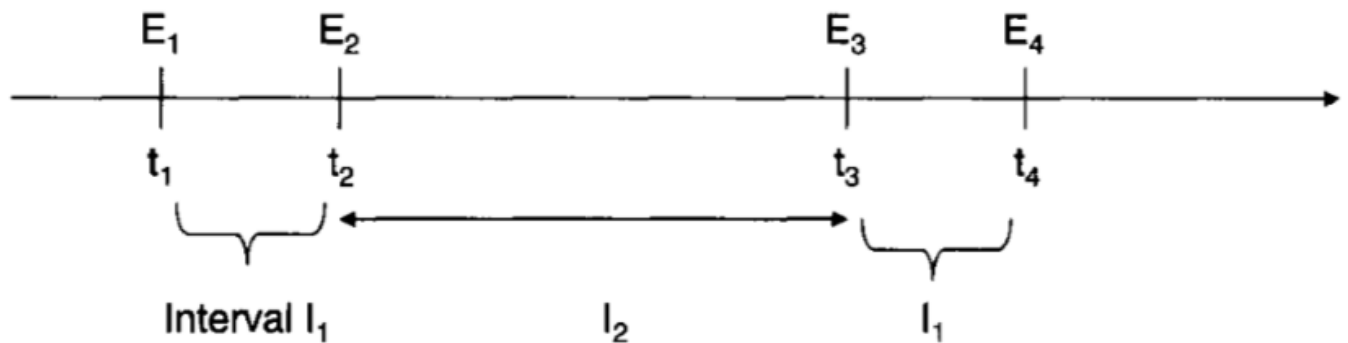
Lesson 228 : Intervals

Overview

- Here, we will learn about the intervals.

Intervals

- An interval represents a period of time.
- System environment is system clocks.
- Instruction execution cycle.
- Intervals are same if they represent the same sequence of events.
- Separate sequence are represented by two intervals then they are unrelated.
- **Intervals**



Summary

- We have learnt about the Intervals.
- Credits: Paul Fortier book.

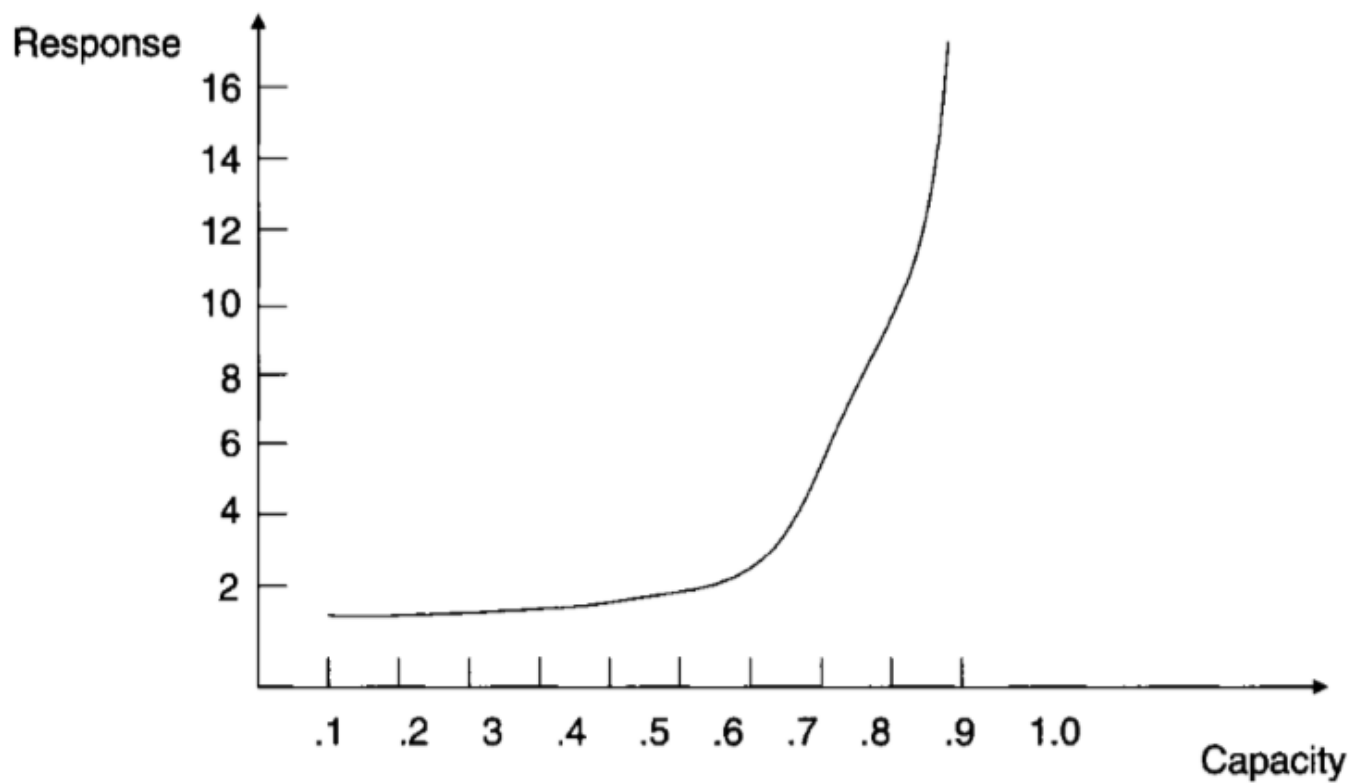
Lesson 229 : Response

Overview

- Here, we will learn about the response.

Response

- Represent a measure of period of time a user or application to wait before task assignment.
- Typical measure may pit the response time.
- **Response**



- Load increases above specific capacity the response increases.
- Study the measurable sequences against system load.
- How these loads impact each other.

Summary

- We have learnt about the response.
- Credits: Paul Fortier book.

Lesson 230 : Independence

Overview

- Here, we will learn about the Independence.

Independence

- Occurrence if one does not influence the outcome of other.
- Important concept while evaluating systems.
- Two programs that cannot run concurrently must be considered independent.
- Running on the same operating system and hardware.
- Cannot interfere each other then they are separate and unrelated.
- Important in modeling to define all elements and their relationship.

Summary

- We have learnt about the Independence.
- Credits: Paul Fortier book.

Lesson 231 : Randomness

Overview

- Here, we will learn about randomness.

Randomness

- Randomness is a property of an event and its reoccurrence.
- Have no pattern.
- Realization of the property of independence.
- It's a mathematical concept.
- External sources can be viewed as random events in systems.
- Important to analyze systems using mathematical concepts.

Summary

- We have learnt about the randomness.
- Credits: Paul Fortier book.

Lesson 232 : Workloads

Overview

- Here, we will learn about the workloads.

Workloads

- Simply we can say load.
- Event or event sequences presented to system to model.
- Represent the number of events offered to system for execution.
- Arranging the requests and perform them by the system.
- The duration could be endless.
- The load is re-entered after prescribed period of time.
- The database community has developed a set of transactional workloads.

Summary

- We have learnt about the workloads.
- Credits: Paul Fortier book.

Lesson 233 : Problems encountered in model development and use

Overview

- Here, we will understand about the problems encountered in model development and use.

Problems encountered in model development and use

- Must start with having concept that what we are doing.
- Determining what to do.
- Two main classes performance assessment.
- First take an existing system and design some experiments.
- Second is either analytical modeling or simultaneous.
- Analyst uses following measures:
- Responsiveness and use levels.
- Mission-ability and dependability.
- Productivity and predictability.
- Analyst must avoid the following mistakes:
- Having no goals.
- Setting biased goals.
- Unsystematic approach to development.
- Choosing of incorrect performance metrics.
- Choosing non-stressful workload.

- To solve these problems the analyst must ask some questions to himself/herself.

Summary

- We have learnt about the problems encountered in model development and use.
- Credits: Paul Fortier book.

Lesson 234 : A Case Study

Overview

- Here, we will understand about how to conduct a case study

A Case Study

- First Step: Define the System that we wish to study
- E.g. Remote pipes vs. Remote Procedures
- Focus on Services – small and large data transfers
- Select Metrics
- Rate of access
- Time for Performance
- Resource requirements per service
- Evaluate the developed model and start over again
- Trial and error
- Conducting our own: Modeling a ? ? ?
- A Bus system
- A railway terminal
- A hospital
- A human cell
- Human brain
- Ecosystem

Summary

- We have learnt about conducting a case study.
- Credits: Paul Fortier book.

Lesson 235 : Introduction to Software Reliability

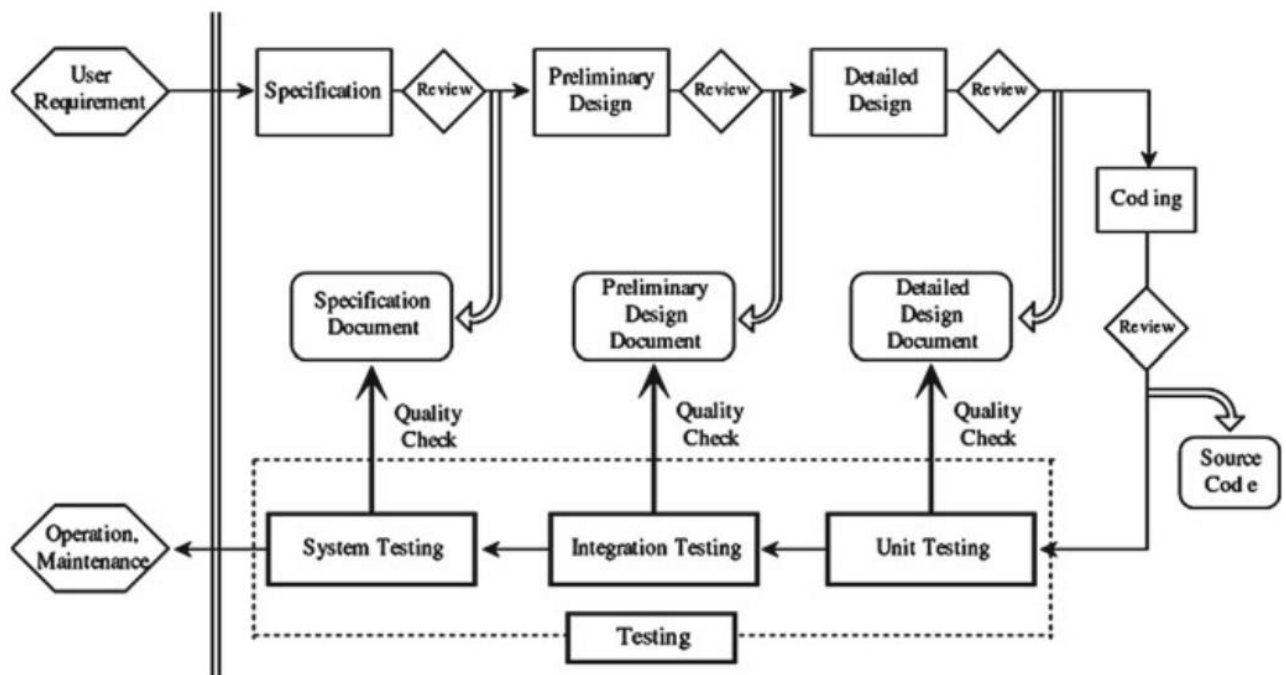
Overview

- Here, we will understand about software reliability

Introduction to Software Reliability

- Software fails on a regular basis
- This is a serious issue
- Especially in case of critical systems – nuclear reactors, patient care etc.
- Problems with X-Ray machine
- Problems with SCUD Missiles
- Problems with rocket launch
- Etc.

Introduction to Software Reliability



Summary

- We have learnt about software reliability.
- Credits: Yamada book.

Lesson 236 : Definitions and Software Reliability Model

Overview

- Here, we will understand about definitions about reliability

Definitions and Software Reliability Model

- Generally, a software failure caused by software faults latent in the system cannot occur except on a specific occasion when a set of specific data is put into the system under a specific condition, i.e. the program path including software faults is executed.
- A software system is a product which consists of the programs and documents produced through a software development process
- Software failure is defined as an unacceptable departure of program operation from the program requirements.
- The cause of software failure is called a software fault.
- Then, software fault is defined as a defect in the program which causes a software failure.
- The software fault is usually called a software bug.
- Software error is defined as human action that results in the software system containing a software fault
- **Definitions and Software Reliability Model**

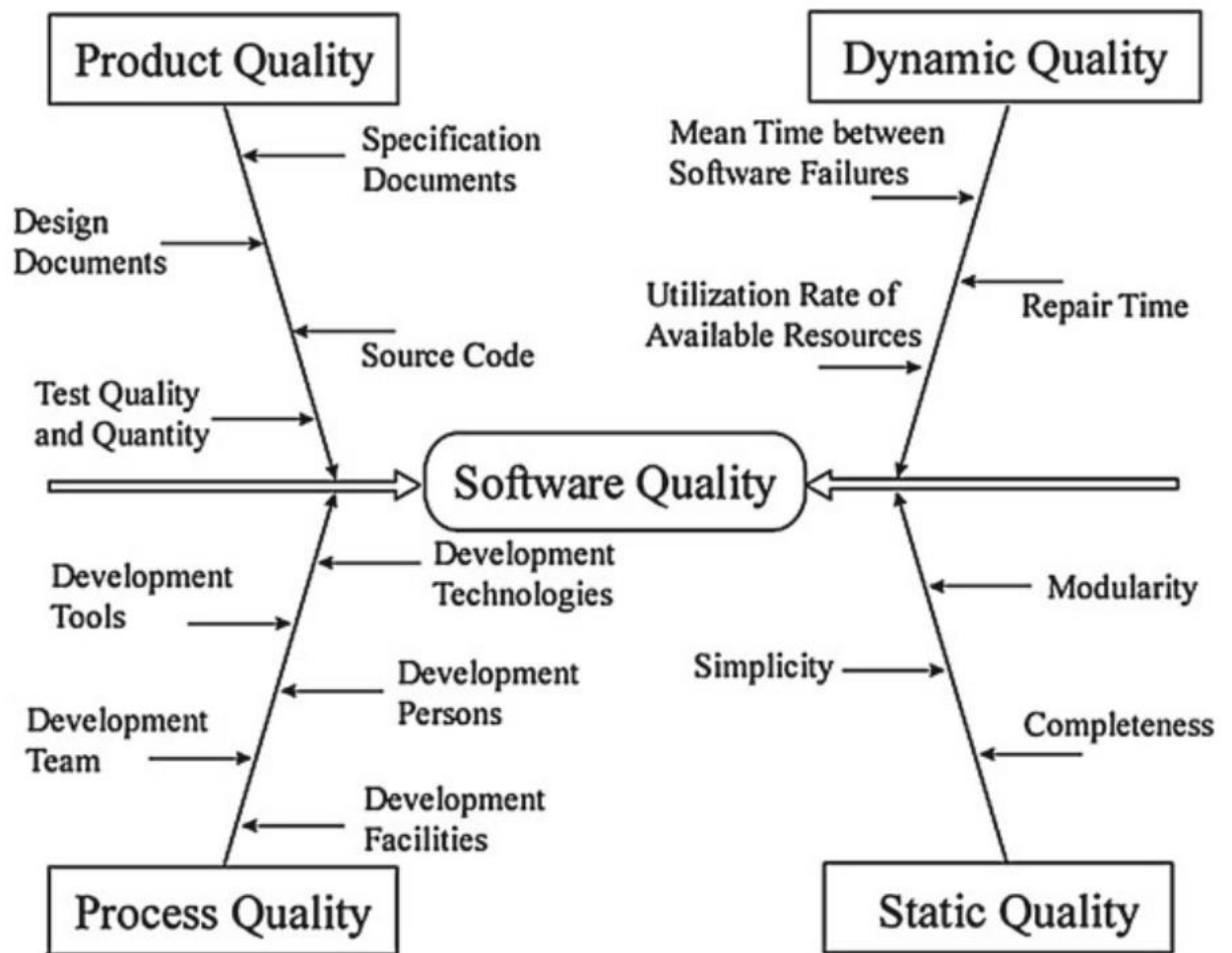
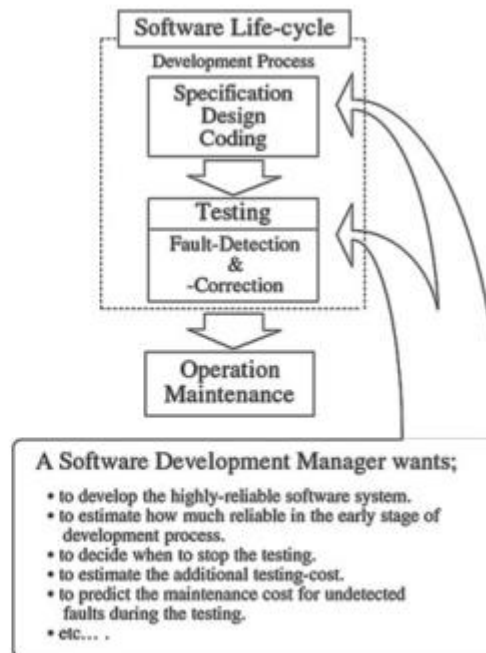


Fig. 1.2 Elements of software quality based on a cause-and-effect diagram

- **Definitions and Software Reliability Model**

Fig. 1.3 Aim of software quality/reliability measurement and assessment



- **Definitions and Software Reliability Model**

Terminologies

- **Software Failure**
— an unacceptable departure of program operation from the program requirements
- **Software Fault**
— a defect in the program which causes a software failure (*software bug*)
- **Software Reliability**
— the attribute that a software system will perform without causing software failures over a given time period under specified conditions
— measured by its probability

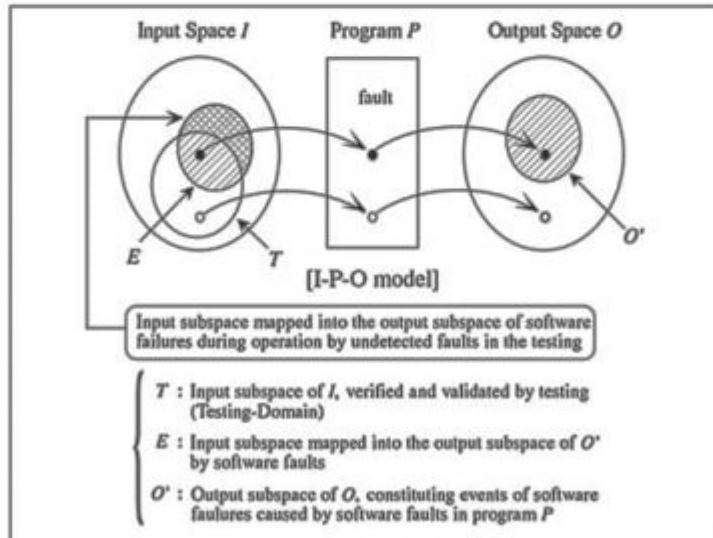


Fig. 1.4 An input-program-output model for software behavior

- **Definitions and Software Reliability Model**

Software Reliability	Hardware Reliability
(1) Software failures can be due to no wearout phenomenon.	(1) Hardware failures can be due to wear.
(2) Software reliability is, inherently, determined during the earlier phase of the development process, i.e. specification analysis and design phases.	(2) Hardware reliability is affected by deficiencies injected during all phases of the development, operation, and maintenance.
(3) Software reliability can not be improved by redundancy with identical versions.	(3) Hardware reliability can be improved by redundancy with identical units.
(4) A verification method of software reliability has not been established.	(4) A testing method of hardware reliability is established and standardized.
(5) A maintenance technology is not established since the market of software products is rather recent.	(5) A maintenance technology is advanced since the market of hardware products is established and the user environment is seized.

Fig. 1.5 Comparison between the characteristics of software reliability and hardware reliability

Lesson 237 : Software Reliability Growth Modeling

Overview

- Here, we will understand about software reliability growth modeling

Software Reliability Growth Modeling

- Generally, a mathematical model based on stochastic and statistical theories is useful to describe the software fault-detection phenomena or the software failure-occurrence phenomena and estimate the software reliability quantitatively.
- **Software Reliability Growth Modeling**

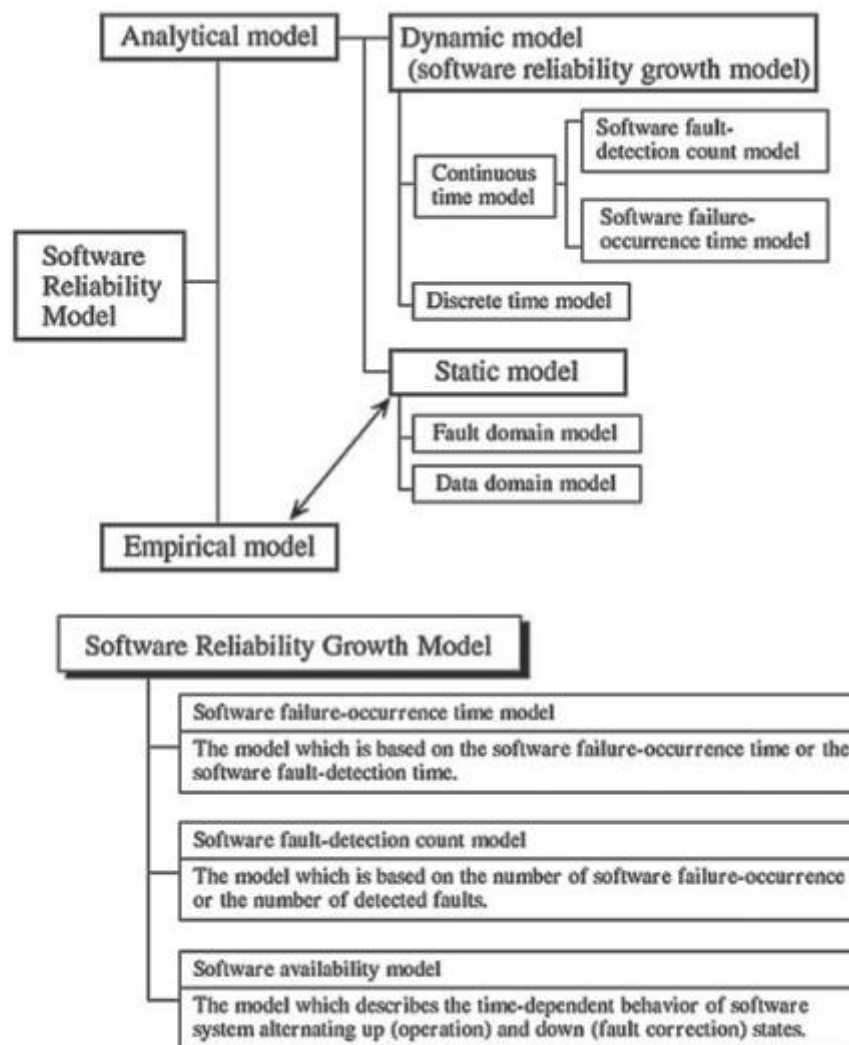


Fig. 1.6 Hierarchical classification of software reliability models

- **Software Reliability Growth Modeling**

$N(t)$ the cumulative number of software faults (or the cumulative number of observed software failures) detected up to time t ,

S_i the i th software-failure occurrence time ($i = 1, 2, \dots; S_0 = 0$),

X_i the time-interval between $(i - 1)$ -st and i th software failures ($i = 1, 2, \dots; X_0 = 0$).

- **Software Reliability Growth Modeling**

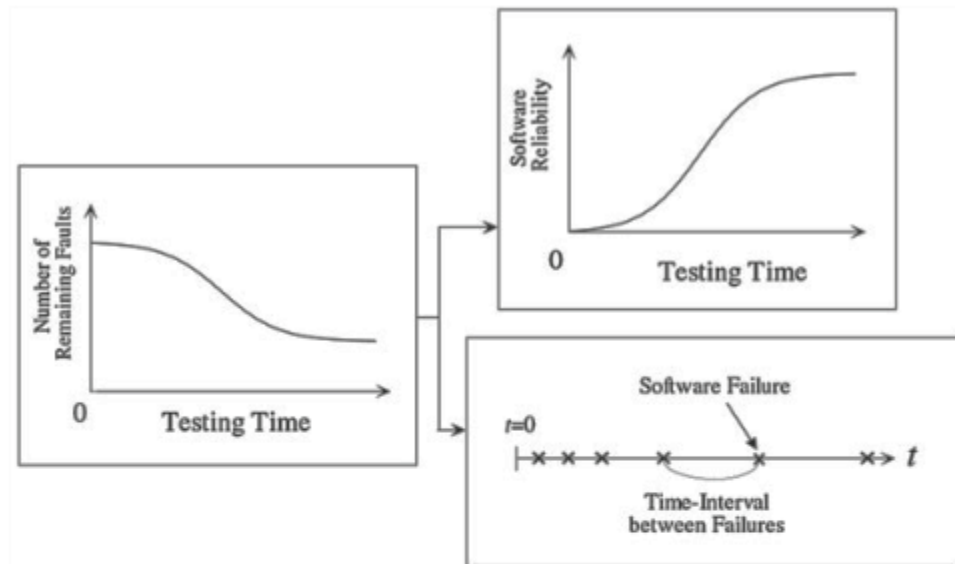


Fig. 1.7 Software reliability growth

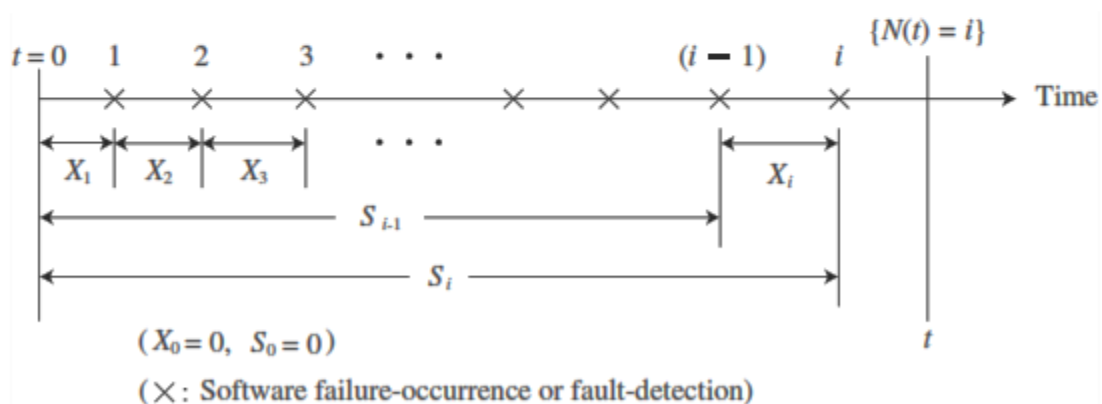


Fig. 1.8 The stochastic quantities related to a software fault-detection phenomenon or a software failure-occurrence phenomenon

Summary

Lesson 238 : Imperfect Debugging Modeling

Overview

- Here, we will understand about imperfect debugging modeling

Imperfect Debugging Modeling

- Most software reliability growth models proposed so far are based on the assumption of perfect debugging
- i.e. All faults detected during the testing and operation phases are corrected and removed perfectly.
- However, debugging actions in real testing and operation environment are not always performed perfectly.
- For example, Typing errors invalidate the fault-correction activity or fault-removal is not carried out precisely due to incorrect analysis of test results
- As a result it is preferable to develop an incorrect debugging model.
- This will result in a better estimation of reliability

Summary

- We have learnt about imperfect debugging modeling.
- Credits: Yamada book.

Lesson 239 : Imperfect Debugging Model with Perfect Correction Rate

Overview

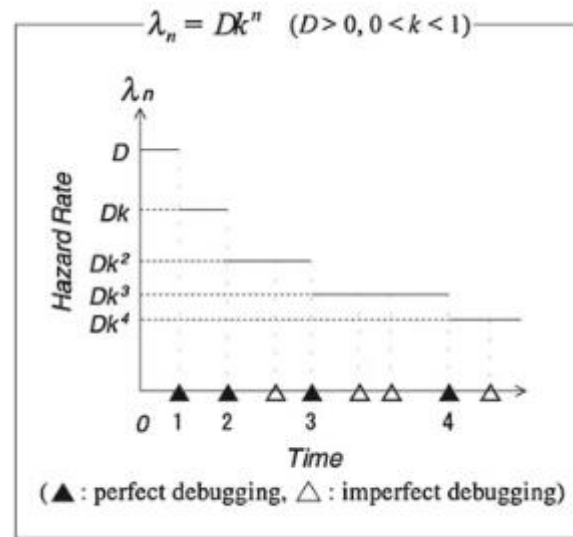
- Here, we will understand about imperfect debugging model with perfect correction rate

Imperfect Debugging Model with Perfect Correction

- To model an imperfect debugging environment, the following assumptions are made:
 1. Each fault which causes a software failure is corrected perfectly with probability p ($0 \leq p \leq 1$). It is not corrected with probability $q (= 1 - p)$. We call p the perfect debugging rate or the perfect correction rate.
 2. The hazard rate is given by (1.5) and decreases geometrically each time a detected fault is corrected (see Fig. 1.14).
 3. The probability that two or more software failures occur simultaneously is negligible.
 4. No new faults are introduced during the debugging. At most one fault is removed when it is corrected, and the correction time is not considered.

- **Imperfect Debugging Model with Perfect Correction**

Fig. 1.14 Behavior of hazard rate



- **Imperfect Debugging Model with Perfect Correction**

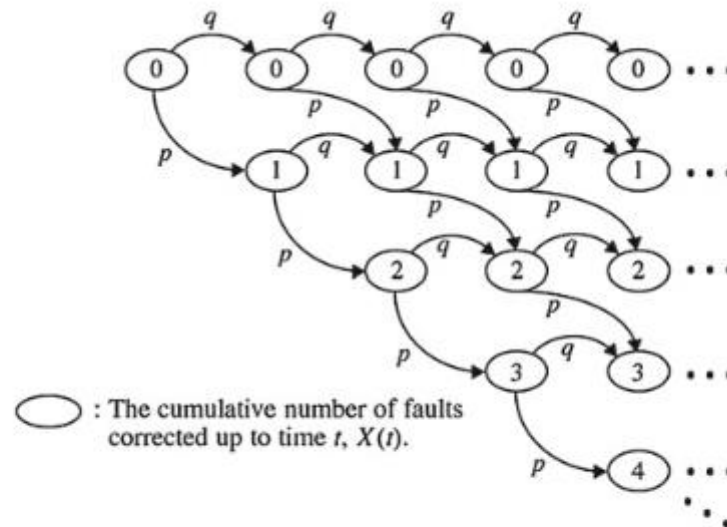


Fig. 1.15 A diagrammatic representation of transitions between states of $X(t)$

- Let $X(t)$ be a random variable representing the cumulative number of faults corrected up to the testing time t . Then, $X(t)$ forms a Markov process.
- That is, from assumption 1, when i faults have been corrected by arbitrary testing time t ,

$$X(t) = \begin{cases} i, & \text{with probability } q, \\ i + 1, & \text{with probability } p, \end{cases} \quad (1.14)$$

Summary

- We have learnt about imperfect debugging modeling with correction.
- Credits: Yamada book.

Lesson 240 : Imperfect Debugging with Introduced Faults

Overview

- Here, we will understand about imperfect debugging modeling with introduced faults

Imperfect Debugging with Introduced Faults

- Besides the imperfect debugging factor above in fault-correction activities, we consider the possibility of introducing new faults in the debugging process.
- It is assumed that two kinds of software failures exist in the dynamic environment
- (F1) software failures caused by faults originally latent in the software system prior to the testing (which are called inherent faults)
- (F2) software failures caused by faults introduced during the software operation owing to imperfect debugging.
- In addition, it is assumed that one software failure is caused by one fault
- And that it is impossible to discriminate whether the fault that caused the software failure that has occurred is F1 or F2.
- As to the software failure-occurrence rate due to F1, the inherent faults are detected with the progress of the operation time.
- In order to consider two kinds of time dependencies on the decreases of F1, let
- $\lambda_i(t) = (1,2)$ denote software failure occurrence rate for F1
- For F2, it is constant ($\lambda > 0$)

Summary

- We have learnt about imperfect debugging modeling with introduced faults.
- Credits: Yamada book.

Lesson 241 : Software Availability Modeling

Overview

- Here, we will understand about software availability modeling

Software Availability Modeling

- Recently, software performance measures such as the possible utilization factors have begun to be interesting for metrics as well as the hardware products.
- That is, it is very important to measure and assess software availability,
- It is defined as the probability that the software system is performing successfully, according to the specification, at a specified time point
- The actual operational environment needs to be more clearly reflected in software availability modeling, since software availability is a customer-oriented metrics.
- **Software Availability Modeling**

- **Software Reliability** : _____
the attribute that software systems can continue to perform and do not cause any software failures *for a given time period*, under a specified environment.

■ software failure

[an unacceptable departure from program operation caused by a software fault remaining in the software system.]

- **Software Availability** : _____
the attribute that software systems are available and performing *at a given time point*, under a specified environment.

- Software fails on a regular basis
- This is a serious issue
- Especially in case of critical systems – nuclear reactors, patient care etc.

Summary

- We have learnt about conducting a case study.
- Credits: Paul Fortier book.

Lesson 242 : Model Description

Overview

- Here, we will understand about model description for software availability modeling

Model Description

- **Assumptions**

1. The software system is unavailable and starts to be restored as soon as a software failure occurs, and the system cannot operate until the restoration action is complete
2. The restoration action implies debugging activity, which is performed perfectly with probability a
 - ($0 < a \leq 1$) and imperfectly with probability $b (= 1 - a)$.
 - a is the perfect debugging rate
 - One fault is corrected and removed from the software system when the debugging activity is perfect.
3. When n faults have been corrected, the time to the next software failure- occurrence and the restoration time follow exponential distributions
4. The probability that two or more software failures will occur simultaneously is negligible

Summary

- We have learnt about model description.
- Credits: Yamada book.

Lesson 243 : Software Availability Measures

Overview

- Here, we will understand about software availability measures

Model Description

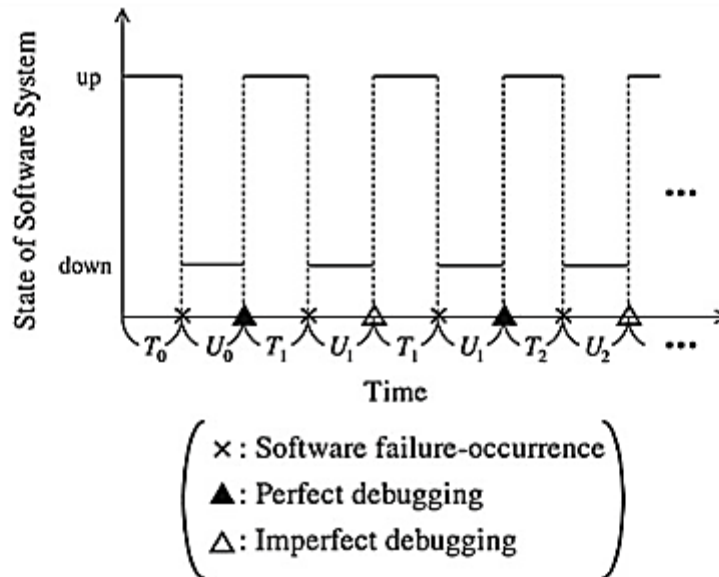


Fig. 1.17 Sample behavior of the software system alternating between up and down states

- Software Availability Measures

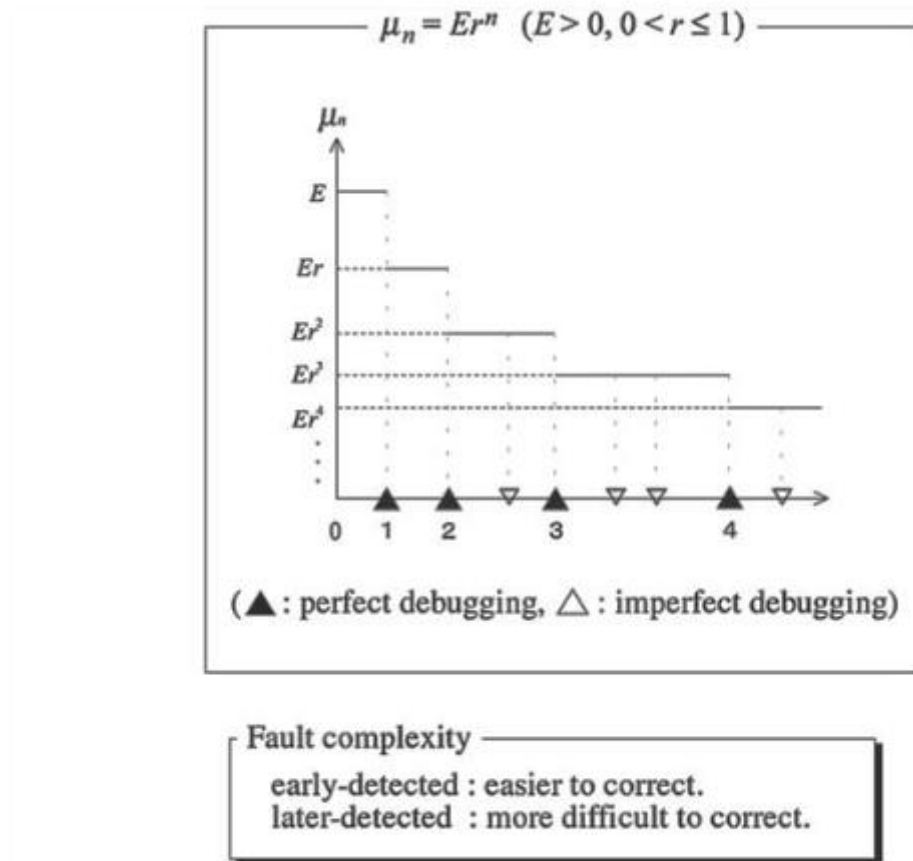
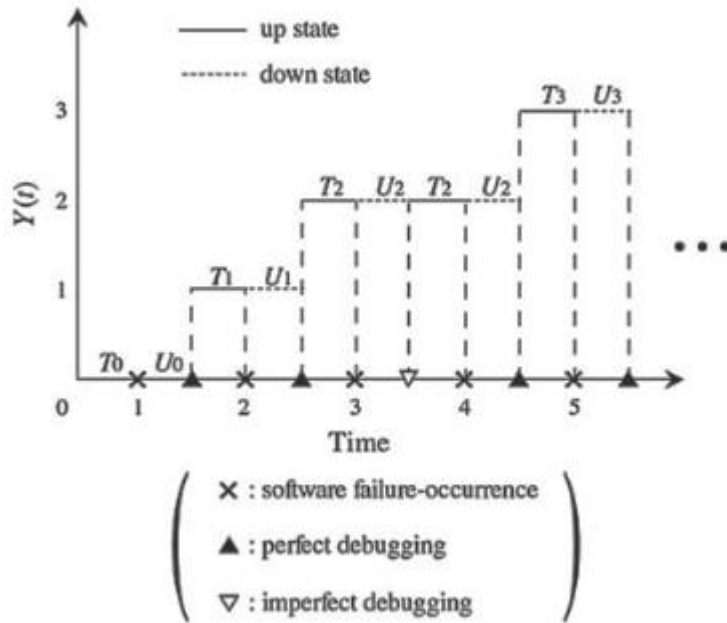


Fig. 1.18 Behavior of restoration rate

- Software Availability Measures



- Software Availability Measures

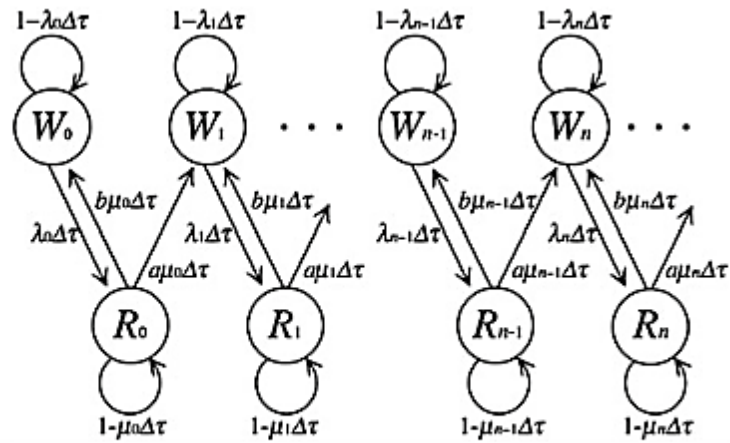


Fig. 1.20 A state transition diagram for software availability modeling

- Software Availability Measures

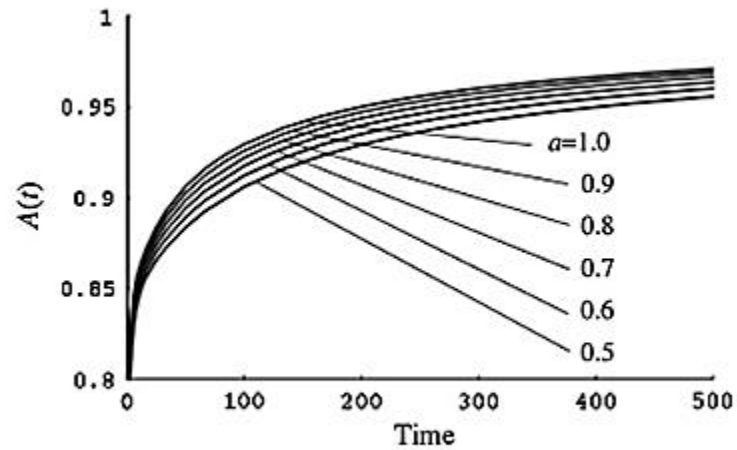


Fig. 1.21 Dependence of perfect debugging rate a on $A(t)$

- **Software Availability Measures**

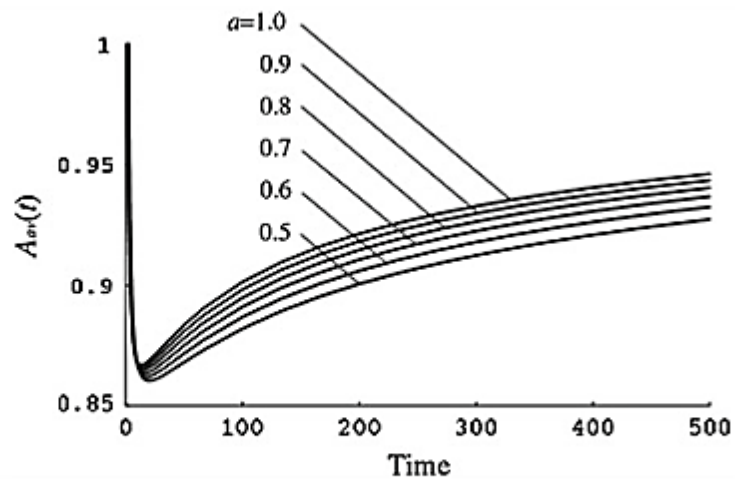


Fig. 1.22 Dependence of perfect debugging rate a on $A_{av}(t)$

Summary

- We have learnt about software availability measures.
- Credits: Yamada book.

Lesson 244 : Application of Software Reliability Assessment

Overview

- Here, we will understand about software reliability assessment

Application of Software Reliability Assessment

- It is very important to apply the results of software reliability assessment to management problems with software projects for attaining higher productivity and quality.
- We discuss three software management problems as application technologies of software reliability models.
- Optimal Software Release Problem
- Statistical Software Testing-Progress Control
- Optimal Testing-Effort Allocation Problem

Summary

- We have learnt about conducting a case study.
- Credits: Paul Fortier book.

Lesson 245 : Optimal Software Release Problem

Overview

- Here, we will understand about optimal software release issues

Optimal Software Release Problem

- Recently, it is becoming increasingly difficult for the developers to produce highly reliable software systems efficiently.
- Thus, it has been necessary to control a software development process in terms of quality, cost, and release time.
- In the last phase of the software development process, testing is carried out to detect and fix software faults, introduced by human work, prior to its release for the operational use.
- The software faults that cannot be detected and fixed remain in the released software system after the testing phase.
- Thus, if a software failure occurs during the operational phase, then a computer system stops working and it may cause serious damage.

- If the duration of software testing is long, we can remove many software faults in the system and its reliability increases. However, this increases the testing cost and delays software delivery.
- In contrast, if the length of software testing is short, a software system with low reliability is delivered and it includes many software faults which have not been removed in the testing phase.
- Thus, the maintenance cost during the operation phase increases.
- It is therefore very important in terms of software management that we find the optimal length of software testing, which is called an optimal software release problem
- These decision problems have been studied in the last decade by many researchers.
- In optimal software release problems we consider both a present value and a warranty period (in the operational phase)

Summary

- We have learnt about optimal software release.
- Credits: Yamada book.

Lesson 246 : Statistical Software Testing-Progress Control

Overview

- Here, we will understand about statistical software testing-progress control

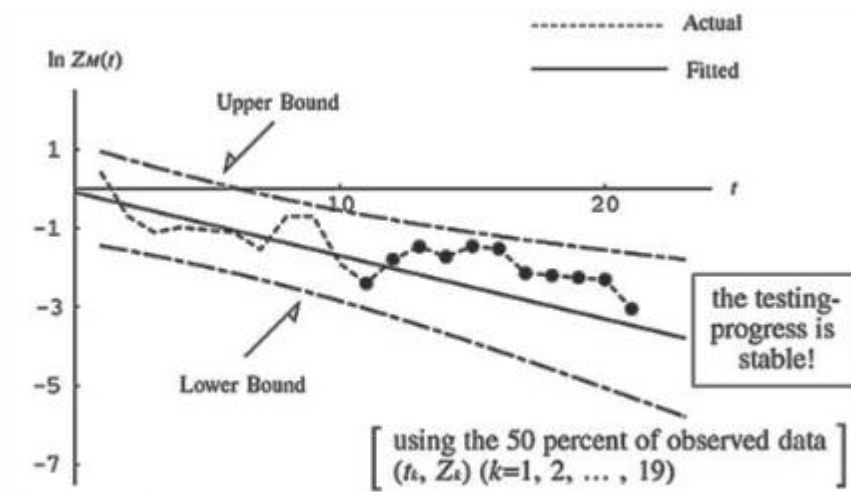
Statistical Software Testing-Progress Control

- As well as quality/reliability assessment, software-testing managers should assess the degree of testing-progress.
- We can construct a statistical method for software testing-progress control based on a control chart method
- This method is based on several instantaneous fault-detection rates derived from software
- **Statistical Software Testing-Progress Control**

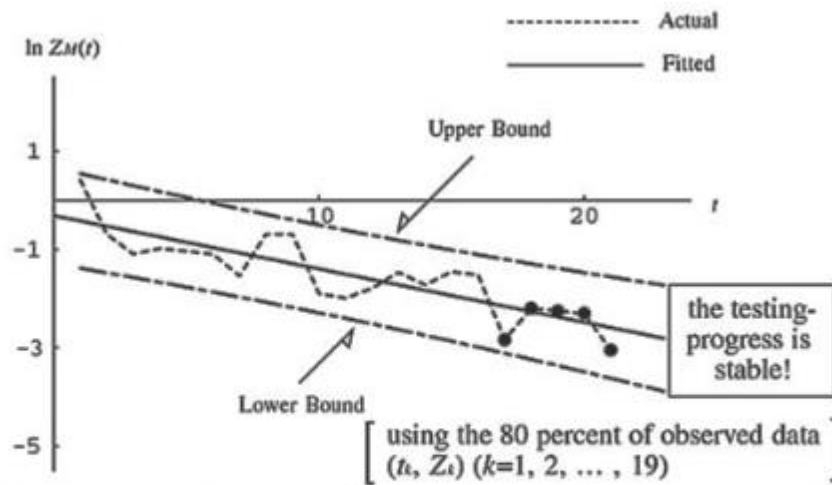
The procedure of testing-progress control is shown as follows:

- Step 1: An appropriate model is selected to apply and the model parameters are estimated by the method of least-squares.
- Step 2: To certify goodness-of-fit of the estimated regression equation for the observed data, we use the F -test.
- Step 3: Based on the result of the F -test, the central line and upper and lower control limits of the control chart are calculated. The control chart is drawn.
- Step 4: The observed data are plotted on the control chart and the stability of the testing-progress is judged.

- Statistical Software Testing-Progress Control



[An example of the control chart for the testing-progress control]



[An example of the control chart for the testing-progress control]

Fig. 1.25 Examples of the control chart for the testing-progress control

Summary

- We have learnt about statistical software testing-progress control.
- Credits: Yamada book.

Lesson 247 : Optimal Testing-Effort Allocation

Overview

- Here, we will understand about Optimal Testing-Effort Allocation

Optimal Testing-Effort Allocation

- We discuss a management problem to achieve a reliable software system efficiently during module testing in the software development process by applying a testing-effort-dependent software reliability growth model
- We take account of the relationship between the testing-effort spent during the module testing and the detected software faults where the testing-effort is defined as resource expenditures spent on software testing, e.g. manpower, CPU hours, and executed test cases.
- The software development manager has to decide how to use the specified testing-effort effectively in order to maximize the software quality and reliability.
- That is, to develop a quality and reliable software system, it is very important for the manager to allocate the specified amount of testing-effort expenditure for each software module under some constraints.
- We can observe the software reliability growth in the module testing in terms of a time-dependent behavior of the cumulative number of faults detected during the testing stage.
- Based on the testing-effort dependent software reliability growth model, we consider the following testing-effort allocation problem:
- Based on the testing-effort dependent software reliability growth model, we consider the following testing-effort allocation problem:
 1. The software system is composed of M independent modules. The number of software faults remaining in each module can be estimated by the model.
 2. The total amount of testing-effort expenditure for module testing is specified.
 3. The manager has to allocate the specified total testing-effort expenditure to each software module so that the number of software faults remaining in the system may be minimized.

Summary

- We have learnt about Optimal Testing-Effort Allocation.
- Credits: Yamada book.

Lesson 248 : Introduction to formal modeling

Overview

- **Here, we will understand about the “formal modeling”.**

Formal modeling

- Petri nets PNs provide graphical tool and notation method.
- Include arrival rate and service rate.
- Individual state information.
- Contention and concurrency.
- Petri Nets have place in computer systems performance assessment.
- Between analytical queuing theory and computer simulation.
- System state more complete than analytical models.
- But not as simulations.
- Have fundamental theory but act like simulations.
- Single entities, movement and affect effect of the state of the entire system.
- Petri Nets was introduced in 1966.
- It describes concurrent systems.
- Followed by continuous improvements.
- Timing of transitions.
- Priority of transitions.
- Types to Token.
- Colors depicting.
- Analysis tools and modelling aid.

Summary

- Here, we understood about the “Testing for Significance of Regression”.
- Credits: Paul Fortier book.

Lesson 249 : Basic notation-??

Overview

- Here, we will understand about the “Basic notation”?

Basic notation

- Petri Nets provides abstraction and information flow.
- Using four fundamental components.
- Places as Circles.
- Transition as bars.
- Arc as line segments.
- Token as dots.
- Places represents useful system components and their state e.g. disk drive.
- Transitions represents events.
- Action of reading an item from disk drive.
- Arcs represents relationship between transitions and places.
- Proved path for activation of transition.
- Tokens define state of Petri net.

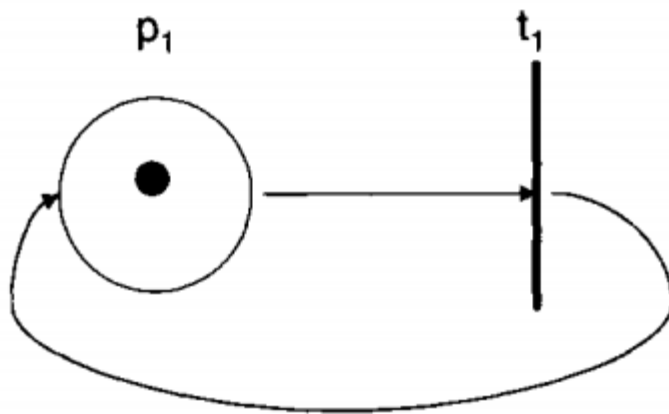
Forever Cycling Petri Net

The place is connected to the transition by an arc, and the transition is likewise connected to the place by a second arc.

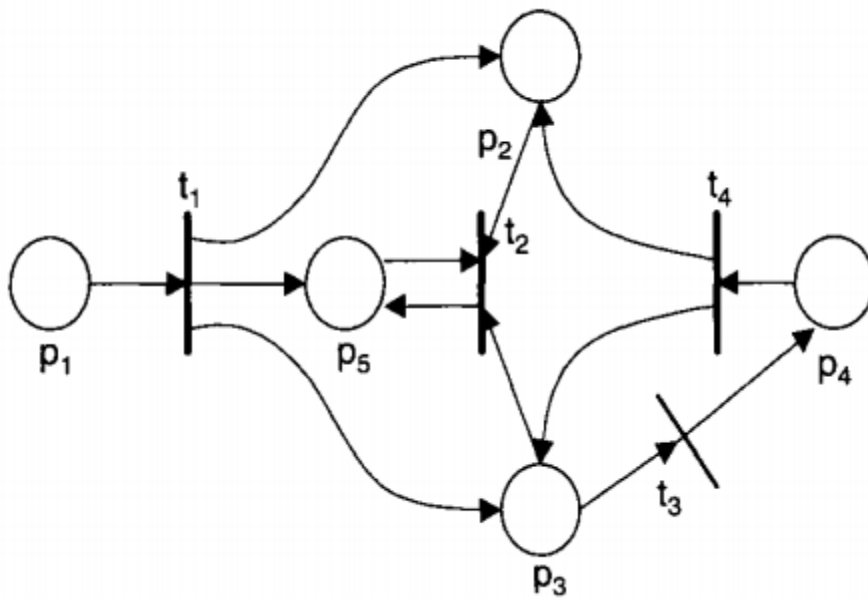
The former arc is an input arc, while the latter arc is an output arc.

The placement of a token represents the active marking of the Petri net state

The Petri net shown represents a net that will continue to cycle forever.



Petri Net Graphical Representation



Places(P), Transitions(T) Arc(input I, Output O), and Tokens for net (MP).

Five tuple $M=(P,T,I,O,MP)$.

P represent set of Places $P\{p_1, p_2, \dots p_n\}$.

P represent set of Transitions $P\{t_1, t_2, \dots t_n\}$.

I as input functions $I = \{I_{t1}, I_{t2}, \dots I_{tn}\}$ and as output functions $O = \{O_{t1}, O_{t2}, \dots O_{tn}\}$.

MP represents marking of places with tokens.

MP_O is represented as ordered tuple of magnitude where n represents number of places in petri net.

Summary

- Here, we understood about Basic notation.
- Credits: Paul Frontier book.

Lesson 250 : Basic notation-II

Overview

- Here, we will understand about the “Basic notation”.

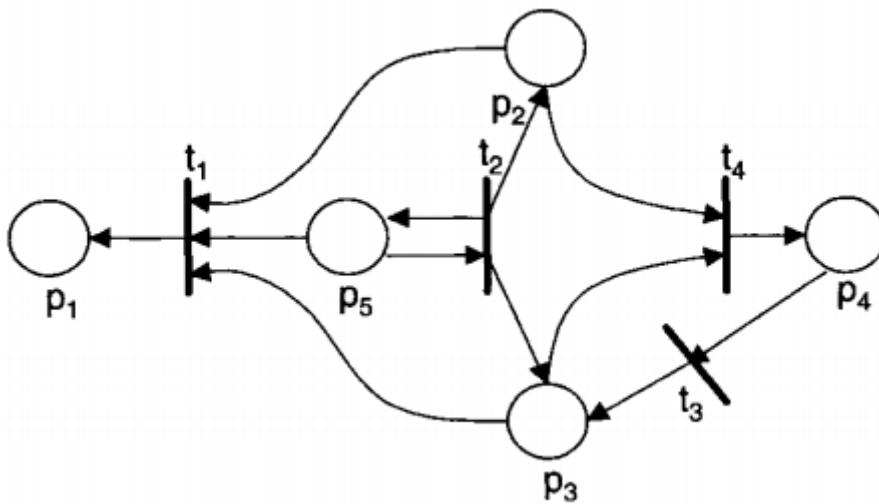
Basic notation-II^{II}

Petri Net Dual

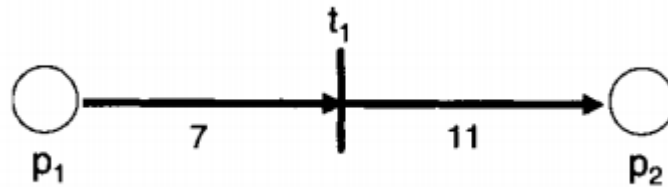
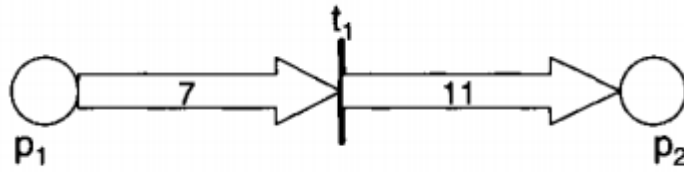
- Petri net model also has dual.
- In dual of petri, transitions are changed to places and places to transitions.
- The input becomes output function and output becomes output function.

Petri Net Inverse

- A petri net model is also defined for inverse.
- Places and transitions are kept same.
- Whereas, input functions switches to output functions.
- **Petri Net Inverse Representation**



- **Petri Net Multigraphs**
- A petri net model is also defined for Multigraphs.
- It implies there could be several arcs between a single place and a transition.
- Can be represented as:
 - Multiple arc as thick arc with numbers inside.
 - A bold arc with number attached to it.
- **Basic Petri Net Multigraphs Representation**



Petri Net State

- A petri net also has a state defined by the cardinality of tokens.
- Represented as function, μ^u .

$$\mu : p \rightarrow N$$

- μ^u can be defined as vectors.

$$\mu = (\mu_1, \mu_2, \mu_3, \dots, \mu_n),$$

Petri Net Multigraphs

- Where.

$$n = |P| \text{ and each } \mu_i \in N, i = 0, \dots, n$$

- Place marking function and vector relation:

$$\mu(p_i) = \mu_i.$$

- Petri net structure:

$$M = (P, T, I, O, \mu_t)$$

- It represents state of petri net at time t.

Summary

- Here, we understood about the “Basic Notations”.
- Credits: Paul Fortier book.

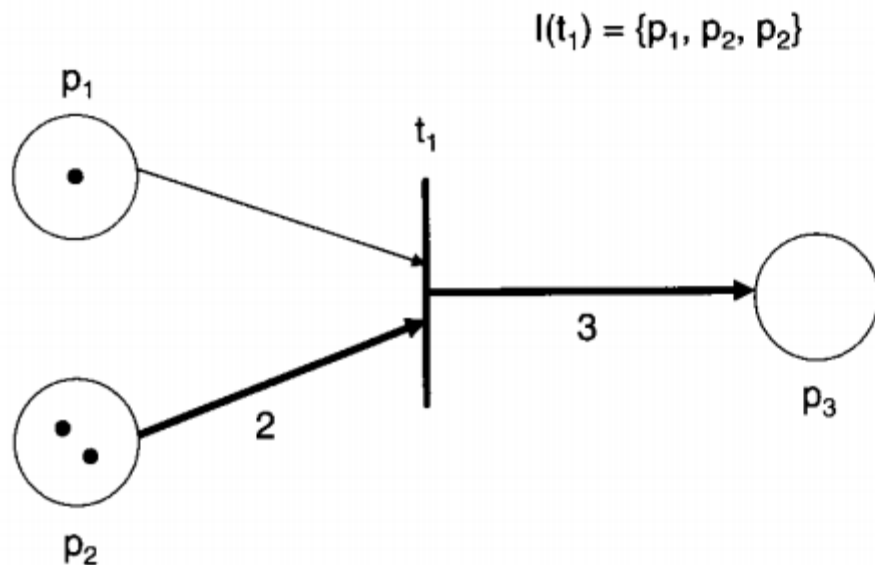
Lesson 251 : Classical Petri Nets-I

Overview

- Here, we will understand about the “Classical Petri Nets”?

Classical Petri Nets

- By using notations Petri nets can be used in modelling.
- A new petri net state transition notation is required.
- These transitions are helpful in movements.
- The exact firing moment can be captured as a clock signal.
- During Clock cycle, all gates start signal execution.
- In petri net all transitions fire once during this cycle.

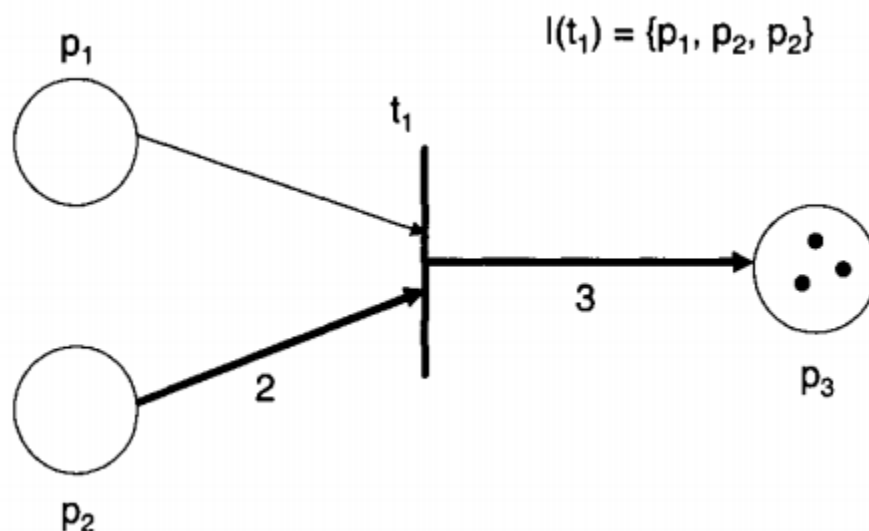


The Petri net shown has the marking $g_0 = (1, 2, 0)$, input function $I(t_1) = \{P_1, P_2, P_2\}$, and output function $O(t_1) = \{P_3, P_3, P_3\}$.

Given this initial marking and the defined input and output functions, transition t_1 is enabled, since it requires one token contributed by P_1 and two tokens contributed by P_2 to have all of its input functions satisfied with tokens available in the named places and in the required quantities.

transition is enabled at the beginning of a Petri net's firing cycle, it will fire the transition, causing the movement of the number of tokens from its input places to its output places. The result of this firing will be a new Petri net state, B_1 . For example, in given the initial state B_0 , transition t_1 will fire at the beginning of the firing cycle, removing one token from place P_1 and two tokens from place P_2 and placing these three tokens into place P_3 .

Forever Cycling Petri Net



The resulting new Petri net state is represented by the marking $B_1 = (0, 0, 3)$, and is depicted in the figure. The firing action is considered an atomic action, in that it appears as if all of the tokens are removed from the input places and deposited into the output places instantaneously.

The firing of the Petri net provides the movement from one state to a new state. The new state is the only state reachable in a single total transition firing of all the enabled transitions. The collection of all possible states that can be represented by this Petri net and its initial markings is called state-space. This implies that a Petri net can only fire enabled transitions, and, after they fire, they cannot fire any newly enabled transitions until the next cycle.

Petri Net Set Notation Representation

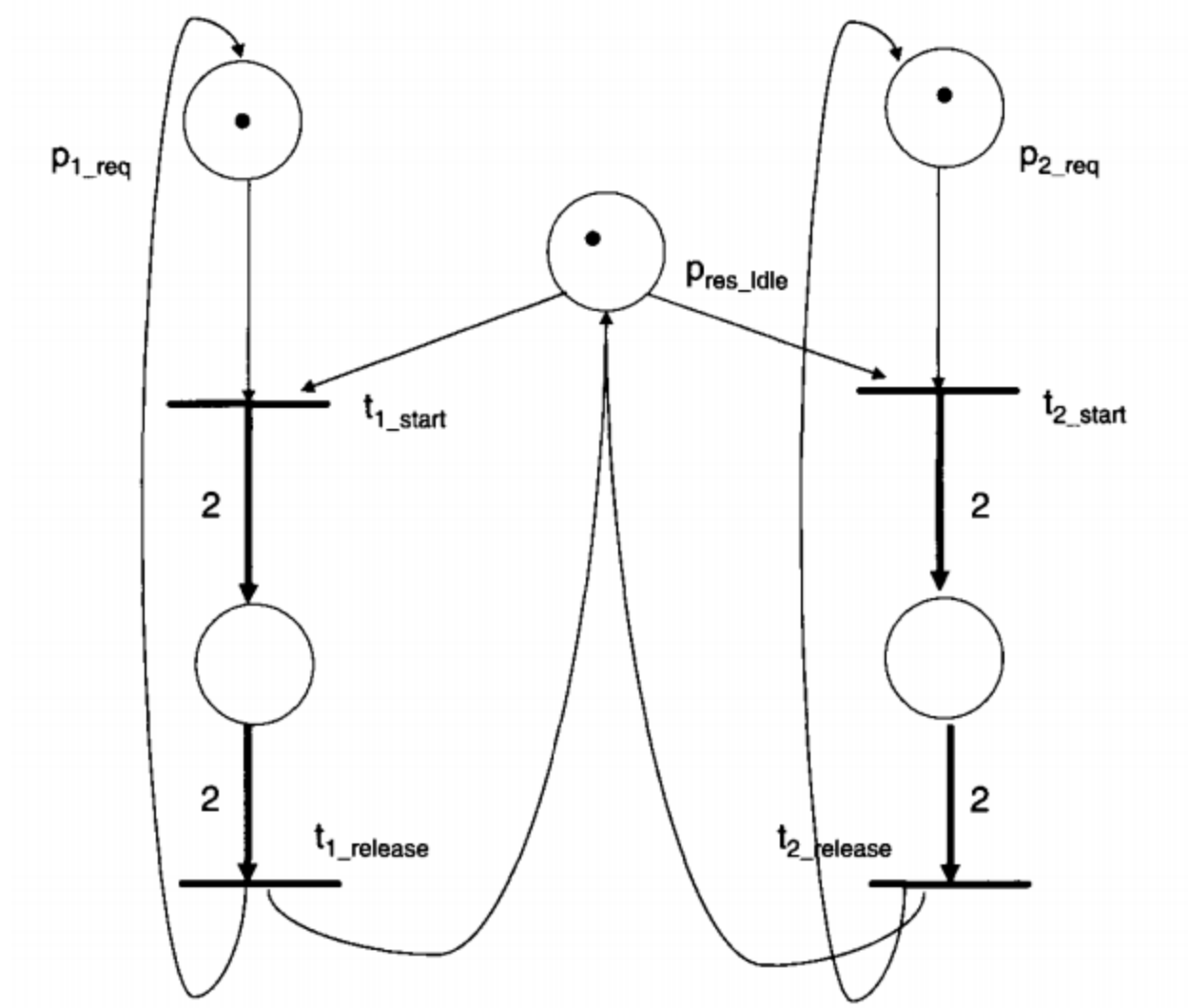
Petri nets enables modeling not available through queuing theory.

- Synchronization.
- Conflict.

- Concurrency.

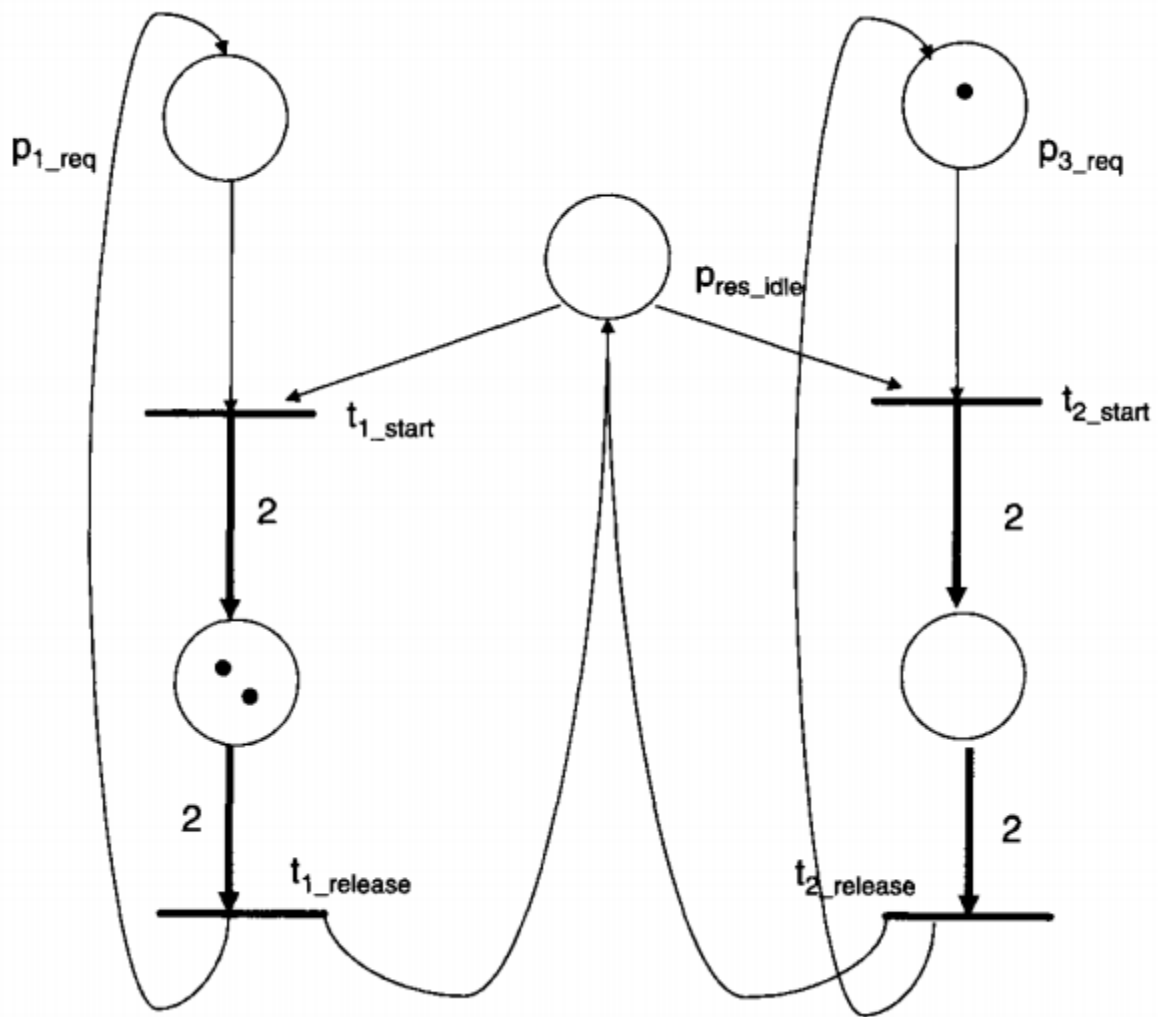
These are not easily defined and modeled by queuing theory.

Resource Sharing



We model two user processes requesting a specific resource (P_{res_idle}). If the resource is idle, there is a token present in the (P_{res_idle}) place. If a process wanting this resource has a token in its place (e.g., P_{p1_req}), and the resource is idle (a token in P_{res_idle}), then the transition (e.g., t_{1_start}) is enabled and can fire on the next cycle. When the transition fires, the resource now becomes unavailable (it is busy), and the second process must wait until the resource is released.

Allocated Resources



Summary

- Here, we understood about Classical Petri Nets.
- Credits: Paul Frontier book.

Lesson 252 : Classical Petri Nets-????

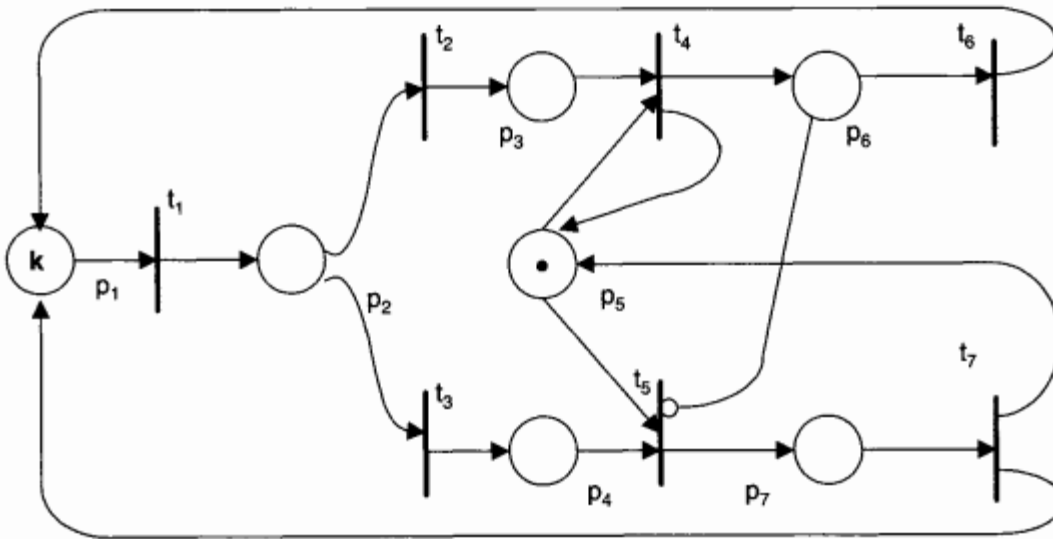
Overview

- Here, We will understand “Classical Petri Nets”.

Classical Petri Nets-II^{II}

Classical Petri Nets

- A Petri net's state, is said to be reachable from some other state, $u^{u'}$.
- There exists some finite number of firings of the Petri net beginning at state u^u .



The reachability set for a Petri net with an initial marking, $\mu_0^{\mu_0}$, is denoted $RS(\mu_0^{\mu_0})$ and is defined as the smallest set of markings, so that:

$$\mu_0^{\mu_0} \in RS(\mu_0^{\mu_0}) \text{ and}$$

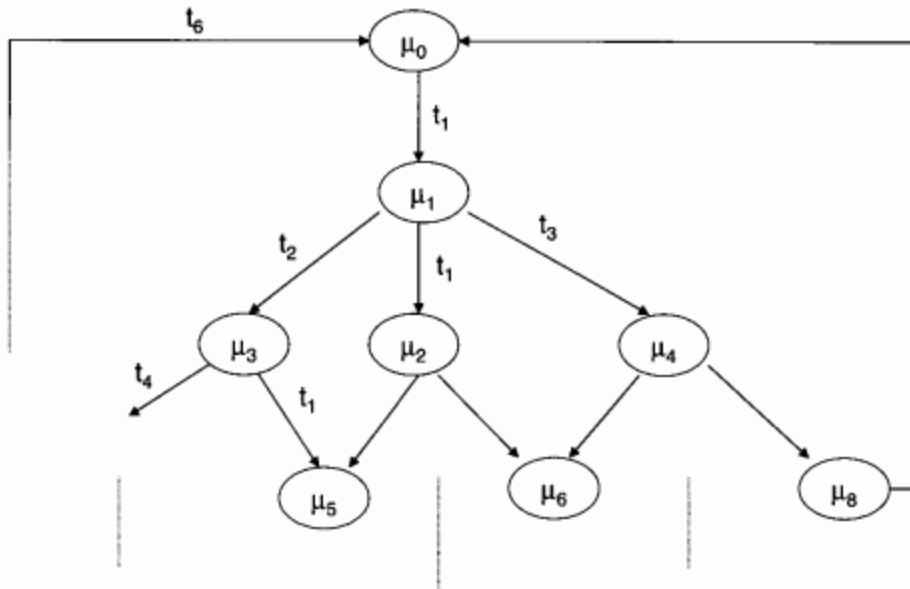
$$\mu_0^{\mu_0} RS(\mu_0^{\mu_0})$$

$$\mu_2 \in RS(\mu_2) \text{ (No } N_0)$$

To determine the reachability set, we must begin from the initial state, $\mu_0^{\mu_0}$, and incrementally define each step possible emanating from this initial state and all states derivable from this state.

The reachability set contains no information about which transitions fired to reach the state markings. Such information can be found in a reachability graph shown next.

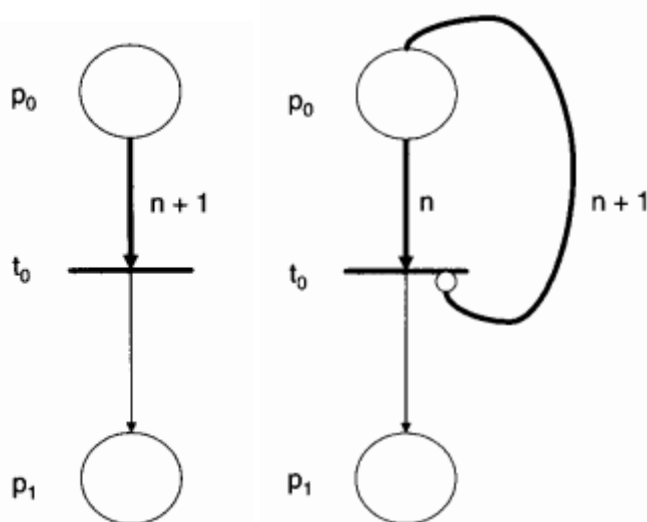
- Reachability Graph**



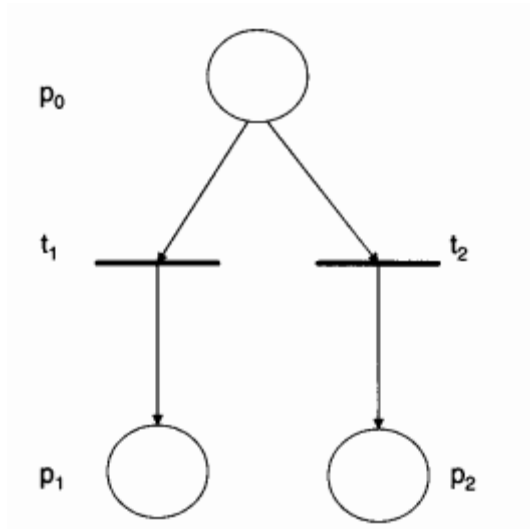
In this graph each node represents a state of the Petri net and each arc represents a direct transition, which is possible from one end of the directed arc to the other end due to a single transitions firing. For example, you can see that if we fire transition t_1 from μ_0 we can get to a new state, μ_1 , where a token has moved from place 1 to place 2.

It is often desirable to model logical conditions. For example, to only fire a transition when there are more than n tokens in a place, we simply need to include an arc with arity $n + 1$. Since the basic condition on a transition firing is that its connected input places meet the conditions of the transitions input function, by including $n + 1$ redundant set items for the input place we can accomplish what we need shown in next figured

- **Component Test**



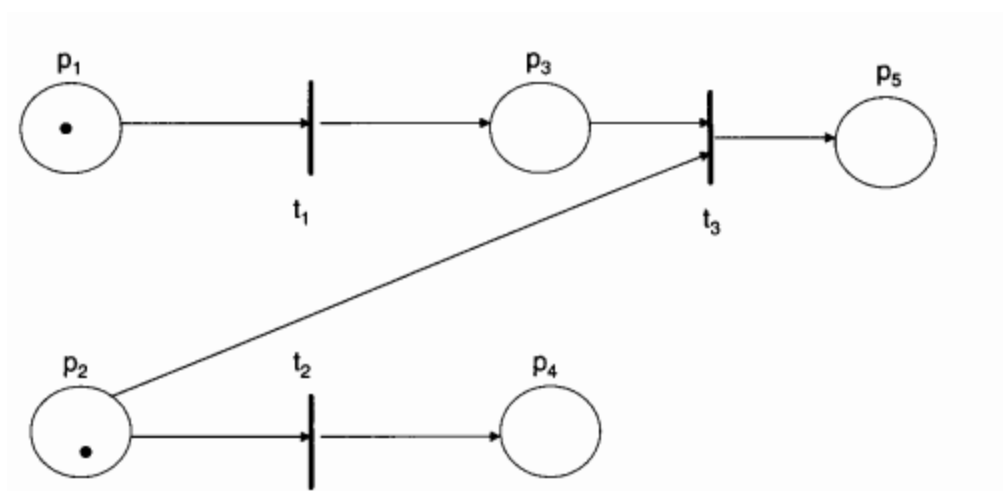
An important property required when modeling computer systems is that of conflict shown next.



In this example, when there is a token in place p_0 , both transitions t_1 and t_2 are enabled.

However, only one of them may fire, since there is only one token available. As soon as one fires, say t_1 , it removes the token from place p_0 and transfers it to place p_1 . As soon as transition

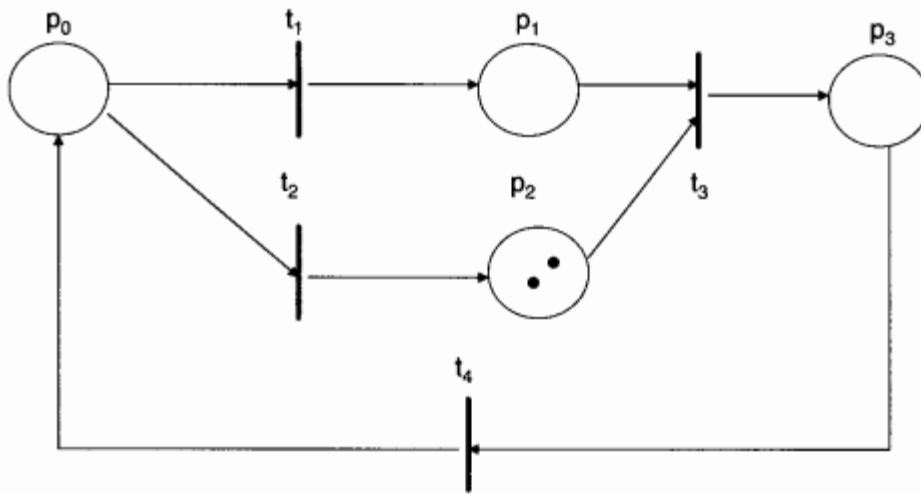
t_1 removes the token, transition t_2 is no longer enabled. If there were two tokens in place p_0 , then both of the transitions would be enabled and could fire during this firing cycle.



Very often a computer system can have both conflict and concurrency occur at the same time as shown.

we have an initial marking, $u = (1,1,0,0,0)$, which results in transitions t_1 and t_2 being enabled, the condition of concurrent transitions. If t_1 fires first, then we now have two transitions enabled, t_2 and t_3 . This then depicts conflict, since the token from p_2 can only satisfy one of the necessary conditions for the two transitions it is enabling at this point in time.

A Petri net can have a variety of other qualities as well.



A Petri net is deadlocked if there are no transitions in the net that are enabled. In the example shown, the net has an initial marking, $u_0 = (0,0,2,0)$. This marking results in no transitions being enabled and no hope after an infinite amount of time of becoming enabled. Conversely, a Petri net is considered live if there are any transitions enabled.

Summary

- Here, we understood about Classical Petri Nets
- Credits, Paul Frontier Book.

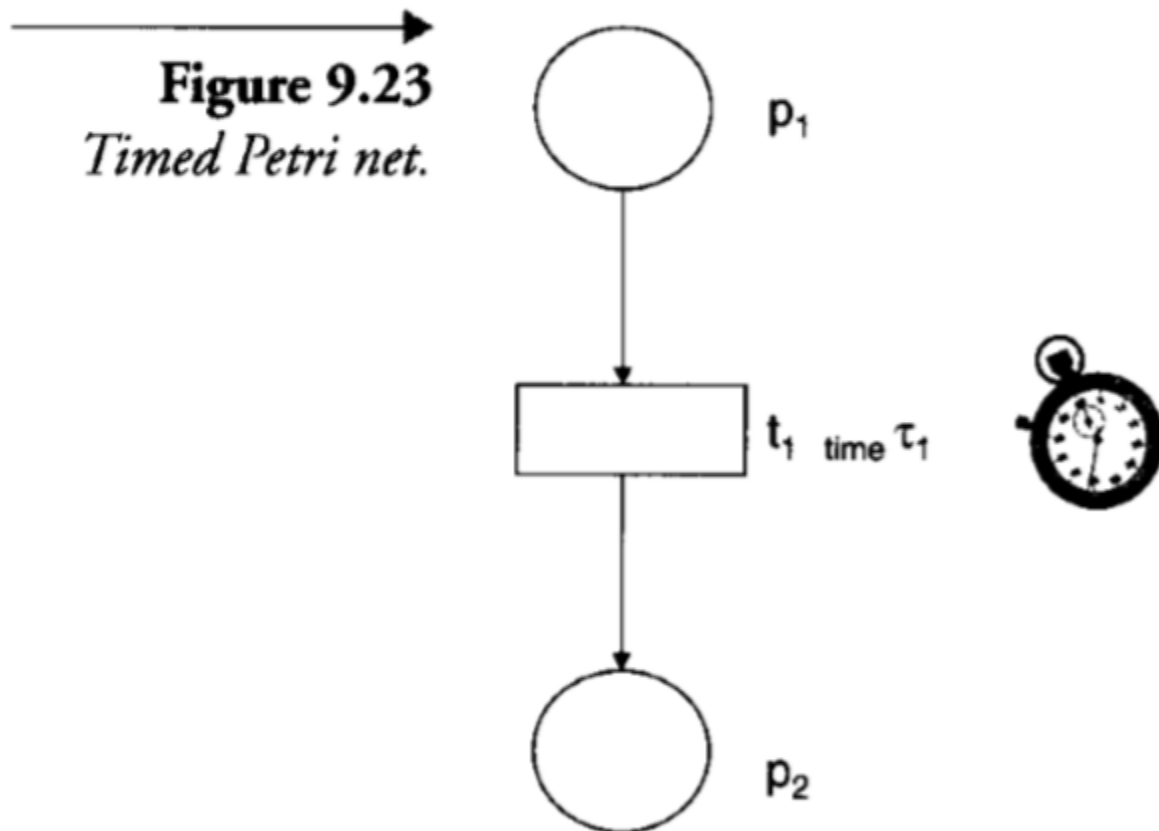
Lesson 253 : Timed Petri nets I

Overview

- Here, we will learn about the timed petri nets.

Timed Petri nets

- Every action takes some time to complete.
- Time addition provides Petri net modeler another powerful tool.
- Time in Petri net modeling is associated with transitions.



Transition $t_1^{\tau_1}$ = timed transition.

Semantic of the firing is different.

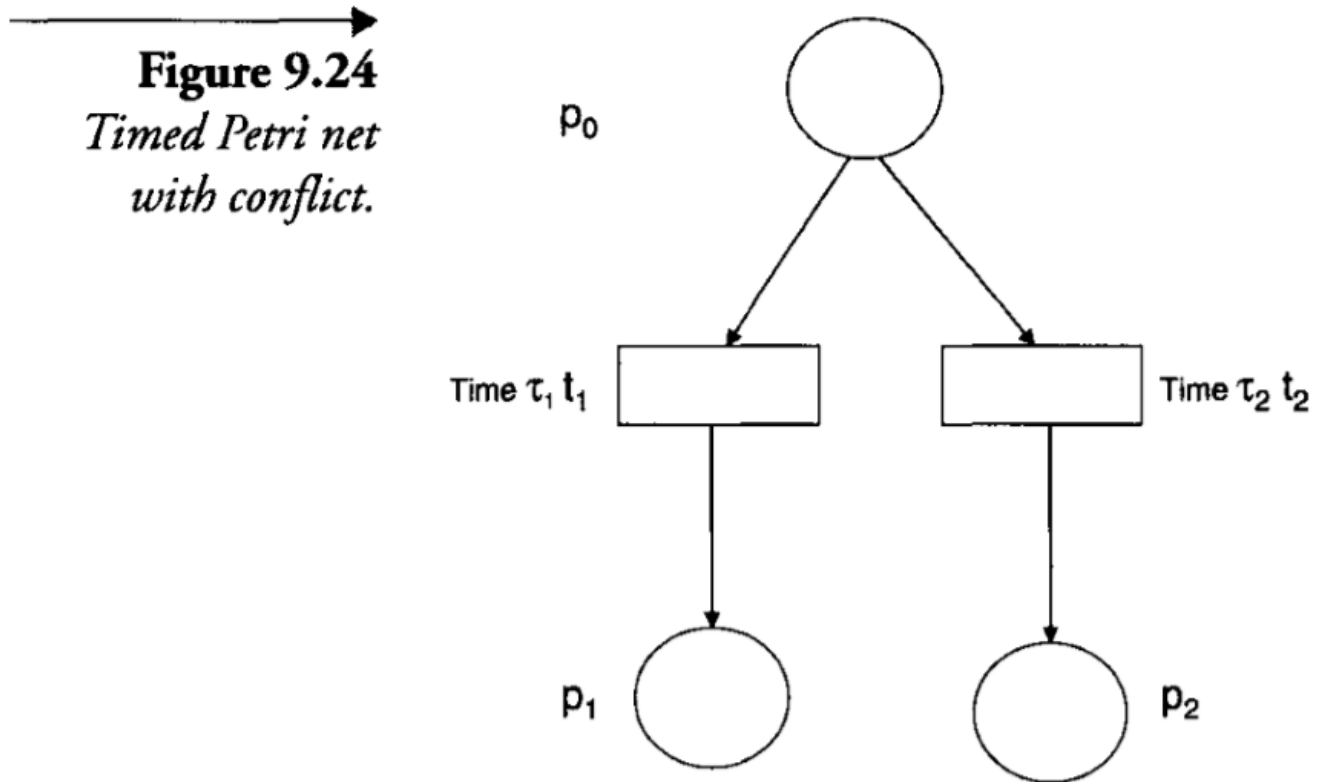
Once transition enabled, its time period clock timer is set.

Transition fires when timer reaches 0.

When token reaches p_1 the timer for transition $t_1^{\tau_1}$ is set to τ_1 .

Token moves to p_2 once t_1 time is passed.

The decrement of the timer must be constant speed.



If $t_1 < t_2$.

Two timers would begin when p_2 receives a token.

When t_1 reached 0 then t_2 becomes disabled.

Reset the timer.

Allowing transition t_2 's timer to maintain the present clock.

Summary

- Here, we have learnt about the two conditions of the timed Petri nets.
- Credits: Paul Fortier book.

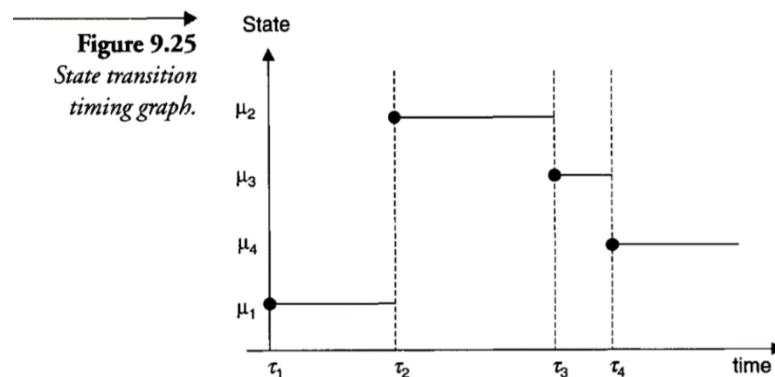
Lesson 254 : Timed Petri nets II

Overview

- Here, we will learn about the timed Petri nets.

Timed Petri nets

- Some Petri net models have proposed using state transition timing graphs.
- Each state μ^u is enumerated.
- Time period is set for each state in sequence.

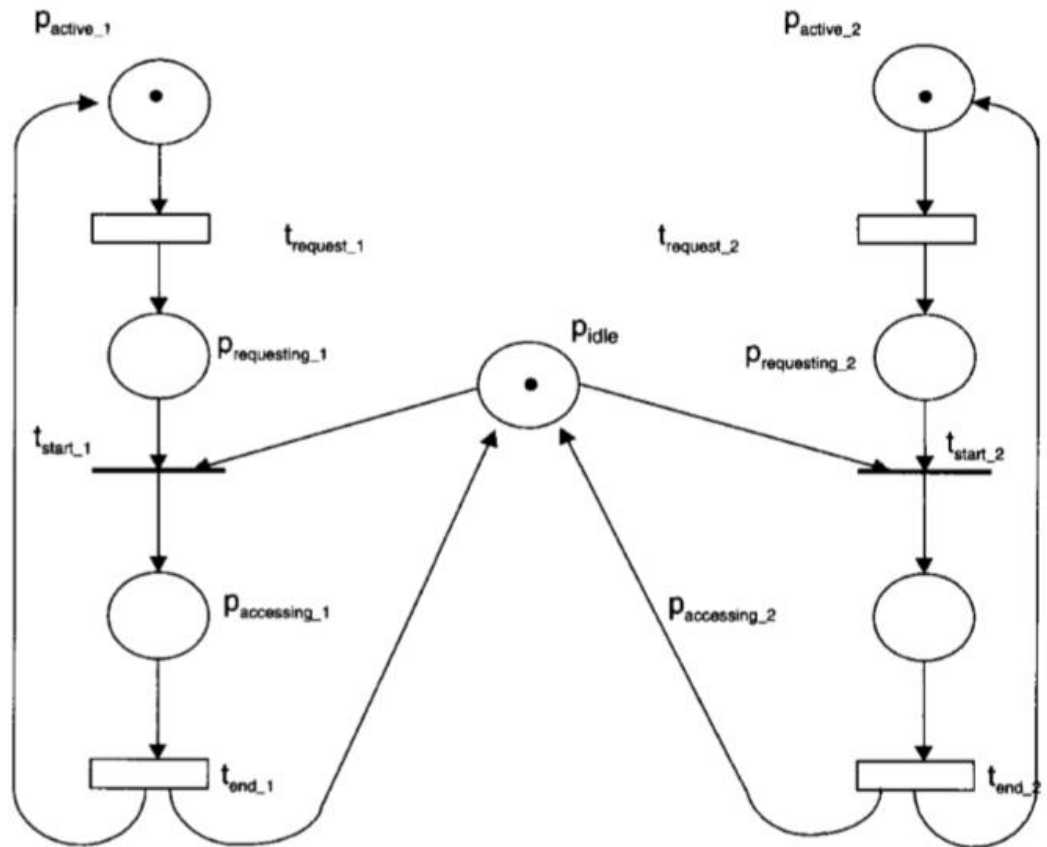


- Time units can be represented using time transition sequences.
- $[\tau_1, t_2; (\tau_2, t_2); (\tau_2, t_2); \dots; (\tau_j, t_j); \dots]$

Time units are required to move from $\mu_1^{\mu_1}$ to $\mu_2^{\mu_2}$.

Transitions can also be represented as immediate transitions.

Figure 9.26
*Timed Petri net
 with immediate
 transitions.*



Immediate transitions are used to capture resources.

Acts like a semaphores.

Active process will win it and other processes have to wait until resources become free.

Summary

- We have learnt about the timed Petri nets.
- Credits: Paul Fortier book.

Lesson 255 : Priority-based Petri nets

Overview

- Here, we will learn about the Priority-based Petri nets.

Priority-based Petri nets

- The Petri net is described by nine tuple.
- $M = (P, T, I, O, H, \Pi, PAR, Pred, \mu)$

P represents the set of places.

T is the set of transitions.

I represents the set of input functions.

O represents the set of output functions.

H represents the inhibitor functions.

Par represents the parameter set for this Petri net.

Pred represents the predicates defining how the parameter set can be distributed.

μ represents the set of markings.

Π represents the priority function

Petri nets are enabled with right amount of tokens.

No additional transitions area enabled.

t_1 has lowest priority at 1.

t_2 has lowest priority at 2.

t_3 and t_4 have the highest priority at 3.

If we start initial marking $\mu = (1, 0, 0, 1)$.

Only t_2 will be enabled.

t_1 cannot be enabled.

Figure 9.27
Priority-based Petri net.

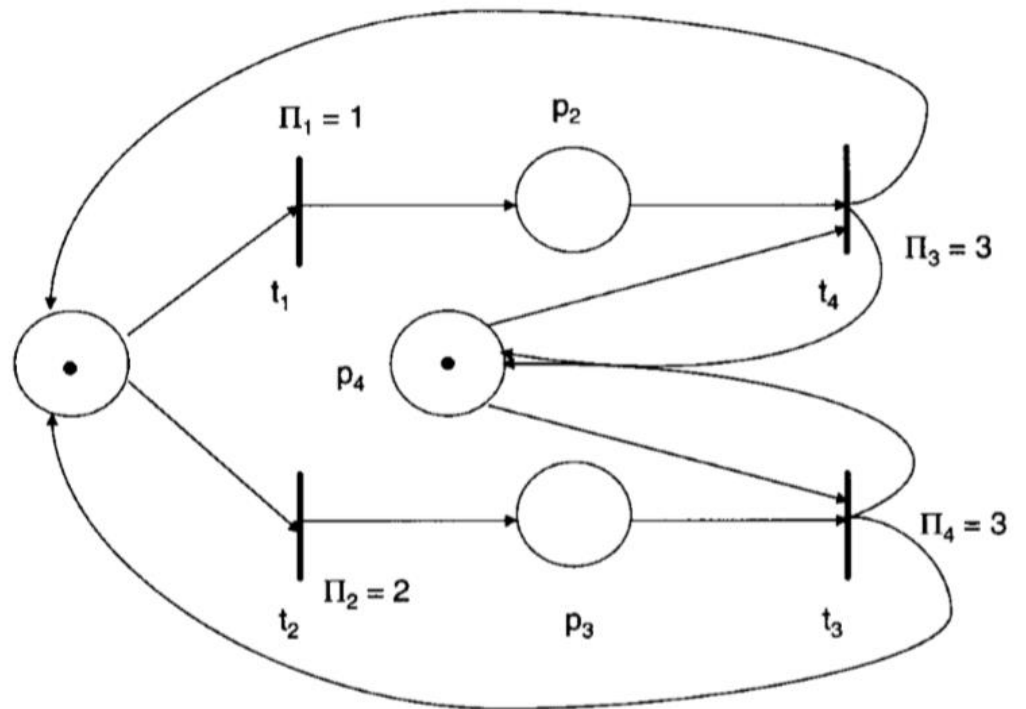
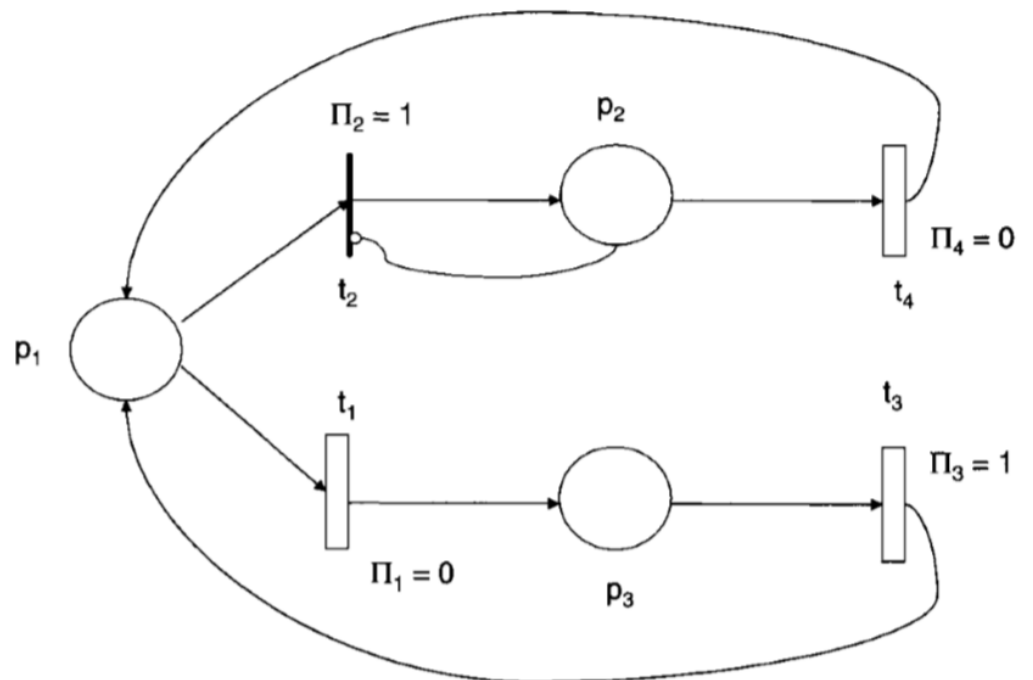


Figure 9.28
Timed and priority net.



Contention, confusion and concurrency condition can also be defined.

This picture has both time and priority features.

System can toggle between two events.

Inhibitor blocks the immediate transition at $t_2^{t_2}$.

In this way it allows $t_1^{t_1}$ to get the service.

Lesson 256 : Colored Petri nets

Overview

- Here, we will learn about the colored Petri nets.

Colored Petri nets

- Different token types.
- Colored tokens are used to represent graphically.

P represents the set of places.

T is the set of transitions.

I represents the set of input functions.

O represents the set of output functions.

H represents the inhibitor functions.

Par represents the parameter set for this Petri net.

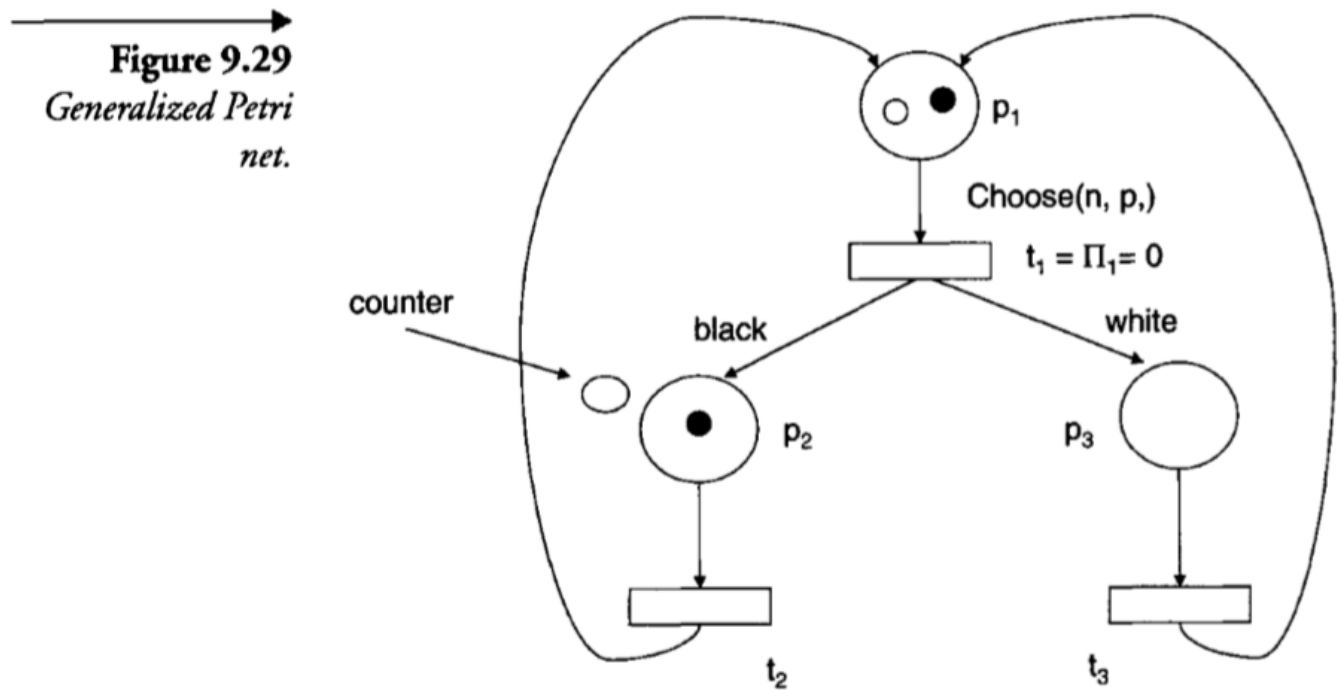
Pred represents the predicates defining how the parameter set can be distributed.

μ^u represents the set of markings.

Π^u represents the priority function

- Cardinality of the token is represented by all colors.
- All definitions of petri nets can be redefined now.

- And firing rules.



Two token types.

Transitions can have priority and time associated with them.

$t_1^{t_1}$ has priority and time.

Arc from $p_1^{p_1}$ to $t_1^{t_1}$ has condition choose.

Other arcs are used to select specific token.

- We can model about any complex condition in systems.

Summary

- We have learnt about the colored Petri nets.
- Credits: Paul Fortier book.

Lesson 257 : Generalized Petri nets

Overview

- Here, we will learn about the Generalized Petri nets .

Generalized Petri nets

- Rate or weight transition function.
- $W(t, \mu)$ must be defined.
- $W(t)$ can refer to this when function needs no definition.
- This is rate of transition.

P represents the set of places.

T is the set of transitions.

I represents the set of input functions.

O represents the set of output functions.

H represents the inhibitor functions.

Par represents the parameter set for this Petri net.

Pred represents the predicates defining how the parameter set can be distributed.

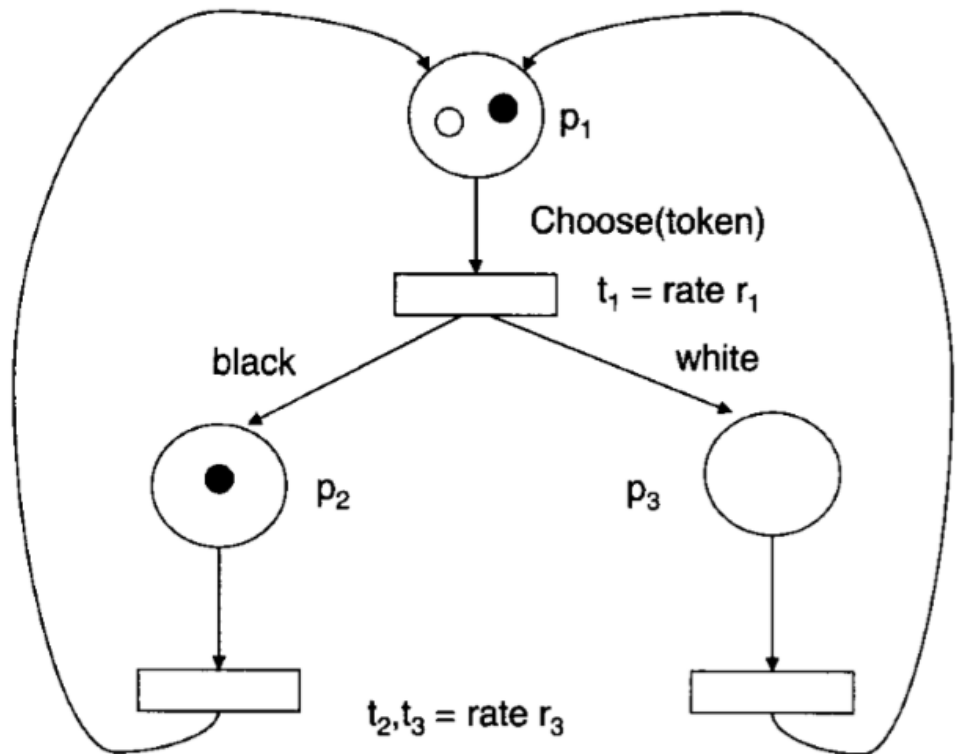
μ represents the set of markings.

Π represents the priority function

Generalized Petri nets

- To compute time to fire transition an exponential function is used.
- Rate must be define.
- Computed timer value is decremented until 0.
- System state changes from one state to another.
- **Generalized Petri nets**

Figure 9.30
Generalized
Petri Net.



initial marking is $\mu_0 = (2,0,0)$.

When t_1 is enabled then it will compute transition rate using mean rate r_1 .

Once timer value reaches 0 then we will use the selected token to choose the path.

- Each state will not be computed at the same time.

Summary

- We have learnt about the generalized Petri nets.
- Credits: Paul Fortier book.